从第一步讲起

数据结构

超级纲要 · 教材式初稿

概念 · 推导 · 原图 · 例题 · 自测

> 先弄懂问题，再看图，再一步一步算。
> 定义、公式、代码、例题和错误，都放在一起解释。

2026-10-25 · sTBook
Wiki 与 Typst 共用正文 · 原图均在图注中注明出处

章节目录

从一小段程序开始，把数据结构学成自己会用的东西 3
1 一段程序凭什么值得相信 5
2 循环究竟跑了几次 8
3 递归怎样变成一张能逐格填写的表 12
4 向量：一排连续格子怎样装下越来越多的数据 16
5 查找：先确定要找的是哪一种位置 20
6 排序时，除了大小次序还在保留什么 23
7 链表：知道位置以后，怎样少搬东西 26
8 栈：先来的事，为什么要等后来的事做完 29
9 队列与单调结构：怎样少看已经失去用途的数据 32
10 二叉树：分叉以后，怎样保证每个节点只访问一次 36
11 Huffman 编码：为什么总把最轻的两个合在一起 40
12 搜索树：每次比较之后，往哪边走 44
13 高级搜索结构，各自在为哪一种成本付账 48
14 搜索一段范围，为什么不能只照搬找一个键 52
15 散列表：算出一个位置以后，还要确认是谁 55
16 图：把“谁和谁相连”变成可追踪的状态 58
17 最短路和最小生成树，究竟在最小化什么 61
18 堆：只保证冠军站在最上面 64
19 字符串匹配：已经比对过的字，怎样不再重读 67
20 划分、选择与步长：排序还有哪些做法 71
21 PA1：从讲解里的几句话，走到能实现的状态 74
22 PS01：把“会背复杂度”变成“自己能数出来” 77
23 PS02：向量里的每一次搬动，都要知道搬了谁 87
24 PS03：链表不是“数组换了几个箭头” 96
25 PS04：先把栈里的东西写出来，再谈递归、表达式和单调结构 103
26 PS05：树的箭头、遍历顺序和编码成本，一步一步核对 111
27 把算法写成程序以后，怎样知道它没有悄悄换题 119
28 概率、摊还和归纳，怎样把省略的一步补上 122
29 把几种工具接起来，完成一整道题 125
30 PS04 的 21 个表达式：每次弹栈究竟算了什么 128
31 换掉数字以后，题目还剩下什么方法 140
32 习题 1-32：把十六段程序逐层数清楚 143
33 教材、习题解析、PS 与 PA，怎样对着解决同一个问题 147
资料覆盖、原图出处与使用说明 151

从一小段程序开始，把数据结构学成自己会用的东西

你可以会写循环，却在看到“均摊复杂度”“秩”“不变量”时卡住。这些词各有准确含义。读到一个新词时，先把它换成一个具体动作，再回来看定义。例如，“循秩访问”就是“给我第 7 个元素”；“循位置访问”就是“我手里已经拿着这个节点，给我它后面的节点”。数组擅长前一个动作，链表擅长后一个动作。这就是课程要研究的事：同样一批数据，换一种摆法，某些操作会便宜，另一些会变贵。

一张成绩表，怎样长出整门课

假设有十万名学生，每条记录有学号和成绩。现在有四种需求。

要按学号查询某个人，可以把记录按学号排好，使用二分查找。每次排除约一半候选，十万条数据只需要约 17 次缩小范围。要不断接收新成绩，直接插入有序数组可能搬动近十万条记录；搜索树把记录放在节点里，用树高换取插入与查找的成本。要随时找最高分，可以维护最大堆；它不保证所有记录有序，却保证根处是最大值。要查名字中是否包含某段字，可以使用字符串匹配，比较的是字符排列而非数值大小。

需求决定你需要维护的关系。不能因为“散列表平均很快”就拿它处理所有问题：散列表通常不能直接回答“在 80 分到 90 分之间有哪些人”；平衡搜索树可以按顺序列出这个区间。也不能因为“链表插入是常数时间”就认为按学号找人后插入也一定是常数时间，寻找插入位置仍可能走过整张表。

先统一几个说法

本书数组下标从 0 开始，长度为 n 的数组合法下标为 0 到 n−1。区间`[lo, hi)`包含 lo，不包含 hi，长度恰好是 hi−lo；空区间允许 lo=hi。除非另说，树高按边数计算：空树高度−1，叶节点高度 0。图有向还是无向、边权是否非负，会在使用算法之前说明。

算法分析默认普通下标、指针、机器字内的加减比较算一次基本操作。若整数的位数会随着 n 增长，就需要单独计算大整数成本。“输入大小 n”也必须说清：字符串题可能指字符总数，图题至少要区分顶点数 V 与边数 E。

关键词“正确”有两层。程序结束时答案满足题目要求，叫部分正确；再证明程序一定会结束，才得到完全正确。测试可以抓出某一个错误输入，数学证明处理的是所有满足条件的输入。两者都要做，它们回答的问题不同。

怎么读，才不会看完答案以为自己会了

读每个例子时，先遮住最后一行。在纸上写出当前数组、栈、队列或树，再执行一步。若执行完以后说不清“为什么这项可以删”“为什么这一半一定没有答案”，应回到前面的条件，补上理由。算法题最容易藏住的缺口，往往就是这一个“为什么”。

第一次可以按 01 到 12 章读，先把线性结构、栈队列和基本树连起来。13 到 20 章继续讨论高级搜索结构、图、堆与串。遇到 PA，直接去 21 章读对应主题，再回到它依赖的章节。PS01 到 PS05 分别在 22 到 26 章；32 章单独拆解习题解析 1-32 的十六段程序。33 章给新旧教材、习题解析与课程材料的对应关系，查资料时要区分原书页码、PDF 页码与本书章号。

每次练习都要留下可检查的痕迹。二分写 `lo`、`hi`、`mid`，动态规划写状态含义与一行表格，树旋转写旋转前后中序序列，图算法写每次选中的顶点与更新的距离。只写最终答案，很难判断你是推出来的还是猜中的。

一个三分钟起步练习

数组是`[4, 1, 7]`，从左到右求和。初始 `s=0`。处理 4 以后 `s=4`，处理 1 以后 `s=5`，处理 7 以后 `s=12`。在处理下标 i 之前，s 等于前 i 项的和。这句话在 `i=0` 时成立，因为前 0 项没有元素，和定义为 0；每次加上当前项后，它又能用于下一轮。

现在把“和”换成“最大值”。不能继续把初始值随手设为 0，因为 `[-5,-2]` 的最大值是 `-2`。可以要求数组非空，令 `best=a[0]`，从下标 1 继续；也可以维护“还没有见到任何元素”的状态。算法的一行初始化，已经在表达它接受哪些输入。

再问：如果数组有十亿项，需要多少额外空间？求和只要一个累计量和一个下标，额外空间为常数；数组本身仍占十亿项空间。“额外空间小”不等于输入数据不占空间。

继续读 01 章时，你已经有了三个抓手：变量保存什么、每一步保持什么、输入边界是什么。后面会把它们用到二分、归并、单调栈、AVL 树和最短路中。

1 一段程序凭什么值得相信

给你数组`[6, -2, 5, 1]`，求所有元素的和。你可以在脑中算出 10；计算机需要明确到“当前访问哪一项”“下次访问哪里”“什么时候停止”。算法就是这样一套可执行的步骤。数据结构决定数据怎样保存，算法决定怎样访问和修改它们。

课件里曹冲称象的例子有一个有用的细节：先记下大象使船下沉后的水位，再换成石头，直到水位相同。水位把“物体的重量”变成可以复现的状态。换物体时必须保持船和测量条件相同；船若漏水，水位相同就未必对应同样的物体重量。算法也依赖条件，证明要把这些条件写出来。

1.1 先把题目说完整

求和的输入可以写成：给定长度为 n 的整数数组 a，n 是非负整数。输出为所有项的数学和。这里允许空数组，其和定义为 0。再加一个工程条件：计算中使用的整数类型能够保存每个中间结果。若题目要求的是对某个模数取余，应明确写在输出定义中，不能把溢出当成取模的替代品。

下面用 Python 展示步骤；Python 的整数会自动增长，因此没有固定 64 位溢出，但大整数变长后，每次相加的成本也会增长。

```
def total(a):
    s = 0
    i = 0
    while i < len(a):
        s += a[i]
        i += 1
    return s
```

输入约定叫前置条件，输出应满足的关系叫后置条件。这个说法看起来正式，其实就是“你给我什么”“我承诺还给你什么”。如果函数需要已排序数组，排序就是前置条件；若调用者给乱序数组，不能再使用那份只针对有序输入的证明。

1.2 例 1：每一轮开始，变量到底代表什么

跟踪`[6, -2, 5, 1]`：

轮次开始的 i	轮次开始的 s	此时已经处理的元素	本轮动作
0	0	没有	加 6，然后 i 变 1
1	6	6	加−2，然后 i 变 2
2	4	6, −2	加 5，然后 i 变 3
3	9	6, −2, 5	加 1，然后 i 变 4
4	10	全部四项	条件不成立，结束

我们要表达的规律是：每轮开始时，`0 <= i <= n`，并且 s 等于`a[0:i]`中所有元素的和。方括号切片`a[0:i]`表示下标从 0 到 i−1 的元素。规律在每轮都保持，所以叫循环不变式。

证明分三步。开始时 i=0、s=0，前 0 项的和确实是 0。假设某轮开始时规律成立，本轮将 a[i]加进去，再让 i 增加 1，s 便等于新的前 i 项之和，规律继续成立。循环结束时条件`i<n`为假，结合`i<=n`可得 i=n，于是 s 就是全部元素的和。

这个证明覆盖了空数组：初始 i=n=0，循环一次也不执行，返回 0。它也覆盖负数，因为证明用的是加法关系，没有偷偷假设每一项为正。

1.3 不变式还没有证明它一定结束

写一个死循环，同样可以保持“s 等于前 i 项之和”：只要循环里什么都不做，s 和 i 便永远符合初始关系。为说明程序会结束，还需要找一个非负整数，证明它每轮严格减小。本例可以用 n−i。它从 n 开始，每轮减 1，最多经过 n 轮就变为 0。这样的量常叫下降量。

“非负整数”不能随便改成“非负实数”。实数序列 1、1/2、1/4、1/8 可以永远减小而不达到 0。下降量的取值空间必须不允许无穷严格下降，才足以推出有限终止。

部分正确性是“如果结束，答案正确”；终止性是“它会结束”。两者合起来，才能保证每个合法输入都得到正确答案。这个区别在递归中也一样：递归调用参数变小，必须小到某个明确的基础情况，而且不能越过基础情况。

1.4 例 2：最大值，错一行初始化就换了一道题

试求`[-8, -3, -5]`的最大值。如果令`best=0`，每一项都无法更新 best，最后得到 0，而输入中根本没有 0。正确写法之一是要求 n>0，让`best=a[0]`，再扫描剩下的元素。

初始 best=−8。读到−3，因为−3>−8，更新 best=−3。读到−5，不更新，结果−3。对应不变式是“处理完前 i 项以后，best 等于这 i 项的最大值”。初始化处理了第 1 项，所以循环应从 i=1 开始。若接口必须接收空数组，可以返回一个明确的“无最大值”结果，或者抛出异常；这属于接口设计，不能悄悄捏造 0 作为答案。

找最大值需要至少检查全部 n 项。若有一项没看过，把那一项改成比其他所有项都大的数，程序的执行仍一模一样，却必须给出不同答案，矛盾。因此普通未排序输入中，最坏至少需要线性数量的读操作。这里的下界来自“尚未读取的位置可以藏着答案”。

1.5 例 3：辗转相除，下降量能把递归讲明白

求正整数 48 与 18 的最大公因数。最大公因数是同时整除两个数的最大正整数。用 48 除以 18，余 12，因此`gcd(48,18)=gcd(18,12)`；接着余 6，得到`gcd(18,12)=gcd(12,6)`；再余 0，得到答案 6。

为什么可以丢掉 48，只留下 18 和 12？因为 48=2×18+12。若 d 同时整除 48 和 18，就也整除 12；反过来，若 d 同时整除 18 和 12，也整除 48。两组数的公因数集合相同，因此最大公因数相同。

```
def gcd(a, b):
    # a >= 0, b >= 0, 且不同时为0
    while b != 0:
        a, b = b, a % b
    return a
```

每轮保存“当前 a、b 的最大公因数等于原输入的最大公因数”。新的 b 是旧 a 除以旧 b 的余数，满足`0 <= 新b < 旧b`。所以 b 是非负整数下降量，必然变为 0。这一次，等式负责正确性，余数范围负责终止性。它们各干一件具体的事。

最坏轮数是对数级，而不是简单由“b 每次变小”直接推出。连续两轮以后较大的参数至少减半，可推出轮数 O(log max(a,b))；更精细的最坏输入与相邻 Fibonacci 数有关。若输入是大整数，整除一次的位运算成本仍应另计。

1.6 机器模型：什么算“一步”

分析数组扫描时，我们常把读`a[i]`、比较和普通整数加法各算常数次。这对应 RAM 模型的一个版本：内存单元能按地址直接访问，每个机器字能容纳下标及通常的数据。该模型适合解释数组与链表等结构的操作成本。

若有一个长度为一百万位的整数，把它和另一个同样长的整数相乘，不能继续说“一条乘法指令，所以 O(1)”。普通处理器一次只能处理有限位数，大整数库需要许多字操作。某些习题明确假设无限字长、移位为单位成本；可以在该模型下求答案，但不要直接把模型结论当作真实 C++的运行时间。

输入大小也要说清。程序枚举 1 到 N 用了 Θ(N)次；可是把 N 以二进制写给程序，只需大约 $\log_2 N$ 位。令输入位数为 b，N 可以接近`2^b`，于是这个算法按输入位数可能是指数时间。一层循环照样可能很慢。

1.7 随机过程与通用停机判定

每轮独立掷公平硬币，正面就停止。恰好第 k 轮停止的概率是`2^(-k)`，永远不出正面的概率为 0，期望轮数为 2。但对于任意有限的上限 m，连续 m 轮反面仍有正概率`2^(-m)`。所以“概率 1 终止”与“有一个统一有限最坏轮数”是不同的承诺。

一般程序的终止性没有万能判定器。假设有程序 H 能对任何程序 P 及输入 x 判断会不会停机。构造 D：先问 H“D 对自身输入会停吗”；H 说会停，D 就无限循环；H 说不会停，D 就马上结束。无论 H 回答什么都会错。这说明不能对所有任意程序实现这样的完美判定器，并不妨碍我们对求和、二分和欧几里得算法分别给出证明。

1.8 再看一次机器：二进制加一怎样在纸带上执行

图灵机把数据写成带子上的符号，读写头一次观察一个格子，并根据当前状态决定写什么、向哪移动、换什么状态。它不会一次看见整串数字，所以“加一”也必须拆成小动作。假设二进制串最高位在左，头开始在最右位。加一时，遇 0 就改 1 并结束进位；遇 1 就改 0，向左继续；若一直越过最高位，则在左侧空格写 1。

对 1011 加一：最右 1 改 0，状态变为仍有进位；再遇 1 改 0；再遇 0 改 1，进位消失，剩下最左 1 不动，结果 1100。对 111 加一，会把三个 1 都改 0，左侧空格写 1，得 1000。对 1000 加一，只需把最后 0 改 1，得 1001。如果要求执行结束后读写头回到某个固定起点，还要把移动回去的成本计入。

这个例子说明，数学上一个“+1”与某台机器的一条指令并不总是同一件事。长度 b 的二进制数最坏有 b 个连续末尾 1，进位会走 Θ(b)格；RAM 字内加法常被模型视为 O(1)，是把有限字长硬件操作作为基本单位。两种分析描述不同模型，不矛盾。

减一也可以自己设计：遇 1 改 0 并结束借位，遇 0 改 1 并继续向左。1000 减一，三个末尾 0 依次改 1，再把最高 1 改 0，得到 0111，按表示约定去前导 0 后为 111。输入 0 若仍要求非负整数，则没有合法结果，应由接口拒绝，不能无限向左借位。

1.9 当“验证答案”比“找答案”容易

给出整数 221，想分解成两个大于 1 的因子。找到 13×17 可能要试许多候选，但别人给出 13 与 17 后，乘一下就能核对 221。验证某一份候选只需检查它满足的关系；验证算法对所有输入正确，还要说明搜索不会漏掉任何可能因子并会终止。

朴素试除只需到平方根：若 N=ab 且 a,b 都大于√N，则乘积会大于 N，矛盾，所以合数至少有一个因子不超过√N。对 221 试 2、3、5、7、11、13 时找到 13，再除得 17。若只试奇数，仍必须单独检查 2。素数输入会一直试到√N 附近，成为这种实现的坏情况。

若 N 以 b 位二进制给出，√N 可接近 `2^(b/2)`，所以试除按数值看是平方根级，按输入位数看仍可能指数。这个具体问题把输入编码、基本操作成本与最好最坏情况连在一起：重复除 2 很快的偶数，不能代表难分解的奇数实例。

测试中找到一份因子可以证伪“它是素数”；测试许多候选都没找到，只在你已经覆盖全部必要候选后才能推出素数。算法证明中的覆盖范围，决定“没找到”能不能变成一个可靠答案。

1.10 自己做，再核对

练习 1。下面的循环想把 a 原地倒置：`i=0,j=n-1; while i<j: swap(a[i],a[j]); i+=1; j-=1`。给出不变式与下降量，并追踪`[1,2,3,4,5]`。

解答：保存一份只用于证明的原数组 b。在每轮开始，i 左边的位置都已满足 `a[k]=b[n-1-k]`，j 右边的位置也满足同样关系，中间`[i,j]`还待处理。第一次交换后数组为`[5,2,3,4,1]`，i=1,j=3；第二次为`[5,4,3,2,1]`，i=j=2。中间只剩一个元素，它本来就在对称中心。下降量可取尚未确定的元素数 j−i+1，执行一轮就减少 2；只在 i<j 时执行，因此不存在无限下降。n=0、n=1 都不进入循环。

练习 2。程序将正整数 x 反复执行 `x=x//2`，直到 x=0。有人说“每次至少减 1，因此时间 Θ(x)”，哪里不够？

解答：至少减 1 只能给出 O(x)上界，不能证明线性下界。取 x=20，状态依次 20、10、5、2、1、0，共 5 轮。执行 k 轮后为 $\lfloor x/2^k \rfloor$；首次为 0 发生在 $k=\lfloor \log_2 x \rfloor+1$，因此时间 Θ(log x)。终止证明与紧确复杂度证明所需的信息量不同：知道会变小足以证明结束，算多快要知道每次缩小的比例。

继续阅读时可以把本章的证明方法贴在每个算法旁边：输入条件、变量含义、保持关系、下降量、结束后得到的结论。具体内容会变，检查的对象不会消失。

2 循环究竟跑了几次

一段程序有两层循环，未必就是平方时间；一段只有一层循环，也可能跑上指数次。要算成本，先选一项要计数的基本操作，再把它执行的次数写出来。通常选比较、赋值、访问元素等操作即可，因为在固定字长模型中，一轮里有限个此类操作只差一个常数倍。

2.1 O、Ω、Θ 分别说了什么

设 T(n)表示规模 n 时的成本。T(n)=O(g(n))表示：存在正常数 c 和阈值 n_0，所有 $n \ge n_0$都有 $T(n) \le cg(n)$。它给上界。Ω 给下界。Θ 同时给两边的常数倍夹逼，说明增长级别恰好相同。

例如 $T(n)=3n^2+7n+20$。$n \ge 1$ 时，$3n^2 \le T(n) \le 30n^2$，所以 $T(n)=\Theta(n^2)$。它当然也属于 $O(n^3)$，只是那个上界较松。写了 $O(n^2)$以后，不能自动改口说“恰好平方级”；你还没有写下界。

大 O 也不自带“最坏情况”含义。可以研究最好成本 B(n)、最坏成本 W(n)、某个概率模型下的期望成本 E[T(n)]，然后分别给它们大 O 或 Θ 界。“平均”必须说明对谁取平均；“摊还”则在分析一串操作的总账，不需要随机输入。04 和 28 章会具体算这两种账。

2.2 例 1：三角形循环，真的数出每一行

```
for i = 0 .. n-1:
    for j = 0 .. i-1:
        do_one_operation()
```

n=5 时，i=0 执行 0 次，i=1 执行 1 次，接着 2、3、4 次，总共 10 次。一般为

$$0 + 1 + \cdots + (n-1) = \frac{n(n-1)}{2}.$$

为什么这个和是平方级？上界：每行不超过 n，共 n 行，所以最多 n^2。下界：后半段约 n/2 行，每行至少约 n/2 次，所以至少是某个常数倍的 n^2。上下界一起给 $\Theta(n^2)$。把正方形切去一半，只改变常数；若要降到线性，必须把总方格数量降到常数倍 n。

循环条件本身也要检查。内层为 0 次时，仍要进行一次条件判断；所有条件判断额外 O(n)，不会改变这里的平方级结论，但在主体很少执行的代码中可能成为主项。

2.3 例 2：变量每次翻倍，数的是指数

```
i = 1
while i < n:
    i *= 2
```

n=20 时，循环开始处 i 为 1、2、4、8、16，5 轮以后变成 32，结束。第 k 轮以后 i=2^k，首次达到 n 需要 k= $\lceil \log_2 n \rceil$ 。因此 $n \ge 2$ 时成本为 $\Theta(\log n)$。n=1 不执行循环，边界值单独处理即可。

如果把上限 n 改成固定常数 1024，循环不超过 10 轮，关于 n 的成本就是常数。出现*=2 不代表必有 log n；得看它在多大的范围里增长。

还有一种容易混淆的程序：`i=1; while i<n: do i operations; i*=2`。各轮成本为 1、2、4、8……，总和小于 2n，因此为 Θ(n)。轮数是对数级，总代价却是线性，因为每轮越来越贵。

2.4 例 3：步长是 i，步数大约是 n/i

```
for i = 1 .. n:
    for j = 0; j < n; j += i:
        do_one_operation()
```

n=8 时，各个 i 对应的内层次数依次是 8、4、3、2、2、2、2、1。别把“j 每次增加 i”读成“内层执行 i 次”。i 越大，一步跨得越远，循环次数越少。

总次数为 Σ `ceil(n/i)`。令调和数 $H_n=1+1/2+\ldots+1/n$。每个向上取整最多多 1，因此

$$nH_n \le \sum_{i=1}^{n} \lceil n/i \rceil < nH_n + n.$$

H_n 为什么是 Θ(log n)？把分母按 1，2 到 3，4 到 7，8 到 15 分组。每组大约有 2^k 项，每项大约是 1/2^k，该组之和在两个正常数之间。共有约 $\log_2 n$ 组，所以总和为对数级。于是整个程序 Θ(n log n)。

积分也能给更精细的夹逼：`ln(n+1) <= H_n <= 1+ln n`。用积分前先观察函数 1/x 递减，才能确定每个矩形在曲线上面还是下面；不需要把积分当成凭空出现的公式。

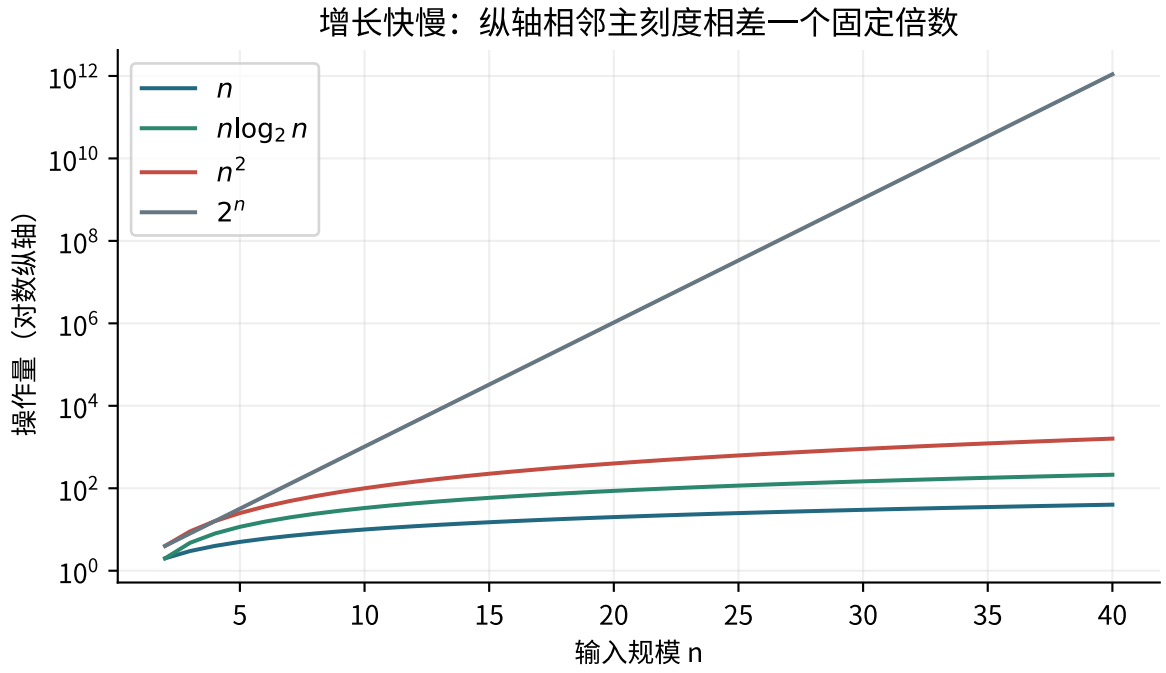


图 1　不同增长函数的操作数量比较

横轴是输入规模，纵轴采用对数刻度，每个主刻度间隔代表乘上一百，而非增加一百。指数曲线看起来接近直线，是取了对数纵轴后的显示效果；它在规模四十时已约一万亿，平方曲线只有一千六百。画图坐标本身也需要阅读，不能只看线弯不弯判断增长。

2.5 常见级数，用状态变化来认

等差和 1+2+…+n 对应每轮多做一点，增长为 Θ(n²)。几何和 1+2+4+…+2^k 由最后一项的常数倍控制，若最后一项约为 n，和就是 Θ(n)。平方和 1²+2²+…+n² 为 Θ(n³)：上界 n×n²，下界后 n/2 项每项至少(n/2)²。一般固定 p≥0 时，`Σ i^p=Θ(n^(p+1))`。

倒数平方和 `Σ 1/i²` 却有常数上界。按 2 的幂分组，第 k 组最多 2^k 项，每项至多 1/2^(2k)，该组不超过 1/2^k；所有组再相加，是收敛的几何级数。因此看到求和号，不能一律乘一个 n。

对数和 `Σ log i` 等于 log(n!)。上界 n log n；后半 n/2 项每项至少 log(n/2)，所以为 Θ(n log n)。后面比较排序的下界会再次出现这个数量。

2.6 例 4：递归树，每层到底付多少钱

归并排序将长度 n 的数组分成两半，递归排好，再用线性时间归并。先假设 n 是 2 的幂，递推为

$$T(n) = 2T(n/2) + \Theta(n), \qquad T(1) = \Theta(1).$$

根这一层合并 n 项。下一层有 2 个问题，每个合并 n/2 项，总共仍 n 项。再下一层 4 个问题，各 n/4 项，总共还是 n。直到问题大小为 1，共 $\log_2 n$ 层非叶合并，每层 Θ(n)，所以 Θ(n log n)。叶子有 n 个，共 Θ(n)，不改变结果。

如果每个节点只做常数工作，递推改成 `T(n)=2T(n/2)+Θ(1)`，每层成本是 1、2、4……，最后一层约 n，所以总成本 Θ(n)。同样两个递归调用，合并成本不同，答案就不同。

如果只有一个子问题：`T(n)=T(n/2)+Θ(n)`，层成本 n、n/2、n/4……，总共 Θ(n)。而 `T(n)=T(n−1)+Θ(n)`的层成本 n、n−1……，则为 Θ(n²)。先画两三层，再套定理，更不容易把公式放错位置。

2.7 主定理是一个有条件的工具

对形如 T(n)=aT(n/b)+f(n)的规则递推，a≥1、b>1 为常数。n^(log_b a)反映叶子的规模。若 f 比它多项式地小，叶子主导；若 f 与它同阶，通常多一个 log n；若 f 比它多项式地大，且满足常用的正则条件，则本层大工作主导。

例如 2T(n/2)+n 符合相等情形，得 n log n；4T(n/2)+n，叶子规模 n^2大于 n，得 n^2；2T(n/2)+n^2，本层为平方级，且 $2(n/2)^2=n^2/2$ 满足正则条件，得 n^2。

T(n)=T(n/2)+T(n/4)+1 不符合“所有孩子大小都为 n/b”的形式，不能把两个孩子硬说成 2T(n/2)。32 章会把 n=2^k 后转成 Fibonacci 递推，得 Θ(n^(log₂φ))。更一般的 Akra–Bazzi 方法可处理不同比例，但其应用同样有连续性、增长与扰动范围等条件；不能把“任意递推都可套”当成结论。

2.8 时间之外，再数同时活着的空间

递归调用树可能总共有 n 个节点，但不会所有调用同时占着栈。深度优先执行时，一条从根到当前节点的路径上的调用尚未返回。二分递归若每层只存常数局部变量，调用栈是 O(log n)。线性递归每次 n−1，栈深可达 O(n)。

归并排序通常还需要一个长度 n 的辅助数组，空间为 O(n)，这比 O(log n)调用栈大。若在每层都复制子数组，某些语言的切片还会带来额外分配和复制；复杂度分析必须对应实际实现。

2.9 对数变换会保留哪些界

若 T(n)≤c f(n)，取对数得到 log T≤log c+log f。要把右侧写成 O(log f)，还需要 log f 最终为正且不会趋近 0，例如 f 最终至少为某个大于 1 的常数。若 f 恒等于 1，log f=0，而 T 恒等于 2 仍有 T=O(f)，但 log T=1 不能是 O(0)。边界条件不能在取对数时消失。

反方向也不成立：log T=O(log f)只给出 T≤f^c 乘常数，不能推出 T=O(f)。例如 $T=n^2$、f=n，log T=2log n，确实同阶；T/f=n 却无界。指数函数会把对数域里的常数倍放大成幂次，这正是信息损失的位置。看到复杂度函数先取对数再比较，应把常数移回原空间重新检查。

小 o 记号比大 O 更强。f=o(g)表示对任意 ε>0，充分大时|f|≤ε|g|，常用正函数下相当于比值趋 0。n=o(n log n)，因为比值 1/log n 趋 0；n=O(n)成立，却不是 n=o(n)。小 ω 反过来表示比值趋无穷。符号相似，但严格增长与常数倍同阶不能混为一谈。

2.10 代入法怎样补一个递归树的直觉

递归树给出 T(n)=2T(n/2)+n 约为 n log n 以后，可用归纳验证上界。假设较小规模 T(m)≤c m $\log_2$m，再代入得 T(n)≤2c(n/2)$\log_2$(n/2)+n=c n $\log_2$n−cn+n。取 c≥1 便不超过 c n $\log_2$n，基础小规模再选足够常数覆盖。下界也能按每层至少 cn 工作与层数给出，不能只凭上界就写 Θ。

有时猜 T(n)=cn 不够。例如 T(n)=T(n/2)+n，代入 cn 得到 cn/2+n，要求 c≥2，可成立；但 T(n)=2T(n/2)+n 代入 cn 会变 cn+n，无论 c 多大都无法压回 cn，说明线性猜测不够。代入失败也可能只是猜式缺一个减常数或低阶项，需结合递归树判断。

非二次幂输入可以把数组分成 floor(n/2)与 ceil(n/2)，最长深度不超过 ceil($\log_2$n)，每层元素总数仍 n，所以同阶证明不依赖完美均分。写教材时先用 2 的幂看结构是简化推理，最后仍应交代一般整数规模。

2.11 封底估算要用增长比，而非只背秒数

假设每秒能做一亿次某种简单操作，这是估算假设，不是所有机器与所有指令的承诺。n=10^5时，n^2约 10^{10}，单靠常数优化通常救不了平方扫描；n $\log_2$n 约 1.7×10^6，则在许多场景里有余地。内存访问、字符串比较与分配比一个整数加法贵，估算应留余量。

指数枚举更要早算：2^{20}约一百万，2^{30}约十亿，2^{40}约一万亿。把 n=40 分两半，各 2^{20}项，能从不可行数量降到可处理列表，这是 PA Gift 的来由。它仍然指数，只是可承受规模区间被改变。

2.12 分数次成本从哪里来，机器明明只做整数次

调和和式里出现 n/i，不表示程序真的执行半次操作，而是用连续量夹住整数计数。若一次循环实际 ceil(n/i)次，便有 n/i≤次数<n/i+1。每项误差小于 1，n 项总误差小于 n，主项若为 n log n，这份误差不会改变阶。把取整界写出来，分数估计就与整数执行建立了联系。

若主项本来只有常数或对数，累计 n 个取整误差反而可能成为主导，不能在所有求和里都随手丢掉。分析 floor(n/i)时，也要保留每项至多 1 的误差，再看总和量级决定是否可忽略。渐近简化是证明之后的结论，不是先把麻烦符号全部删掉。

对数底数同样只在固定底数之间差常数倍：log_a n=log_b n/log_b a，a、b 为固定大于 1 常数，因而同 Θ 级。若底数随 n 变化，例如 log_n n=1，就不能再把它与 $\log_2 n$ 当同阶。32 章迭代对数也必须固定使用的底与停止阈值后才谈精确次数。

2.13 固定多项式间隔，比任何固定对数幂都强

比较 n log n 与 n 的一点零一次方，较小规模可能前者更大，但当 n 无界增长，log n 相对 n 的零点零一次方最终趋于零。因此前者渐近更小。阈值可能极其巨大，渐近优劣并不保证手边规模上的实际快慢。

将 n 写成二的 k 次方后，问题变成比较 k 与某个固定底数大于一的指数增长；指数最终超过线性或任何固定次多项式。这是主定理里“多项式地小或大”的来由：需要固定正指数间隔，而非只知道某项偶尔小一点。若比较 n 与 n 除以 log n，两者只差对数因子，不属于固定多项式间隔，不能直接塞进最简单主定理的第一种情况。

最后还要分清算法与问题。一个算法平方时间，并不证明这个问题必须平方时间；最大区间从立方降到线性就是反例。要给问题下界，必须约束所有符合模型的算法，而不是只分析手中这一段代码。

2.14 练习与完整答案

练习 1。 `i=0,j=1; while i<n: i+=j; j+=1` 执行几轮？列出 n=10 的状态。

解答：每轮结束后 i 依次 1、3、6、10，j 依次 2、3、4、5，共 4 轮。k 轮后 i=k(k+1)/2，所以最小 k 满足 k(k+1)≥2n，k=Θ(√n)。只看到“while i<n”无法决定次数；i 每轮的增量由另一个状态 j 控制。若 j 从 0 开始，则第 1 轮 i 不动，精确轮数不同，渐近阶仍为平方根。

练习 2。 比较 A：`T(n)=3T(n/3)+n`，B：`T(n)=T(n−1)+1`。哪一个调用树节点更多？哪个时间更长？

解答：A 每层总工作 n，约 $\log_3 n$ 层，时间 Θ(n log n)；它的满三叉调用树叶子约 n，节点总数 Θ(n)。B 是一条 n 层长链，每层常数工作，节点数 Θ(n)，时间 Θ(n)。两者节点数同阶，节点本身的工作不同，时间可以不同。A 的调用栈深 Θ(log n)，B 的调用栈深 Θ(n)，也再次说明总调用数与同时占用的栈深是不同指标。

练习 3。 有人对 `1+2+4+…+n` 写出 O(n log n)，理由是“约 log n 项，每项最多 n”。这个答案是否错误？

解答：作为上界，它成立；作为紧确增长级别，它太松。设 `n=2^k`，总和`=2^(k+1)−1=2n−1`，所以 Θ(n)。分析时允许先写一个松上界帮助判断，但最终题目若问渐近复杂度，应继续找匹配下界，或者给精确和式再化简。

单独计算上界以后，再问一个合法输入能否达到同阶成本，能帮助你区分松上界与紧确界。

3 递归怎样变成一张能逐格填写的表

一个数组[-2, 3, -1, 4, -5, 2]，哪一段连续元素的和最大？“连续”要求选一段相邻位置，不能跳过中间的-1。这里最佳是[3,-1,4]，和为 6。先用这一个具体问题，观察同一件事怎样从反复枚举，变成只保留必要状态。

3.1 例 1：最大连续区间，从三重循环走到一遍扫描

最直接的方法，枚举左端 l，再枚举右端 r，每次重新求 a[l:r+1]的和。有 $\Theta(n^2)$个区间，每次最多加 n 项，总时间 $\Theta(n^3)$。n=6 时有 6×7/2=21 个非空区间；规模变成十万时，不能再指望逐个重新加。

先把“重新求和”删掉。固定 l 以后，令 s=0，右端每向右移动一位，就做 s+=a[r]。同一个左端下，每项只加一次，总时间降为 $\Theta(n^2)$。这一步没有改变要检查的区间，只是复用了上一次算出的部分和。

再问一个更窄的问题：所有恰好在下标 i 结束的非空区间，最大和是多少？记为 end[i]。这样的区间只有两种：只取 a[i]，或者把某个在 i−1 结束的区间接上 a[i]。要接，当然选前一类里最大的，所以

$$end[i] = \max(a[i], end[i-1] + a[i]).$$

令 best[i]为前 i+1 项中任意非空区间的最大和，则 best[i]=max(best[i-1],end[i])。两个状态一个管“必须在这里结束”，另一个管“此前任何地方结束”，含义不能混用。

i	a[i]	end[i]的候选	end[i]	此前最佳 best
0	−2	只取−2	−2	−2
1	3	3 与−2+3	3	3
2	−1	−1 与 3−1	2	3
3	4	4 与 2+4	6	6
4	−5	−5 与 6−5	1	6
5	2	2 与 1+2	3	6

每一项只读取一次，时间 $\Theta(n)$；若只要最大和，用两个变量替代整张表，额外空间 $O(1)$。要返回区间端点，还需保存当前候选起点，以及 best 被更新时的左右端点。

这份递推为什么没有漏？任一以 i 结束的非空区间，要么长 1，要么去掉末项后确实以 i−1 结束；后者若不是前一状态中的最大候选，把它换成更大的候选不会改变末端且只会增大总和。所有可能性都已涵盖。

空区间是否允许会改变初始化。若允许空区间，答案至少 0，可设 end=best=0，递推取 max(0,end+x)。若要求非空，[-4,-2,-7]应答-2，不能初始化为 0。题目改了一个词，算法就要跟着改。

3.2 同一个问题，分而治之怎样想

把数组切成左右两半，最大区间可能完全在左边、完全在右边，或者跨过中点。前两类交给递归；跨中点的区间一定由“左半的最大非空后缀”和“右半的最大非空前缀”拼成。各从中点向外扫一次，就能求出它。

在原例中，切为[-2,3,-1]与[4,-5,2]。左半最大后缀为[3,-1]，和 2；右半最大前缀为[4]，和 4，跨中点候选为 6。三类候选比较后得到 6。递推 T(n)=2T(n/2)+θ(n)，成本 $\Theta(n \log n)$。它比一遍扫描慢一些，但训练了“分类覆盖全部可能解”的证明方法。

还能让每个子问题返回四项：总和、最大前缀、最大后缀、最大区间。左右结果合并只需常数时间，得到 $\Theta(n)$的分治；这四项也正是后面线段树维护最大区间时会使用的摘要。选择状态时，要保留未来拼接所需的信息。

3.3 例 2：Fibonacci 递归，重复的到底是哪一项

定义 $F_0=0$，$F_1=1$，$F_n=F_{n-1}+F_{n-2}$。直接翻译成递归很容易读懂，但求 F_5时，F_3会在求 F_4时算一次，又作为 F_5的另一个孩子再算一次。更深的 F_2、F_1重复得更多。递归树的节点数约按 φ^n 增长，其中 $\varphi=(1+\sqrt{5})/2$。

重复的原因不是递归语法本身，而是同一个参数 n 的答案被多次重算。加一个记忆表，第一次遇到 k 就计算并保存 F_k，以后直接读取。共有 n+1 种参数，每种只计算一次，普通字成本模型下时间 O(n)、表空间 O(n)，递归栈仍可能深 n。

如果按 k 从小到大填表，F_k 只依赖已经算好的 F_k_{-1}和 F_k_{-2}，就可以不用递归。求 F_6的过程：

准备生成	较前的值	较后的值	新值
F_2	0	1	1
F_3	1	1	2
F_4	1	2	3
F_5	2	3	5
F_6	3	5	8

每次把两个变量向前挪一格，额外空间 O(1)。如果要打印 F_0到 F_n，输出本身就有 n+1 项；若要精确大整数，F_n 有 Θ(n)位，逐次相加的位成本总和不再是 O(n)。说复杂度时应同时给出模型。

3.4 例 3：最长公共子序列，先说清“可以跳”

字符串 ABCBDAB 和 BDCABA 有公共子序列 BCBA，长度 4。子序列可以跳过字符，但保留相对顺序；子串要求连续，不能混淆。这里不要求列出所有公共子序列，只求最长的长度并恢复其中一种。

记 D[i][j]为第一个串前 i 个字符与第二个串前 j 个字符的最长公共子序列长度。注意 i、j 表示前缀长度，所以字符位置是 i−1、j−1。只要有一个前缀为空，公共子序列只能为空，故 D[0][j]=D[i][0]=0。

如果末字符相等，可以取这一个共同字符，得到 `D[i-1][j-1]+1`；如果末字符不等，公共子序列不可能同时使用这两个末字符，至少舍弃其中之一，于是取 `max(D[i-1][j],D[i][j-1])`。相等情形也有最优解使用这对末字符：若最优解未用其中一个，可以把它的最后一个相同字符推到末端，不会破坏次序。

用更小的 ABC 与 BAC 逐格填：

D	空	B	BA	BAC
空	0	0	0	0
A	0	0	1	1
AB	0	1	1	1
ABC	0	1	1	2

例如 D[2][2]比较 AB 与 BA，末字符 B 与 A 不同，取上格 D[1][2]=1 和左格 D[2][1]=1 中的较大者，得到 1。D[3][3]末字符同为 C，取左上 D[2][2]+1=2。答案可以是 AC，也可以是 BC。

恢复时从右下角往回走：末字符相等就记录该字符并斜走；否则走向提供较大值的上格或左格。记录顺序是从后往前，最后反转。若两格相等，任选一条可恢复一种答案；要列出所有不同答案，需要处理分支和重复，输出量甚至可能很大。

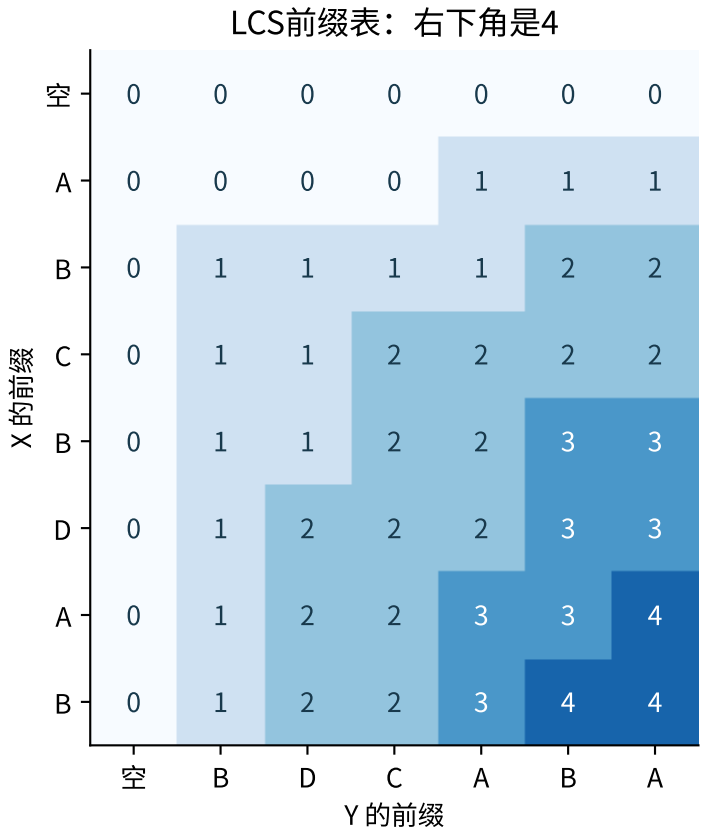


图 2　LCS 的每格只依赖已经填好的格子

图使用较大的例子 X=ABCBDAB、Y=BDCABA，纵向为 X 前缀，横向为 Y 前缀，首行首列为空前缀。颜色越深表示该格长度越大，右下值为 4。图中没有画依赖箭头，读每一格时自己对照其上、左、左上三格：字符相同时取左上加一，否则取上或左较大者。坐标标签的字符表示前缀刚加入的末字符，而格子索引本身是前缀长度。

n×m 个状态，每格常数次比较，时间 Θ(nm)，完整表空间 Θ(nm)。只求长度时保留上一行和当前行，空间 O(min(n,m))；若还按上述路径恢复序列，简单滚动数组会丢失路径信息，需要保留决策或使用另一个空间优化恢复方法。

3.5 动态规划的状态必须够用，也不能乱加

动态规划适用的共同结构是：同一个小问题会重复出现，而大问题的答案能由这些小问题按明确规则组合。状态要包含足够决定未来的信息。在最大区间里，“前缀最佳值”不够推下一格，因为最佳区间可能早已结束；所以另设“恰好在此处结束”。在 LCS 里，两个前缀长度足够，因为剩余问题只取决于这两个前缀，不取决于之前怎样跳字符。

状态一旦定义，就逐项检查基础值、转移覆盖、计算顺序与答案位置。若转移读取尚未算好的状态，要么调整顺序，要么明确使用记忆化递归。若依赖形成环，则不是直接按表从左到右就能解决。

3.6 从递归调用看“返回以后还要做什么”

数组求和也可以递归：sum(lo,hi)在空区间返回 0，否则将 a[lo]与 sum(lo+1,hi)相加。求[4,1,7]时，外层保存“返回后还要加 4”，下一层保存“还要加 1”，再一层保存“还要加 7”，到空区间返回 0，依次得到 7、8、12。计算步数线性，等待中的调用深度也是线性。

如果改成分两半，左右求和后相加，叶子仍是每个元素一次，工作总量 Θ(n)，但栈最长路径只约 $\log_2 n$。调用树总节点数与某一时刻尚未返回的节点数不同，所以时间同阶、栈空间可以不同。切片若复制子数组，又会增加额外复制，必须按实际代码分析。

最大区间的四元组也可以从这份“返回什么”出发设计。对子数组保存总和 sum、最大非空前缀 pre、最大非空后缀 suf、最大非空区间 best。左右结果 L、R 合并：sum=L.sum+R.sum；pre=max(L.pre,L.sum+R.pre)；suf=max(R.suf,R.sum+L.suf)；best=max(L.best,R.best,L.suf+R.pre)。每项都对应一个完整分类。

对左段[−2,3,-1]，sum=0、pre=1、suf=2、best=3；右段[4,-5,2]，sum=1、pre=4、suf=2、best=4。合并总和 1，最大前缀 max(1,0+4)=4，最大后缀 max(2,1+2)=3，最大区间 max(3,4,2+4)=6。与线性扫描得到同一答案，而且这四项能继续与别的相邻段组合。

为什么单独保存 best 不够？段[5,-100]与[−100,5]的 best 都为 5，但它们连接右段[6]后的最大区间不同：前者为 6，后者可为 11。是否能跨边界接上，取决于后缀而不是内部最佳值。状态缺信息时，两个看似相同的状态会在同样未来输入下产生不同答案，这是判断状态不充分的直接反例。

3.7 LCS 恢复路径，真的走回一条答案

再看 ABC 与 BAC 的表。右下 D[3][3]=2，末字符 C 相同，先记录 C，斜走到 D[2][2]=1。这里 B 与 A 不同，上格 D[1][2]=1、左格 D[2][1]=1 并列。若选上格，就到只比较 A 与 BA；末字符 A 相同，记录 A，斜走到空前缀。记录顺序 C、A，反转得到 AC。若先走左格，则得到 BC。

路径相等时有多条选择，不代表当前表算错，也不代表最长长度不确定。长度是 2，具体序列可以有多个。若题目要求字典序最小的 LCS，仅仅“并列时总往上”未必够，需要进一步比较后续可恢复的字符或使用适合的状态信息。输出要求会反过来影响恢复策略。

滚动数组只保留两行，能算长度，但覆盖掉的旧格子仍可能是恢复路径需要读取的。因此“空间从 nm 降到 m”要同时说明你放弃了简单回溯能力。若题目只要长度，这正合适；若要序列，可保留决策表，或使用分治恢复法重新计算必要信息，交换时间、空间与实现复杂度。

3.8 编辑距离与 LCS，名字相近时重新读操作集合

A=ABC、B=ADC。LCS 为 AC，长度 2。只允许插删时，删 B 再插 D 需 2 步，公式 n+m−2L=2。若允许成本 1 替换，直接 B 改 D 就 1 步。两种动态规划都有二维前缀状态，但转移不同；不能因为都叫字符串相似度便共用答案。

对只插删距离 D，边界 D[i][0]=i、D[0][j]=j。字符相同允许对角免费，字符不同的对角替换不在操作集合内，不能写 D[i−1][j−1]+1。把允许操作逐个列出，再将每个动作翻译成一条转移，能避免无意换题。21 章 Filename 会利用预算小进一步只算带状区域。

3.9 练习与答案

练习 1。 对`[-3,4,-1,2,-6,5]`手算非空最大连续区间，同时给出 end 和 best 两列。

解答：end 依次为−3、4、3、5、−1、5；best 依次为−3、4、4、5、5、5。一个最佳区间是`[4,-1,2]`，另一个是末尾单项`[5]`，和都为 5。读到−6 时 end=5−6=−1，但 best 保留 5；下一个 5 应重新起段，因为只取 5 比−1+5=4 更大。若要固定输出规则，例如最短区间或最左区间，必须在相等时一并比较端点，不能只保存和。

练习 2。 对 `AB` 和 `ACB` 写出 LCS 表，并说明为何最长公共子串不是同一个答案。

解答：空行是 0、0、0、0；A 行是 0、1、1、1；AB 行是 0、1、1、2。末格 2 对应子序列 `AB`，第二个串跳过了 C。公共子串必须连续，在 `ACB` 里 A 后面不是 B，最长长度只有 1。两类问题的状态与转移不同：公共子串常用“必须分别在 i、j 结束”的状态，失配时长度归 0，而 LCS 失配时可以从上或左继承。

练习 3。 已经会记忆化，为什么还要学自底向上填表？

解答：记忆化只计算被访问的状态，适合状态稀疏或依赖不易排序的情况；递归会占调用栈并有调用开销。自底向上明确计算顺序，容易滚动压缩与观察每个状态；若可达状态只有少数，却把整个巨大表填满，也会浪费工作。选择取决于状态图，不取决于哪种写法看上去更高级。

4 向量：一排连续格子怎样装下越来越多的数据

现在有一份待办清单，开始只有[读书，交作业，买面包]，后来还会加入新事项。数组提供一块固定长度的连续内存；向量在数组外面增加管理信息，满了就换一块更大的内存，让调用者看到一个可以增长的序列。所谓“第 r 项”，r 就是秩，在本书中从 0 开始。若每项占 w 个字节，起始地址为 B，则第 r 项地址为 B+rw。地址可以直接计算，所以按秩访问是 O(1)。这个结论要求元素大小固定或数组保存固定大小的引用；字符串内容本身长短不一时，还要区分引用与字符串所占空间。

4.1 size 与 capacity，先分清已经放了多少和最多能放多少

向量通常保存三个量：指向底层数组的地址，现有元素数 size，已经分配的格数 capacity。需要始终保持 `0 <= size <= capacity`。只有`[0,size)`是当前有效元素；`[size,capacity)`是预留空间，不能当作已经存在的元素来读取。

例如 size=3、capacity=8 时，访问第 2 项合法，访问第 5 项不能因为底层有空间就算合法。C++的`operator[]`常不主动检查越界，`at()`则会检查；“程序暂时没崩”不代表访问正确。

接口是使用者能做什么，例如读取、追加、在秩 r 前插入、删除某段。实现是用什么内存和代码完成。相同接口可以换实现，算法分析仍需针对具体实现。例如“插入一个元素”的接口，并没有承诺所有位置都一样快。

4.2 例 1：在中间插入，为什么必须从后往前搬

底层容量 8，当前内容`[10,20,30,40]`，想在秩 1 前插入 15。先让 40 从下标 3 移到 4，再让 30 从 2 移到 3，再让 20 从 1 移到 2，最后把 15 写在 1，size 从 4 变 5。

```
开始     10 20 30 40  _  _  _  _
移40     10 20 30 40 40  _  _  _
移30     10 20 30 30 40  _  _  _
移20     10 20 20 30 40  _  _  _
写15     10 15 20 30 40  _  _  _
```

暂时出现重复值没有关系，最终有效区间会恢复正确内容。若从前往后搬，先把 20 写到 2，会覆盖尚未搬走的 30；随后再读 a[2]，拿到的已经是 20。方向由“目的地是否会覆盖未来还要读的数据”决定。

在长度 n 的向量秩 r 前插入，需要搬 n−r 项，时间 Θ(n−r+1)，另加可能的扩容成本。尾部追加 r=n，不用搬已有元素；头部插入 r=0，则可能搬全部 n 项。

删除`[lo,hi)`时，后面的每项要向左移 hi−lo 格。这次应从前往后搬，因为左侧空位不会覆盖后面尚未读取的数据。删除后 size 减少 hi−lo；逻辑长度减少不一定立即归还底层容量。

4.3 例 2：满了以后加倍，贵的一次怎样摊开

设初始容量 1，只做尾部追加，满了就容量加倍。追加 A，直接写入；追加 B 前容量 1 不够，分配 2 格，复制 A，再写 B；追加 C 前分配 4 格，复制 A、B，再写 C；追加 D 直接写；追加 E 前分配 8 格，复制 4 项。

追加后的 size	容量	本次复制旧元素数	本次含写入的成本
1	1	0	1
2	2	1	2
3	4	2	3
4	4	0	1
5	8	4	5
6	8	0	1
7	8	0	1

追加后的 size	容量	本次复制旧元素数	本次含写入的成本
8	8	0	1
9	16	8	9

第 9 次确实很贵，不能说每次最坏 O(1)。但前 n 次追加，总写新元素 n 次；复制旧元素数最多 1+2+4+…，最后一项小于 n，故总复制小于 2n，总成本小于 3n，再加常数初始化。因此任意前 n 次追加总成本 O(n)，平均分到每次为 O(1)，叫摊还常数时间。

这里没有假设追加次数随机、输入随机或“绝大多数时候运气好”。它对每一条合法的纯追加序列成立。真正贵的一次由之前许多便宜操作共同承担。

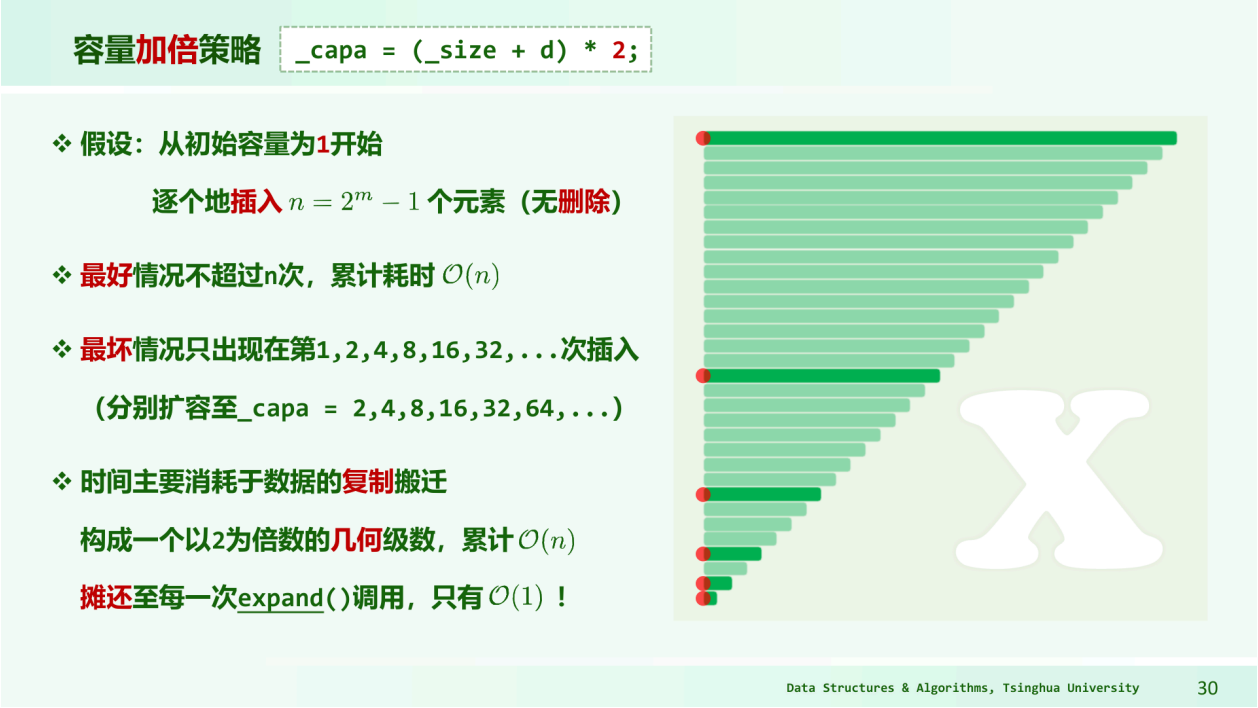


图 3　容量加倍的教师原图

图中每条横条表示一次复制规模，颜色较深的条是发生扩容的时刻；长条之间夹着许多不需要大规模复制的操作。把复制量按 1、2、4、8 相加，就是上面的几何级数。图中的具体实现用(_size+d)*2 表示预留 d 项后再加倍，触发点可能与我们的简化表差一两项，常数比例增长的总账相同。

扩容时通常有一小段时间新旧数组同时存在。若旧容量 C、新容量 2C，迁移期间可能占约 3C 个元素槽，不能把稳态容量 2C 误当瞬时峰值。元素若有昂贵复制构造，基本操作的单价也要相应调整。

4.4 例 3：每次只加固定格数，为什么总账变差

假设满了只加 2 格。容量依次 2、4、6、8、10……。做 n 次追加时，复制量约为 2+4+6+…+n，是 $\Theta(n^2)$。单次普通追加仍然只写一项，但扩容过于频繁，旧数据被反复搬迁。摊还到每次是 Θ(n)，而不是常数。

n=10 时，加 2 策略复制 2+4+6+8=20 项；加倍从容量 2 开始，复制 2+4+8=14 项。小例子差距还不大，规模增长后一个平方级、一个线性级，差距会持续放大。

更一般地，按固定比例 r>1 增长，总复制量仍是几何级数，为 O(n)，但空间与复制常数随 r 改变。r 接近 1，闲置空间较少，扩容次数更多；r 较大，扩容较少，但空位与迁移峰值更大。“加倍”是常见选择，不是唯一正确比例。

4.5 缩容为什么要留一段缓冲区

若容量 C，只要 size 降到 C/2 就立即缩成 C/2，而一满又扩成 2 倍，会发生抖动。容量 8、size4，删除到 3 可能缩成 4；再追加到 5 又扩成 8；删一项又触发缩容。少量反复增删，就可能每次搬动 Θ(C)项。

常见规则是满时扩成 2 倍，低于四分之一满时才缩成一半。例如容量 16、size4 时可缩到 8；缩完仍只占一半容量，需要再删除约 2 项才会接近下一次缩容，也需要追加约 4 项才会再次扩容。昂贵搬迁之间保证有足够多便宜操作支付成本。

这个“两个阈值分开”的做法常叫滞回。最低容量也要设下限，避免空容器缩成 0 后无法按乘法增长。若要严格证明任意增删序列的总成本，可以用势能法；28 章会把账本等式逐项展开。

4.6 去重也体现了数据布局的成本

无序向量去重，可以逐个查看当前项以前是否出现，发现重复就删除。但每次删除要搬后面的元素，查找本身也可能线性，整体最坏平方时间。若允许改变顺序，可以将待删位置与末项交换后弹出，删除便宜了，但顺序不再保留。

有序向量的相同值相邻，可以用读指针扫描、写指针压紧。例如`[1,1,2,2,2,5]`：保留第一个 1；第二个 1 跳过；第一次遇 2 写入下标 1；其余 2 跳过；遇 5 写入下标 2，最后有效区间为`[1,2,5]`。每项读一次，总时间 Θ(n)，没有逐次删除带来的反复搬移。改变维护的信息，算法就能省下一大段工作。

4.7 复制、移动与引用，扩容时元素身份怎样变化

设向量中放三条大字符串，底层格子保存字符串对象。扩容会在新数组中构造相应对象，再销毁旧对象；语言可能采用移动以转交内部缓冲区，也可能复制全部字符。若每条字符串长度 L，一次元素迁移不再必然只花与 L 无关的时间。通常分析向量时把“一次元素复制或移动”作为单位，再由元素类型决定单价。

异常也可能在迁移中发生。健壮容器不能复制到一半就丢掉旧数组，应该保证失败时仍能恢复一个合法状态，或明确接口提供的异常保证。教学伪代码省略内存申请失败，并不意味着真实实现可以忽略对象生命周期。

预留空间 reserve 只改变 capacity，不改变 size；调整长度 resize 则可能创建或销毁逻辑元素。预留十格以后写 a[9]，若 size 仍 0，按通常向量接口仍不合法。把“内存已经申请”与“对象已经构造并进入逻辑序列”分开，才能正确使用非平凡类型。

4.8 精确看一个扩容边界

初始容量 1，满时加倍，做 n 次追加。若 n 恰为 `2^k`，旧元素复制总数为 `1+2+...+2^(k-1)=n-1`；若 `n=2^k+1`，刚触发下一次扩容，复制总数变为 `2^(k+1)-1`，接近 2n。累计曲线会在边界跳跃，却始终被 2n 包住。某次实验看到 n 只多 1 而耗时接近翻倍，不与总账线性矛盾。

不同语言的标准容器未必严格加倍，常数因子可能不同，标准也可能只保证摊还性质而不指定具体增长公式。要分析教师实现，就用教师实际代码；要依赖标准库行为，就依接口保证，别把某次观察出的容量序列当跨版本承诺。

4.9 插删交替，反例必须真的触发昂贵操作

若说“每次删到半满就缩容会抖动”，需要把阈值与触发关系写精确。设容量 8、size5；删成 4 时立即缩到 4；再追加一项，size 从 4 到 5 前因满扩到 8；再删到 4 又缩到 4。两次操作就搬约 4 项，反复做 m 轮总 Θ(m×4)。把 4 换成大容量的一半，少量操作可以反复支付与容量成比例的成本。

低于四分之一才缩容时，容量 8 删到 4 不会缩；在 size4、5 间反复操作只付常数动作。若继续删到 1 再缩到 4，搬迁前已经经历足够多次删除，能分担账单。滞回阈值不是为了美观，而是切断上述短周期的反复迁移。

4.10 一次批量删除，怎样避免反复搬相同元素

要删掉向量里所有偶数，逐次调用 remove 可能每次都搬整个后缀，最坏平方。可用 read 从左扫、write 标记下一个保留位置：遇奇数就写到 a[write]并 write 加 1，遇偶数只前进 read。输入[2,7,4,9,6,1]最终前缀为[7,9,1]，逻辑长度改为 3。每项读一次、保留项写一次，总线性，且保留相对顺序。

不变式是已读前缀中的所有应保留项，按原顺序恰好在[0,write)；write≤read 保证写入不会破坏尚未读取的后缀。这个证明同时说明为什么可以原地压紧。它与有序去重的双指针相似，但“是否保留”的判断来自不同条件，不能把两者混成只适合相邻重复的算法。

4.11 练习与答案

练习 1。 容量 4、size4、内容`[2,4,6,8]`，在秩 2 插入 5。按加倍策略写出扩容、搬移与结果，哪些已有迭代器可能失效？

解答：先分配 8 格，将 4 个旧元素迁移。再从后往前搬 8 到下标 4、6 到下标 3，写 5 到下标 2，得到`[2,4,5,6,8]`，size5。扩容换了底层地址，指向旧数组元素的指针、引用和迭代器通常全部失效。即使没有扩容，插入点及其后的元素也发生位移，不能继续假设旧迭代器仍代表原逻辑元素；具体失效规则依容器语言规范。

练习 2。 已知接下来会追加 10000 项，提前分配 10000 格和每次加倍有什么差别？

解答：提前预留使后续追加无需扩容迁移，容量申请一次，适合规模可靠且内存足够的情况。每次加倍不要求预先知道总数，仍有线性总搬迁保证，却可能多申请空位和产生数次迁移。提前预留改变的是容量，不应误把 size 直接变成 10000；否则容器中会多出 10000 个逻辑元素。若预估远大于实际需求，会长期浪费空间。

练习 3。 给有序向量`[2,2,3,3,3,4]`原地去重，写指针初值为何可以设 1？空向量又如何？

解答：非空时第 0 项必然保留，令 write=1；从 read=1 开始，若`a[read] != a[write-1]`，就把当前值写到 a[write]并增加 write。扫描完成得到前 3 项`[2,3,4]`。空向量没有第 0 项，必须先判断 n=0 并返回 0，不能读取`a[0]`或`a[write-1]`。不变式是已扫描前缀的不同值，按原顺序恰好保存在`a[0:write]`。

5 查找：先确定要找的是哪一种位置

数组[1,3,3,3,7]中查找 3，可以有四种合理要求：返回任意一个 3，返回第一个 3，返回最后一个 3，或者返回第一个大于 3 的位置。它们会得到不同下标。把接口说清楚，往往比背二分模板更能减少错误。无序数组通常只能逐项比较，最坏检查 n 项。有序数组额外提供了一个关系：某项小于目标时，它左边所有项也不会更大。二分查找就是用这份顺序信息，一次排除一整段候选。

5.1 例 1：找第一个不小于 x 的位置

定义 lower_bound：在非降数组 a 中，返回最小下标 p，使 a[p]≥x；若没有则返回 n。p 也就是把 x 插进去仍能保持有序的最左位置。返回 n 是合法的“尾后位置”，但不能读取 a[n]。

```
def lower_bound(a, x):
    lo, hi = 0, len(a)
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if a[mid] < x:
            lo = mid + 1
        else:
            hi = mid
    return lo
```

用 a=[1,3,3,3,7]、x=3 追踪：

lo	hi	mid	a[mid]	动作
0	5	2	3	它可能是第一个，保留 mid，hi=2
0	2	1	3	仍可能更靠左，hi=1
0	1	0	1	这一项太小，lo=1
1	1	无	无	结束，返回 1

不变式用排除区间表达很清楚：[0,lo)全小于 x，[hi,n)全大于等于 x；尚未决定的区间为[lo,hi)。当 a[mid]<x，mid 及其左侧全部太小，故 lo 可以变成 mid+1。否则 mid 与右侧全部大于等于 x，可以令 hi=mid。两个分支都让 hi−lo 严格减小，最终 lo=hi；此时边界左边全小于 x，右边全不小于 x，恰好得到所求位置。x=0 会返回 0；x=8 会返回 5；空数组直接返回 0。统一使用半开区间，边界条件通常可以自然覆盖，不必另造四五个补丁。

5.2 例 2：重复值的范围与前驱

在同一个数组中，upper_bound 返回第一个大于 x 的位置。只需把判断改为 a[mid] <= x 时移 lo。对 x=3，lower=1，upper=4，所以所有 3 占据[1,4)，个数 4−1=3。

最后一个小于等于 x 的位置是 upper_bound(x)−1。查 x=4，upper 仍为 4，前驱下标 3，值 3；查 x=0，upper=0，前驱位置为 −1，表示不存在。某些课程版 search 接口恰好返回前驱而非“等值匹配位置”，读教材程序时要先看返回值约定。

若要判断 x 是否存在，先算 p=lower_bound(x)，再检查 p<n and a[p]==x。不能只看 p 是否小于 n，因为查 x=2 时 p=1，a[1]=3，插入位置存在不等于目标值存在。

同样一份有序数组，可以支持区间计数：落在[L,R]内的元素个数为 upper_bound(R)−lower_bound(L)，前提 L≤R。这里 R 无需实际出现在数组中。若 L>R，结果按接口约定应为 0，不能无条件相减得到负数。

5.3 一次比较和两次比较，不要藏进同一个词

版本 A 常先判断相等，再判断大小，某些失败路径会对同一元素做两次比较。lower_bound 式写法每轮只用一个大小关系，结束后若要确认存在再判断一次。它们都为 O(log n)，但比较次数的主项或常数可能不同。

成功与失败的平均查找长度也需要区分。成功在元素节点上停止；失败在相邻元素之间的 n+1 个间隙中停止。不同树形、n 是否为 2 的幂减 1、成功目标分布与失败间隙分布，都会影响精确均值。即使两者都为 Θ(log n)，也不能写成“精确平均次数总相等”。

举 n=3 且平衡地先比中间元素。成功所需大小判定层数为 1、2、2，等概率均值 5/3；失败的 4 个间隙都需要走完 2 层，均值 2。这个例子已经足以否定“总是精确相等”。若教材把等值判断另计，需按同一成本口径重新统计，不能混用。

5.4 例 3：答案不是数组元素，也可以二分

有一台机器，每分钟生产 3 件产品，要生产至少 20 件，最少需要几分钟？检查 t 分钟是否够用，谓词为 `3t>=20`。t 从 0 增加时，结果先假后真；我们找第一个真，答案 7。

更贴近算法题：包裹重量`[2,3,4,5]`，保持顺序，最多分成 2 批运输，求每批容量最小值。容量必须至少为 5，最多为 14。给定容量 C，用贪心依次装：当前批装不下才开新批。C=8 时，第一批 2+3=5，第二批 4，再遇 5 需第三批，因此不够；C=9 时，第一批 2+3+4=9，第二批 5，够用。最小容量 9。

为什么可以二分？若容量 C 能完成，那么更大容量也能完成，谓词单调。为什么检查函数的贪心正确？在相同批数下，每批尽量向右延伸，不会比任何其他合法划分更早停止；归纳可得贪心用的批数最少。二分与贪心各需要自己的证明。若检查函数本身不准确，外层二分再正确也无济于事。

5.5 位图：用一位回答“有没有”

当键来自较小整数范围`[0,U)`，只需要保存是否出现，可以给每个可能值分配一位。键 k 对应字节下标 k//8，以及该字节内偏移 k%8。若约定低位在前，掩码为 `1 << (k%8)`。设置用按位或，测试用按位与，清除用与上反掩码。位次约定必须统一，与机器大端小端是不同层面的概念。

U=16，要标记 2、5、13：第 0 字节置第 2 位与第 5 位，二进制为 00100100；第 1 字节置第 5 位，为 00100000。查询 5 读第 0 字节与 00100000 相与，非 0，存在；查询 6 与 01000000 相与为 0，不存在。标记相同值两次不会计成两份，位图保存的是集合而非出现次数。

空间约 U/8 字节，设置与查询为 O(1)。但清空整个新位图需要 Θ(U)位操作或 Θ(U/w)机器字操作。若 U 巨大而实际只出现少量键，位图可能比散列表浪费得多。某些“快速初始化”技巧用已初始化前缀与互相校验的索引，避免每次全清，但实现需要额外数组与合法性判断，不等于未初始化内存可以随便读取。

5.6 例 4：筛质数，标记从哪里开始

寻找 2 到 30 的质数。开始把所有数设为未合成，0 和 1 另排除。处理 2 时标记 4、6、8……；处理 3 时标记 9、12、15……；处理 5 时标记 25、30。处理到 $\lfloor\sqrt{30}\rfloor=5$ 即可，剩余未标记者为 2、3、5、7、11、13、17、19、23、29。

从 p^2开始，是因为小于 p^2的 p 的倍数 p×k 中 k<p，早已被更小的质因子标记。只处理尚未标记的 p，经典埃拉托斯特尼筛的时间 O(U log log U)，空间 O(U)位；证明这个更精细的时间界需要质数倒数和估计，不能把“每个数只被标一次”当理由，合数实际上可能被多个质因子标记。

实现时用 `p <= U//p` 判断 $p^2\le U$，可以避免固定宽度整数中的 p*p 溢出；枚举倍数前也要确保起点可表示。

5.7 在纸上设计二分，不从模板反推题意

任何二分都可以先画一个布尔序列：假、假、假、真、真，目标是第一个真。让 lo、hi 把已经确认的假区和真区夹住；每次看 mid，真就把真区向左扩，假就把假区向右扩。数组查找中谓词是 a[mid]≥x，容量问题中谓词是“这个容量能否完成”。不同题共用的是单调边界，不是同一条数值公式。

如果想找最后一个真，可以转换成找第一个假再减 1，或者另外设计区间约定。不要在半开模板里临时把 hi 改 n−1、把结束条件改≤，却仍沿用原更新式。合法区间、返回意义与更新规则是一套整体。

算 mid 时`(lo+hi)/2` 在固定宽度有溢出风险，常写 `lo+(hi-lo)/2`。但若 lo、hi 可分别为极大正负数，hi−lo 也未必安全；常见数组下标 $0\le lo\le hi\le n$ 的前提才让差值范围受控。边界类型若混有有符号与无符号，−1 前驱也需明确表示，不能让它绕成巨大正数。

5.8 比较查找长度，先固定树和计费方式

对 7 项的理想中点二分，比较层分别有 1 项深 1、2 项深 2、4 项深 3。若成功目标等概率、每层算一次主比较，平均成功层数为(1+4+12)/7=17/7。8 个失败间隙在这棵满分支决策结构里都需要走完 3 层，失败均值 3。它们均为 Θ(log n)，主导增长一致，精确均值不同。

若算法每层先判等再判大小，成功在根只判一次，失败或向下路径可能判两次；此时应该将每条路径的实际判断逐条加起来。所谓平均查找长度有时按“访问节点数”，有时按“关键码比较次数”，两种教材口径不能直接相减。

Fibonacci 查找试图选择不完全对称的比较位置，匹配不同分支的成本。插值查找按数值在区间中的比例估位置，均匀分布条件下可能更快，但偏斜值如指数增长数组可能让它表现很差，最坏可线性。它们都进一步使用了输入或成本模型信息，不是二分随意换个 mid 就自动更优。

5.9 位图快速清空的一个可实现替代

若每轮只标记少量键，可额外记录本轮真正从 0 变 1 的键列表，结束时仅清这些位。比如 U 很大，本轮只置 2、5、13，记录这三个键，清空只做三次。若重复置 5，不要重复记录，否则列表可能随操作数增长。这个方法的清空成本与本轮不同键数有关，但第一次申请与初始化底层位图仍可能花 Θ(U/w)。

另一做法为每键保存时间戳，当前轮号为 epoch，时间戳等于 epoch 视为已出现。清空只增加 epoch；但每键不再是一位，内存增大，epoch 溢出时要真实重置。将清空从每轮移到偶尔一次，并不等于完全没有初始化或溢出成本。

5.10 把不存在的边界也当作合法答案

对第一个真位置，所有项都假时返回 n；所有项都真时返回 0。两端都不需要读取虚拟元素。若你为了简化证明假设 a[−1]小于所有值、a[n]大于所有值，那是数学哨兵，不等于代码真的可以访问越界位置。程序只在 lo<hi 时计算 mid，因而 mid<n，读取始终位于真实数组内。

许多二分错误来自把“候选答案位置”与“可访问元素下标”混为一谈。返回 p 以后先检查 p<n，再读 a[p]；查前驱返回−1 以后先检查非负。一个函数能用 n 表示尾后边界，另一个接口可能用 null 或可选类型，调用者也要相应转换，不能只因值都是整数就直接当作数组下标。

5.11 谓词只在指定范围内单调也够用

例如判断整数 x 的平方是否至少十六，在非负整数区间上是假后真，可二分第一个真得到四；若范围包含负数，结果会是真、假、真，不再是一个单边界。公式相同，搜索域不同，二分资格也不同。先证明题目实际搜索范围内单调，再设置上下界。

空数组的返回位置为零，但它仍没有可读取的第零项。

5.12 练习与答案

练习 1。 a=[2,2,4,6,6]，分别求 lower_bound(6)、upper_bound(6)、lower_bound(5)和≤1 的最后位置。

解答：依次 3、5、3、−1。6 的范围是[3,5)，有 2 项；5 不存在，但插入下标 3 前仍有序；≤1 的元素没有，所以前驱为−1。若把 upper_bound(6)的结果 5 拿去读 a[5]就越界；它表示边界，不表示一个可读取元素。

练习 2。 数组[1,5,3,7]能否使用同一二分查找 3？写一个失败过程。

解答：按 lower_bound 代码，lo0,hi4,mid2，a[2]=3，hi 变 2；接着 mid1，a[1]=5，hi 变 1；mid0，a[0]=1，lo 变 1，返回 1，a[1]=5，误判 3 不存在。算法依赖的“a[mid]≥x 则右半都不小于 x”在乱序输入中不成立。二分不是对任意数组都有效的机械折半。

练习 3。 U=10^9、实际只有 1000 个整数，要做存在查询。位图是否总比散列表好？

解答：位图需要约 125MB，还可能花时间初始化；1000 个键的散列表通常只需远少于这个空间。位图适合范围可控、键较密集或位运算批处理有价值的场景。也可先将出现的键排序，查询用二分，空间与实际键数相关。选择应比较范围 U、实际数 n、更新频率与可用内存。

6 排序时，除了大小次序还在保留什么

同一班同学先按姓名排好，再按成绩从高到低排。若同分学生仍保持原来的姓名顺序，第二次排序就是稳定的。为看出相等值的身份，本章把记录写成(关键码,标签)，比较时只比较关键码。

稳定性讨论的是相等关键码的相对顺序，不是算法每次运行结果是否一致，也不是排序有没有越界。原地排序通常指只用常数数量或很少的额外存储；它同稳定性也是两个指标，不能混为一谈。

6.1 例 1：冒泡排序，为什么相等时不交换

给[(3,A),(1,B),(3,C),(2,D)]升序排序。第一趟从左向右比较相邻项：3A 与 1B 交换，得到 1B,3A,3C,2D；3A 与 3C 相等，不交换；3C 与 2D 交换，得到 1B,3A,2D,3C。最大值 3C 到了末尾。

第二趟只需处理前三项：1B 与 3A 不动，3A 与 2D 交换，得 1B,2D,3A,3C。第三趟没有交换，可以提前结束。两个 3 仍然是 A 在 C 之前。

若仅在左项严格大于右项时交换，相等的两项不可能直接越过彼此。相对顺序要改变就必须在某次相邻交换中交叉，而这次交换不会发生，所以算法稳定。若条件写成大于等于，相等项会被交换，稳定性就丢了。

每一趟把当前未排前缀中的最大项推到末尾，证明不变式可写为“最后 k 项已经处在最终位置”。普通固定趟数版本比较(n−1)+(n−2)+…+1 次，最好与最坏均 $\Theta(n^2)$；记录一趟是否交换的版本，已排好输入一趟即可，最好 $\Theta(n)$。这些是不同版本，不能不看代码就说冒泡最好一定线性。

还能记录最后一次交换的位置，下一趟只扫描它以前的区域，因为后缀已无逆序。这会改善部分输入，却不改变最坏平方界。

6.2 例 2：插入排序，把新牌插到左侧有序区

数组[4,2,3,1]，先认为前 1 项[4]已排序。拿出 2，4 右移，2 放入首位，得[2,4,3,1]；拿出 3，4 右移，3 放在 2 后，得[2,3,4,1]；拿出 1，4、3、2 依次右移，1 放在首位，完成。

```
def insertion_sort(a):
    for i in range(1, len(a)):
        x = a[i]
        j = i
        while j > 0 and a[j-1] > x:
            a[j] = a[j-1]
            j -= 1
        a[j] = x
```

每轮开始前，a[0:i]已经有序，并含原前 i 项的同一批元素。内层将比 x 大的项右移，空位最后停在所有≤x 的项之后。等值旧项不会被跨越，故稳定。若条件改为>=x，新的等值项会插到旧项前面，稳定性就变了。

已排好输入，内层每次立即停止，时间 $\Theta(n)$；严格倒序，移动 1+2+…+(n−1)项，时间 $\Theta(n^2)$。可以二分找到插入位置，将比较次数减到 $O(n \log n)$，但数组搬移仍可能 $\Theta(n^2)$。减少比较不等于减少移动。

6.3 逆序对把“离有序还有多远”变成整数

逆序对是下标 i<j、数值 a[i]>a[j]的一对元素。[4,2,3,1]的逆序对有(4,2)、(4,3)、(4,1)、(2,1)、(3,1)，共 5 对。相等元素不算逆序。

一次交换相邻且逆序的两个元素，恰好消除它们之间的 1 个逆序，不改变它们与其他元素组成的逆序总数。因此只靠相邻逆序交换排好数组，交换次数正好等于初始逆序数 I。

插入排序每右移一个比 x 大的旧元素，对应原来一对“旧元素在前且比 x 大”的逆序。总移动数恰好 I；加上每项的扫描和放回，成本 $\Theta(n+I)$。这个表达式解释了它为何适合接近有序的输入，也解释了随机互异排列中平均 $\Theta(n^2)$：每一对元素逆序概率 1/2，共 n(n−1)/2 对，期望 I=n(n−1)/4。

6.4 例 3：归并，一边排序一边统计逆序

两个已经有序的序列 L=[2,5,8]、R=[1,3,7]。比较两端未取出的最小项：先取 1，此时 L 的 2、5、8 都大于 1，新增 3 个跨区逆序；再取 2，没有新增；取 3 时 L 还剩 5、8，新增 2 个；取 5 无新增；取 7 时 L 剩 8，新增 1 个；最后取 8。

归并输出[1,2,3,5,7,8]，跨区逆序共 6。因为 L 有序，当 L[i]>R[j]时，L[i]以及它后面的每一项都大于 R[j]，可以一次增加 len(L)−i，无需逐项配对。相等时先取左侧，既保持稳定，也不会把相等值算进逆序。

递归把原数组切成左右两半，逆序对分为左内部、右内部、跨左右三类，彼此不重叠且覆盖全部。递归统计前两类，归并统计第三类，总时间 Θ(n log n)，额外数组 O(n)。n 很大时逆序数最多 n(n−1)/2，计数类型要够宽，即使每个元素本身只需 32 位，计数也可能需要 64 位。

归并排序的稳定性也能归纳：左右各自稳定；合并相等值时左先右后，原来位于左半的等值元素本来就在右半之前，所以不会跨越。只说“归并天然稳定”不够，实际比较分支必须采用相应规则。

6.5 例 4：选择排序，“取最后一个最大值”仍可能不稳定

选择排序每轮在未排区找最大项，与未排区末项交换。即使在相等最大项中选最后一个，也不能保证稳定。考虑[(2,A),(1,B),(2,C),(1,D)]。

第一轮选最后一个最大项 2C，与末项 1D 交换，得到 2A,1B,1D,2C。第二轮在前三项里选最大 2A，与 1D 交换，得到 1D,1B,2A,2C。两个关键码 1 的记录已经从 B 在 D 前变成 D 在 B 前。最大项的等值选择规则没有阻止另一个关键码的记录被远距离交换越过。

若改成取出选中项，再把中间项整体平移，最后放到目标位置，可以做稳定的选择过程，但移动成本会改变。链表中也可通过摘下节点再接入目标位置实现不同的稳定版本。判断稳定性要跟着具体动作走，不能只跟算法名字走。

选择排序即使输入已排序，仍需扫描每个未排区找最大，比较次数 Θ(n²)；交换通常只有 O(n)次。若元素交换特别贵而比较便宜，它可能有自己的使用理由，但大型一般排序通常更偏向其他方法。

6.6 什么时候用哪一种

小数组、接近有序、插入到已有序列：插入排序常易实现且能利用较少逆序。需要稳定且能接受辅助数组：归并排序有可靠的 n log n 最坏界。需要理解相邻交换与逆序：冒泡是清楚的教学模型。若只要最小的 k 项或第 k 项，完整排序可能做了超出需求的工作，18 与 20 章会介绍堆与选择。

比较排序还存在 Ω(n log n)最坏下界，但它不适用于所有输入限制。计数排序对小整数键可用额外范围空间换时间；位图甚至只保存是否出现，不能直接替代带重复计数与记录标签的稳定排序。先看问题允许读取哪些信息，再谈下界。

6.7 相邻交换的逆序变化，为什么恰好只有一对

相邻位置放 x>y，交换为 y,x。二者之间的逆序消失。对左侧任意 z，它与 x、y 的两次比较总贡献只与数值有关，不受 x、y 内部顺序影响；右侧同理。因为中间没有第三项，不会出现其他关系变化，所以逆序总数恰减 1。若交换不相邻，就可能一次消除或新增许多逆序，不能继续把交换数等同初始 I。

例如[3,2,1]有 3 个逆序，直接交换首尾便得到[1,2,3]，只用 1 次任意交换。冒泡需要 3 次相邻逆序交换。两种“交换次数最少”的问题允许动作不同，因此答案不同。28 章的置换环 n−c 下界适用于任意两位置交换，而不是相邻交换。

6.8 归并的循环不变式可以写到每个指针

设左右子列 L、R 已排好，i、j 指向尚未取出的首项，输出前缀 B 已排序且恰含两列中已经取出的元素。比较 L[i]与 R[j]，较小者不大于所有尚未取出的元素，因此接到 B 尾仍有序。取完一项只前进对应指针，元素不会重复或丢失。某列耗尽后，另一列剩余本就有序，直接接上。

统计逆序时还要加一句：当前累计值恰好等于已输出右侧元素与尚未输出左侧元素之间，在它们被取出时确认的跨区逆序总数。若取 R[j]且 R[j]<L[i]，新增 L 剩余数量，所有这些对都会且只会在此时计一次。若取 L[i]，不新增，因为尚未输出的右侧元素均不小于它。

6.9 多关键字排序，稳定性的用途可以直接看见

学生记录为(班级,成绩,姓名)。先按姓名排序，再稳定地按成绩排序，再稳定地按班级排序，最终以班级为第一关键字、成绩为第二、姓名为第三。后一次稳定排序保留相同新关键字内部已经排好的旧顺序，所以应从最低优先级开始。

若先按班级，再稳定按成绩，最终第一关键字会变成成绩，而非原先班级。稳定性保存的是等值组里的旧次序，不会阻止后一次排序把不同新关键字的记录跨班移动。把三条记录写出来就能检验：甲班 90、乙班 80、甲班 70，最后按成绩全局排序会混开班级。

如果语言排序支持比较器直接比较元组，也可以一次按(班级,成绩,姓名)排序；若比较器在相等关键字上又引入了与原顺序无关的判定，就要重新看它是否满足严格弱序等要求。比较函数自相矛盾，例如同时认为 a<b、b<c、c<a，会让排序的基本假设失效。

6.10 随机排列的平均逆序数，逐对算最清楚

n 个互异元素均匀随机排列，给任意一对数 x<y，x 在 y 前与 y 在 x 前各一半。为这对定义指标 X，逆序取 1，否则 0，期望 1/2。全体逆序数是所有 n(n−1)/2 个指标之和，期望为 n(n−1)/4。各对指标并不相互独立，但线性期望不需要独立。

因此稳定插入排序的平均移动数为二次级，尽管它在有序输入上线性。随机输入假设只用于这个平均结论，最坏平方与近有序 Θ(n+I)是另外两种描述。若现实数据本来由昨天的排序结果局部修改而来，均匀随机排列未必是合理模型。

6.11 每个输出记录也需要被读取

即使输入已经有序，要确认并输出全部记录，通常仍需线性遍历。最好时间可以线性，不可能仅因没有交换就把整个排序检查叫零成本。

6.12 练习与答案

练习 1。 给 `[3,1,2,1]` 列出所有逆序对，并预测稳定插入排序的元素右移总次数。

解答：3 与后面的 1、2、1 组成 3 对；第一个 1 与后面不构成逆序；2 与最后的 1 组成 1 对，共 4。插入第一个 1 时移 3 一次；插入 2 时移 3 一次；插入最后 1 时移 3、2 两次，共 4 次。两个 1 因严格大于时才移动，原先的 1 仍在后来的 1 前面。比较次数还包括发现无需继续移动的那一次，因此不必恰好等于 4。

练习 2。 归并 L=`[1,3,3]` 和 R=`[2,3]`，跨区逆序多少？相等时取右侧会发生什么？

解答：取 1 不增；取 2 时 L 剩 3、3，增 2；以后全是相等 3，不新增逆序，答案 2。若相等时先取右侧，数值排序依然正确，但 R 里的 3 会越过 L 里的 3，失去稳定性。若统计代码把“右侧被取出”一律视为产生逆序，还会把相等项误算，必须把严格大小与相等分开。

练习 3。 对 n 项互异数组，逆序对为 0 能否推出已升序？I 很小能否推出每个元素离最终位置都很近？

解答：I=0 意味着任意 i<j 都有 a[i]<a[j]，因此升序。I 小能约束总的错位，但不保证每一项距离都小。例如 `[2,3,4,5,1]` 只有 4 个逆序，末项 1 却需要向左走 4 格。说“几乎有序”之前应固定度量，逆序数、最大位移与无序段长度并非同一指标。

7 链表：知道位置以后，怎样少搬东西

向量把元素紧挨着放，按下标访问方便，但中间插入要搬动后面的项。链表把每个元素放在一个节点里，节点保存数据和通向邻居的链接。要在已知节点后插入，可以只改附近几根链接，其他节点不必搬家。这里的“已知节点”很重要。若任务是“在第九千个元素后插入”，链表通常得先从头走九千步。插入动作是常数成本，找到位置仍是线性成本。学习数据结构时，应把“找位置”和“改结构”分开记账。

7.1 先画一个双向链表

一个数据节点可有三个字段：data 保存元素，pred 指向前驱，succ 指向后继。若 p 的后继是 q，应同时有 `p.succ=q` 和 `q.pred=p`。本书用 header、trailer 两个不保存用户数据的哨兵节点，把边界处理统一起来：

```
header <-> A <-> B <-> C <-> trailer
```

空表不是没有任何节点，而是 `header.succ=trailer`、`trailer.pred=header`。header 没有合法数据前驱，trailer 没有合法数据后继；不能拿哨兵的 data 参与用户数据比较。哨兵的作用是让“插到第一个元素之前”与“插到中间”使用同一套四根链接。

访问位置 p 时，你拿到的是节点地址或迭代器。秩 r 只说明这是序列第几项，两者不相同。向量由秩直接算地址；链表由节点中的链接找到邻居。把链表“封装成 a[r]”并不会让它突然拥有 O(1)随机访问。

7.2 例 1：在 B 之前插入 X，每根链接都要有着落

原结构 `A <-> B`，设 q 指向 B，p=q.pred 指向 A。先申请新节点 x，让 `x.pred=p`、`x.succ=q`；再令 `p.succ=x`、`q.pred=x`。结果成为 `A <-> X <-> B`。

```
x.pred = q.pred
x.succ = q
q.pred.succ = x
q.pred = x
```

最后两行的顺序不能随意颠倒。若先做 `q.pred=x`，再执行 `q.pred.succ=x`，第二行实际改的是 x.succ，把它指向 x 自身，原前驱 A 的后继仍指向 B。为了不依赖复杂表达式的旧值，工程代码常先把 p 保存到局部变量，再清楚地更新四根指针。

不变式是：除局部的 p、q 及新节点外，所有节点的邻接关系不变；更新结束后，p 的后继为 x，x 的前驱为 p，x 的后继为 q，q 的前驱为 x。size 增加 1。若 q 是 trailer，就是尾部插入；若 q 是原首节点，就是头部插入；空表也同样成立，因为 p=header、q=trailer。

7.3 例 2：删除节点，先接好邻居再释放

从 `header <-> A <-> B <-> C <-> trailer` 删除 B。设 p=B.pred、q=B.succ，做 `p.succ=q` 与 `q.pred=p`，结构变成 `A <-> C`。若接口需要返回 B 的数据，应先保存它；随后释放 B 并将 size 减 1。

不能先释放 B 再读 B.succ。释放后的地址可能暂时仍有原数值，但对象已经不再可用，这叫释放后使用。也不能假设外部保存的 B 迭代器会自动改成 C；它现在失效了。对其他节点的引用通常无需因这次局部删除而改变，但具体容器规则仍应以实现为准。

边界检查有两层：p 确实属于当前链表，而且 p 不是哨兵。把另一张表的节点交给 remove，可能破坏两张表；只检查指针非空还不够。

从前往后删除所有值为 2 的节点，原表为`[2,1,2,2,3]`。应先保存 next=p.succ，再判断是否删除 p，最后 p=next。过程为删首 2→`[1,2,2,3]`；保留 1；删 2→`[1,2,3]`；再删 2→`[1,3]`；保留 3。若删除后直接做 `p=p.succ`，就会读取已释放节点；若误跳两次，则会漏掉连续重复值。

7.4 例 3：反转链表，方向变了以后往哪里走

双向链表可以将每个节点的 pred 与 succ 交换，最后交换 header 和 trailer 的角色。但遍历时必须先保存旧后继，否则“下一步”会沿着刚反转过的链接走回去。

以 `H <-> A <-> B <-> T` 为例，先保存 H 的旧 succ=A，再交换 H 的链接；处理 A 时保存旧 succ=B，再交换；接着处理 B 和 T。全部节点都交换完成后，旧 T 作为新 header，旧 H 作为新 trailer，用户数据顺序变成 B、A。

单向链表没有 pred 字段，可用 prev、cur、next 三个指针：保存 next=cur.next，将 cur.next 改成 prev，再让 prev=cur、cur=next。对 `A->B->C->null`，第一步得到 `A->null`、cur=B；第二步得到 `B->A->null`、cur=C；第三步得到 `C->B->A->null`、cur=null，新的头是 prev=C。

证明时把已经反转的前缀与尚未处理的后缀分开。prev 指向已反转前缀，cur 指向未处理后缀；每轮从后缀取一个节点接到前缀前面。未处理节点数每次减 1，时间 Θ(n)，辅助指针为常数数量。

7.5 排序时搬值还是搬节点

链表插入排序可以遍历未排序节点，在已排序前缀中寻找位置，将当前节点摘下并接进去。寻找位置最坏 O(n)，摘接本身 O(1)，总时间仍可能 $\Theta(n^2)$。链表让“挪动一个已经找到的节点”便宜，却没有免除比较。

取`[(3,A),(1,B),(2,C),(1,D)]`。先保留 3A；把 1B 接到 3A 前；把 2C 接到 1B 和 3A 之间；最后将 1D 插在已有 1B 之后、2C 之前，得到 1B、1D、2C、3A。若寻找插入点时越过所有≤当前值的旧节点，等值顺序保持，排序稳定；若停在第一个相等节点前，D 会跑到 B 前面。

链表归并两个已排序子表，可以不断比较头节点，把较小的节点摘下接到输出尾部，避免数组里的大规模搬移。一次归并仍是线性，因为每个节点只取出一次；递归归并排序总时间 Θ(n log n)。若通过切片复制节点数据再排序，就不再是同一份“只改链接”的空间账。

7.6 深复制与析构，数据结构还要负责谁活着

浅复制只复制头指针，两个对象会共同指向同一批节点。若两者都认为自己拥有节点，析构时可能释放两次；一方删节点，另一方也会持有失效指针。深复制会新建每个节点，并按原顺序拷贝数据，两个链表拥有不同节点。

数据字段若本身是指针，还要继续定义它指向的对象归谁所有。复制节点并不自动等于复制所指对象。实际 C++可以用明确所有权的智能指针或遵守拷贝/移动规则的容器设计解决，但不能靠“我只复制三个字段”回避接口含义。

循环链表让尾节点链接回头部，适合轮转。遍历不能再用 `p!=null` 退出，应知道回到哪个哨兵，或准确走 size 步。若遍历条件仍等待 null，就会永远绕圈。判断链表是否意外成环，可以用快慢指针：慢每次走 1 步，快走 2 步；若有环，进入环后相对位置每轮改变 1，最终会相遇。若无环，快指针先到空。

7.7 什么时候链表有优势

已知节点附近频繁插删、需要稳定节点地址、需要把整段节点在容器间摘接时，链表可能合适。大量按下标访问、顺序扫描紧凑数据、对内存占用敏感时，向量常更好。链表每项多存链接，节点可能分散在内存，缓存局部性通常较差；渐近同为 O(n)的遍历，实际耗时仍可能明显不同。

单向与双向也有区别。给定 p，要删 p 的后继在单向表中容易；要删 p 本身，通常还需要知道它的前驱。某些“把后继数据复制到 p 再删后继”的技巧不适用于末节点，也会改变节点身份，不能作为通用删除替代品。

7.8 整段摘接，O(1)的前提要逐一数出来

假设已知一段连续节点首 a、尾 b，以及目标节点 q，想把[a,b]移到 q 之前。先保存旧前驱 p=a.pred、旧后继 r=b.succ，将 p 与 r 相连；再保存 q 的旧前驱 t，将 t 接 a、b 接 q，并修正反向链接。节点数据完全不用复制，已知所有端点且目标不在被移动段内部时，只改常数根链接。

若 a、b 只由秩给出，先找到它们可能要线性遍历；若需要更新两张列表的 size，却不知道这段长度，还要先数节点。若 q 在[a,b]内部，直接按普通摘接操作可能形成环或断链，需要明确拒绝或视为空操作。所谓链表 splice 为常数时间，通常隐含了位置已知、必要计数已知或接口不要求那笔工作的条件。

用 H↔A↔B↔C↔D↔T，把[B,C]移到 A 前：断 A 与 B 之间、C 与 D 之间，接 A↔D；再把 H↔A 之间插入整段，接 H↔B、C↔A，结果 H↔B↔C↔A↔D↔T。B 与 C 内部链接不改，证明可以按原段内部、原段外部与新边界三部分检查。

7.9 有序链表二分，为什么比较少了却仍要走路

你可以先数出长度，再找中点比较，保留一半，再找下一中点。若每次只在保留区间内顺序走到中点，路径长度可能为 n/2+n/4+...=O(n)，比较只有 O(log n)，总时间仍 O(n)，没有变成向量的 O(log n)。若每次又从全表头重新走中点，甚至可能重复更多路程。

因此“链表不能二分”更准确的意思是：它缺少按秩常数时间定位，普通二分无法得到同样的时间优势。若比较对象特别昂贵，减少比较仍可能有意义，但别把指针移动成本删掉。跳转表给链表增加多层索引，才改变快速到达远处位置的能力。

7.10 链表调试中的回路检查

双向表从 H 沿 succ 走到 T，数到的用户节点应恰好 size；从 T 沿 pred 走回 H 也应恰好 size，正反序列应互为逆。对每个非边界节点，检查 p.succ.pred==p 与 p.pred.succ==p。这些条件联合起来能抓住漏改一根链接、误自环与 size 没有同步更新。

只检查正向打印可能漏掉反向链接错误；只检查每节点局部互指又可能没发现一批节点从主表脱离成另一个环。结构验证需要同时检查可达性、节点数量与双向一致。修改代码时，先用空表、单节点、两节点和连续删除追踪，通常比一开始就用长随机表更易定位问题。

7.11 环上删除与约瑟夫递推，计数起点仍要明确

若固定每次数到第 k 人删除，剩余者从下一人重新起数，零下标幸存者满足 J(1,k)=0，J(n,k)=(J(n−1,k)+k) mod n。缩小问题中零点对应原环删人之后的下一位，所以恢复原编号要平移 k，而不是随意加 k 减一。

例如 n=5、k=2，依次删除编号 1、3、0、4，幸存 2；递推从一人到五人得到 0、0、2、0、2。链表模拟便于理解删除，递推只求最后幸存者可省去逐次走环。若题目步长每轮变化、恢复的是空座秩，不能不核对规则就套同一递推；21 章 Interview 处理的是更一般的动态空位定位。

只有一个数据节点的循环表，删除后必须让哨兵重新指向自己；若无哨兵，则头指针应变空。不能留下指向已释放节点的头指针，因为下一次入队或遍历会把无效地址当作合法位置。

7.12 练习与答案

练习 1。 空双向链表含 H、T 两个哨兵。在 T 前依次插入 7 和 9，再删除 7，写出所有关键链接。

解答：初始 H.succ=T、T.pred=H。插 7 后 H.succ=7、7.pred=H、7.succ=T、T.pred=7。插 9 后 7.succ=9、9.pred=7、9.succ=T、T.pred=9。删 7 后 H.succ=9、9.pred=H，其余 9.succ=T、T.pred=9 保留。size 依次 0、1、2、1。若删除后 H 仍指 7，就说明只改了反向链接，结构不完整。

练习 2。 已知单向链表头指针，想删掉第 n/2 项并重复 1000 次。为什么不能直接说总成本 O(1000)？

解答：每次先从头找到待删节点的前驱，最坏要走 Θ(n)步，局部改链虽 O(1)，整次仍 Θ(n)。若 size 随删除减少，准确总成本是各次当前长度的和的一部分，通常仍与 1000n 同阶。只有外部已经持有每次需要的前驱节点，且这些指针在操作后仍有效，才可只计算常数次改链。

练习 3。 反转单向表时先写 `cur.next=prev`，随后用 `cur=cur.next` 前进，会出现什么？

解答：第一次若 prev=null，cur.next 被改成 null，随后 cur 也变 null，算法立即停止，原后半段丢失。应在覆盖链接之前保存旧 next，再用它前进。用三节点小图追踪，比记住“三指针口诀”更容易看见错误究竟发生在哪一行。

8 栈：先来的事，为什么要等后来的事做完

编辑器遇到左括号，要等到对应右括号才知道内部是否闭合；函数调用另一个函数，调用者要等被调用者返回；计算 `2*(3+4)`，乘法要等括号里的加法完成。这些任务都需要保存“还没做完的事”，并优先处理最近放进去的那一件。

栈只允许在同一端插入与删除，叫后进先出。push 是压入，pop 是弹出并返回，top 只查看顶部。空栈不能 pop 或 top，接口必须给出报错或失败结果。用向量尾部实现时，扩容追加摊还 O(1)，弹出与查看通常 O(1)；一旦底层换成不同结构，也应重新核算成本。

8.1 例 1：多种括号，栈保存的是“正在等谁”

输入`{[()()]}`。读到`{`压入，栈为`{`；读`[`后为`{ [`；读`(`后为`{ [ (`；读`)`时栈顶是`(`，匹配后弹出，回到`{ [`；再处理一对`()`，然后`]`弹`[`，最后`}`弹`{`，输入结束且栈为空，因此合法。

输入`([)]`为何不合法？读完`([`，栈顶是`[`，下一个`)`需要`(`，类型不匹配。只统计左右括号数量无法发现这个错误：它们数量相同，却交叉闭合。

算法读到左括号就压入；读到右括号时检查栈非空且类型匹配，再弹出。结束后还要检查栈空。`(()`的每个已读右括号都匹配，但仍剩一个未闭合左括号，所以也非法。

不变式是：栈从底到顶恰好保存已读前缀中尚未闭合的左括号，越靠顶越晚打开。因此当前右括号必须与栈顶匹配，否则没有任何合法嵌套能补救。每个字符入栈或出栈至多一次，时间 Θ(n)，最坏空间 Θ(n)，例如全部为左括号。

8.2 函数调用栈，与自己写的栈有何关系

求 `factorial(4)`，调用会依次等待 factorial(3)、factorial(2)、factorial(1)。每层要保存返回到哪里、自己的参数及还未执行的乘法。最深处返回 1 后，依次乘 2 得到 2、乘 3 得到 6、乘 4 得到 24。栈帧是一次尚未返回的函数调用所需的现场。

把阶乘改成循环，只需一个累计积；递归不是一定必需。尾递归指递归调用之后已没有其他工作，有些编译器或语言会优化复用栈帧，但 C++并不承诺所有尾递归都会被优化。`return n*factorial(n-1)`还要乘 n，本身不是尾调用。

用显式栈改递归时，需保存同样的待办信息。二叉树后序遍历不能只压节点而不记录“左右孩子是否已经处理”；否则弹出时不知道下一步应访问节点还是继续走孩子。10 章会具体演示。

8.3 例 2：逆波兰表达式，运算数排在运算符前面

普通中缀 `3+4*2`，先乘后加；逆波兰表达式 RPN 写成 `3 4 2 * +`。遇数字压栈，遇二元运算符弹出两个数，计算后把结果压回。

读到的符号	动作	栈底到栈顶
3	压入 3	3
4	压入 4	3,4
2	压入 2	3,4,2
*	弹右 2、左 4，算 8	3,8
+	弹右 8、左 3，算 11	11

减法和除法最能暴露顺序错误。`8 3 -`要算 8−3=5；先弹出的是右操作数 3，后弹出的是左操作数 8。若直接把第一次 pop 减第二次 pop，结果会变−5。

二元表达式结束时应恰有一个值。`2 +`在遇`+`时操作数不足；`2 3`结束剩两个值，也不合法。检查这两个条件，能把输入格式错误与算术错误分开。阶乘是单目后缀，只弹一个数；双阶乘 `5!!=5*3*1=15` 与`(5!)!`完全不同。

8.4 例 3：中缀转 RPN，运算符栈为何要看优先级与结合性

将 2+3*4^2 转成 RPN。读 2 直接输出；+压入运算符栈；3 输出；比+优先级高，先压入；4 输出；^比优先，压入；2 输出；输入结束，从栈依次弹出^、*、+。结果 2 3 4 2 ^ * +，求值为 2+3×16=50。

若读到新运算符 op，栈顶运算符优先级更高，应先输出栈顶；优先级相同时，要看 op 的结合性。减法左结合，8-3-2 等于(8-3)-2，读第二个-时要先弹第一个-。教师原版优先表把^遇到^写为先弹栈，即左结合，所以原程序将 2^3^2 解释成(2^3)^2=64，RPN 是 2 3 ^ 2 ^。若题目要求数学常用的右结合，则要修改相等优先级的处理：读第二个^时先压栈，不提前弹第一个^，于是得到 2 3 2 ^ ^，值为 512。两种语法都能明确实现，做课程题先按给定表，不能把自己习惯的约定当作原代码。

左括号压栈但不输出；右括号到来时，一直弹到对应左括号，并丢弃这一对括号。输入结束若还有左括号，说明未匹配。右括号到来却找不到左括号，也应报错。负号还要区分一元与二元：-2^2、(-2)^2 的值依语法优先级可能不同，必须先固定规则，不能靠符号外形猜。

30 章对 PS04 的 21 式逐项给出中间值。这里使用的^表示数学乘方；C++里的^是按位异或，直接把中缀表达式粘进 C++源代码会换掉含义。

8.5 栈混洗：输入顺序固定，输出能有多少种

输入 1、2、3，按这个顺序进栈，可以随时弹出。输出 231 可行：压 1、压 2、弹 2、压 3、弹 3、弹 1。输出 312 不可行：为了先弹 3，必须先压 1、2、3；弹 3 后栈顶是 2，不能跳过 2 去弹 1。

判断一个给定输出是否可行，可以顺序扫描目标：若栈顶不是所需元素，就继续把尚未输入的项压入；若已经没有可压入项仍不匹配，则失败。每个元素入、出各一次，时间 O(n)，最坏空间 O(n)。这个贪心有必然性：要输出目标 x 之前，若栈顶不是 x，唯一合法动作就是继续压入，不能先弹其他值，否则输出序列就已经错了。

8.6 例 4：Catalan 计数，从第一个元素在哪里弹出拆开

n 个不同元素依序入栈，所有合法压入弹出操作序列共有 C_n 种。把 push 记作左括号，pop 记作右括号：任何前缀里 pop 不能多于 push，最后两者相等。于是问题等价于合法括号串计数。

$C_0=1$，表示空序列也有一种不做操作的安排。看第一个左括号对应的右括号：其内部含 k 对括号，后面含 n−1−k 对，内部与后部可独立选择，因此

$$C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}.$$

$C_1=1$；$C_2=C_0C_1+C_1C_0=2$；$C_3=C_0C_2+C_1C_1+C_2C_0=2+1+2=5$。三对括号为(((())))、((()()))、((())())、(()(()))、(()()())。对应栈混洗 5 种，不等于全部 3!=6 种排列。

还可从全部含 n 个左括号、n 个右括号的串中扣除非法者。总数为 binom(2n,n)；将首个出现“右括号比左括号多 1”的前缀左右反转，可与含 n+1 个左括号、n−1 个右括号的串建立双射，非法数为 binom(2n,n-1)，差得到 C_n=binom(2n,n)/(n+1)。这个反射论证要说明映射可逆，不能只说“减掉一半”。

C_n 增长约为 4^n/(√π n^(3/2))，很快超过固定宽整数；精确计算应选足够大的类型或大整数。若只求模数余数，直接除以 n+1 未必合法，要使用递推或符合模数条件的逆元。

8.7 运算符表是语法的一部分，逐格读一次

中缀转换的栈顶运算符 a 与当前输入 b 之间有三类关系：先弹 a 并输出，或先压 b 等待，或括号配对后同时消去。以栈顶、当前+为例，乘法应先完成，所以弹；栈顶+、当前则应先读乘法右边，所以压；栈顶(、当前)是配对，不输出括号。

同级结合性就藏在对角格。-对-先弹，得到左结合；^对^先弹也为左结合；若要求右结合就改成先压。每修改一格，要重新检查连续链与括号外的关系，不能只改展示的一个例子。后缀阶乘遇下一运算符应先完成，但阶乘操作数必须是合法非负整数；这属于算术域检查，不是优先级表能替代的。

具体转换 8-3-2：输出 8，压-，输出 3，读第二个-时弹旧-并输出，然后压新-，输出 2，结束弹-，得 8 3 - 2 -，值 3。若错压第二个-不先弹，会得到 8 3 2 - -，值 7。一个三数链即可分辨结合性错误。

8.8 栈混洗计数中的空情况不能漏

Catalan 递推里 $C_0=1$，是“没有元素时有一种空操作序列”，并非说有一个空元素。若把 C_0设 0，C_1的递推会变 0，后面全部归零。很多组合递推依赖空对象作为拼接单位，正如加法空和为 0、乘法空积为 1。

把首个左括号配对拆开时，内部与后部各可以为空。n=3、内部 k=0 给()后接 2 对，贡献 2 种；k=1 给(())后接 1 对，贡献 1；k=2 给外括号包 2 对，后部空，贡献 2。逐个列出 5 串后，卷积式中的每项都有具体对应，不会把形式相似的“首弹元素分类”也盲套成同一卷积。

若按第一个弹出的输入编号 k 分类，初始必须先压 1 到 k 再弹 k，栈里已有 k−1 项，后面不再是两个无约束的独立合法括号串。需要考虑初始栈高度与剩余入栈数的 ballot 路径约束。分类变量变了，计数对象的边界条件也会变。

8.9 空间峰值为何要注明算哪个栈

表达式处理至少可能有运算符栈、数值栈以及递归解析器的调用栈。RPN`a b + c +`数值峰值 2，不说明中缀原式需要多少运算符等待；原式加了许多多余括号，运算符栈会更深，但 RPN 数值峰值不变。计数是否包含底部哨兵也会使精确答案差 1，先写约定再比较公式。

8.10 字符与记号，数字十二不能当成一和二

RPN 应先分词，再逐记号执行。字符串`12 3 +`有两个数值记号，答案十五；若逐字符压入一、二、三，最后会剩多余值。中缀中的小数、科学记数法、一元负号与双阶乘，也都要求词法层先明确边界。

例如`3!!`应识别为数字三与一个双阶乘记号，而不是两个阶乘；`-2`可以被定义为带符号数字，也可以解析成一元负号作用于二，两种设计会影响乘方优先关系，必须与语法一致。运算符表只决定已经识别出的记号怎样结合，不负责猜原始字符属于哪个记号。

实际求值还要检查除零、负整数阶乘和不支持的指数域。栈中操作数够了，只说明语法局部可执行，并不保证算术结果在指定数域里存在。

对 RPN`2 3 4 * +`，一共压入三个数，先把三与四合为十二，再与二合为十四；栈峰值三。换成`2 3 + 4 *`，先合成五再压四，峰值只有二，结果二十。记号集合相同，顺序同时决定运算结构、答案和空间，这就是不能只数运算符个数预测峰值的一个小例。

单目运算消耗一个值再放回一个值，二元运算消耗两个值再放回一个值，可用栈长度变化检查格式。

8.11 练习与答案

练习 1。 求 RPN`10 2 3 * - 4 /`，并写栈峰值。

解答：10 入栈；2 入栈；3 入栈，峰值 3；*弹 3、2 得 6，栈为 10,6；−弹 6、10 得 4；4 入栈；/得 4/4=1。若使用整数除法，此例仍为 1，但换成`9 2 /`时数学有理数为 9/2，C++整数除法为 4，必须固定语义。

练习 2。 输出 2413 能否由输入 1234 经一个栈产生？

解答：为输出 2，压 1、2 再弹 2；为输出 4，继续压 3、4 再弹 4；此时栈为 1,3，顶是 3，目标却要 1，而且已经没有新输入可压，故不行。失败位置比单说“不符合栈规则”更有信息：是 4 已经输出后，3 挡在 1 上面。

练习 3。 为什么二元 RPN 求值栈峰值不一定等于表达式树高？

解答：计算`(((a+b)+c)+d)`，先算 a+b 留下一个值，再加 c，再加 d，峰值只要 2，树却很高。计算`a+(b+(c+d))`时，a、b、c 需要依次等待，压入 d 时峰值 4。二者运算数与运算符数量相同，树形与既定求值顺序不同，空间就不同；31 章会写出对应递推式。

9 队列与单调结构：怎样少看已经失去用途的数据

排队买饭时，先来的人先离开。普通队列从一端入队、另一端出队，叫先进先出。它与栈的差别只有出口规则，却足以改变能解决的问题：广度优先搜索需要按到达层次处理，适合队列；表达式中最近未完成的运算优先，适合栈。

9.1 例 1：环形数组队列，绕回去不代表清空

用容量 C 的数组，保存 front 与当前元素数 size。第 k 个逻辑元素放在 `a[(front+k)%C]`，入队写 `a[(front+size)%C]`，再增加 size；出队读 a[front]，front=(front+1)%C，再减 size。空是 size=0，满是 size=C。

C=5，先入 A、B、C、D，数组为 `A B C D _`，front0,size4。出 A、B 后 front2,size2；再入 E 写下标 4，再入 F 写下标 0。物理数组为 `F B C D E`，有效逻辑顺序却是 C、D、E、F，旧 B 位只是无效残留。不能从数组下标 0 开始把所有格子打印成队列内容。

另一种实现只保存 head 与 tail，并留一个空槽区分满与空：head=tail 为空，`(tail+1)%C=head` 为满。这种设计容量 C 只能容纳 C−1 项；它与保存 size 的设计不可混用判定条件。

9.2 例 2：两个栈怎样模拟一个队列

准备 in 与 out 两个栈。入队只向 in 压入。出队时若 out 非空，弹 out；若 out 空，将 in 全部弹出压入 out，再弹 out。翻转一次，最早进入 in 的元素便来到 out 顶。

依次入 1、2、3，in 底到顶为 1,2,3；第一次出队，将 3、2、1 依次移入 out，out 底到顶为 3,2,1，弹出 1。再入 4，此时 in 只有 4，out 顶仍是 2，所以后两次出队得到 2、3；out 空后再转移 4，得到 4。

为什么不能每次出队都把 in 转入 out？out 里若还有 2、3，把新来的 4 压到其上方，下一次就可能先弹 4，破坏先进先出。转移只能在 out 为空时发生。队列的逻辑顺序是“out 从顶到底”，后接“in 从底到顶”。

单次出队可能搬动 k 项，成本 Θ(k)，最坏不是 O(1)。但每个元素一生最多入 in 一次、从 in 移到 out 一次、从 out 弹出一次，所以 m 次操作总成本 O(m)，摊还 O(1)。这是对每条合法操作序列的总账，和输入随机无关。28 章会用势能法将这份逐元素记账变成等式。

9.3 例 3：滑动窗口最大值，为什么旧的小数能删掉

数组 `[2,1,5,3,4,2]`，窗口宽 3。所有窗口最大值依次为 5、5、5、4。朴素法每个窗口扫 3 项；一般宽 w 时成本 O(nw)。我们想维护一个候选队列，只留下将来仍可能成为最大值的下标。

队列从头到尾下标递增，对应数值单调递减。每读到新下标 i，先从队头删掉已经离开窗口的下标；再从队尾删掉所有值≤a[i]的旧下标；最后将 i 放入队尾。窗口形成后，队头就是最大值。

i	新值	删过期后、删弱候选后再入队的下标	对应值	输出
0	2	0	2	未满
1	1	0,1	2,1	未满
2	5	2	5	5
3	3	2,3	5,3	5
4	4	2,4	5,4	5
5	2	4,5	4,2	4

为什么新 5 能删掉旧 2 与 1？5 更大，而且更晚离开窗口。在未来任何同时含旧 2 和新 5 的窗口里，旧 2 都不可能当最大值；当新 5 离开时，旧 2 早就离开了。删掉的是已经失去竞争资格的候选，而非随便扔数据。

若相等时也删旧项，留下更晚过期的新项，最大值依旧正确。若题目还要求最早最大值下标，就不能照搬这个去重规则，应保留旧相等项或调整输出约定。队列通常保存下标，才能判断过期；只存数值会难以区分两个相同值来自哪个位置。

每个下标入队一次，最多从头或尾删除一次，总时间 Θ(n)，空间 O(w)。某一轮可能连续弹出很多项，但总弹出次数不超过 n，这也是摊还分析。

9.4 最大栈：同时保存当前前缀的最大值

如果需要栈在 push、pop 以外还能 O(1)查询最大值，可以每项保存(值,压入此项后栈内最大值)。依次压 2、5、3，节点记录为(2,2)、(5,5)、(3,5)。弹 3 以后，顶部(5,5)仍给最大值 5；再弹 5，顶部(2,2)给 2。

这需要每项多存一个最大值，总空间仍 O(n)。也可以只在最大值变化时保存"影子记录"，但重复最大值要计数或允许重复压入，避免弹出一个 5 就误以为所有 5 都离开。用两个最大栈组成队列时，队列最大值为两栈最大值的较大者，空栈另行处理。

单调队列更积极地删除被支配的候选，空间在随机输入下可能较小；但最坏递减窗口会保存 w 项。固定一个随机连续分布的窗口，其候选数期望约 H_w，不能据此声称无限长运行过程的峰值也只有 O(log w)。28 章会解释观察时间为何影响峰值。

9.5 例 4：直方图最大矩形，栈里存"还没遇到右边界"的柱子

柱高[2,1,5,6,2,3]，每柱宽 1。最大矩形可以取高度 5、覆盖第 2 与第 3 柱，面积 10。枚举每个区间求最低高度会很慢；单调栈维护高度不降的候选柱，一旦遇到更矮柱，就能确定若干旧柱的最远右边界。

用下标栈，开始压一个虚拟下标−1，结束补高度 0。处理过程：

- i=0，高 2，压 0，栈为−1,0。
- i=1，高 1，比 2 矮，弹 0；弹后左边界−1，宽度 1−(−1)−1=1，面积 2。再压 1。
- i=2，高 5，压 2；i=3，高 6，压 3。
- i=4，高 2，先弹 3，高 6，左侧剩 2，宽 4−2−1=1，面积 6；再弹 2，高 5，左侧剩 1，宽 4−1−1=2，面积 10。高 1 低于 2，停止，压 4。
- i=5，高 3，压 5；i=6 补 0，依次弹 5 得 3，弹 4 得 8，弹 1 得 6，最大仍 10。

宽度为什么是 i−left−1？i 是右边第一个严格更矮柱，不能包含；弹出后栈顶 left 是左边尚留的不高于当前柱的候选位置，本次先不包含它。中间从 left+1 到 i−1，共 i−left−1 根柱子，它们都至少与当前弹出高度一样高。若 left 与当前柱等高，本次结算的只是较窄范围，不是当前高度能向左延伸的最终最大范围；更靠左的等高代表随后弹出时会覆盖完整宽度。例如[2,2,0]先弹右侧 2 只得面积 2，再弹左侧 2 得到面积 4。因此保留等高的版本不能把每次弹后的 left 都称作"严格更矮"。

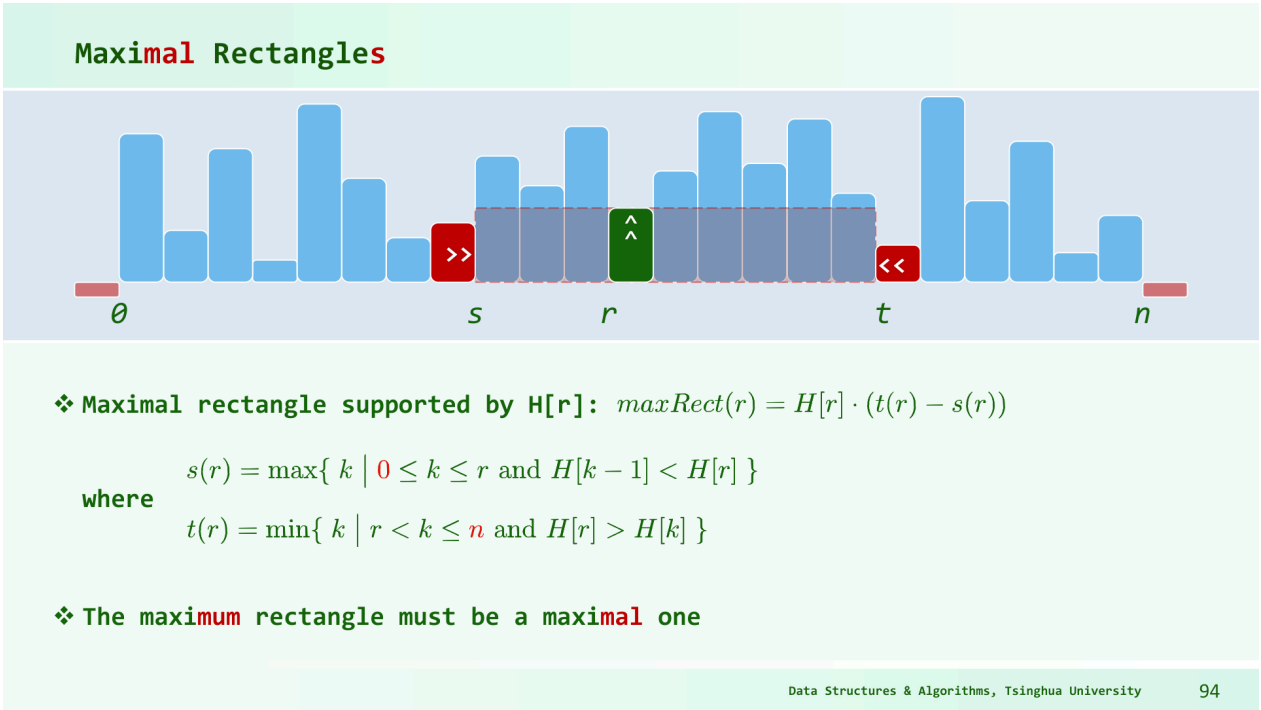


图 4　教师的单调栈示意

图中绿色柱 r 给出矩形高度，左侧红柱比它矮，右侧红柱也比它矮；中间被阴影覆盖的所有柱都不低于绿色柱。原图把左端第一个可用位置记 s，因此宽为 t−s；对图中这组严格更矮边界，若将左侧不可用柱下标记 left、右侧记 i，则同一完整范围宽为 i−left−1。算法保留等高柱时，某次弹出的 left 可能等高，需按上段先结算较窄候选、再由左侧等高代表取得完整宽度的规则理解，不能把每次栈顶都套成图中的严格更矮边界。单调栈的用途是及时找到这些边界，弹出时先结算矩形面积，再移除候选。

相同高度可采用“仅在更高时弹”或“相等也弹”的不同版本，宽度处理与保留代表项的规则需要配套。上面采用严格更高才弹，多个等高项会保留，后面较左者最终获得更宽范围。每项入栈一次、出栈一次，时间 Θ(n)，最坏空间 Θ(n)。

9.6 前缀和是另一种复用，不要见窗口就只想单调队列

若求每个宽 w 窗口的和，维护当前和，右端新值加上，左端旧值减去即可，时间 Θ(n)、额外空间 O(1)，前提能访问即将离开的旧项。若是多次任意区间求和，前缀 P[0]=0，P[i+1]=P[i]+a[i]，则 `sum([l,r))=P[r]-P[l]`。最大值没有一般的“减去离开者”逆运算，所以需要维护候选或额外结构。不同聚合量拥有不同代数性质，直接决定可以怎样压缩状态。

9.7 直方图边界，用一个候选高一次算完

以柱高[3,1,3,2,2]为例。高度 1 可以覆盖所有 5 柱，面积 5；左端 3 只能宽 1，面积 3；下标 2 的 3 右边遇 2，宽 1 面积 3；高度 2 可以覆盖下标 2 到 4 共 3 柱，面积 6。最大为 6。单调栈会在读到第一个 2 时结算高 3，在末尾 0 时结算高 2，让某一个等高代表获得完整宽 3。

若实现弹出条件为>=，新来的等高柱会替换旧代表，则必须让它继承旧代表能向左延伸的起点。例如处理第二个 2 时，不能把可用左界重设到自己，否则漏掉下标 2 的高 3 与前一个 2 构成的宽范围。下标栈的某些版本通过前一个更矮元素自然保留该信息，显式(高度,起点)版本则要手动继承。相等分支可以不同，配套边界不能断。

面积用高度乘宽度，宽度最大 n。即使高度、n 各在 32 位内，乘积可能超过 32 位。末尾哨兵 0 只适用于柱高非负；如果题目允许负高度，“矩形面积”本身的几何语义要重新定义，不能默认同一个补 0 规则覆盖。

9.8 轮转调度，用队列模拟一段真实过程

任务 A 需 3 单位、B 需 1、C 需 2，时间片为 1。初始队列 A3,B1,C2。执行 A 一单位后剩 A2，接到尾，队列 B1,C2,A2；执行 B 完成，队列 C2,A2；执行 C 后剩 C1，队列 A2,C1；执行 A 后剩 A1，队列 C1,A1；执行 C 完成，再执行 A 完成。完成时刻分别 B 为 2、C 为 5、A 为 6。

队列只管理轮到谁，不决定每个任务的总需求。若新任务在执行中到达，应明确入队时刻与当前任务重新入队的先后规则；相同时刻的约定可能改变精确等待时间。用一个时间轴与队列状态同步写，就不会把先进先出误读成每个任务一口气执行完。

9.9 单调结构的证明与普通队列的证明有何不同

普通队列从不删除中途任务，因此只要证明入出顺序一致。单调队列主动从尾部扔掉某些仍在窗口里的元素，必须额外证明这些元素在任何未来窗口都不会胜过留下的新元素。支配关系含两个方向：新值不小于旧值，新下标不早于旧下标。少了后一个条件，只留下最大值会在它过期后失去次大候选；少了前一个条件，就可能扔掉真正最大者。

9.10 双端队列是接口，单调队列是使用它的规则

双端队列允许头尾插入删除，普通队列只用尾入头出，单调队列还会从尾删除被支配候选。一个双端容器本身不会自动维护最大值，必须由每次入队时的比较与弹出规则建立单调性。

窗口过期条件若窗口为下标 i 减 w 加一到 i，队头下标小于 i 减 w 加一才过期，等于左端仍属于窗口。写成小于等于会早删一项。用宽一窗口检查最容易发现：每读一项就应输出它自身，不能刚加入就被当过期丢掉。

宽度等于数组长度时，滑动最大只输出一项，应等于全局最大；宽度大于长度或为零，接口应拒绝或明确定义为空结果，不能让负下标或除零决定行为。检查窗口题时，把宽一、宽等于长度、全相等三个边界先算一遍，能覆盖不同的过期与并列分支。

候选下标严格递增，是判断过期次序的依据。

9.11 练习与答案

练习 1。宽 3 窗口扫描`[4,4,2,5]`，相等时删旧候选，写出每步候选下标。

解答：读第 0 个 4，队列[0]；读第 1 个 4，旧 4 被新 4 支配，删 0 后为[1]；读 2，得[1,2]，第一个窗口最大 4；读 5，先检查窗口为下标 1 到 3，1 尚未过期；再从尾删 2、1，得[3]，第二个窗口最大 5。若要最早最大值下标，第一个窗口应答 0，当前删旧规则无法提供它，需要调整相等分支。

练习 2。柱高`[2,2,2]`最大矩形多少？用严格更高才弹的版本追踪结尾。

解答：依次将 0、1、2 压栈，补 i=3 高度 0。弹 2 时左 1，宽 1，面积 2；弹 1 时左 0，宽 2，面积 4；弹 0 时左−1，宽 3，面积 6。相等高度并未使算法错误，只是最大宽度在最左代表项弹出时才结算。

练习 3。双栈队列操作为入 A、入 B、出、入 C、出、出。写出出队顺序及每个元素转移次数。

解答：第一次出队把 B、A 从 in 倒到 out，弹 A；入 C 仅进 in；第二次出队直接弹 out 的 B；第三次 out 空，倒 C 后弹 C。顺序 A、B、C，每个元素只从 in 转到 out 一次。若每次出队先无条件倒 in，C 可能越过 B，这正是转移条件不可省的原因。

10 二叉树：分叉以后，怎样保证每个节点只访问一次

文件夹里套着子文件夹，表达式里套着子表达式，一场淘汰赛由小比赛组成更大的比赛，这些关系都适合画成树。树的每个非根节点有唯一父节点；沿父链接不断向上会到根，没有环。一般树的孩子数不受限制，二叉树每个节点最多有左、右两个孩子，而且左右位置有区别。

本章树高按边数算：叶高 0，空树高−1；节点深度是根到它的边数。树高是所有节点深度的最大值。“第几层”有的资料从 1 数，有的从 0 数，因此涉及公式时优先用深度，避免差一。

10.1 先固定一棵树，所有算法都对它动手

```
      A
     / \
    B   C
   / \   \
  D   E   F
```

A 为根；B、C 是 A 的孩子；D、E 是兄弟；D、E、F 是叶子。共有 6 个节点、5 条边。B 子树含 B、D、E，大小 3，高 1。整树高 2。二叉树大小 n 与高度 h 之间有 `h+1 <= n <= 2^(h+1)-1`，前者由最长根叶路径至少含 h+1 个节点得到，后者由深度 d 最多 `2^d` 个节点相加得到。

有 n 个节点的非空二叉树共有 2n 个孩子槽，非空孩子槽恰好 n−1 个，对应 n−1 条边，所以空槽为 n+1。这个计数会在带空标记的序列化中再次出现。

10.2 例 1：先序、中序、后序，区别是何时处理根

对任意子树，都要处理根、左子树、右子树。先序是根左右；中序是左根右；后序是左右根。对固定小树，先序 `A B D E C F`，中序 `D B E A C F`，后序 `D E B F C A`。

后序为什么适合计算目录总大小？只有先知道孩子子树的大小，才能算父目录。先序为什么适合复制树？先创建父节点，再递归建立它的孩子。中序本身不保证数据有序；只有二叉搜索树额外满足左边键小、右边键大的规则时，中序才有序。

递归中序可以写得很短：空树直接返回；遍历左孩子；访问当前节点；遍历右孩子。正确性对节点数归纳：空树无节点，结论成立；非空树按归纳假设左右子树各被恰好访问一次，根单独访问一次，三部分互不重叠，因此整树恰好一次且次序正确。每节点常数动作，时间 Θ(n)，调用栈 O(h)。退化链时 h=n−1，不能一概写 O(log n)空间。

10.3 例 2：显式栈中序，栈到底存了哪些待办

```
def inorder(root):
    stack = []
    cur = root
    while cur is not None or stack:
        while cur is not None:
            stack.append(cur)
            cur = cur.left
        cur = stack.pop()
        visit(cur)
        cur = cur.right
```

在小树中：先沿左链压 A、B、D，cur 变空；弹 D 并访问；D 无右孩子，再弹 B 访问；转入 B 的右孩子 E，压 E、弹 E 访问；再弹 A 访问；转入 C，压 C、弹 C 访问；转入 F，压 F、弹 F 访问。访问序列正是 D、B、E、A、C、F。

栈中每个节点都在等待“左子树处理完以后轮到我”。弹出时，它左边已经全部访问；访问自身，再把右子树交给同样的过程。每节点入栈出栈各一次，双层 while 不是平方时间，而是 Θ(n)总动作。栈深至多 h+1。

后序需要区分节点当前处在哪个阶段。简单可靠的写法是栈里存`(节点,是否已展开)`：第一次弹到节点，将`(节点,已展开)`压回，再按右、左顺序压孩子；再次弹到已展开标记时访问。由于栈后进先出，左孩子先处理，

右孩子后处理，最后才访问根。每个节点只有常数次栈动作，仍为 Θ(n)时间；也可以用 lastVisited 指针实现，但要清楚它记录的是“刚刚完成的子树根”。

10.4 例 3：层次遍历，把还没轮到的节点放队列

从队列[A]开始，出 A，入 B、C，队列[B,C]；出 B，入 D、E，队列[C,D,E]；出 C，入 F，队列[D,E,F]；随后依次出 D、E、F。层序为 A、B、C、D、E、F。

队列保持“已经发现但尚未访问的节点”，先进先出保证浅层先于深层，同层按孩子入队次序访问。时间 Θ(n)，空间由最大层宽控制，最坏 Θ(n)。一条很长的链层宽只有 1；完美二叉树最后一层却约占总节点一半。树高和层宽控制不同遍历的空间。

完全二叉树除最后一层外都满，最后一层从左到右连续填充。按层序从下标 0 存数组，左孩子 2i+1、右孩子 2i+2、父节点 $\lfloor (i-1)/2 \rfloor$ 。这不是任意二叉树紧凑存储的通用公式：若稀疏树仍按这种下标放，可能留下大量空格。

“满二叉树”在不同资料中可能指每个内部节点恰有两个孩子，也可能指所有层都满。后者本书称完美二叉树，节点数为 `2^(h+1)−1`。做题先看资料定义，别让同名术语替你决定条件。

10.5 例 4：先序加中序，根的位置把问题分开

给互异标签的先序 `A B D E C F` 与中序 `D B E A C F`，先序首项 A 是根。A 在中序中把序列分成左段 `D B E` 与右段 `C F`，因此左子树大小 3。先序紧接 A 的 3 项 `B D E` 属于左子树，剩余 `C F` 属于右子树。

再看左子树：先序首 B 为根，中序 `D B E` 分出左 D、右 E。右子树根 C，中序 `C F` 说明 C 左空、右 F。递归即可唯一重建。

若每次都线性寻找根在中序的位置，退化情况总时间可能 $\Theta(n^2)$。先用字典建立“标签→中序下标”，每次 O(1)定位，并传区间边界而非复制子数组，总时间可为 O(n)，额外字典 O(n)、递归栈 O(h)。该字典需要标签互异；若标签重复，同一符号在中序可能有多个位置，唯一性与实现都要重新讨论。

后序加中序同理，根在后序末项。先序加后序一般不唯一：先序 A B、后序 B A，既可能 A 只有左孩子 B，也可能 A 只有右孩子 B。若已知每个内部节点恰有两个孩子且标签互异，信息又会增加，可在相应条件下重建，不能把两种情形混用。

10.6 带空标记的遍历，为什么能只靠一个序列

把空孩子也输出为^。小树增强先序为 `A B D ^ ^ E ^ ^ C ^ F ^ ^`。解析时读到普通标签就创建节点，并递归读取左、右子树；读到^就返回空。每个标签都明确占据一个待填槽，左右空位不会丢失，所以能唯一恢复。

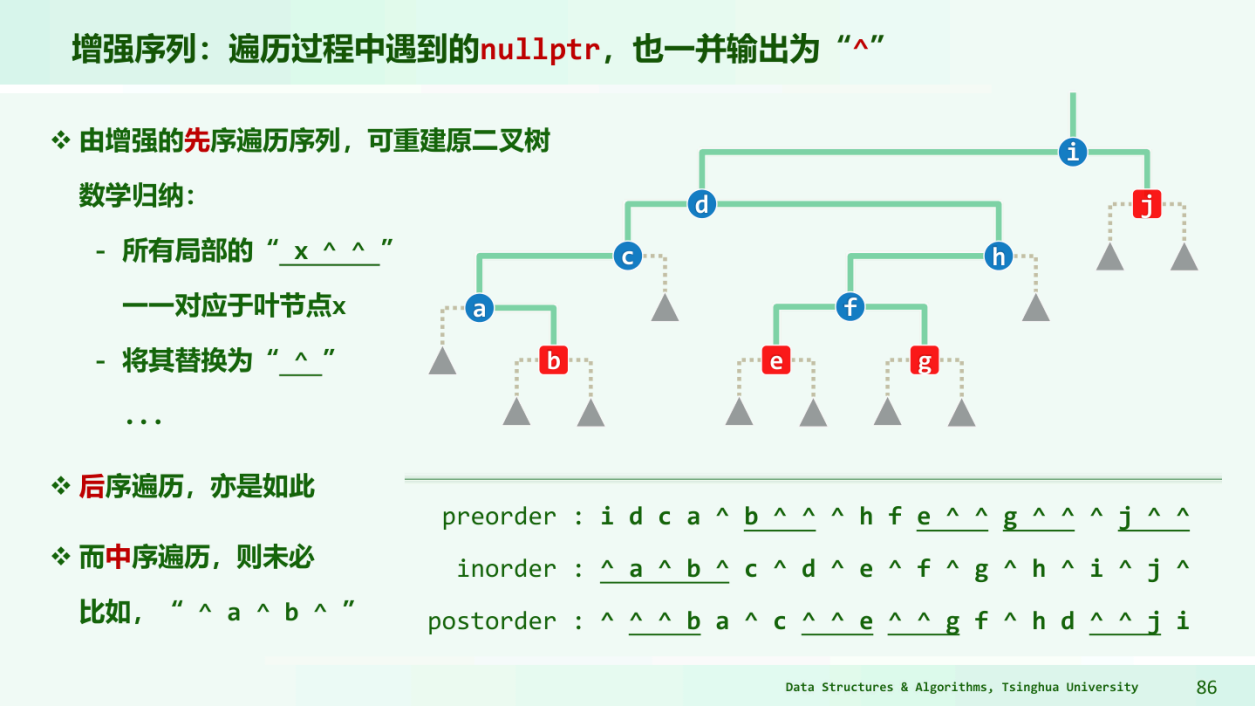


图 5　教师关于增强遍历的原图

图中三角形代表空孩子，红色叶节点后跟着两个空标记。增强先序中，叶子对应一段 x ^ ^，可以将它收缩成一个空位，再递归理解整棵树。增强后序也能从相反方向解析；增强中序却不能唯一编码树形，因为它总是空标记与节点交替，缺少根与子树分界的足够信息。

n 个节点有 n+1 个空槽，因此完整增强先序长度为 2n+1 个记号。这是校验条件之一，但长度对仍可能结构非法。解析器还需检查是否恰好填完所有槽、输入是否有多余记号。

10.7 中序后继：不用重走整棵树

若节点 x 有右子树，中序后继是右子树里最左节点。若没有右子树，沿 parent 向上走，直到第一次从某个祖先的左边返回，该祖先就是后继；若一直从右边向上到根，还没有找到，x 就是最后节点，无后继。小树中 E 无右子树，先从 E 到 B，E 是 B 右孩子，继续；从 B 到 A，B 是 A 左孩子，所以后继 A。C 有右子树 F，后继就是 F。单次后继最坏 O(h)，但顺着后继遍历整树，边只会被上下经过常数次，总时间 O(n)，不能把每次最坏 h 机械乘 n 得到唯一结论。

10.8 外部空叶与普通叶子，数的是两类对象

把每个缺失孩子都补成一个外部叶，包括原来叶节点缺少的两个孩子。原树 n 节点有 2n 个孩子槽，已占 n−1，缺 n+1，因此扩充后有 n+1 个外叶，每个原节点都变成恰有两个孩子的内部节点，内部节点数为 n。若原双孩子节点 n_2、单孩子 n_1、原叶 n_0，则原树 $n_0=n_2+1$；$n=2n_2+n_1+1$，所以外叶 $n+1=2n_2+n_1+2$。

例如原树根 A 只有左孩子 B，n=2。A 右边补一个外叶，B 左右各补一个，共 3 个；A 和 B 都变双分支内部节点，共 2 个。只给 A 补右孩子会漏掉 B 的两个空槽，得到的是另一种变换，不能拿来解释增强先序中的空标记。

10.9 重建时发现输入矛盾，要及时拒绝

先序 A B C，中序 B A D，标签集合不同，不可能表示同一棵树。即使集合相同也要检查根在当前中序区间内：递归的边界分配一旦矛盾，不能从全局字典随便取一个位置继续切片。还应检查先序使用项数恰好等于 n，中序每标签只出现一次，构造结束无剩余项。

这些校验不是数学唯一性证明自动帮代码完成的。证明接受的是合法输入对，解析器若面向外部数据，必须确认前置条件。带空标记解析同样要限制读完后没有额外记号、读取时不提前耗尽，避免“前半能成树”便忽略后面垃圾。

10.10 后序求子树大小，走一个依赖完整的例子

固定小树中，D、E、F 为叶，size 各 1；B 处理完两个孩子后得 1+1+1=3；C 左空大小 0、右 F 大小 1，得 2；A 最后得 1+3+2=6。若在先序访问 A 时就读孩子尚未更新的 size，可能拿到旧值。遍历顺序的选择来自数据依赖，不是偏好。

同样的后序可以求 height=max(左高,右高)+1，空高−1；求表达式值则先求左右操作数再施运算；释放整树也通常先释放孩子，再释放父，避免父被释放后找不到孩子地址。一个遍历模式能复用，但访问动作必须符合对象生命周期。

10.11 叶数与双孩子节点数，从边数推出

设无孩子节点数为 n 零，一个孩子为 n 一，两个孩子为 n 二。节点总数 n=n 零+n 一+n 二，边数一方面为 n 减一，另一方面每个父贡献孩子数，所以为 n 一加二倍 n 二。两式相减得 n 零=n 二加一。

这个关系对任意非空二叉树成立，单孩子节点多少不影响等式。链有一个叶和零个双孩子节点，成立；完美七节点树有四叶、三个双孩子节点，也成立。空树不属于这一非空计数前提，不能把零等于一当作公式错误。

完全二叉树数组的最后一个非叶下标是长度除二取下整再减一。只有一项时没有非叶节点，建堆下滤循环应直接跳过。

访问空孩子时直接返回。

10.12 练习与答案

练习 1。 先序 `R P Q S`，中序 `P Q R S`，求后序并画出左右关系。

解答：根 R，中序左段 P Q、右段 S；左子树根 P，中序 P Q 说明 P 左空、右 Q。右子树只有 S。后序左子树为 Q P，右子树 S，再根 R，得到 `Q P S R`。写出 P 右 Q 这条边很重要，不能因为只有两个节点就随意画在左侧。

练习 2。 增强先序 `A ^ B ^ ^`代表什么？`A ^ ^ B ^ ^`为何非法？

解答：第一串表示 A 左空、右孩子 B，B 左右空。第二串前三个记号已经形成一棵完整的单节点 A 树，后面又多出 B，无法属于同一棵树的先序编码。只检查每个节点后有没有两个空标记不够，应按待填槽解析：开始 1 槽，普通节点占 1 槽并新增 2 槽，净增 1；空标记占 1 槽，减 1；过程中不得提前归零后继续读，结束必须恰好归零。

练习 3。 一棵 10000 节点的退化树，递归中序与层序的辅助空间各多少？

解答：若每个节点只有一个孩子，树高 9999，递归调用栈 Θ(10000)，有栈溢出风险；层序队列中通常只有 1 个待处理节点，空间 O(1)。二者访问时间都 Θ(n)。算法同样正确，空间却受树形影响。

11 Huffman 编码：为什么总把最轻的两个合在一起

四个符号 A、B、C、D 的出现次数分别为 5、7、10、15。若每个符号统一用 2 位编码，总长度为 2×37=74 位。频率高的符号若能用更短编码，通常可以减少总位数；但编码必须能被无歧义地解读。

前缀无关编码要求：任何符号的编码都不是另一个符号编码的前缀。比如 `0,10,110,111` 符合；`0,01,11` 不符合，因为读到 0 时不知是否已经完成一个符号，还是后面还有 1。将 0、1 分别当作树的左、右边，所有符号放在叶子，根到叶的路径就是编码，自然具有前缀无关性。

11.1 例 1：完整合并一次，看两种成本怎样相等

初始权重为 5、7、10、15。取最小 5 与 7 合并成 12，森林剩 10、12、15；取 10 与 12 合成 22，剩 15、22；再合成 37。若左边记 0、右边记 1，可得到 D=0、C=10、A=110、B=111。

编码长度分别为 A3、B3、C2、D1，带权路径长度为

$$5 \times 3 + 7 \times 3 + 10 \times 2 + 15 \times 1 = 71.$$

也等于每次合并权重之和：12+22+37=71。因为合并两棵子树，相当于让其中所有叶子深度加 1，新增位数恰好是这两棵树的权重总和。每个叶子的权重会在它到根的每次合并中各算一次，所以两种记账一致。

解码 `0110111`：从根读 0 到 D，回根；再读 110 到 A，回根；再读 111 到 B，得到 DAB。每到叶子立即完成一个符号，无需在符号之间额外加分隔符。若只剩一个符号，数学上可用长度 0 编码，但实际文件格式通常还需保存消息长度，或约定用一位码；这一边界需由格式规定。

11.2 贪心选择的理由，要先证明“存在一棵最优树这样做”

对正权重的最优前缀编码树，内部节点不需要只有一个孩子：若存在这种节点，将它绕过，其后所有叶子都少走一条边，总成本严格减小。所以可以考虑每个内部节点恰有两个孩子的树。

在任意这样的最优树中，取最深的一对兄弟叶子。将最轻的两个符号 x、y 交换到这两个叶子上，不会增加成本。为什么？若 w_x≤w_z、深度 d_x≤d_z，交换后成本变化为`(w_z-w_x)(d_x-d_z) <= 0`。把较轻者放得更深，不会变差。重复交换，可得到一棵仍最优、且最轻两符号为最深兄弟的树。

将这对兄弟收缩成权重 w_x+w_y 的一个伪符号，原树成本减去 w_x+w_y。收缩后的树必须对缩小后的权重集合也最优；否则用一棵更便宜的树替换它，再展开 x、y，便得到比原最优更便宜的树，矛盾。

于是原问题可安全地先合并最小两个，再递归处理。注意证明说的是“至少有一棵最优树具备这个结构”，并没说所有最优树形完全相同。频率相同、左右孩子互换，都可能产生不同编码而总成本相同。

11.3 例 2：不按最小两项合，成本会变多少

权重 1、2、8、9。Huffman 先合 1+2=3，再合 3+8=11，再合 11+9=20，总成本 34。若先合 8+9=17，再合 1+2=3，最后合 17+3=20，总成本 40。

第二种把高频 8、9 一起埋到更深层，每出现一次都多花位数。这个反例说明“任意两棵合起来最后总权相同”不意味着总编码成本相同。根权重总是 20，但叶子的深度不同。

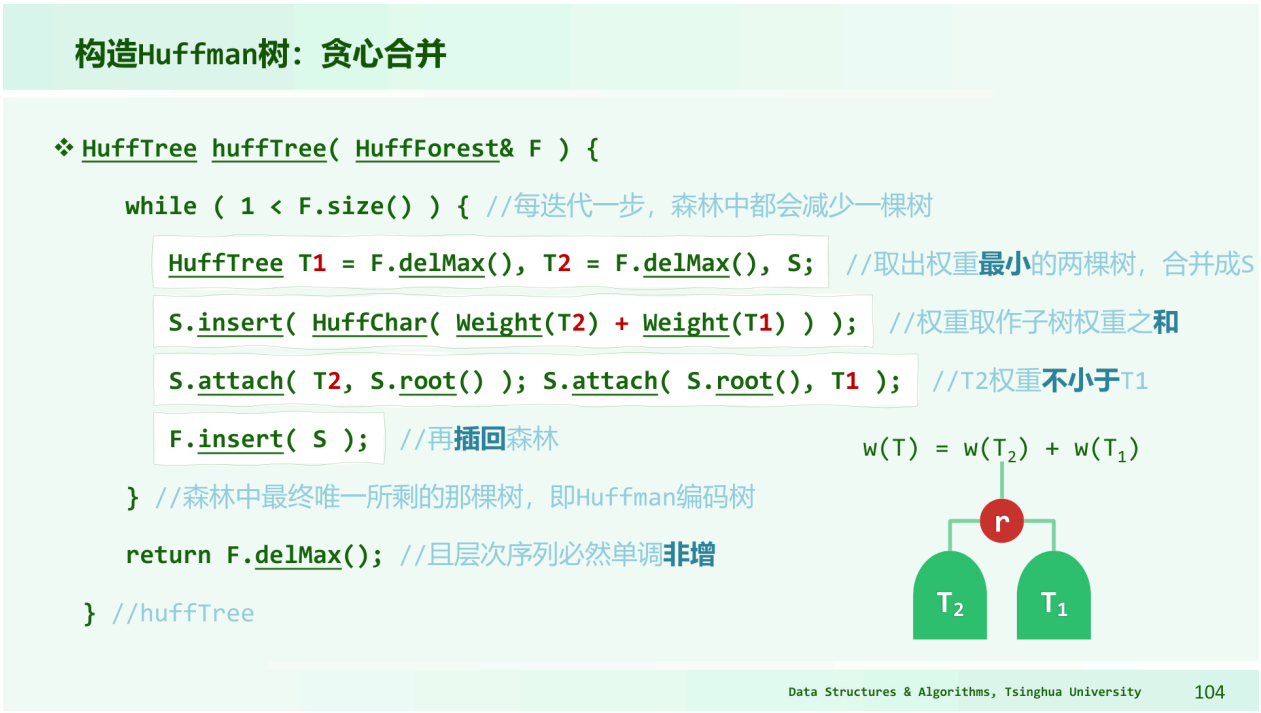


图 6　教师原图中的 Huffman 合并

原图把两棵取出的最小权树接到新根下，新根权重等于两者之和。注意图中接口名可能写 `delMax`，而注释说取最小，这是通过比较器或权重优先级的定义实现的；读程序应核对比较规则。若直接用默认最大堆取最大的两个，就变成上面的错误策略。

11.4 构造的时间，取决于怎样找最小

用无序表，每次扫一遍找最小两项，n−1 次合并共 $\Theta(n^2)$。用二叉最小堆，初始建堆 O(n)，每轮两次删除最小、一次插入，总时间 O(n log n)，树节点 O(n)。每次输出完整编码串还要计入输出位数，不能把所有字符串复制成本都塞进常数。

如果输入权重已经非降排序，可用两个队列。一个放尚未使用的原权重，另一个放新合成权重；每次从两个队头中取较小者两次，合并后放到第二队尾。新合成权重会非降出现，因此第二个队列也有序。所有权重进出常数次，构造为 O(n)。如果原输入无序，先排序通常要 O(n log n)，不能把这一步悄悄删掉。

11.5 例 3：两个有序队列怎样执行

权重 `[2,3,7,8]`。原队列 Q1 为 2,3,7,8，合成队列 Q2 为空。取 2、3 得 5，Q1 剩 7,8，Q2 为 5；下一次先取 Q2 的 5，再取 Q1 的 7，合成 12，Q1 剩 8，Q2 为 12；再取 8、12 合 20，总成本 5+12+20=37。

最小两个可能同来自 Q1、同来自 Q2，或者一边一个。不能每轮固定取“一个原符号、一个合成符号”。统一比较两个队头，才与全局取最小两项一致。

11.6 比较排序下界：为什么不能靠比较把所有输入都线性排好

n 个互异元素有 n!种可能的大小次序。只允许两项大小比较的确定性算法，每次比较最多产生两个结果，可画成二叉决策树。不同输入次序必须到达能区分它们的叶子，否则同一输出次序至少会对一种输入错误。

高度 h 的二叉树最多 `2^h` 个叶子，所以 `2^h≥n!`，有 $h \geq \log_2(n!)$。后半约 n/2 个因子都至少 n/2，因此 $\log_2(n!) \geq (n/2)\log_2(n/2) = \Omega(n \log n)$。这给最坏比较次数下界。

n=3 时有 6 种次序，2 次比较最多区分 4 种结果，故至少某些输入要比较 3 次。它并没说每一个输入都一定 3 次，也没说所有非比较算法都需要 3 次；模型限制是论证的一部分。

若输入是均匀随机排列，平均深度同样有 $\Omega(n \log n)$下界，可以用前缀码的 Kraft 不等式证明。若输入分布偏向少数有序情况，平均下界要改用其信息量，不能直接继续套 log(n!)。

11.7 归约时，箭头方向必须读对

若问题 A 能用 O(n)时间转成问题 B，并能从 B 的答案 O(n)恢复 A 的答案，就说 A 可线性归约到 B。若 B 有线性算法，A 便也有线性算法；因此已知 A 有 n log n 下界时，可以将下界传给 B。

要证明 Huffman 构造在任意未排序实权重的比较模型下有相应下界，必须明确你要求输出的对象、从中恢复排序的成本及相关归约细节；不能只因为常用实现先排序，就宣布所有算法必先排序。排序好的权重可线性构造已经说明条件会改变复杂度。本节先给比较树与归约方向；PS05 第 37 题用强分离权重构造至少 n!/2 类可区分带标签输出，进一步展开 Huffman 特定下界。阅读时仍应保持该题的比较模型、输入权重范围与输出对象约定。

11.8 权重相同，树不唯一却仍可严格核对

四个符号权重全 1，先任取两项合 2，再取另两项合 2，最后合 4，总成本 2+2+4=8，每项深 2。若第二步错误地合 1 与刚产生的 2，得到 3，再与剩 1 合 4，总成本 2+3+4=9，比最优差。即使初始权重相同，“每次最小两项”仍有实质约束。

在最优成本固定的情况下，左右交换、相同权重叶子互换会改变码字。自动检查不应只与某一份码字文本完全相等，应验证前缀无关、每个符号都编码、解码可还原、带权长度达到要求。如果题目指定规范 Huffman 码，才额外按长度与符号顺序生成确定的码字。

11.9 交换论证中的成本差，再代一组数

权 1 的 x 在深度 1，权 5 的 z 在深度 3，原贡献 1×1+5×3=16。互换后 1×3+5×1=8，下降 8。一般变化为(w_z−w_x)(d_x−d_z)，这里 4×(−2)=−8。先用数看清符号，再写不等式，就能避免把“轻的放深”证明反了。

把最轻两项放到最深兄弟位置时，可能一次涉及两个已有符号交换，应逐次应用不增成本原则；若它们本来已占其中位置，就少做一次。证明不要求每次严格下降，因为相等权重或深度相同可能使变化 0，仍保留最优。

11.10 两队列线性构造，为何新合并权重不会倒退

本次取出的最小两项为 a≤b，合并 a+b。剩余项都不小于 b，下一轮取出的两项 c≤d 至少满足 c≥b；若其中包含刚生成的 a+b，也仍不小于 b。因为上次 a≤b，下一次合并总值 c+d≥2b≥a+b，因此合成队列非降。

这个不变式保证只看两个队头就够。若允许负权，a+b 可能小于 a，使队列顺序与编码最优化解释都需要重审；普通频率与出现次数为非负，正权证明可将零频符号按格式约定先处理或保留，不要直接把带负权的“成本”套入同一贪心定理。

11.11 输出码字也是工作

如果某个 Huffman 树退化得很深，逐叶输出完整 0/1 字符串的总字符量可达平方级，即使构造树本身用堆只有 n log n。只返回树的节点链接与权重，是 O(n)规模的输出；打印所有根叶路径，是另一种输出要求。复杂度陈述应写清你构造了什么，而不是让“编码”一词同时代指树、长度表与所有完整字符串。

11.12 编码树中的未使用叶子，可以怎样理解

前缀无关编码不一定把整棵二叉空间占满。例如只有编码零与一零，仍可解码，但一一对应的分支空着。若两个符号权重都正，将第二个码缩为一，总成本会下降；所以最优二进制编码可去掉无用单孩子分支，得到真二叉树。

若权重为零，缩短它可能不改变成本，最优树仍可能有多种无成本差异的形状。实际压缩格式还需传送码表、频率表或树结构，这部分头部开销不在单纯带权路径长度中。只有很短的消息时，省下的正文位数可能还不够支付头部；算法优化的目标量要与文件总大小区分。

Huffman 针对已知符号频率的逐符号码最优，不说明它能捕捉任意跨符号相关性。例如两个字符总成对出现，把一对作为一个复合符号建模可能得到另一种压缩效果；模型变了，最优性范围也随之改变。

两个符号频率分别九与一，最优前缀码仍各一位，总十位；不能给常见符号空码、罕见符号一位，因为空串是任何字符串的前缀，解码会丢失符号边界。单符号的空码约定不能推广到两个符号。

11.13 练习与答案

练习 1。 权重 2、2、3、6，写合并序列、总成本和一种编码长度。

解答：2+2=4，3+4=7，6+7=13，总成本 24。权 6 叶深 1，权 3 叶深 2，两项权 2 叶深 3，所以 6×1+3×2+2×3+2×3=24。左右路径可互换，因此具体码字不唯一，长度与成本在该构造下如上。

练习 2。 只给码字长度 1、2、2，能否安排前缀无关二进制编码？长度 1、1、2 呢？

解答：前者可为 0、10、11，成立；后者若两项长度 1，已占 0 和 1，任何更长二进制串都以其中一个为前缀，无法再放第三项。也可用 Kraft 和检查：前者 1/2+1/4+1/4=1，后者 1/2+1/2+1/4>1。

练习 3。 有人拿已经排序权重的 O(n)Huffman 算法，声称推翻比较排序下界，为什么不成立？

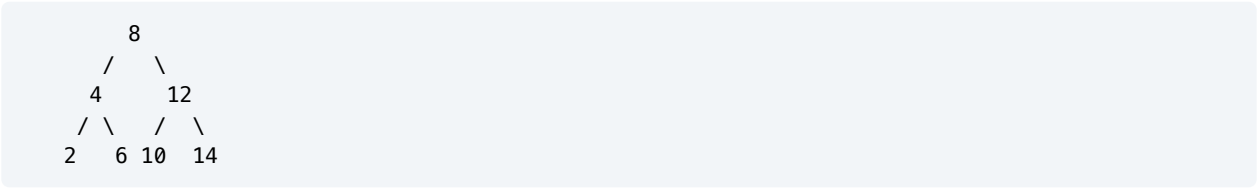
解答：算法接收的输入已含顺序信息，省去了原本必须确定的 n!种顺序。比较排序下界讨论任意未排序互异输入。使用更强的输入前提，并没有解决原来同一个问题。任何复杂度冲突，先检查模型、输入范围与输出要求是否一致。

12 搜索树：每次比较之后，往哪边走

有序向量查找快，却可能为一次插入搬动很多项。链表局部插入快，却很难快速找到按键值的位置。二叉搜索树把“有序”放进分叉关系：每个节点左子树的所有键较小，右子树的所有键较大。为了先讲清关系，本章假设键互异；重复键可以用计数，或统一放某一侧，但接口和证明要配套。

12.1 例 1：查找与插入，共用同一条比较路径

依次插入 8、4、12、2、6、10、14，得到：



查找 6：6<8 向左到 4；6>4 向右到 6，找到。查找 5：向左到 4，向右到 6，再向左到空，失败。这个空槽正是插入 5 的位置。路径上每个比较都排除另一侧的整棵子树，因为那里的所有键都不可能等于目标。

不变式是“若目标存在，则它仍在当前子树内”。每步进入一个严格更小的子树，最终到目标或空槽。时间 O(h)，h 为树高。若树退化成链，h=n−1，查找最坏线性；仅仅名字叫“树”不保证对数级。

中序遍历先列左侧小键，再根，再右侧大键，所以 BST 中序严格递增。反过来，若二叉树中所有互异键的中序严格递增，也能推出每个节点左右满足搜索性质。因此旋转是否保序，可以通过中序序列检查。

12.2 例 2：删除节点，要分清它有几个孩子

删叶子 2，直接断开父 4 的左链接。删只有一个孩子的节点，让父直接接到它的唯一孩子。两个孩子都在的节点，不能把左右孩子随便挤在同一个位置。

在原树删 8，可以找它的中序后继 10，也就是右子树的最左节点。把 10 的键与对应记录移到根的位置，再从原位置删掉 10。因为后继没有左孩子，后一次删除只会遇到 0 或 1 个孩子。

若节点存`(学号,成绩)`记录，替换时必须复制整个对应记录，不能只改学号却留下旧成绩。若外部持有节点身份，复制记录与真正交换节点也有不同语义；接口应说明外部引用会怎样变化。

后继为何没有左孩子？它已是右子树最左节点，有左孩子便还能向左走，与“最左”矛盾。为何替换后仍保序？后继是所有大于原根键的键中最小者；左子树所有键小于原根，也小于它；右子树去掉后继后其余键仍大于它。

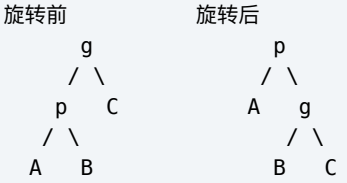
删除总路径长度 O(h)，还可能需要沿祖先更新高度或子树大小。若有父指针，删除后孩子的 parent 也必须更新，不能只改向下链接。

12.3 平衡是在控制高度，不要求左右节点数相等

按 1、2、3、4、5 顺序插入普通 BST，会形成一路向右的链。AVL 树要求每个节点的左右子树高度差绝对值不超过 1。平衡因子可定义为 `height(left)-height(right)`，合法值−1、0、1。

高度 h 的 AVL 树最少节点数 N(h)满足 `N(h)=1+N(h-1)+N(h-2)`，因为要尽可能少，就让两棵子树高度分别 h−1 与 h−2。取 N(−1)=0、N(0)=1，得到 1、2、4、7、12……，与 Fibonacci 同阶，故 n 至少指数于 h，反过来 h=O(log n)。这才是 AVL 查找对数级的理由。

12.4 例 3：一次右旋，为什么不破坏大小关系



旋转前的键关系是 A 里的键<p<B 里的键<g<C 里的键，旋转后这条中序关系完全相同。改动是让 p 升上去，g 成为 p 的右孩子，B 转接成 g 的左孩子。不能丢掉 B，也不能把它接到 g 右边。

若 g 原来不是整树根，g 原父节点指向这棵局部树的链接要改成 p；若 g 是根，要更新 root。再更新 B 的 parent、g 的 parent、p 的 parent。高度必须先更新降下去的 g，再更新升上来的 p，因为 p 的新高度依赖 g。



图 7　教师关于局部旋转的原图

图中左右两幅互为逆操作，X、C、V 等方块代表整棵子树，不是单个值。旋转只改少量链接，整块子树内部不动，所以一次旋转 O(1)。中间子树在两个节点之间重新挂接，是保持中序不变的关键。

12.5 例 4：四种插入失衡，跟着三个节点的方向判断

插入 30、20、10，节点 30 左侧过高，路径为左、左，即 LL。对 30 右旋，得到根 20、左 10、右 30。

插入 10、20、30，路径右、右，即 RR。对 10 左旋，同样得到 20 为根。

插入 30、10、20，路径左、右，即 LR。先对 10 左旋，局部变成 30 左孩子 20，20 左孩子 10；再对 30 右旋，得到 20 为根、10 与 30 为左右孩子。

插入 10、30、20 为 RL，先对 30 右旋，再对 10 左旋。双旋可以理解为把三个关键节点按中序排成 a<b<c，让 b 成为局部根，四棵间隔子树依序挂回。不要仅靠画出的左右倾斜外观判断，应该追踪从失衡祖先 g 到较高孩子 p，再到本次插入所在孙辈 v 的方向。

插入修复从新节点向根回溯，找最低失衡祖先。修复后这棵子树高度恢复到插入之前，故更高祖先不会因这次插入继续失衡；需要更新沿路高度，但旋转修复次数为常数。

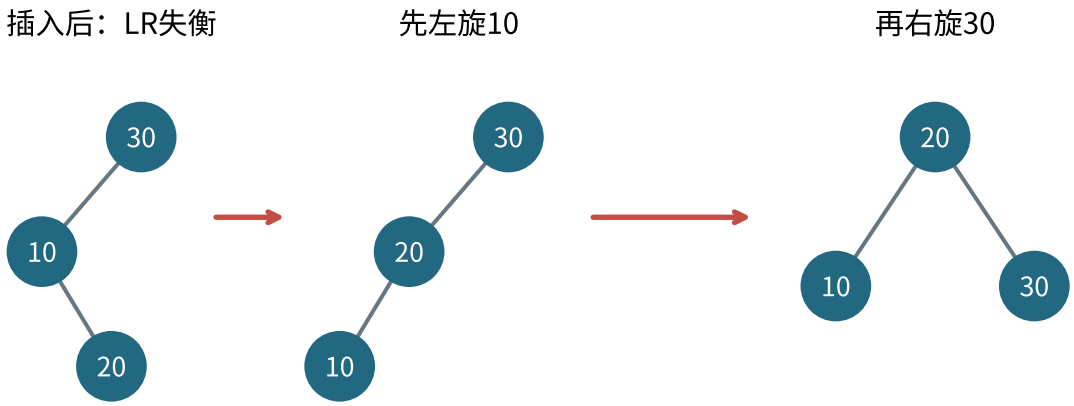


图 8　LR 失衡的两步旋转

左图三十的左孩子是十，十的右孩子是二十，形成内折。第一根箭头表示先在十处左旋，使二十上升；第二根箭头表示再在三十处右旋，最终二十成为局部根。三个节点的中序始终是十、二十、三十，图只省略了空子树；真实树中还要按值域接回四棵侧子树。

12.6 删除修复与插入不同

删除可能让子树高度减少 1，修复后仍比原来矮，因此失衡可能继续向上传播，最坏需要 O(log n)层的旋转修复。不能把“AVL 插入最多一次局部重构”直接套到删除。

考虑根 4，左子树为 2 带孩子 1、3，右孩子 5 为叶。删 5 后，4 的左高 1、右高−1，平衡因子 2；较高孩子 2 的左右等高。对 4 右旋后根 2，左 1，右 4 且 4 左 3，树重新平衡。这里孩子平衡因子为 0，是删除中需要特别覆盖的一种情况；若只写插入时的 LL/LR 判断，可能遗漏。

每次回溯先重新计算高度，再判断因子，旋转后继续从新局部根的父节点向上。实现最好将“更新高度”“重新连接父链接”“局部旋转”分别封装，避免四种情况各写一份不一致的指针操作。

12.7 子树大小还能支持第 k 小

每个节点额外存 size=1+左 size+右 size。要找从 0 起的第 k 小，令 L 为左子树大小：k<L 就去左边；k=L 返回当前节点；k>L 就去右边，新的 k 减去 L+1。每次只走一条根叶路径，在 AVL 中 O(log n)。

例如原 7 节点树找第 4 小，根 8 左边 3 项，4>3，去右子树，k 改为 0；节点 12 左子树含 10 一项，0<1，去 10，返回 10。旋转不改变局部元素总数，但改变各节点子树范围，所以必须像更新高度一样更新 size，先下后上。

12.8 AVL 删除为什么可能一层接一层修复

插入使局部高度增加，最低失衡点重构后常恢复插入前高度，因此上方不再继续受影响。删除使局部高度降低，旋转后这棵子树可能仍比删除前矮 1，父亲就可能因另一侧相对变高而失衡。回溯必须继续，不能看到第一次旋转成功就直接返回。

准确实现时，从被物理移除节点的父亲开始，而不一定从最初指定删除键的位置开始。两孩子删除若用后继替换，真正少掉节点发生在右子树最左端；沿那里的祖先链更新才覆盖实际高度变化。若只从原根位置改一次高度，会漏掉右子树内部的修复。

12.9 三节点四子树重构，把双旋展开成一次有序接回

设按中序排序的三个关键节点为 a<b<c，它们之间四棵子树 T_0、T_1、T_2、T_3也按值域依次排列。重构后 b 为局部根，a 为左孩子、c 为右孩子；a 左右接 T_0、T_1，c 左右接 T_2、T_3。先更新 a、c 的 height 与 size，再更新 b，最后把 b 接回原祖父。

例如 LR 结构根 30，左 10，10 右 20。若 10 左 T_0、20 左 T_1、20 右 T_2、30 右 T_3，则重构根 20，左 10 挂 T_0,T_1，右 30 挂 T_2,T_3。四个值域从未改变相对顺序，所以搜索性质保留；中间两子树不再依原父节点身份挂接，而依它们数值区间挂接。这比只画 10、20、30 三个圆更接近真实指针操作。

12.10 有序统计的 rank 查询，和第 k 小方向相反

若问键 x 之前有多少键，比起查第 k 小，需要沿查找路径累计。当前节点键小于 x 时，它的整个左子树和自己都可计入，答案增加左 size+1，再去右侧；当前键≥x 时去左侧，不增加。不存在 x 也能得到它的插入秩。

原树键 1、3、6、8、10，查 x=7：根 8 太大去左，节点 3<7，计左 1 及自己，共 2；去 6，6<7 再计 1，总 3。恰有 1、3、6 三个键小于 7。若重复键以计数保存，当前节点贡献是 count 而非 1，size 也应统计所有记录数，接口到底返回不同键秩还是记录秩要先说明。

12.11 搜索失败不代表没有可用信息

BST 查找 5 最终停在 6 的空左槽，路径已经给出前驱与后继范围。可在下降时保存“最后见到的小于目标节点”与“最后见到的大于目标节点”，失败后仍能返回边界。平衡树除了是否存在，还能回答 lower_bound、upper_bound 与范围遍历，这些能力来自全局顺序，不是散列桶可以直接提供的。

12.12 等值键保存为计数，删除一个与删除全部不同

若节点键五的 count 为三，removeOne 只将 count 减到二，树形无需改变，但路径上 size 都要减一；removeAll 才需要物理删除节点，并将相关记录数量从增广 size 里扣除。若 size 统计不同键数，而查询要求含重复的第 k 小，两者就不匹配。

例如键一一份、键五三份、键九一份，记录序列为一、五、五、五、九。第五小是九，第三小是五；在节点五处，左边记录数一，自身覆盖接下来的三个秩。用左 size 加一替代左 size 加 count，会把后面秩算错。多重集合不是只允许重复插入这一个改动，查找、秩与删除语义都需一致。

搜索树的键应满足一致的比较顺序。若同一对键这次比较小于、下次又大于，或者键在树内被外部修改，原来指向左子树的范围可能不再成立。修改键通常应先删除旧记录，再按新键重新插入；只改节点字段而不重新定位，会让查找沿错误分支走失。浮点特殊值也可能不满足普通全序假设，比较器需明确定义它们怎样排列。

旋转前后还应核对总记录数不变，局部根接回以后，原祖先的孩子指针与新根的父指针必须互相对应。

12.13 练习与答案

练习 1。 依次插入 40、20、30，写出失衡类型与两次旋转后结构。

解答：40 左孩子 20，20 右孩子 30，是 LR。先 20 左旋，让 30 挂到 40 左边、20 成为 30 左孩子；再 40 右旋，让 30 成为根，左 20、右 40。中序始终 20、30、40，旋转后高度 1，左右高 0。

练习 2。 删除 BST 中有两个孩子的节点时，能否用前驱而非后继？

解答：可以。前驱是左子树最右节点，它不可能有右孩子，所以替换后也归约到 0 或 1 孩子删除。左右论证对称。应复制完整记录并更新原前驱位置的链接、parent 与增广信息。若重复键策略不同，前驱或后继替换需与等值放置规则一致。

练习 3。 左右子树节点数为 7 与 3 的节点，能否是 AVL 合法节点？

解答：可能。7 节点子树若高度 2，3 节点子树若高度 1，高度差 1，合法。AVL 约束高度，不约束节点数相等。反过来，节点数相近也不保证高度接近：某侧若退化为链仍可能很高，但它本身就违反 AVL 递归平衡条件。判断时必须检查每个节点的高度关系。

13 高级搜索结构，各自在为哪一种成本付账

AVL 用严格的高度差控制最坏查找时间，但真实任务还会关心别的事：刚访问的键会不会很快再访问？数据是否在磁盘上，一次读一页能取回很多键？是否希望结构简单，允许用随机性换实现成本？伸展树、B 树、红黑树与跳转表回应这些不同需求。

它们都支持按键查找，但维护的关系不同。读这一章时，每换一种结构，就重新确认它保存什么、不变量是什么、一次操作为什么能完成，以及给的是最坏、期望还是摊还保证。

13.1 伸展树：把刚访问的节点转到根

伸展树仍满足二叉搜索树的中序关系。查到节点 x 后，用旋转把 x 移到根。若查找失败，常把最后访问的非空节点伸展到根；具体接口应明确，以便后续插入与分裂使用。

只有父节点而无祖父时，转一次，叫 zig 或 zag。若有祖父 g 与父 p，x、p 同向为 zig-zig，例如 g 左 p、p 左 x；先转 g，使 p 升上去，再转 p，使 x 升上去。若方向相反为 zig-zag，例如 g 左 p、p 右 x；先转 p，再转 g。

13.2 例 1：为什么同向时要成对转

有左链 30⟶20⟶10，访问 10。先对 30 右旋，得到 20 为局部根、10 左、30 右；再对 20 右旋，得到 10 为根、20 右、30 为 20 右。每次中序仍 10、20、30。

这一次看上去只是把链倒过来，但更大树中的双旋会重新分布侧子树，摊还分析正利用这种整体路径缩短。若每次只把 x 与父转一下，反复逐层提根，仍能把 x 放到根，却不等于标准伸展策略，也不能自动照搬标准摊还界。

下一次再访问 10，它就在根，成本 O(1)；访问附近经常使用的键也可能因局部性而便宜。不过伸展树不是时刻平衡的。一条长链中访问最深节点，单次可能 Θ(n)。对一串 m 次操作，标准伸展操作有 O((m+n)log n)型总界，常说摊还 O(log n)，初始结构的势能成本要一并计入。它不承诺每次最坏对数。

13.3 例 2：分裂与合并，不必重新插入全部节点

对一棵键集合为{1,3,5,7,9}的伸展树，按键 5 分裂。先查到 5 并伸展到根，此时左子树恰好所有小于 5 的键，右子树恰好所有大于 5 的键。断开左右链接，即得到两部分，5 可按接口约定放入其中一边或单独返回。

若要合并两棵树 L 与 R，前提是 L 的所有键小于 R 的所有键。把 L 最大节点伸展到根，此根没有右孩子；令它的右孩子为 R，更新 parent 与大小，完成。若两集合的键范围交错，例如 L 含 1、8，R 含 3、9，就不能这么接，BST 性质会坏。

插入可借分裂完成，删除可把目标伸展到根，丢掉根后合并左右。每种操作都复用同一个保序结构关系，比逐项重插更省工作。

13.4 B 树：一次读回一整页，就让一个节点保存许多键

磁盘或外存一次访问的单位常是一页，读一个键与读同一页里的几十个键，I/O 次数可能一样。B 树让一个节点保存有序键序列和多条子树链接，减少根到叶要跨越的页数。

用“最小度数 t”约定：非根节点至少 t−1 个键，至多 2t−1 个键；内部节点孩子数等于键数+1；除根外内部节点至少 t 个孩子；所有叶子深度相同。根可更少，空树另行处理。有的教材用“阶 m”表示最多 m 个孩子，转换时要看最少孩子取整规则，不能把 t 与 m 直接当同一个变量。

节点内键 $k_1<k_2<\dots<k_r$，孩子依次容纳小于 k_1、介于 k_1与 k_2、……、大于 k_r 的键。查找先在当前页中定位间隙，再读相应孩子页。高度 O(log_t n)，这描述页访问数；页内线性找键为 O(t)，二分找键为 O(log t)，CPU 成本还需另算。

13.5 例 3：t=2 的 B 树插入，分裂时中键上升

从空树依次插 1、2、3，根叶里是[1,2,3]，达到最多 3 键。插 4 前先分裂满根：中键 2 升到新根，左叶[1]、右叶[3]。再把 4 插右叶，得到[3,4]。插 5 后右叶[3,4,5]。插 6 前分裂这个满孩子，中键 4 升到根，根变[2,4]，三片叶[1]、[3]、[5]；再将 6 插末叶得[5,6]。

中键只保留在父页，分裂出的两个孩子各保留 t−1 项；内部节点分裂还要把 2t 个孩子按顺序分成两组。所有叶深仍相同，键的范围关系也不变。若采用“先插入造成溢出再向上分裂”的课程版本，触发时刻与临时键数不同，应按那套容量约定追踪，最终仍需满足相应 B 树条件。

删除更复杂：目标在内部节点，可换成前驱或后继；进入键太少的孩子之前，需要向兄弟借键或与兄弟合并，保证删除后不低于下限。例如 t=2、根[2]、两叶[1]与[3]，删 1 时两兄弟都只有最少 1 键，可把左叶、父键 2、右叶合成[1,2,3]，根下降；删 1 后根叶[2,3]。根是唯一允许通过变空而降低高度的位置。

13.6 红黑树：颜色是对多叉树结构的一种编码

红黑树保持 BST 顺序，并给节点颜色。使用常见约定：根黑；空孩子视为黑；红节点的孩子必须黑；从任意节点到各个后代空孩子的路径含相同数量黑节点。最后一条称黑高一致，计不计当前节点可另定，但同一证明中要统一。

没有连续红节点，说明路径上红数不超过黑数附近的常数项；黑高为 b 的子树至少有 2^b−1 个内部节点。因此最长路径至多约 2b，得到高度 O(log n)。它允许比 AVL 更宽松的局部高度差，却仍有最坏对数查找保证。

把红孩子与黑父收缩成一个多键节点，可以把红黑树理解成 2-3-4 树的一种二叉表示。颜色修复是维护多叉节点容量的另一种写法。这个联系能解释为什么会出现“叔叔红就变色、叔叔黑再旋转”，而不必先背一整页情况表。

13.7 例 4：插入 10、5、15、1，为什么先变色

插 10 为根，涂黑。插 5 为红左孩子，插 15 为红右孩子，均不违反红父限制。插 1 为 5 的红左孩子，出现红父 5 与红子 1；叔叔 15 也是红。将 5、15 涂黑，将祖父 10 涂红，再因为它是根涂回黑。整树根 10 黑，左右 5、15 黑，1 红。

若叔叔黑或空，例如依次插 10、5、1，则是红父 5、红子 1、黑叔叔。对 10 右旋，让 5 升上去，再使 5 黑、10 红，得到根 5 黑，左右 1 红、10 红。若为内折 10、1、5，先对 1 左旋把它变成同向，再做外侧旋转。

删除黑节点可能造成某条路径少一个黑节点，修复要处理兄弟颜色、兄弟孩子颜色等情况。不能简单“删后把孩子涂黑”便保证所有场景正确。理解黑高缺口以后再读实现，逐项检查每种变色与旋转是否让所有出口路径的黑数一致。

13.8 跳转表：在有序链表上加几层快车道

底层保存所有有序键；每个节点独立随机决定是否升到上一层，例如每层以 1/2 概率继续升高。上层是下层的子序列，越高越稀疏。查找从最高层最左开始，只要向右仍不超过目标就向右，不能再右进时下降一层。

13.9 例 5：在三层索引中查找 7

底层为 1、3、5、7、9；中层为 1、5、9；顶层为 1、9。查 7 时顶层从 1 看向 9，9 太大，下降；中层从 1 走到 5，下一个 9 太大，下降；底层从 5 走到 7，找到。上层让我们跳过一批底层节点，下降后又保留精确定位能力。

插入时先查出每一层位于新键之前的节点，保存在 update 数组，再按新节点随机高度接入这些层。删除则在各层断开它的链接。所有层必须保持相同底层键的相对顺序，且高层节点必有对应低层节点。

理想独立抛币模型下，总链接数期望 O(n)，查找、插入、删除期望 O(log n)。最坏抛币结果可能几乎没有高层快车道，查找退化 O(n)。若教材使用确定性跳表或限制层数的版本，保证形式会改变。随机性必须来自定义好的层高生成过程，不能只写“平均很快”。

13.10 B 树删除的三个修复动作，键必须经过父节点

下面都取最小度数 t=2，即非根至少 1 键、至多 3 键。进入一个只有最少键数的孩子之前，先保证它能安全删一项。借键不是兄弟把一项直接扔给它：兄弟与当前孩子之间隔着父键，顺序必须通过父键轮换。

向左兄弟借：父节点[20]，左孩子[5,10]，右孩子[30]，准备在右边删 30。左兄弟有多余键，让父键 20 降到右孩子开头，左兄弟最大键 10 升到父亲。得到父[10]、左[5]、右[20,30]，再删 30，右变[20]。若直接把 10 塞到右孩子而父仍 20，右孩子含 10<20，搜索范围就错了。

向右兄弟借：父[20]，左[5]，右[30,40]，准备删 5。父 20 降到左孩子末尾，右兄弟最小 30 升父，得到父[30]、左[5,20]、右[40]，再删 5，左剩[20]。内部孩子借键时，相关的最左或最右子树指针也要一起转移，否则键范围虽然看似对了，子树却挂错。

合并：父[20]，左右孩子[5]、[30]都只有最少 1 键，谁也不能再借。将左、父分隔键、右拼成[5,20,30]，父失去 20 及一个孩子链接。若父原来就是根且无键，直接让合并节点成为新根，树高降 1。随后删 5，根叶为[20,30]。若父不是根，它变少可能继续引发上方修复，所以删除路径必须保留祖先信息，或采用下降前预修复策略。

删除内部键也有明确分支。父[20]左孩子[5,10]足够大时，用左边前驱 10 替换 20，再递归删左子树的 10，结果根[10]、左[5]、右[30]。若右子树足够大，可用后继；若两边都最少，就先合并成含 20 的大孩子，再在其中删除 20。每个动作保持键范围与叶深一致，不能只保证节点数没少错。

根允许比其他节点少，是高度变化能够发生的位置。所有其他节点若随意变空，会让一次向下选择失去分隔关系；所有叶深若不一致，外存查找路径界也不再成立。B 树维护的容量条件和同深条件，分别约束宽度与高度。

13.11 红黑删除：把少掉的一层黑色当作一个待处理缺口

普通 BST 删除 0 或 1 孩子节点后，若被移除的是红节点，黑路径数不变；若删黑节点而替代孩子是红，只需将该孩子涂黑。真正困难的是删掉黑色后，由一个黑孩子或空孩子 x 接替，使这条路径比兄弟方向少一个黑。常将这个暂时状态叫“双黑”，它是算法记账，节点并没有第三种真实颜色。

下面假定 x 是父 p 的左孩子，右兄弟为 w。近侄是 w.left，远侄是 w.right；若 x 在右边，所有方向镜像。每次修复都要比较从 p 出去的所有路径黑数，目标是消掉缺口或把缺口推向更高祖先。

第一种，兄弟 w 为红。此时 p 必黑，w 两孩子必黑。将 w 涂黑、p 涂红，再对 p 左旋。x 仍在原 p 左边，新兄弟变成原 w 的左孩子，它是黑色。这个动作没有立即补黑，而是把红兄弟情形转换成下面的黑兄弟情形。不能旋转以后仍拿旧 w 当兄弟。

一个合法原树可取根 10 黑，左 5 黑，右 20 红，20 的左右 15 黑、25 黑。删 5 后左侧出现缺口。对 10 左旋并换色，根 20 黑，左 10 红且 10 右 15 黑，右 25 黑。x 的新兄弟是 15 黑；它两个孩子空黑，接着适用第二种，将 15 涂红，缺口提升到红父 10，再把 10 涂黑消去缺口。最终各条根到空的路径黑数重新一致。

第二种，w 黑且两个侄子都黑。w 变红，让兄弟方向也少一个黑，两个方向在 p 下方重新相等；但以 p 为根整棵子树仍少一个黑，因此将缺口转移到 p。若 p 红，涂黑即可吸收；若 p 黑，继续向上；若到根，可直接丢掉这份整体欠账，因为所有根路径一起减少一黑仍等高。

例如根 10 黑，左右 5 黑、15 黑，删 5。w15 两孩子空黑，将 15 涂红，缺口到根 10 后结束，结果根 10 黑、右 15 红。每条根到空路径黑数相同。若误将 15 也涂黑而不处理，就保留了左右黑数不一致。

第三种，w 黑、近侄红、远侄黑。先将近侄涂黑、w 涂红，对 w 右旋。新的兄弟是原近侄，它为黑，原 w 成为它的红右孩子，形成“远侄红”，转入第四种。它是一个转换动作，做完不能直接宣布全部修好。

例如根 10 黑，左 5 黑，右 20 黑，20 左 15 红、右空。删 5 后，先在 20 处右旋并换色，得到 10 右 15 黑、15 右 20 红。此时 x 的兄弟 15 黑，远侄 20 红，正好进入下一种。

第四种，w 黑且远侄红，近侄可红可黑。令 w 继承 p 原颜色，p 涂黑，远侄涂黑，再对 p 左旋，缺口被消除。对上例操作后，根 15 黑，左 10 黑，右 20 黑，左右黑高一致。若 p 原来红，w 继承红可以保持整棵局部树对祖先贡献的黑数不变，不能无条件把所有节点都涂黑。

另一个直接进入第四种的例子：根 10 黑，左 5 黑，右 15 黑且 15 左右 12 红、20 红。删 5 后转一次，得到根 15 黑、左 10 黑、右 20 黑，10 右 12 红。逐条数黑节点即可核对。所谓“近红、远红”只是帮助定位的词，真正判断要看 x 在父的哪侧、相应兄弟的两个孩子颜色。

13.12 伸展树的势能为什么选子树大小的对数

令 s(v)为节点 v 子树大小，rank r(v)=$\log_2 s(v)$，势能 Φ 为所有节点 rank 之和。子树大小反映一次访问后有多少节点被放到某节点下面；取对数使“规模翻倍”变成“rank 增加 1”，便于将一长串旋转的成本与树形变化相抵。

单次伸展的访问引理给出：实际旋转步数加势能变化，不超过常数倍 r(根)−r(x)再加常数，常见表述为 3 倍加 1。因为根子树大小为 n，x 大小至少 1，rank 差至多 $\log_2 n$，于是得到摊还 O(log n)。这里只说明引理怎样推出总界；要完整证明引理，需逐一检查 zig、zig-zig、zig-zag 中改变 rank 的几个节点，不能把这一步叫作“显然”。

这也解释了为什么只做单旋提根不能随便复用同一引理：成对旋转的局部结构决定势能怎样下降。某次长路径花很多旋转，但它改变许多子树大小，使势能下降足以抵消大部分实际成本。若初始势能不为 0，总账要保留 Φ 初这一项；n 个节点每项 rank≤log n，初始势能上界为 n log n。

用三节点链 30 左 20 左 10 看账：原子树大小分别 3、2、1，Φ=$\log_2 3+1$；伸展 10 成反向链后大小仍 3、2、1，只是归属换了，势能总和相同，这次两旋仍需要实际支付 2，不是每次都能靠势能下降变便宜。摊还界容许常数或对数支出，并不宣称每次 Φ 都会降。

13.13 访问引理的三种局部账，逐式看常数怎样消掉

下面把一次旋转成本计 1，r 为旋转前 rank，r′为旋转后 rank，δ=r′(x)−r(x)≥0。只有参与旋转的 x、父 p、祖父 g 的 rank 可能变化，其他节点的子树元素集合不变。对数有一条常用不等式：若 a+b≤c，则 $\log_2 a+\log_2 b\le 2\log_2 c-2$，这是 $ab\le((a+b)/2)^2$得到的。

zig 只有 x、p 变化，p 子树缩小，因此摊还成本 A=1+[r′(x)−r(x)]+[r′(p)−r(p)]≤1+δ。每次完整伸展最多在最后发生一次 zig，所以这一个加 1 不会被每层重复累积。

zig-zag 后 x 占据原 g 整棵子树，故 r′(x)=r(g)。实际两旋加势能变化，约去这两项，得 A=2+r′(p)+r′(g)−r(p)−r(x)。旧 p 包含旧 x，所以 r(p)≥r(x)，得 A≤2+r′(p)+r′(g)−2r(x)。新 p 与新 g 为新 x 两棵不相交子树，大小和小于新 x 大小，故 r′(p)+r′(g)≤2r′(x)−2。代入得 A≤2δ≤3δ。

zig-zig 也先约去 r′(x)=r(g)，得到同样的 A 式。再用 r(p)≥r(x)、r′(p)≤r′(x)，得到 A≤2+r′(g)+r′(x)−2r(x)。这次旧 x 子树与新 g 子树在新 x 内不相交，二者大小和不超过新 x 大小，所以 r(x)+r′(g)≤2r′(x)−2。代入即 A≤3r′(x)−3r(x)=3δ。

将一连串双旋及至多一个单旋的 A 相加，节点 x 在本步后的 rank 正是下一步前的 rank，中间项抵消。最终 x 成为根，rank=$\log_2 n$，得到整次伸展摊还成本≤1+3($\log_2 n$−原 r(x))。这证明了旋转阶段的对数界；查找路径长度与相应伸展步数成常数倍关系，所以查找也纳入这份界。插入、删除还需考虑新节点加入、旧节点移除及分裂合并带来的势能变化，标准实现可据同一大小与 rank 关系继续得到摊还对数总界。

原书习题 8-2 在官方合体 PDF580 到 582 页使用同类势能分析。读总账时保留初始势能：Σ 实际=Σ 摊还+Φ 初−Φ 末。非负势能只保证实际≤摊还+Φ 初，不能省去任意初始树的 Φ 初就说实际总不超过摊还总。

13.14 练习与答案

练习 1。伸展树单次查找最坏 Θ(n)，是否与摊还 O(log n)矛盾？

解答：不矛盾。摊还约束一串操作的总成本，并允许某次很贵。访问深节点后结构发生改变，后续操作可能便宜；分析还要计入初始树势能。如果只拿一棵精心构造的深树做一次访问，再忽略构造与初始势能，就无法用它否定标准总账结论。

练习 2。最小度数 t=3 的 B 树，非根节点键数范围是多少？为什么孩子数比键数多 1？

解答：键数 2 到 5，内部孩子数 3 到 6。r 个有序分隔键把数轴切成 r+1 个间隙：最左、相邻键之间以及最右，因此对应 r+1 个子树范围。叶子没有实际孩子，但容量限制仍适用。根可少于 2 键，例如仅 1 键 2 孩子。

练习 3。跳转表中所有节点恰好只在底层，查找最后一个键要多久？红黑树会出现同样的 n 层链吗？

解答：跳表要走过近 n 个节点，Θ(n)，这是一种可能的最坏随机结果。合法红黑树不能形成任意长单链，因为黑高一致和无连续红节点会被破坏；其高度始终 O(log n)。一个给期望界，一个给最坏界，选择时要看是否需要严格延迟上限。

14 搜索一段范围，为什么不能只照搬找一个键

预约系统保存许多时间区间，要问“下午 3 点哪些预约正在进行”；数组需要反复求某段总和；地图要找矩形框里的点。这些任务都在查询范围，但数据对象和输出不同：区间可能互相覆盖，数组位置固定，平面点有两个坐标。因此要先区分问题，再选择区间树、线段树、范围树或 kd 树。

本章默认区间端点规则会明确给出。闭区间[1,3]包含 3，半开区间[1,3)不含 3。两段是否相交、边界点属于哪边，都依赖这个约定。

14.1 例 1：区间相交，先把判定写出来

闭区间[a,b]与[c,d]相交，当且仅当 `max(a,c) <= min(b,d)`，也等价于 `a<=d and c<=b`。例如[1,4]与[4,6]在 4 相交。半开区间若长度为正，相交应改成严格小于；[1,4)与[4,6)不相交。

对区间[1,3]、[2,8]、[6,9]、[10,12]，查询点 7，答案是[2,8]与[6,9]。朴素扫描全部 n 段，成本 Θ(n)。若查询很多次且区间集固定，值得预处理。

一种区间搜索树按左端点建 BST，每个节点再存子树最大右端点 maxEnd。查找与查询区间[qL,qR]相交的任意一段时：当前节点若相交便返回；否则若左子树 maxEnd<qL，左边所有区间都在查询左侧结束，可整棵跳过；若左侧仍可能相交，先去左边。与平衡 BST 结合，找一个相交区间可 O(log n)。

这里“找一个”与“报告全部”不同。全部输出 k 项至少要 Ω(k)时间；遍历剪枝算法的界还取决于具体增广结构。不能将“有一个 maxEnd”自动当作所有报告查询均 O(log n+k)的充分证明。

课程里的中心分割区间树是另一种结构：选中心点 c，跨过 c 的区间留在当前节点，左边完全小于 c 的递归进左树，右边完全大于 c 的进右树；当前节点的跨中心区间分别按左端升序、右端降序保存。查询 q<c 时，只需在左端有序表中报告左端≤q 者，随后走左子树；q>c 时对称。合适的平衡构造能达到对数定位加输出成本，读课件时注意它与 maxEnd 增广 BST 不是完全同一种树。

14.2 例 2：线段树，每个节点保存一段数组的摘要

数组`[2,1,3,4,5,2,6,1]`，要查询半开区间[2,7)的和，并支持单点修改。线段树根代表[0,8)，左右孩子代表[0,4)、[4,8)，继续二分直到单点。节点存其区间和。

查询[2,7)时，根与查询部分重叠，继续分解。左子树[0,4)中，[0,2)不相交，跳过；[2,4)完全包含在查询中，直接取 3+4=7。右子树[4,8)中，[4,6)完全包含，取 5+2=7；[6,8)继续分，取[6,7)的 6，跳过[7,8)。答案 7+7+6=20。

完全覆盖时为何可以不继续？节点保存的摘要已经等于该整段的答案，继续访问叶子只会重复工作。不相交时为何可跳过？这段没有任何位置属于查询，因此贡献为加法单位元 0。

单点把下标 3 的 4 改成 10，只需要更新叶[3,4)及它到根的祖先。[2,4)从 7 变 13，[0,4)从 10 变 16，根从 24 变 30。每层只改一个节点，高度 O(log n)，因此单点更新 O(log n)。构建时每个节点只合并一次，总节点 O(n)、总时间 O(n)。

求和可以换成最小值、最大值、最大连续区间四元组等，只要能从左右摘要合成父摘要。组合顺序可能重要，例如字符串拼接不可交换，但只要按左后右顺序合并仍可使用。查询空区间时要给出适当单位元，最小值查询通常用正无穷，而不是 0。

14.3 区间修改，懒标记究竟懒了什么

若把[0,4)所有元素加 3，逐叶更新要做 4 次；更一般宽区间可达 Θ(n)。节点已知自己覆盖长度 len，整段加 d 会让 sum 增加 d×len。可在这个节点记一个 lazy=d，表示“该加法已计入本节点和，但尚未传给孩子”。

以后若查询或修改只覆盖它的一部分，就必须先将 lazy 下推：两个孩子 sum 分别增加 d×各自长度，各自 lazy 加 d，再把父 lazy 清零。若整段仍被完全覆盖，可以直接使用父 sum，不必展开。lazy 保存的是尚欠孩子的动作，不能重复给父 sum 加一次。

在长度 8 数组的根上加 2，根 sum 增加 16 并记 lazy2；随后查[0,4)，需要先将左右孩子各加 8 并记 lazy2，父 lazy 清零，再返回左 sum。下推之后根总和不应再改变，它早已算过这一笔。

加法标记可相加，赋值标记却会覆盖旧赋值；赋值与加法混合时先后顺序不能交换。若题目要两种操作，标记需表示一个明确的函数组合，不能只多放两个变量而不定义它们的优先关系。

14.4 例 3：二维矩形查询，范围树利用两层顺序

点为(1,4)、(2,1)、(3,3)、(4,5)、(5,2)，查询 x 在[2,5]、y 在[2,4]内的点，答案(3,3)、(5,2)。只按 x 排序，可先二分出 x 范围，再逐项看 y，最坏仍扫很多项。

二维范围树先按 x 建立平衡树，每个 x 子树再存一份该子树点按 y 排序的辅助结构。查询 x 区间可拆成 O(log n)棵完整子树，对每棵辅助表二分 y 区间并输出。基础版本预处理空间 O(n log n)，查询 $O(\log^2 n+k)$，k 为输出数量。分数级联可继续改进二分开销，但需要额外链接及证明，不是把公式里的平方直接删去。

每个点会出现在从叶到根的 O(log n)份辅助表里，所以空间增加。它用重复保存顺序信息换查询速度。若数据频繁动态更新，维护这些辅助表比静态构建复杂得多，应确认题目是静态还是动态。

14.5 例 4：kd 树，轮流按坐标切开空间

二维 kd 树在根按 x 切，下一层按 y 切，再下一层按 x 切，轮流继续。通常选当前子集的中位数附近分割，使左右点数接近。节点对应一个包围盒，查询矩形 Q 时：若盒与 Q 不交，整棵跳过；若盒完全在 Q 内，整棵输出；部分相交才递归。

例如一组点被分在 x<3 与 x≥3 两盒，查询框 x∈[4,6]时，左盒可完全排除。到右盒若按 y=2 分成上下两片，而查询 y∈[3,5]，下片又可排除。它用空间包围关系剪枝，不是对两个坐标同时做普通二分。

对平衡二维静态 kd 树，轴对齐范围报告经典界为 $O(\sqrt{n}+k)$，空间 O(n)，比二维范围树省空间但查询界不同。最近邻搜索时可用“查询点到包围盒的最短距离”作为下界：若该下界已不小于当前最佳距离，盒内所有点都不能改善答案，便可剪枝。最近邻最坏仍可能访问 Θ(n)点，不能保证每次 O(log n)；高维时剪枝还可能明显变差。

距离计算可用平方距离比较，避免开方；若坐标为大整数，差的平方也要防溢出。坐标相等时的分割与叶容量要有规则，否则全相等数据可能让递归无法缩小。

14.6 选择前先写清问题

数组区间求聚合并更新，用线段树时节点的区间是位置范围。已有区间做穿刺查询，用区间树时节点里保存的是区间对象。平面正交范围查询，范围树或 kd 树利用多维坐标。它们名字里都有“树”和“范围”，并不代表可以互相替代。

输出敏感复杂度里的+k 是实际输出数量，而不是可有可无的附注。如果查询要报告全部十万个点，任何算法都得花时间写出十万条结果；若只问个数，可能通过节点计数直接汇总，输出需求的变化又带来不同优化。

14.7 中心区间树，查询点左移时到底扫描哪张表

区间为[1,4]、[2,8]、[5,6]、[7,9]，选中心 c=5。跨中心的[2,8]与[5,6]留在根，完全左侧[1,4]进左树，完全右侧[7,9]进右树。根里左端升序表为[2,8]、[5,6]，右端降序表也恰好同序。

查 q=3<c，根里所有区间右端至少 5，因此只要左端≤3 就包含 3。扫描左端表，报告[2,8]，遇左端 5>3 便停止；再走左树，报告[1,4]。不访问右树，因为那里所有区间左端都>5>3。查 q=7>c，则根里所有左端≤5，只需右端≥7，报告[2,8]，遇右端 6<7 停止，再走右树报告[7,9]。

每层只走一边，扫描表中除停止项外都产生输出，因而合适平衡构造下总成本是树高加输出数。若只保存一张按左端排序的表，q 在右边时无法遇到第一个不满足便停止，可能需扫描很多不输出项；两种排序正是在支付两侧查询的不同需求。

14.8 线段树的查询为何只有对数数量的摘要块

看某一层，查询区间中间被完全覆盖的节点可以直接取摘要；只有包含左端点与右端点的最多两个边界节点还要继续拆开。下一层从这两个节点产生至多四个孩子，其中中间完整块又可停止，仍只有两端继续。树高 O(log n)，每层只新增常数数量需要处理的节点，所以单个连续区间查询为 O(log n)。

若查询的是任意散乱的一万个位置，就不能继续沿用“每层最多两个边界”的证明；那已是多个区间或点集查询。算法界来自查询形状，而非只要叫 query 就自动对数。

懒标记也只对能从摘要快速更新的操作成立。整段加 d 更新 sum 只需 len；若只存区间最小值，整段平方不能在含正负值时仅凭旧最小值推出新最小值。例如[−5,2]与[−5,4]旧最小值同为−5，平方后的最小分别 4、16。状态缺少信息，就不能强加一个懒函数。

14.9 kd 最近邻，实际比较一次下界

查询点 Q=(4,4)，先找到候选 P=(5,4)，平方距离 1。某子树盒为 x∈[0,2]、y∈[0,6]，Q 到盒的最近横差 2、纵差 0，平方下界 4，大于当前 1，所以整盒可跳过。另一盒 x∈[3,6]、y∈[0,2]，最近纵差 2，下界也 4，可跳过。若盒 x∈[3,6]、y∈[3,6]包含 Q，下界 0，即使盒中心离 Q 较远，也不能剪枝。盒内可能恰好有 Q 本身，距离 0。计算下界时每维取：点在区间内贡献 0，在左边贡献到左边界的距离，在右边贡献到右边界的距离，再平方相加。这是可靠几何下界，盒中心距离不是。

通常先搜索包含查询点或更近的一侧，尽早找到小的当前上界，让另一侧更容易被剪掉。这改变实际访问量，不消除最坏可能访问所有点的情况。最近邻算法的“平均表现不错”不能代替题目需要的最坏延迟保证。

14.10 练习与答案

练习 1。 线段树保存`[1,2,3,4]`，整段加 5 后再把[1,3)加 2，最终总和与各项是多少？

解答：第一次变为[6,7,8,9]，总和 30；第二次变为[6,9,10,9]，总和 34。若根第一次记 lazy5，第二次部分覆盖前必须下推到左右孩子；再在各自相关叶或区间加 2。最后父 sum 由左右重新合并。若不下推就更新旧孩子值，合并后会丢掉第一次的加 5。

练习 2。 maxEnd 增广 BST 查询[8,9]，左子树最大右端点为 7，可以跳过吗？若为 8 呢？

解答：闭区间情形，7<8 可排除；若为 8，可能有区间恰在 8 结束，与查询在 8 相交，不能跳过。若输入改成半开区间，边界比较也要改；区间端点规则会直接进入剪枝条件。

练习 3。kd 树最近邻已经找到距离 5 的点，另一盒到查询点的最近距离为 6。盒里是否还需要逐点查看？下界若为 4 呢？

解答：下界 6 意味着盒内每一点距离至少 6，不可能优于 5，可跳过；下界 4 只说明可能存在更近点，并不保证有，需要继续检查。剪枝使用的是可靠下界，不能拿盒中心距离当下界，因为盒边缘可能更接近查询点。

15 散列表：算出一个位置以后，还要确认是谁

希望用学号查成绩，且不要求按学号排序输出。直接用学号当数组下标可能浪费巨大空间；散列函数把键映射到较小的桶号，例如 h(k)=k mod 7。不同键可能得到同一桶，叫冲突。冲突不是偶然的程序错误，只要键范围比桶数大，就不可避免。

散列表首先通过哈希值缩小候选，最终仍要比较真实键是否相等。两个对象哈希相同，不代表对象相等；若对象相等，则哈希必须相同。键若在入表后改变参与哈希的字段，后续按新值查找可能找不到原位置，因此通常要求键在表内保持相应不可变性。

15.1 例 1：拉链法，每桶放一串候选

桶数 m=7，插入 10、17、24、11。前三者都余 3，进入桶 3；11 余 4，进入桶 4。查 17，先算余 3，再沿桶 3 比较 10、17，找到。查 31 也余 3，要比较 10、17、24 后失败。

装载因子 α=n/m 表示平均每桶有多少元素。在散列足够均匀等假设下，拉链查找期望 O(1+α)。当 α 保持常数，称期望 O(1)；若所有键都挤在同一桶，最坏 O(n)。不能只背“散列表 O(1)”而删掉概率条件与最坏情况。

字符串键的哈希计算还要读取字符，长度 L 的键通常至少 O(L)处理成本；若已缓存哈希值，可复用一部分，但相等性比较也可能读到很多字符。字典操作的常数时间结论常默认键本身可在常数时间哈希与比较。

删除某一键时只删对应节点，不影响同桶其他键。若支持相同键更新值，插入前需要先在桶内确认已有键，不能每次直接追加而产生多个相同逻辑键，除非接口本来就是多重映射。

15.2 例 2：线性探测，空槽为什么有两种

开放定址把所有记录放在数组中。桶 m=7，线性探测依次试 h(k),h(k)+1,...对 m 取模。插 10 放 3；插 17 先看 3 被占，放 4；插 24 再看 3、4，放 5；插 11 的本位 4 已占，再看 5，放 6。

查 24 沿 3、4、5 找到。若删除 17 后把 4 直接设成“从未使用”，查 24 会在 4 提前停止，误判不存在。因此删除槽通常标成墓碑：查询遇墓碑继续，遇从未使用槽才知道这条探测链上不可能有目标。

插入可记住沿途第一个墓碑作为候选空位，但仍需继续查到真正空槽或确认不存在已有同键。否则可能在墓碑处先插一个 17，而后面其实已有 17，造成重复键。若扫描一整圈都无从未使用槽，也要有明确终止条件，避免满表无限探测。

墓碑不算有效记录，却会增加查找路径。重建触发条件往往不仅看有效装载因子，还看有效记录加墓碑占据的比例。桶数远大于 n 也不必然快：若几乎所有槽都被墓碑覆盖，失败查找仍可能走很长。

15.3 线性聚集与其他探测方式

线性探测形成连续占用块，新键撞到块中任一位置都会并入这个块，使块越来越长，这叫主聚集。二次探测用平方偏移，双重散列用第二个函数给步长，试图分散探测位置。

双重散列(h1(k)+i*h2(k)) mod m 要确保步长与 m 互质，才能遍历全部桶。例如 m=8、步长 2 只会访问 0、2、4、6，另一半槽即使全空也到不了；m 为质数且步长介于 1 与 m−1，则互质。二次探测是否能找到空位也依桶数与装载条件，不能把“改变步长”当作完整正确性证明。

在理想均匀探测模型下，失败查找期望探测数可由装载率估计为不超过约 1/(1−α)。α 接近 1 时成本急剧上升。线性探测的具体期望公式与均匀独立探测模型不同，不能把一个模型的公式搬给所有冲突策略。

15.4 例 3：扩容以后，为什么所有键都要重新算位置

旧桶数 7，键 10 在 3、17 在 4、24 在 5。扩成 17 桶以后，h(k)=k mod17，三者本位分别 10、0、7。若只是把旧数组复制到新数组前三格相同位置，查 10 会先去 10 而找不到。

重散列需要新建空表，遍历所有有效记录，按新桶数重新插入，不复制墓碑。若表内 n 项且新表规模 O(n)，重建成本 O(n)期望或与具体散列规则相应的成本。按固定比例扩容并控制装载阈值，可以把这种昂贵重建摊还到之前的多次插入。

它同时包含两层不同分析：冲突处理的成本通常是期望界；扩容总账是摊还界。因此完整说法可能是“期望摊还 O(1)插入”，而不是两种条件被一个 O(1)遮住。

取模前的哈希也会影响分布。若整数键都为某个因子的倍数，桶数选择可能造成大量冲突。字符串多项式哈希按字符顺序累积，例如 `h = h*b + code(c)`，比简单相加能区分换位字符，但仍可能碰撞。密码学抗攻击要求另有安全设计，课程普通哈希不能自动承担密码学用途。

15.5 Trie：键是字符串时，沿字符路径找

词典保存 `to`，`tea`，`ten`，`in`，`inn`。Trie 根按首字符分出 t 与 i；t 后是 o 或 e，e 后再分 a 与 n；i 后 n 对应单词 in，这个节点还能继续 n 成为 inn。每个节点要有“这里是否恰好结束一个单词”的标记，否则无法区分一个词与另一个词的前缀。

查 `te` 会走到一条存在路径，但若结束标记为假，它不是已保存单词；前缀查询则可报告路径存在。查 `ten` 经过 t、e、n 三条边并检查结束，找到。时间与键长 L 有关，而非词典总词数 n 的对数；孩子若用平衡树或散列表存，字符转换成本也要纳入。

字母表很大时，每节点开全套孩子数组很浪费，可用压缩分支、映射或基数树。路径压缩把只有一个孩子的连续路径合成一条带字符串片段的边，节省节点，但查找需逐字符比较该片段。

15.6 例 4：用散列表计数，输出需求会改变下一步

输入 `[4,2,4,7,2,4]`，依次更新计数：4⟶1，2⟶1，4⟶2，7⟶1，2⟶2，4⟶3，最后映射{4:3,2:2,7:1}。若只问 4 出现几次，直接查 4。若要按数值从小到大输出频数，普通散列表不提供这种顺序，还需把键排序，得到 2:2、4:3、7:1。

若键范围只有 0 到 9，数组计数可能更直接；若还要找前缀相同的字符串，Trie 更贴合；若要随时问最小键和前驱，平衡搜索树更方便。字典只是接口，底层结构应按需要选择。

15.7 为什么随机选哈希函数能帮助，而不是把输入称作随机

设从一个函数族中随机选 h，保证任意两个不同键冲突概率不超过 1/m。固定要查的键 x，以及已经存入的 n 个不同键，为每个其他键 y 定义冲突指标。该桶候选数期望至多 1+(n−1)/m=O(1+α)，由线性期望直接得到，不需要假设输入键本身均匀。

这个模型将随机性放在哈希函数选择上。若始终使用固定简单取模，输入恰好全是 m 的倍数，就不能引用上述随机函数族保证。若对手能看见 h 以后专门选键，假设也可能改变；课程一般散列分析与面向恶意输入的哈希设计需区分。这里要学的是概率到底对哪个对象取，而非把“随机”当万能形容词。

15.8 重新散列也要检查删除后的负载

假设表容量 16，最多容纳某阈值比例有效项。大量删除后只有 2 项，却留下 12 个墓碑。只看 α=2/16 似乎很空，但失败查询可能穿过连续墓碑长链。重建到一个新空表，重新插入这 2 项，才清除历史探测痕迹。

如果每次有一个墓碑就立即重建，会让频繁删插序列反复支付全表成本。可在有效加墓碑比例过大、墓碑相对有效项过多时触发，并与容量增长、收缩阈值配合留缓冲。每条策略都需要总账，不是只说“偶尔重建”就得到摊还常数。

15.9 装载因子同样大，冲突形状仍会影响成本

容量 8、4 项，α=1/2。若占位为 0、2、4、6，各新失败探测可能很快遇空；若为 0、1、2、3，哈希到 0 的失败查找会检查 5 格才遇空。α 描述全局密度，不完整描述具体簇形。期望公式依赖散列与探测的随机模型，单个具体表仍可能偏离期望。

拉链法 α 可以大于 1，因为每桶可有多个节点；开放定址有效项不能超过容量，α 接近 1 时探测成本暴涨。把两种结构的相同字母当作相同使用范围，会得出满表仍能常数插入之类错误结论。

15.10 Trie 删除与前缀报告，逐项保留仍有用途的节点

词典含 car、cart、cat。删除 car 只清 car 末节点的结束标记，不能删其节点，因为 cart 仍沿它向下。再删 cart 后，t 节点无孩子且不结束任何词，可删；向上 car 的 r 节点也无孩子且已非词尾，可删；到 a 节点仍有通往 cat 的 t 分支，停止。每层检查是否仍被别的词需要，才不会拆掉共享前缀。

报告前缀 ca 下所有词，先用两步走到 ca，再遍历它的后代并在词尾输出；时间至少包括输出字符总数。若每次拼接复制整个前缀字符串，成本可能超过只按节点数算的界，通常用可回退字符缓冲区记录当前路径。

15.11 迭代顺序也是接口承诺的一部分

普通散列表遍历通常不按键大小，扩容重散列后内部顺序还可能改变。有些语言字典另外承诺插入顺序，那是实现额外维护的关系，不是哈希函数自动产生有序。

若程序依赖按成绩从低到高输出，应显式排序键或选择搜索树，不能拿一次本地桶顺序碰巧递增当保证。相同集合经过不同插入、删除历史，内部布局也可能不同，但存在查询仍应给同样结果。

墓碑保留的是探测历史，不是一个仍然存在的用户键，计数与遍历都应跳过它。

15.12 练习与答案

练习 1。 m=5，线性探测插入 1、6、11，再删 6。查询 11 访问哪些槽？随后插 16 应怎样利用墓碑？

解答：1 在 1，6 在 2，11 在 3，删 6 后 2 为墓碑。查 11 从 1 到 2 再到 3，不能在 2 停。插 16 从 1 开始，记住 2 为首墓碑，仍继续检查 3 以免后面已有 16；到 4 发现从未使用槽，确认不存在后将 16 放回 2。有效记录仍沿同一探测规则可达。

练习 2。 相同对象的哈希能否不同？不同对象的哈希能否相同？

解答：按相等性定义判为相同的键必须产生相同哈希，否则它们会被送到不同桶而无法正确更新和查找。不同键可以同哈希，这正是冲突，需要真实相等比较处理。哈希值只用于定位候选，不能单独证明键相等。

练习 3。 Trie 中已存 `a` 与 `ab`，删除 a 能否直接删掉字符 a 对应的节点？

解答：不能，那会同时断掉 ab 的路径。先清除 a 节点的结束标记；只有节点不再标记单词且没有孩子时，才可向上回收无用节点。一个节点可以同时是某个单词的终点和另一个单词的内部路径。

16 图：把“谁和谁相连”变成可追踪的状态

地图中的地点与道路、课程与先修关系、计算任务与依赖，都是图。顶点代表对象，边代表关系。无向边表示双向连接；有向边 u→v 只承诺从 u 到 v。图 G=(V,E)中，|V|为顶点数，|E|为边数；本章讨论有限图，算法细节会区分是否允许重边与自环。

树可以看作没有环的连通无向图，但一般图允许环、多个父方向与不连通部分。树遍历直接递归孩子不会重复；图遍历必须记录访问状态，否则绕一圈又回到起点，可能无限递归。

16.1 怎样存边，决定遍历要付多少成本

邻接矩阵用 V×V 表格，a[u][v]记录是否有边或边权，判断两点是否相邻 O(1)，空间 $\Theta(V^2)$。枚举 u 所有邻居通常扫一整行 Θ(V)。邻接表为每个顶点保存出边，空间 Θ(V+E)，枚举所有邻居总成本与实际边数有关。

无向边在邻接表中通常保存两份，u 表中有 v，v 表中有 u。于是所有度数和为 2E。有向图中出度总和与入度总和都为 E。复杂度写 O(V+E)时，通常默认邻接表等能按实际边数枚举邻居的表示；同一算法换成矩阵，遍历可能 $\Theta(V^2)$。

16.2 例 1：BFS，队列保证无权最短路

无向图边为 A-B、A-C、B-D、C-D、D-E。源点 A，初始化 dist[A]=0，其他未发现。先将 A 入队，并立即标记已发现。

出队顶点	新发现的邻居	新距离	出队后的队列
A	B,C	B=1,C=1	B,C
B	D	D=2	C,D
C	无，D 已发现	不变	D
D	E	E=3	E
E	无	不变	空

为什么入队时就标记，而非出队时才标记？B 和 C 都邻接 D，若 D 入队后仍保持未发现，C 会再次将 D 入队，造成重复处理。第一次发现时设置 dist 和 parent，记录它由谁到达，既避免重复，也能恢复路径。E 沿 parent 倒走 E-D-B-A，反转得到 A-B-D-E。

队列中顶点距离按非降顺序出现。处理距离 d 的点时，只会新发现距离 d+1 的邻居；若某点存在更短路径，它的前驱距离更小，应已更早出队并发现它，矛盾。因此 BFS 首次发现给无权图的最少边数距离。边权不同就不再成立，例如 A 到 B 直边权 100，经 C 两条权 1 的路更短，BFS 只最小化边数。

邻接表中每点入队出队一次，每边检查常数次，时间 O(V+E)、额外空间 O(V)。若图不连通，从 A 只得到 A 所在连通分量；要遍历全图，外层再对每个未发现顶点启动一次 BFS。

16.3 例 2：DFS，进入与退出时间分别说明什么

深度优先搜索沿一个邻居继续走，走到没有未发现邻居才回退。顶点可以分白、灰、黑：白为未发现，灰为已进入但尚未完成，黑为所有出边已检查完。进入时记发现时间 d[u]，退出时记完成时间 f[u]。

对有向边 A→B、A→C、B→D、D→B、C→D，按字母邻居次序从 A 开始。进入 A，进入 B，进入 D；D 看到灰色 B，发现一条指向当前递归祖先的回边 D→B，说明有环 B→D→B。D 完成，B 完成；回 A 后进入 C，C 看到黑色 D，D 不在当前递归祖先链上，是跨到已经完成部分的边；最后 C、A 完成。

DFS 树中，一个节点子树的进入退出区间嵌套在其祖先区间内。对任意两个节点，区间要么嵌套，要么分离。灰节点恰好是当前尚未退出的递归路径，指向灰节点的有向边因此构成环。

无向图每条树边有反向记录，孩子看到父亲时不能把这条原树边当新环。如果允许平行边，应按边编号跳过“进入时那条边的反向副本”，而不能跳过所有通向父顶点的边，否则会漏掉另一条平行边形成的环或影响桥判定。

16.4 例 3：拓扑排序，顺序必须尊重箭头

课程依赖：A→C，B→C，C→D，B→E。意思是 A 和 B 必须先于 C，C 必须先于 D，B 必须先于 E。拓扑序并不唯一，例如 A B C D E 或 B E A C D 都合法。

Kahn 算法先计算各点入度。初始 A、B 入度 0，入队。出 A，删除 A→C，C 入度从 2 变 1；出 B，C 变 0、E 变 0，入队 C、E；出 C 使 D 变 0；最后得到 A、B、C、E、D。每次挑入度 0 顶点，说明剩余图里没有任何未完成前驱挡住它。

若最后输出少于 V 个点，剩余图每个点入度至少 1。在有限图中不断往前追前驱，必有重复顶点，从而存在环。反过来，有环上的点不可能在环尚未破坏时变成入度 0，所以无法全部输出。时间 O(V+E)。

另一种方法是 DFS 完成时间逆序。对 DAG 任意边 u→v，DFS 若从 u 走到 v，则 v 先完成；若 v 已完成，也仍有 f[v]<f[u]；不可能指向灰祖先，否则有环。因此按完成时间递减排列，每条边都从前指向后。应先检查回边，不能对含环图照样输出然后叫拓扑序。

16.5 强连通：彼此都能走到，与无向连通不同

有向图中，u 可达 v 且 v 可达 u，二者属于同一强连通分量。把每个强连通分量缩成一个点，得到的凝聚图一定是 DAG；否则分量之间形成有向环，沿环便能互达，应合为一个更大的强连通分量。

Kosaraju 算法先在原图做 DFS，记录完成次序；再将所有边反向，按原图完成时间递减顺序启动 DFS，每棵新搜索树给一个强连通分量。为什么要用这个次序？凝聚图中的“靠源头”的分量会在适当完成次序中先被选到；反图里它没有通向尚未处理其他分量的出口，因此一次搜索不会越过分量边界。反向图与次序必须配套，不能只记“两次 DFS”便随意改变方向。

例：A→B、B→A、B→C、C→D、D→C、D→E。强连通分量为{A,B}、{C,D}、{E}，凝聚图是三点链。A 能走到 E，但 E 不能回到 A，所以绝不能把整个弱连通图当作一个强连通分量。

Tarjan 强连通算法也用 lowlink 和栈，但它与下面无向图割点算法的栈条件不同，尤其要判断邻居是否仍在分量栈中。两个算法都出现 low 变量，不代表更新规则可以互换。

16.6 例 4：无向割点，哪一个节点拿掉会断开

无向图由三角形 A-B-C-A，连接边 C-D，以及三角形 D-E-F-D 组成。删 C 后，A、B 与右侧分开；删 D 后，E、F 与左侧分开。因此 C、D 是割点，C-D 是桥。三角形内部任意一条边都不是桥，因为还有另一条绕路。

DFS 进入时记录发现序号 dfn[u]。定义 low[u]为：从 u 子树中的节点出发，沿若干树边向下，再最多用一条指向祖先的非树边，能到达的最小发现序号。它描述这棵子树能不能绕过父连接到更早祖先。

对子树孩子 v，搜索完成后做 `low[u]=min(low[u],low[v])`。遇已发现且不是父边的邻居 w，在无向 DFS 中可用 `dfn[w]` 更新 low[u]。不要用 low[w]替代 dfn[w]乱传，因为定义只允许一条向祖先的非树边，不允许继续借别的子树绕多次。

非根节点 u 若有某个孩子 v 满足 `low[v] >= dfn[u]`，说明 v 子树无法绕到 u 的严格祖先；删 u 会把该子树与上方分开，所以 u 是割点。等号仍是割点条件：子树只能回到 u，u 被删就断。

树边 u-v 若满足 `low[v] > dfn[u]`，则 v 子树连 u 本身也绕不到，删除这条边就断，因此是桥。这里必须严格大于；若 low[v]=dfn[u]，说明另有回到 u 的边，这条树边处在环里，不是桥。

DFS 根没有“上方”部分，要单独判断：根有至少 2 个 DFS 树孩子才是割点。注意是树孩子数，不是邻接顶点数。三角形从 A 出发时，A 可能邻接 B 和 C，但 DFS 经 B 已经发现 C，根只有一个树孩子，不是割点。

16.7 双连通分量，为什么边栈比顶点集合更清楚

点双连通分量常指极大的、没有割点的连通子图。不同分量可以共享割点，却不会共享普通边。按本章“极大无割点连通子图”的约定，孤立顶点也单独构成一个单点分量；边栈从未入边时无法自动输出它，需要在外层对度为 0 顶点另外报告。若课程采用仅报告含边分量的约定，应明确不输出这些单点。DFS 时将经过的边压栈；当某孩子 v 满足 `low[v]>=dfn[u]`，从边栈弹到树边 u-v，弹出的边构成一个分量。

在两个三角形通过 C-D 相连的例子中，分量包括三角形 A B C、桥边 C D 以及三角形 D E F，割点 C、D 可出现在多个分量里。是否将桥单独视为一个二顶点分量，是资料中的定义约定；做题应对照课程，而不能因为“两个顶点不够形成环”就强行删除它。

算法每顶点、每边处理常数次，时间 O(V+E)。实现时递归深度最坏 V，大图可能栈溢出；可以用显式栈保存当前顶点、父边与下一条邻边位置，但必须保留“子调用返回后更新 low”的阶段信息。

16.8 Kosaraju 在一个有方向的例子上完整跑两遍

边 A→B、B→A、B→C、C→D、D→C、D→E，从 A 开始按给出的邻接次序 DFS。进入顺序 A,B,C,D,E；E 先完成，再 D、C、B、A，因此完成逆序为 A,B,C,D,E。反图里有 A↔B、C→B、C↔D、E→D。

第二遍先从 A 走反图，只能到 B，不能从 B 走到 C，因为原 B→C 反成 C→B，得到分量{A,B}。接着 C 未访问，从 C 可到 D 和已处理 B，忽略后者，得到{C,D}。最后 E 得到{E}。如果第二遍仍用原图，A 会一路走到 E，错误合成一团，说明反向边是隔离分量的关键。

对一般凝聚 DAG，可用完成时间证明该隔离次序：若原图有不同分量 X→Y，则两个分量的最大完成时间满足 F(X)>F(Y)。第二遍选尚未处理的最大完成时间分量，在反图里它不可能走到另一个尚未处理分量，否则原图会有该分量指向它并具有更大 F，矛盾。因此一次搜索恰好覆盖一个分量。每遍 O(V+E)，建立反向邻接表也 O(V+E)。

16.9 双连通分量的边栈，真的弹一次

在两个三角形 A-B-C-A 与 D-E-F-D 之间接 C-D。DFS 先压 A-B、B-C、C-A，再沿 C-D 进入右侧，压 D-E、E-F、F-D。右三角形回边让 low[F]、low[E]回到 dfn[D]，当 E 返回 D，low[E]≥dfn[D]成立，从边栈弹 F-D、E-F、D-E，得到右三角形分量。

D 返回 C 时，low[D]>dfn[C]，再弹 C-D，得到单桥边分量。最后左三角形返回根的对应孩子时，弹 C-A、B-C、A-B，得到左分量。每条边只入栈出栈一次，总线性；割点 C 和 D 可由多组弹出的边共同出现。

如果用顶点栈而不区分分量边界，容易把共享割点从前一个分量弹掉，后面缺了它。点双连通与边双连通也不同：去掉桥后的连通块是边双连通的一种分解，不能直接冒充去掉割点条件下的点双连通分量。

16.10 遍历次序不唯一

邻接表中邻居排列不同，DFS 树、BFS 父亲与拓扑序可能不同。检查正确性应验证层数、依赖关系与分量，而非要求每次打印同一串顶点。

16.11 练习与答案

练习 1。 有向边 A→B、A→C、B→D、C→D，给出两个拓扑序。再加 D→A 后，Kahn 算法发生什么？

解答：A B C D 与 A C B D 均可。加 D→A 以后，每个点入度至少 1：A 来自 D，B、C 来自 A，D 来自 B、C，初始入度 0 队列为空，输出 0 点，检测到环。不能因 A 原先是源点便继续把 A 强行放队列。

练习 2。 无向链 A-B-C，根选 B。B 有几个 DFS 树孩子？A、C 的 low 与割点结论是什么？

解答：B 分别搜索 A、C，两者互不连通，各为一个树孩子，共 2 个，所以 B 是割点。A、C 无返回祖先的其他边，low 等于自身 dfn；两条树边都满足孩子 low 严格大于父 dfn，是桥。根判定不能使用非根的“有一个孩子 low≥dfn”条件，否则单个孩子的根也会被误判。

练习 3。 BFS 若在出队时才标记，结果距离一定错误吗？为什么仍不推荐？

解答：某些配套实现仍可最终得到正确距离，但会重复入队，若反复覆盖 parent 或 dist 可能产生错误，复杂度保证也更难保持。标准实现发现时就标记，使每点只入队一次，证明与代码都更直接。要评价一个非标准版本，应读完它处理重复条目的全部规则，而不能只凭一行断言必错或必对。

17 最短路和最小生成树，究竟在最小化什么

从宿舍到图书馆，要让这一次走的路尽量短，是最短路径。给所有宿舍铺网络，要让整个连通网络的总线长尽量短，是最小生成树。前者关心从一个源点到目标的路径长度，后者关心选出的全部边的总权重。它们都可能用贪心与优先级队列，但不能互换答案。

以 A-B 权 2、A-C 权 3、B-C 权 2 为例，从 A 出发的最短路树会保留 A-B、A-C，总权 5；最小生成树选 A-B、B-C，总权 4，却让 A 到 C 的树上路径长 4，大于原图直接走 3。这个三点例子足以区分两种目标。

17.1 松弛：当前找到的路还能不能变短

距离数组 dist[v]记录目前已知从源点 s 到 v 的一条路径长度上界。初始 dist[s]=0，其他为正无穷。若有边 u→v、权 w，且 dist[u]+w<dist[v]，就更新 dist[v]并令 parent[v]=u，这个动作叫松弛。

它不会凭空低估答案，因为每次新距离都来自一条真实可走路径；但尚未发现的更短路可能让当前值过大。各种最短路算法的差别，在于按什么顺序做松弛，以及什么时候可以宣布某个值不会再变。

17.2 例 1：Dijkstra，每次拿出当前最小距离

有向边 A→B 权 4、A→C 权 1、C→B 权 2、B→D 权 1、C→D 权 5。源 A。

本轮确定的点	松弛后的 B	C	D	理由
A	4	1	∞	经 A 两条边
C	3	1	6	经 C 到 B 为 1+2，到 D 为 1+5
B	3	1	4	经 B 到 D 为 3+1
D	3	1	4	无更短更新

最终 A 到 D 为 4，路径 A-C-B-D。每次在未确定顶点中选 dist 最小的 u，将它确定，再松弛出边。

为什么 u 可以确定？假设还有一条更短路。沿那条路从源走，取第一个尚未确定的点 y，其前驱 x 已确定；处理 x 时就已给 y 一个不大于这条路前缀的候选。边权非负，前缀长度不大于整条路，因此 dist[y]会小于当前 dist[u]，与 u 最小矛盾。证明用到了非负边权，负边会破坏这个前缀比较。

用邻接表和支持 decrease-key、每顶点至多一条记录的二叉堆，时间 O((V+E)log(V+1))，空间 O(V+E)。不用堆而每次扫描全部顶点找最小，时间 O(V^2+E)，稠密图中也可能合适。若堆不支持 decrease-key，可以每次更新后再插一条(新距离,v)，出堆时发现距离不是 dist[v]就丢弃旧条目；不能把过期条目再次当作确定事件。这个懒堆可能同时保存 O(E)条新旧记录，一般界可写 O(V+E log(E+2))，堆空间 O(E+1)；对简单图 E≤V^2时可化到常见的 log V 量级，允许任意多平行边时不能无条件删掉 E 的对数。

17.3 例 2：一个负边就能让 Dijkstra 的确定过早

边 A→B 权 2、A→C 权 5、C→B 权−4。Dijkstra 先确定 A，再选距离 2 的 B；但真正最短路 A-C-B 长 1。错误出在“距离 2 的 B 不可能再改善”的判断，而不是松弛公式。

含负边而没有相关负环时，可以用 Bellman–Ford。每轮扫描所有边做松弛，经过 k 轮以后，所有使用至多 k 条边的最短路径都已被考虑，理由是将最后一条边拆出来归纳。若最短路可选为简单路径，最多 V−1 条边，所以 V−1 轮足够。再做一轮若仍能降低某个从源可达的距离，则存在源可达负环。

负环不是对所有目的地都让答案无穷小。只有从源可达该负环，并能从该负环再到达目标，目标的最短路才可通过反复绕环无限下降。图中完全不可达的负环不影响当前源点可达范围的有限答案。

Bellman–Ford 时间 O(VE)，可在整轮无更新时提前结束，但最坏界不变。SPFA 是队列式调度优化，某些输入快，最坏仍可很差，不能把它宣传为普遍线性算法。

17.4 DAG 与全点对：结构能换一种顺序

有向无环图可按拓扑序处理，每到一个点，其所有前驱都已处理，直接松弛出边即可。允许负边，时间 O(V+E)，因为没有环，无需反复回头。若图不是 DAG，拓扑顺序不存在，不能只取某个 DFS 序代替。

Floyd–Warshall 维护 D[i][j]，逐步允许编号更小的顶点作中间点。加入中间点 k 后，最短路要么不用 k，保持 D[i][j]；要么经过 k，候选 D[i][k]+D[k][j]。因此 `D[i][j]=min(D[i][j],D[i][k]+D[k][j])`。k 必须在最外层，确保读到的状态具有正确的“允许中间点集合”含义。时间 $\Theta(V^3)$、空间 $\Theta(V^2)$，可发现 D[i][i]<0 的负环现象。

正无穷加权时要防溢出，不能拿最大整数加正数后绕成负数。若距离使用浮点数，相等与误差规则也应根据题意谨慎处理。

17.5 最小生成树：无向连通图，选 V−1 条边连通全部点

生成树不含环，并覆盖所有顶点。最小生成树 MST 在所有生成树中使边权总和最小。边权可以为负，MST 不受 Dijkstra 的非负要求限制；若图不连通，得到的是最小生成森林，不能承诺一棵覆盖所有点的树。

割把顶点分成两组。设已选边集合 A 能包含在某棵 MST 中，且 A 没有边跨过当前割，称该割尊重 A。取割上最轻边 e，它可以安全加入：先取一棵包含 A 的 MST T，若 T 已有 e，结论直接成立；否则加 e 成环，环上必有另一条跨割边 f。因为割尊重 A，f 不在 A 中，所以删 f 不会破坏已选集合。新树仍包含 A 与 e，且 $w(e)\le w(f)$，成本不增，因此仍是 MST。这个交换论证同时保住“最优”与“之前已选边”，是 Prim 与 Kruskal 的共同基础。

17.6 例 3：Prim 只关心跨出当前树的边

无向图边 A-B2、A-C3、B-C1、B-D4、C-D2。从 A 开始，当前树{A}，候选 A-B2、A-C3，选 A-B2。现在树{A,B}，跨割候选 B-C1、A-C3、B-D4，选 B-C1。树{A,B,C}，到 D 的最轻是 C-D2，完成，总权 5。

Prim 对未进入树的顶点 v 保存 key[v]：它与当前树之间最轻连接边的权重。Dijkstra 的 dist[v]则是从固定源点走到 v 的路径总长。上例 C 加入前 Prim key[C]=1，而从 A 经 B 到 C 的路径长是 3；若把松弛写成 `key[u]+w`，就把 Prim 改成了另一种问题。

使用堆时成本 O(E log V)量级，稠密图用矩阵与线性选点可 $O(V^2)$。初始起点可以任意，多个最优树可能不同，但最小总权一致。

17.7 例 4：Kruskal 与并查集，避免选边绕成环

将同一图的边按权排为 B-C1，A-B2，C-D2，A-C3，B-D4。先选 B-C，分量为{B,C}；选 A-B，分量变{A,B,C}；选 C-D，所有点连成一体，已选 3=V−1 条，完成。

若继续看到 A-C，两端已属同一分量，加上会成环，跳过。并查集用 find 判定分量代表，用 union 合并分量。按秩或大小合并加路径压缩后，m 次操作有近线性的逆 Ackermann 摊还界；不必把这个极慢增长函数当真正常数，常写 $O(m\,\alpha(V))$。

排序通常 O(E log E)，其余并查集成本较小。对于简单无向图 $E\le V^2$，可写 O(E log V)同阶上界；若允许任意多平行边，应保留 E 的表达更稳妥。重边可以被算法自然处理，自环永远不能帮助生成树连通两个分量。

17.8 权重相等与“必须选”的区别

割上唯一最轻边会出现在每一棵 MST 中；若最轻边有并列，只能保证存在某棵 MST 包含其中某条，不能说每条都必选。环上唯一最重边不属于任何 MST；若最重并列，也不能随意断言每条都不出现。

例如三角形三边全为 1，任取两条都是 MST。没有一条边是唯一必须选的，但每个算法仍能稳定得到某棵最优树。证明中的“存在”与“所有”不能互换。

17.9 Bellman−Ford 逐轮表，证明的状态与原地实现稍有不同

用同步数组演示：每轮只从上一轮 dist 读取，写入新数组，状态恰好代表至多 k 条边的最短路。边 S→A 权 4、S→B 权 5、A→B 权−2、B→C 权 3、A→C 权 10。初始 S0，其余无穷；第 1 轮 A4、B5、C∞；第 2 轮 B 降 2、C 降 8；第 3 轮 C 降 5；第 4 轮无变化。最短到 C 为 S-A-B-C，长 5，使用 3 条边。

原地扫描可以在同一轮中连续传播多条边，因此某个顶点可能比同步表更早变好；“k 轮以后至少已涵盖至多 k 边路径”的保证仍成立，但“恰好只含这些路径”就不一定成立。读证明时用同步版本最清楚，实现原地版本时不要拿轮号当精确边数。

负环例子 A→B 权 1、B→C 权−3、C→A 权 1，绕一圈总−1。只要源可达 A，反复绕圈可让 A、B、C 距离持续下降；第 V 轮仍有松弛，反映不存在有限最短值。若要输出受负环影响的所有顶点，可从仍可更新的点再沿有向边传播标记，不能只标记环上一个被更新点。

17.10 Floyd 的一格更新，允许中间点集合必须一致

三点 A,B,C，初始 A→B 为 3，B→C 为 2，A→C 为 9，对角 0，其他无边无穷。允许 A 作中间点暂不改善；允许 B 时，D[A][C]取 min(9,3+2)=5；允许 C 时保持 5。D[A][C]由两段都只允许先前中间点的路径拼成，正好覆盖经过 B 的候选。

为什么 k 放最外层？状态定义是“当前统一允许哪些中间点”。如果 i、j 在外，某些格已经用了编号较大的点，另一些格还没用，同一轮读取的信息不再对应同一个限制集合。个别小图即使错序也算对，不能据此证明该循环顺序正确。

恢复路径可存 next[i][j]，表示从 i 去 j 的第一跳；经 k 改善时赋 next[i][j]=next[i][k]。打印时从 i 反复跳向目标，若无路径要先判断，若存在相关负环也不能当有限最短路无限打印。距离值和路径证据需要同步更新。

17.11 并查集合并后，代表编号不代表最小顶点

集合{A,B}与{C,D}合并，按大小可能把 C 挂在 A 下，也可能相反；find 返回的是内部根，不承诺字典序最小元素。若题目要查询分量最小编号，必须额外在根保存 min 字段并随 union 合并。

路径压缩把访问链上的节点直接连到根，只改变内部表示，不改变集合划分。Kruskal 只关心两端是否同集合，因此可安全使用。若还维护距离、奇偶关系或历史回滚，简单压缩规则可能不再足够，额外信息也要随父关系变化调整，不能直接拿普通并查集复制过去。

17.12 零权边与相等距离，不需要禁止

Dijkstra 允许零权边，证明只要求路径前缀不会比整条路径更长，不要求严格增加。多个点距离相同，可以任意顺序确定；如果还要求某种字典序最小路径，需要额外处理并列，而非假设最小堆自然给出全部路径规则。

无穷距离的点从堆里永不出现，代表源不可达；矩阵扫描版本若选出的最小值仍为无穷，可停止当前源搜索。不能把无穷当普通大数去松弛，否则溢出或哨兵相加可能伪造一条很短的路径。

恢复最短路径时，从目标沿父链接到源，再反转。不可达目标没有这条父链，应报告无路，而非打印一个未初始化的父编号。

同距时不随意覆盖父链。

17.13 练习与答案

练习 1。边 S→A 权 10、S→B 权 1、B→A 权 1。Dijkstra 中 A 可能在堆里出现几次，怎样避免旧条目干扰？

解答：处理 S 时插(10,A)，处理 B 后更新 dist[A]=2，再插(2,A)。先出(2,A)并正确处理；以后出(10,A)时发现 10≠当前 dist[A]，直接跳过。不能每次出堆都把旧值重新写回 dist，否则会把正确距离覆盖成更大值。

练习 2。三点边权 A-B 为−2、B-C 为 1、A-C 为 5，MST 是什么？存在负边是否导致无法求解？

解答：选 A-B 与 B-C，总权−1。生成树只选两条且不成环，负边不会造成反复绕圈的问题。最短路允许重复走路时负环会导致无下界，生成树不允许环，是另一个优化集合。

练习 3。无向图上有一条负权边，若把每条无向边理解成两条有向边用于最短路，会怎样？

解答：沿该边过去再回来，总权为负的两边回路，可重复降低路径长度。因此普通允许重复边的最短路会出现可达负环。若题目要求简单路径、不能重复顶点，则定义不同且问题性质改变，不能继续直接套普通最短路模型。

18 堆：只保证冠军站在最上面

任务队列要不断插入新任务，并取出优先级最高者。每次完整排序做了很多不需要的工作；只用无序数组则每次找最高都要扫描。堆维持一种较弱的顺序：父节点优先级不低于孩子，因此最高项在根，但左右子树之间不必有序。

优先级队列是接口，可以用多种结构实现；二叉堆是常见实现。最大堆取最大，最小堆取最小。本章以最大堆为主，切换到最小堆时所有比较方向都要一起反转。

18.1 完全二叉树的形状，让链接变成下标算式

堆按层序紧凑存在数组里，节点 i 左孩子 2i+1、右孩子 2i+2，父为 $\lfloor (i-1)/2 \rfloor$，根下标 0。最后一层必须从左到右填满，形状由 size 唯一确定，所以无需为每节点另存孩子指针。

数组 `[9,6,8,2,4,7]` 是合法最大堆：9≥6、8；6≥2、4；8≥7。但数组并未有序，8 在 6 后面，7 在 2、4 后面。若要列出所有项从大到小，仍需反复删除最大或另做排序。

18.2 例 1：插入，只有一条向上路径可能出错

在上述堆插 10，先放到末尾下标 6，形状仍完全，但 10 可能大于父 8。与 8 交换，数组变 `[9,6,10,2,4,7,8]`；10 再与父 9 交换，变 `[10,6,9,2,4,7,8]`，到根结束。

为什么不检查其他路径？新项之外所有旧父子关系未变；每次把较小父项向下移到新项原位置，该父项仍不小于原来新项的旧兄弟子树，因为它原先就在这些后代之上。唯一尚可能违反的是新项与新父之间。因此沿祖先不断修复即可。

树高 $\lfloor \log_2 n \rfloor$，最多交换这么多次，最坏 O(log n)。如果新值不大于父，只需一次比较，最好 O(1)。底层数组还可能扩容，插入的完整账可以包含向量的摊还扩容成本。

18.3 例 2：删除根，洞口应往哪边走

从 `[10,6,9,2,4,7,8]` 删除最大 10。将末项 8 移到根并缩小 size，得 `[8,6,9,2,4,7]`。8 小于两个孩子中的最大 9，因此与 9 交换，得 `[9,6,8,2,4,7]`；此时 8 只有孩子 7，已不小于 7，结束。

为何必须与较大的孩子交换？若 8 与 6 换，9 仍在根的另一个孩子位置，根 6<9，堆性质立刻失败。选择较大孩子可确保升上来的新父同时不小于两个孩子，只留下被下移项与它的新孩子之间可能冲突。

可以用“挖洞”减少交换：保存末项 x，根留下洞；不断把较大孩子提上来，直到适合放 x 的位置，再写入 x。比较逻辑相同，减少了反复交换带来的赋值次数。它仍是最坏 O(log n)，修改常数不改变阶。

18.4 例 3：自底向上建堆，为什么不是 n log n

从普通数组 `[3,1,6,5,2,4]` 建最大堆。最后一个非叶节点下标 $\lfloor n/2 \rfloor - 1 = 2$，从它向前逐个下滤。

i=2，值 6 与孩子 4 比较，不动。i=1，值 1 与孩子 5、2 比较，5 升上来，得 `[3,5,6,1,2,4]`。i=0，3 与孩子 5、6 比较，6 升根，3 下到 2，再与孩子 4 比较，4 升上来，最终 `[6,5,4,1,2,3]`。

处理 i 时，它的左右子树都已经是堆，所以一次下滤即可使整个 i 子树成堆。叶子不需要下滤。若把每个节点都按最坏 log n 相乘，会得到 O(n log n)的松上界；实际上大多数节点靠近底层，最多只下移一两层。

高度至少 h 的节点数约不超过 n/2^h，因此总下移工作可按高度上界求和：

$$\sum_{h \ge 1} O\left(\frac{n}{2^h}\right) = O(n).$$

也可将高度恰为 h 的节点数乘 h，得到 n 乘收敛级数 Σ h/2^h。所有元素至少要读一次，故建堆为 Θ(n)。这里的关键是按节点实际可能下移的高度记账，而不是所有节点都支付根的成本。

18.5 堆排序：根与末尾交换后，缩小有效堆

最大堆根是当前最大值，与有效区间末项交换，最大值便到最终位置。有效堆 size 减 1，对新根下滤，再重复。数组后缀逐渐变成升序的最终位置，最后得到整体升序。

对[6,5,4,1,2,3]：6 交换到尾部，前 5 项下滤成[5,3,4,1,2]，后缀[6]；再把 5 移到下标 4，前 4 项下滤；持续至剩 1 项。总共 n−1 次删除根，每次 O(log n)，时间 O(n log n)，加上线性建堆仍同阶。额外空间可 O(1)，但通常不稳定，因为根与末项的远距离交换会改变相等记录的相对顺序。

18.6 第 k 大与前 k 大，不必把全部排好

先约定有限输入有 n 项，且 1≤k≤n；k=0 不表示合法名次，k>n 没有第 k 大，应由接口明确报告不存在。数据流尚未读到 k 项时，也只能报告“尚不足 k 项”。维护一个容量 k 的最小堆，保存当前最大的 k 项。读到新值 x，堆未满就插入；满后若 x≤堆顶，它不可能挤进前 k 大，跳过；若 x 更大，替换堆顶再下滤。扫描 n 项时间可统一写 O(n log(k+1))，空间 O(k)。k≥2 时通常简写 O(n log k)，k=1 时仍须扫描全部 n 项，成本 Θ(n)，不能把 log1=0 代入后误判为零工作。结束时堆顶为第 k 大，其余堆内是前 k 大集合，但未按大小顺序排列。

例如 k=3，输入 5、1、8、3、9。先保留 1、5、8，顶 1；读 3 替换 1，保留 3、5、8；读 9 替换 3，保留 5、8、9，第 3 大为 5。重复值是否分别计入名次，应由题目说明；若要不同值的第 k 大，还需额外去重。

18.7 左式堆：如果主要需求是合并两堆

二叉堆的紧凑数组很适合取极值，但合并两堆通常需要重新组织全部元素。左式堆是一种可合并的堆序二叉树，额外维护空路径长度 npl：空节点记−1，非空节点 1+min(npl(left),npl(right))，并要求左 npl 不小于右 npl。

合并两个最小左式堆，先比较根，较小根保留；递归合并它的右子树与另一堆；回来时若左右 npl 次序反了，交换两孩子，再更新 npl。沿右链递归，右链长度 O(log n)，因为 npl 为 r 的树至少有指数于 r 的节点。这里平衡约束的是右链，不要求整树高度 O(log n)，左侧可很深。

插入等价于与单节点堆合并，删除根等价于合并左右堆。最坏合并 O(log(n+m))量级。要处理父指针、内存所有权和重复优先级，仍需与普通树一样认真检查。

18.8 优先级变化，不是改完数值就结束

若最大堆中某节点优先级增加，应上滤；若减小，应下滤。首先需要找到它的数组位置。只给任务 ID，额外映射“ID→当前堆下标”可 O(1)期望或按映射结构相应成本定位；每次堆交换都要同步更新映射。

也可以用懒删除：每次更新插一条新记录，出堆时检查版本是否过期，过期则跳过。Dijkstra 常采用这种方式。它简化修改接口，却让堆里可能暂存多条旧记录，需要计入空间和总操作数。

18.9 左式堆合并，走完所有返回与交换

有最小左式堆 H_1，根 2、左叶 5、右叶 8；H_2根 3、左叶 6、右叶 9。两个根比较，保留 2，把它右子树 8 与 H_2合并。8 与 3 比，保留 3，再把 3 的右叶 9 与 8 合并；保留 8，让 9 成为它的右孩子，回来时左空、右非空，交换成 8 左 9、右空。

回到 3，左 6 的 npl 为 0，右 8 的 npl 也为 0，不必换，3 的 npl 为 1。回到 2，左 5 的 npl0，小于右 3 的 npl1，交换左右，最终根 2 左 3、右 5，3 左 6 右 8，8 左 9。每处仍父≤子，而且左 npl≥右 npl。总节点六个，一个也没复制或丢失。

npl 为 r 意味着最短到空路径长度为 r+1；左右 npl 至少 r−1，因此至少有 2^(r+1)−1 个节点，推出右链长 O(log n)。左链可以长，不影响合并只沿右链递归的界。若把 npl 误写成整树高度再强制左高右低，会维护另一种更苛刻结构，不能照搬左式堆证明。

18.10 自底向上建堆的下界，不是所有节点都要下移

线性建堆说总工作 Θ(n)，不表示每个节点都移动一次。有些输入本来已成堆，只需比较确认；约一半节点是叶，无需下滤。但算法至少要读取并纳入全部输入，且最后非叶需要进行相应检查，总工作仍为线性量级。

按高度求和的精细版本：高度至少 1 的约 n/2 个节点，各可能付第一步；高度至少 2 的约 n/4 个，再付第二步；高度至少 3 的约 n/8 个，再付第三步。把每个节点的下移路径分成一层一层的账，所有账合为 n/2+n/4+...<n。这避免了“每个节点都是根”那种重复收取 log n 的松估计。

18.11 优先级相等时的服务次序

堆本身通常不稳定，同优先级任务的出队顺序未必与入队一致。若系统要求同优先级先到先服务，可把比较键设计为(优先级,反向到达序号)供最大堆使用，或最小堆比较(负优先级,到达序号)。要小心序号溢出与比较方向，写两三个同优先级任务测试比只看说明可靠。

例如优先级均为 5 的 A 先入、B 后入，最小堆键(-5,0)、(-5,1)，A 先出。后来高优先级 7 的 C 键(-7,2)，C 先于 A、B，这同时满足主优先级与并列到达次序。额外序号改变比较关系，不改变堆的操作复杂度。

18.12 任意位置删除，末项替换以后可能上也可能下

删除堆中下标 i，不必搬移整个后缀。取末项 x 填入 i 并缩小 size；若 x 比新父更大，最大堆应上滤；否则可能比某孩子小，应下滤。不能无条件只做下滤，因为末项来自另一分支，未必小于 i 的父。

例如最大堆根一百，左分支父五十、其孩子十与二十，右分支父九十、其孩子八十与七十。删左下的十，末项七十填进去，七十大于父五十，必须上滤。堆序只约束祖先后代，不能比较不同分支推断替换方向。

18.13 堆数组下标变化，需要同步任何外部定位表

任务编号到下标的映射若只在插入时写一次，后续上滤交换后便过期。更新某任务优先级时，可能改到另一任务。每次交换数组项，也交换它们在映射中的位置记录；删除时清除被删编号，替入末项也更新它的新下标。

调试可对每个数组下标检查 map[id]恰等于当前位置，再反向检查映射指向的记录编号相同。堆性质正确但映射错误，取最大可能一直对，按 ID 修改却错，必须分别验证这两份不变量。

堆空时 getMax 和 delMax 没有有效记录，应该返回失败或抛出错误，不能读数组首格。容量尚在、size 为零也仍是空堆。相反，size 为一时删除根后只需变空，无孩子可下滤；若代码仍拿左右孩子比较，会访问越界。单元素与双元素测试能直接覆盖这些分支，比只用大型随机堆更容易看清结构变化。

若优先级不允许改变，可省去定位映射；接口需求少一项，实现也可以更简单，不能把额外索引开销当成堆必有成本。

18.14 练习与答案

练习 1。 在最大堆`[8,5,7,2,3]`插 6，写出数组变化。

解答：6 追加在下标 5，父为下标 2 的 7。6≤7，无需上滤，得到`[8,5,7,2,3,6]`。不能因为 6 比另一个孩子 5 大就交换，它不是 5 的孩子；堆约束只比较真实父子关系，不要求同一层有序。

练习 2。 最小堆能否 O(1)获得最大值?

解答：一般不能。最大值必在某个叶子，但叶子之间没有大小顺序，最坏需检查线性数量叶子。堆只承诺根极小，不承诺另一端极值。若两端都要高效查询，需要同时维护额外结构或使用双端优先级队列。

练习 3。 维护前 2 大，扫描`[4,9,4,7]`，最后小根堆保存什么?

解答：前两项堆为 4、9；第三项 4 不大于堆顶 4，可跳过，仍有两个最大的记录 4、9；读 7 替换 4，保留 7、9，顶 7 为第 2 大。若要求所有与第 2 大并列的记录，堆容量固定 2 不足以完整输出全部并列者，输出规格要先确认。

19 字符串匹配：已经比对过的字，怎样不再重读

文本 T=ABABABAC，模式 P=ABABAC，想找 P 第一次完整出现的位置。朴素法先把 P 对齐 T 下标 0，前 5 个字符匹配，最后 C 对上 B 失败；然后右移一格，从头再比。失败之前得到的“前面若干字符相同”是一份信息，KMP 会用它跳过不可能的对齐位置。

字符串是字符序列，长度 n、m 分别表示文本与模式的字符数量。本章先把字符看作可在常数时间比较的符号。真实 UTF-8 字节、Unicode 码点与用户看到的字形不是同一个单位；实现查找中文时必须知道接口返回的是字节偏移还是字符下标。

19.1 例 1：朴素匹配，最坏为何会接近 nm

T=AAAAAAAB，P=AAAAB。对齐下标 0，先匹配 4 个 A，再在 B 处失败；对齐 1，仍先匹配 4 个 A 再失败；继续右移。每次失败都重比大量相同 A，若文本长 n、模式长 m，可达到 Θ(nm)量级。

朴素算法并非所有输入都慢。若每次第一字符就失败，总时间近线性；模式很短、数据量小，简单实现也可能足够。KMP 提供的是最坏线性保证，并避免文本指针后退。

空模式的定义要提前决定。常见约定认为空串在下标 0 匹配；查找所有匹配则空串可在每个边界出现。模式长于文本时无非空匹配。这些边界不应等代码崩了以后再补。

19.2 前缀函数：一段字符串自己与自己能重合多少

对模式 P，π[i]表示 P[0:i+1]的最长真前缀，同时也是后缀的长度。“真”指不能取整段自身。例如 ABABA 的前后都可取 ABA，长度 3；整段长度 5 不允许，否则失配时不会缩小。

P=ABABAC，逐项得 π=[0,0,1,2,3,0]。第 0 项只有 A，没有非空真前缀，得 0；AB 没有；ABA 前后都是 A，得 1；ABAB 前后都是 AB，得 2；ABABA 得 3；加 C 后 ABABAC 没有以 C 结尾的非空前缀，得 0。

19.3 例 2：计算 π 时，失败候选也沿着同一条链退

```
def prefix_function(p):
    pi = [0] * len(p)
    for i in range(1, len(p)):
        j = pi[i-1]
        while j > 0 and p[i] != p[j]:
            j = pi[j-1]
        if p[i] == p[j]:
            j += 1
        pi[i] = j
    return pi
```

以 P=ABABAC 的最后 C 为例，上一前缀 ABABA 的最长边界长 3，候选是 ABA，期望再接 P[3]=B，但当前为 C，失败。下一候选长度 π[2]=1，期望 P[1]=B，又失败；再退到 0，比较 P[0]=A 仍失败，最终 π[5]=0。

为什么下一候选必须是 π[j−1]，不能随意 j−1？当前长度 j 的前缀已经等于之前的后缀，若还要缩短并继续保持前后相同，新的候选必定是这段长度 j 前缀自身的边界。π[j−1]给其中最长者，沿链可枚举所有可能长度，既不漏也不重试不可能项。

每次外层 i 只增加，j 在匹配时至多增加 1，在 while 里严格减小。总增加不超过 m，总减少也不超过总增加，所以构造 π 为 O(m)。有嵌套循环仍是线性，总账要看 j 的所有变化，而非每轮拿 m 作上界相乘。

19.4 例 3：KMP 匹配，失配时文本指针原地等一下

```
def find_all(t, p):
    if not p:
        return list(range(len(t) + 1))
    pi = prefix_function(p)
    ans = []
    j = 0
```

```
    for i, ch in enumerate(t):
        while j > 0 and ch != p[j]:
            j = pi[j-1]
        if ch == p[j]:
            j += 1
        if j == len(p):
            ans.append(i - len(p) + 1)
            j = pi[j-1]
    return ans
```

对 T=ABABABAC、P=ABABAC：读前 5 个字符 ABABA 后 j=5，表示模式前 5 项匹配了文本末尾 5 项。读 T[5]=B 时，本来期望 P[5]=C，失配；令 j=π[4]=3。此时 P 前 3 项 ABA 等于已经读过的文本后 3 项，不必重比。当前 B 与 P[3]=B 匹配，j 升 4；接着读 A 升 5；读 C 升 6，完整匹配，起点 7−6+1=2。

不变式是：读过的文本前缀中，当前 j 为某个与其后缀相同的模式前缀长度；失配退到下一边界后仍保持这个关系。未被尝试的中间对齐位置不存在合适的前后重合，故跳过去不会漏答案。整个文本只向前读，j 总上升 O(n)、下降 O(n)，匹配 O(n)，加 π 构造共 O(n+m)。

完整匹配后 j 不能无条件归 0，否则会漏重叠答案。T=AAAA、P=AA，应得起点 0、1、2。每匹配一次退到 π[1]=1，保留末尾 A 作为下次模式开头，于是能继续找到重叠。

19.5 教师图中的 next，与本章 π 不是同一个数组定义

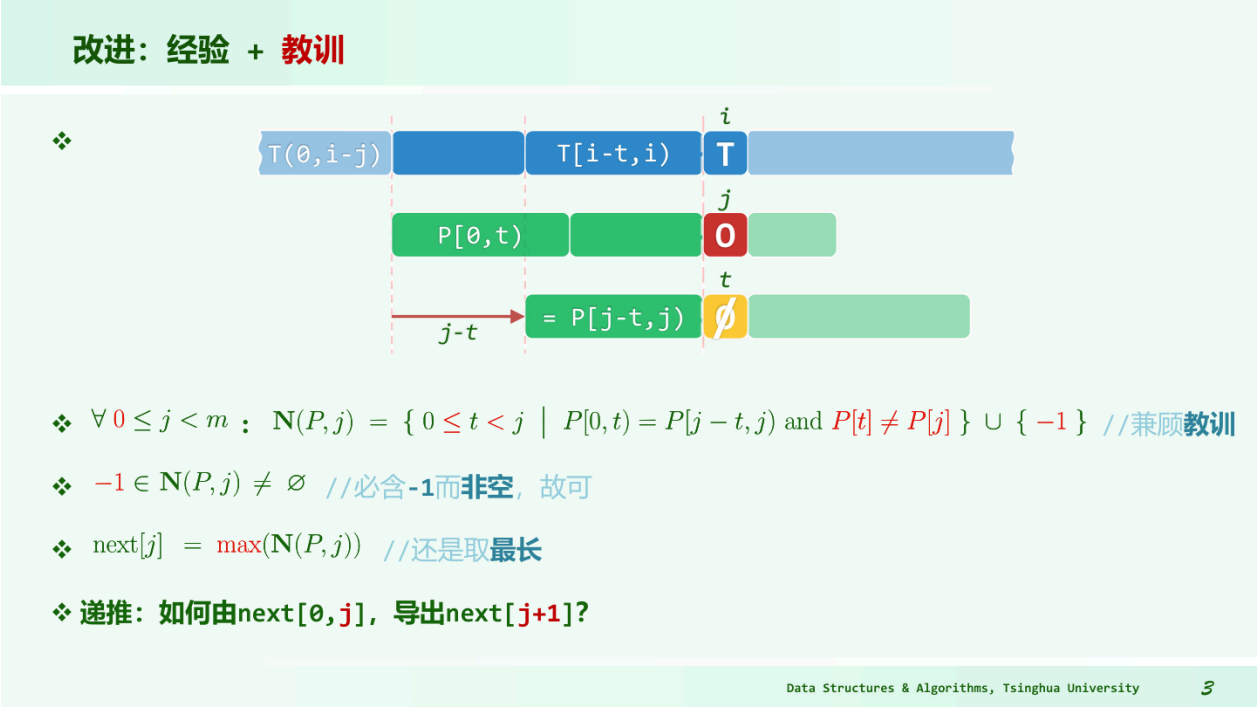


图 9　教师展示的改进失配跳转

图中 i 是文本当前位置，j 是模式中正要比较的位置。上方已知文本片段等于模式前缀；失配以后，模式滑动，把一个前缀与已知匹配后缀重合。图中特别要求候选位置 t 满足 `P[t] != P[j]`，因为若 P[t]与刚失败的 P[j]相同，再拿它比同一个文本字符必然还失败，可以继续跳。

这是改进 next 的定义，常带−1 哨兵；本章 π[i]只是前缀 `P[0:i+1]` 的最长边界长度，允许下一个字符相同。两者数值不能逐项强行相等。使用课程代码时，先写清 next[j]描述“位置”还是“长度”、失配时 i 是否增加、−1 如何处理，再做对照。

19.6 Boyer–Moore 与滚动哈希，换一种利用失败信息的方法

Boyer–Moore 从模式右端往左比。坏字符规则利用文本中导致失败的字符在模式中的最后出现位置来跳；好后缀规则利用已经匹配的模式后缀寻找其他对齐。它常能跳过许多文本位置，但具体最坏保证依版本与附加规则，不能把经验上的快自动叫 O(n)。

滚动哈希把长度 m 窗口编码成数，窗口右移时减去旧字符贡献、乘基数、加新字符，从而 O(1)更新某种固定模数哈希。哈希不相等可确定字符串不同；哈希相等仍可能冲突。若要求确定正确，应在候选处再逐字核对，最坏核对成本可能增大；双哈希降低碰撞概率，也不把概率变成数学上的 0。

例如用十进制直观表示数字串，窗口 123 右移为 234：先减 1×100 得到 23，再乘 10 加 4 得到 234。真实字符哈希常在模数下做同样操作。若模数太小，不同窗口容易碰撞；如果攻击者能选择输入，简单随机假设更需要谨慎。

19.7 多模式匹配：Trie 加失败链接

若同时找 `he,she,his,hers`，每个模式单独跑 KMP 会重复扫描文本。Aho–Corasick 在 Trie 节点上加失败链接：当前前缀失配时，跳到它的最长后缀、且该后缀也是某模式前缀的节点。这个思路与 KMP 的边界链相通，但状态是 Trie 节点，不再只是一个整数前缀长度。

输出还要处理“某个状态对应多个模式后缀”的情况。例如读到 she，同时可能命中 he。实现复杂度依转移表、失败构建与输出方式，若实际命中 k 次，至少要计入 O(k)输出。先掌握单模式 KMP 的状态含义，再学多模式结构，会更容易检查失败链接究竟保存了什么。

19.8 一次坏字符跳转，先把对齐下标画对

模式 P=ABCD，先对齐文本 T=ABXDABCD 的下标 0，从右向左比较。D 对 D 相同，接着模式位置 2 的 C 对文本位置 2 的 X 失败。X 在模式中未出现，最后位置记−1，坏字符规则可让模式起点向右移动 j−last[X]=2−(−1)=3。新起点 3 不是直接就匹配，还要重新比较；随后可继续跳到起点 4 找到 ABCD。

如果失配字符在模式中最后出现在 j 右侧，j−last 可能非正，至少仍需向右移 1，不能让模式倒退。好后缀规则还能利用已匹配的 D，可能给更大的安全位移；最终取安全规则中的较大值，但每个规则都要独立保证不漏匹配。

Boyer–Moore 的表格定义有多种版本，last 通常以字符为下标、返回最后出现位置；KMP 的 π 以模式位置为下标、返回长度。两个数组都叫“跳转信息”，却不是同一种输入输出函数。先写一个表项的中文含义，再抄公式，能避免把字符编码拿去索引长度 m 数组的越界错误。

19.9 哈希窗口也可以有一个很短的碰撞反例

以十进制数对 7 取模作教学哈希，串 12 的值 12 mod7=5，串 19 的值 19 mod7 也=5。它们长度相同、哈希相同，字符却不同。真实滚动哈希基数与模数更大，只是让某些模型下碰撞概率更低，并没有改变“有限值空间可能碰撞”的事实。

查询模式 12 时若直接接受哈希 5，窗口 19 就会成为假阳性。再逐字比较能排除它，但如果恶意构造许多碰撞窗口，核对总成本可能很大。要给严格最坏线性且确定正确的单模式匹配，KMP 提供了不同性质的保证；要用哈希，便明确报告概率假设或验证成本。

19.10 从匹配结果恢复位置，字符单位必须一路相同

假设模式长度 m=3，在读到文本下标 i=7 时 j 达到 3，起点为 i−m+1=5。这里 i 与 m 都按同一种符号单位计数。若 i 按 UTF-8 字节数、m 按 Unicode 字符数，公式相减就无意义。中文“数据结构”有四个常见汉字，但 UTF-8 通常每个占多个字节，工具返回的字节偏移不能直接当第几个汉字。

如果用户看到的一个字形由多个码点组成，按码点切片也可能切开显示单元。课程算法通常抽象为符号序列，不要求先解决所有文本规范化问题；实际应用必须在读入、比较、长度与返回偏移之间统一层次。字形看似相同也可能有不同编码序列，是否先规范化取决于产品语义。

19.11 模式预处理一次，多个文本可以复用

前缀函数只依赖模式，不依赖文本。固定关键词反复搜索多份文件，先构造一次 π，每份文本从 j 为零开始扫描，节省重复预处理。如果把上一份文件的 j 直接带到下一份，会把两个独立文件当成拼接文本，可能报告跨文件边界的假匹配；只有明确处理连续分块的同一文本时才应保留 j。

流式处理同一大文本时，块尾已匹配前缀长度正是下一块需要的状态，无需保存全部旧文本。要报告全局起点，另维护已经读过的总字符数；局部下标必须加块偏移。模式本身及 π 仍需保存，空间随模式长而非文本总长增长。

如果只要第一次匹配，找到即可返回，输出成本常数；若要所有匹配，结果个数可达文本长度量级，例如重复字符，输出列表也占相应空间。说算法额外空间仅为模式长度时，要说明是否把输出空间排除。
文本结束而 j 小于模式长，表示最后只有一个未完成前缀，不应输出匹配。模式比文本长时，即使所有文本字符都与模式开头相同，也不能把这段前缀当完整命中。
找到首次命中后立即返回，适用于只问第一次出现的接口。

19.12 练习与答案

练习 1。 写出 P=AAAA 的 π，并用它找 T=AAAAA 中的全部匹配。

解答：π 为 0、1、2、3。前 4 字符匹配后输出 0，j 退到 3；读第 5 个 A 再升 4，输出 1。答案 0、1。若匹配后归 0，第二个重叠出现会漏掉。重复字符不是特殊补丁，而是边界定义自然覆盖的输入。

练习 2。 P=ABABC，最后字符 C 到来时，π 构造的候选怎样退？

解答：此前 ABAB 的边界长 2，候选下一个字符 P[2]=A，与 C 不同；退 j=π[1]=0，再比 P[0]=A 仍不同，最终 π[4]=0。注意不是从 2 逐个试 1 再 0，因为长度 1 不是 ABAB 的边界，根本不可能接上。

练习 3。 两个窗口哈希相同，可以直接认定字符串相同吗？

解答：一般不能。有限哈希值比可能字符串少，必有不同串碰撞。若题目允许概率算法，要清楚说明随机选择与碰撞风险；若要求严格确定答案，应做足够的核对或使用无碰撞的结构。不能把“本地测了一百万次没撞”替代正确性证明。

20 划分、选择与步长：排序还有哪些做法

归并排序先分成两半，再花时间合并；快速排序先花时间划分，使轴点左侧不大于它、右侧不小于它，再各排各的。两者都用分治，但本层工作发生的位置不同。读快排实现时，最关键的不是记住递归两行，而是弄清划分函数到底返回什么边界。

20.1 例 1：三路划分，重复值不必反复递归

以轴值 v=3 划分`[3,1,4,3,2,3]`，维护四段：`[lo,lt)`小于 3，`[lt,i)`等于 3，`[i,gt)`尚未检查，`[gt,hi)`大于 3。初始 lt=i=lo，gt=hi。

读 a[i]若小于 v，与 a[lt]交换，lt 与 i 都加 1；若等于 v，只让 i 加 1；若大于 v，先 gt 减 1，与 a[gt]交换，i 不动，因为换来的项还没看。

```
初始   [3,1,4,3,2,3] lt0 i0 gt6
等于   [3,1,4,3,2,3] lt0 i1 gt6
小于   [1,3,4,3,2,3] lt1 i2 gt6
大于   [1,3,3,3,2,4] lt1 i2 gt5
等于   [1,3,3,3,2,4] lt1 i3 gt5
等于   [1,3,3,3,2,4] lt1 i4 gt5
小于   [1,2,3,3,3,4] lt2 i5 gt5
```

结束 i=gt，未知段为空，返回两条边界 lt=2、gt=5。只递归`[lo,lt)`与`[gt,hi)`，等于区间无需再排。每轮未知段长度 gt−i 至少减少 1，因此本次划分 $\Theta(n)$。这里返回的是一段等值区间；某些二路划分返回轴点最终秩，另一些返回左右分界，递归区间不能互换。

20.2 快排为什么平均快，又为什么会平方

若每次轴点接近中间，递推约为 $2T(n/2)+\Theta(n)$，得到 $\Theta(n \log n)$。若总选到最小值或最大值，一边为空、一边 n−1，总代价 n+(n−1)+…，为 $\Theta(n^2)$。用第一个元素当轴，对已排输入的某些二路实现就会遇到这种情况。

随机从当前区间选轴，在互异元素、随机选择独立且均匀等假设下，期望比较次数为 $O(n \log n)$。这给每个固定输入上的随机期望，而不是最坏保证。三路划分能避免大量相等值把递归搞坏，但仍不能让所有不平衡轴点的最坏情况自动消失。

实现上可每次递归较小半边，较大半边改成循环，使调用栈最坏 $O(\log n)$，即使总运行时间仍可能平方。这是时间与栈空间分别优化的例子。工业实现还常用内省排序，递归过深就切换堆排，小区间用插入排序。

20.3 例 2：第 k 小，只追答案所在的一边

要找`[7,2,9,4,1,6]`中从 0 起第 3 小，即第 4 个数。选轴 4，划分为小于区`[2,1]`、等值`[4]`、大于区`[7,9,6]`。前面有 2 项，4 本身秩 2；目标秩 3 在右侧，因此只进入右侧，并将局部秩改成 3−3=0，求右侧最小值，答案 6。

Quickselect 每次只递归一边，随机轴的期望时间 $O(n)$，最坏仍 $O(n^2)$。不能把快排的“两个孩子”递推直接照搬给选择，也不能因只递归一边就无条件说线性：若每次只丢掉 1 项，仍是平方和。

确定性线性选择可用“中位数的中位数”：每 5 项一组，求各组中位数，再递归选这些中位数的中位数作轴。忽略常数边界后，至少约 3n/10 项不小于轴，也至少约 3n/10 项不大于轴，最大递归侧不超过约 7n/10。递推`T(n) <= T(n/5)+T(7n/10+O(1))+O(n)`可证明 $O(n)$。它牺牲常数与实现简洁，换来最坏界。

20.4 例 3：希尔排序，先让远处的元素接近位置

希尔排序选一串逐渐减小、最后为 1 的步长 gap。固定 gap 时，把下标相差 gap 的元素分成子序列，各做插入排序。大步长让元素一次跨越较远距离，最后 gap1 时整个数组已较接近有序。

数组`[8,7,6,5,4,3,2,1]`，先 gap4，分别排下标(0,4)、(1,5)、(2,6)、(3,7)，得到`[4,3,2,1,8,7,6,5]`。再 gap2，偶数位置 4,2,8,6 排成 2,4,6,8，奇数位置 3,1,7,5 排成 1,3,5,7，得`[2,1,4,3,6,5,8,7]`。最后 gap1 插入，得到 1 到 8。

gap 有序并不等于整体有序，最后必须包含 1。较小 gap 排序后仍保持此前某些 gap 有序性质，但复杂度强烈依赖步长序列。朴素折半序列的最坏界可达平方级；其他序列有不同理论界。不能笼统给所有希尔排序写一个统一的 n log n。它通常原地但不稳定，因为远距离移动会越过相等记录。

20.5 计数排序：键范围小，就直接数有多少个

若 n 个键都是`[0,U)`整数，可以统计每个键出现次数，再计算前缀位置，将记录稳定放入输出数组。时间 Θ(n+U)、额外空间 O(n+U)，跳过了只做两两比较的模型，因此不受比较排序 Ω(n log n)下界直接约束。

例如记录`(2,A),(1,B),(2,C),(0,D)`，计数为[1,1,2]，前缀累计[1,2,4]。从右往左放：2C 放输出下标 3 并减 2 的累计；0D 放 0；1B 放 1；2A 放 2。得到 0D、1B、2A、2C，两个 2 仍保持 A 在 C 前。若用“每种值打印次数”替代放完整记录，就会丢掉标签等附加数据。

U 若远大于 n，比如 64 位任意整数范围，直接开计数表不可行。可以做基数排序，按若干位一轮稳定排序；每轮时间与基数有关，总成本还取决于位数。固定 32 位与随 n 增长的长整数，是不同的模型。

20.6 例 4：最低位优先基数排序，为什么每一轮要稳定

排序两位十进制数`[21,13,12,22]`。先按个位稳定排序，得到 21、12、22、13；再按十位稳定排序，得到 12、13、21、22。第二轮同十位内部保留上一轮已确定的个位顺序，所以整体正确。

如果第二轮不稳定，十位为 1 的 12、13 可能被翻成 13、12，前一轮信息丢失。对最低位优先法，稳定性就是把新一位加到已有低位次序上的桥梁。最高位优先法则先按高位分桶，再在各桶内递归处理低位，结构不同，不能把两者的过程表混用。

20.7 比较算法时，先确定实际工作

完全排序输出全部有序序列，第 k 小只输出一个秩位置，前 k 大只关心一个集合，找众数关心频率。这些输出要求不同。图方便而先把全部排序，可能多做 n log n 工作；但若后面还要反复区间查询，一次排序也可能值得。

稳定性、额外内存、输入近有序程度、重复值数量、最坏延迟与元素移动成本，都影响选择。复杂度是重要限制，却不是只拿一个大 O 就能替代的全部条件。

20.8 线性选择的好轴，怎样保证至少丢掉常数比例

每 5 项一组，组内排序取第 3 项作中位数。再递归求这些组中位数的中位数 p。至少约一半组的中位数≥p，每个这样的完整组至少有 3 项≥其中位数≥p；另一半对称。因此至少约 3n/10 项不小于 p，至少约 3n/10 项不大于 p，扣除轴所在组和末尾不足 5 项组只损失常数。

递归选择 p 花 T(n/5)，分组和划分花 cn，目标侧至多约 7n/10+常数。要证明总 O(n)，猜 T(n)≤An，代入右边得到约 A(n/5+7n/10)+cn=0.9An+cn。选 A≥10c 并用足够大的基础阈值吸收常数，就能压回 An。决定可行的是两个递归比例之和 0.9 小于 1，不是“选了中位数”这几个字。

如果每 3 项一组，简单比例分析可能得到 T(n/3)+T(2n/3)+cn，比例之和等 1，不能用同一代入证明线性。更细算法可能另有性质，但不能把 5 组法证明中关键常数删掉再声称所有分组大小都一样。

20.9 快排的期望比较数，也能按元素对来数

把互异元素按最终秩标 1 到 n。秩 i<j 的两项会直接比较，当且仅当在秩区间[i,j]中，最先被选作轴的是 i 或 j。若先选中间某项，它会把两项分到不同子问题，此后不再直接比较；若端点先成为轴，则比较一次。均匀随机轴下概率为 2/(j−i+1)。

将所有对的比较指标取期望并求和，得到 `Σ_{i<j} 2/(j−i+1)=O(n log n)`。这个证明不要求不同对是否比较相互独立，线性期望即可。它也说明随机化为何有效：固定输入顺序不影响最终秩区间内哪个轴最先出现的均匀性。

有大量重复时，普通两路比较与这份互异秩证明不直接匹配。三路划分将所有等于轴的项一次归并成不再递归的区间，更适合这种输入。对全相等 n 项，扫描一次后两侧都空，时间线性；若实现仍逐个将等值项留到一侧，就会与这个结论不同。

20.10 希尔排序为什么可能打乱相等项

考虑相等记录 5A 在前、5B 在后，中间穿插其他值。大 gap 子序列里某一 5 可能因与更大值比较向前移动多格，而另一 5 位于不同子序列保持不动，二者可以跨越而从未直接比较。这与稳定插入排序只移动严格较大且相邻逻辑区间的情况不同。

具体用 gap2 再 gap1 处理`[(2,A),3,1,(2,B)]`。gap2 的偶位置 2A、1 排好后，数组变`[1,3,2A,2B]`；奇位置 3、2B 排好后变`[1,2B,2A,3]`。最后 gap1 即使采用稳定插入排序，也只会保持当前等值次序 B 在 A 前，原来 A 在 B 前的顺序已无法恢复。

所以“每一趟子序列插入排序稳定”并不能推出整个希尔排序稳定：不同子序列之间在全局顺序上交错，跨步移动会改变它们的相对位置。需要稳定性时，除非使用带原始序号的完整比较键或专门变体，不应依赖普通希尔排序。

20.11 划分轴值与数组元素分开保存

划分过程中会交换数组，若只保存指向某格的轴引用，而该格被交换覆盖，后续比较可能在用一个变化的轴值。教学代码通常先把 pivot 的值或稳定记录拷贝出来，或者保证轴所在格在最终归位前不被改变；两种设计都可以，但实现要兑现这个不变式。

若元素很大，复制轴记录可能有成本，可保存比较键或使用合适引用策略；若键又随记录修改，则还要确认比较期间不变。泛型算法的元素类型假设，决定某个看似简单赋值是否真能视为常数。

20.12 第 k 小的 k，必须说明是否从零开始

六项数组从零起第 3 小是排序后的第四项，从一起第 3 小是第三项。进入右子问题时，零基秩减去小于区长度与等值区长度；若用一基，判定与减量要同步调整，不能只在入口减一次后又把内部条件改回一基。等值区长度大于一时，目标只要落在这个整段秩范围内，返回轴值即可，无需再挑其中一项递归。若要返回原记录身份，多个相同键之间必须另定并列次序。数值答案唯一与记录答案唯一，也不是一回事。快速排序的空区间与单项区间应立即返回。若递归区间包含轴点本身，轴在边界时可能再次传入原区间，造成无限递归；划分后大小必须严格缩小，这个终止检查比仅看输出是否有序更早发现问题。

20.13 练习与答案

练习 1。 三路划分遇到 a[i]>v，与右端交换后为何不增加 i？

解答：从未知区右端换来的值尚未被分类，可能小于、等于或大于 v。若直接 i++，这项会被错误归入等值区或已处理区。只有看过它并按规则放好，才可缩小未知段左边界。与左端交换时换来的是已知等值项，因此那个分支可同时增加 i。

练习 2。 Quickselect 求第 0 小，恰好每次选最大轴，会怎样？

解答：每次最大轴落在末尾，目标仍在大小 n−1 的左侧，只丢掉 1 项。划分成本 n、n−1、……，总 $\Theta(n^2)$。只递归一边不足以证明线性，需要证明被保留规模在整体上缩得足够快，随机期望或确定性好轴都在解决这件事。

练习 3。 计数排序 n=100、U=10^9，时间 $\Theta(n+U)$是否比比较排序更好？

解答：这里 U 主导，约十亿规模的初始化与扫描很不划算，比较排序的几百次比较量级通常更合适。渐近表达式中两个变量都要代入，不能只看到计数排序“线性”二字便丢掉 U。

21 PA1：从讲解里的几句话，走到能实现的状态

编程作业会同时检验几件事：题目究竟允许什么操作，状态是否足够表达答案，规模是否允许你的复杂度，以及代码边界是否与证明一致。本章对照已取得的 PA1 讲解 PDF 与 PPTX，讨论 Filename、Interview、Gift、Graphics 与二维偏序。示例另设小数据，便于把每步算清；正式输入格式、约束与提交要求仍按课程实际题面。

当前课程文件清单可核验，但独立作业枚举仍出现 `LEARN_HOMEWORK_ENUMERATION_UNVERIFIED`，所以不能把这些主题叫作已确认的全学期全部 OJ 题。这里不给未经验证的“已通过 OJ”承诺。

21.1 Filename：只准插入和删除，不能悄悄用替换

如果把串 A 变成串 B，每次允许插入一个字符或删除一个字符，最小操作数与最长公共子序列 L 有关。保留最长公共子序列，删掉 A 里其余 n−L 项，再插入 B 缺的 m−L 项，成本 n+m−2L。

反方向也成立：任何插删方案最终保留下来的原字符按顺序构成 A、B 的公共子序列，最多 L 项，因此至少删 n−L、插 m−L。两边合起来证明最优值，不只是构造了一个可行方案。

例：A=ABC，B=ADC。保留 AC，删 B、插 D，共 2 步。若正式题面允许成本 1 的替换，B→D 只需 1 步，上式便不再等价。相似题名与“编辑距离”一词不足以确定操作集合。

定义 D[i][j]为 A 前 i 字符变成 B 前 j 字符的最少插删数。基础值 D[i][0]=i、D[0][j]=j。若末字符相同，可从 D[i−1][j−1]直接继承；无论是否相同，都可删除 A 末字符得到 D[i−1][j]+1，或插入 B 末字符得到 D[i][j−1]+1。完整表时间 O(nm)。

21.1.1 例 1：预算 k 小，为什么只看对角线附近

A=ABC、B=ABXC，k=1。沿匹配 A、B 到状态(2,2)，插 X 到(2,3)，再匹配 C 到(3,4)，总成本 1。任何状态(i,j)如果`|i−j|>k`，仅前缀长度差就至少需要这么多次插删，不可能属于总成本≤k 的路径。

因此只计算`|i−j|<=k`的带状区域，带外当作无穷大。每行最多 2k+1 格，时间 O((n+m)(k+1))量级，滚动行空间 O(k+1)。若`|n−m|>k`，可立即判超过预算。

带内不代表成本必≤k，例如 `ABC` 与 `XYZ` 各前缀长度相同，仍需很多插删。带约束只是必要条件，格中仍要计算真实最小值。滚动数组还要防止上一行带的位置变化，让旧缓存被误当当前列值；实现时应明确每个槽对应的 j，以及当前行边界外如何置无穷大。

教师讲解中关于带内目标有一处“大于k”的文字，应按递推意图理解为保留可能达到“不超过k”的状态。回到长度差下界，比凭一句话猜更可靠。

21.2 Interview：环上第 k 个空位，先把“从哪里开始数”固定

恢复座位过程时，常需要找到从某位置之后的第 k 个空座。朴素扫描每格，最坏 O(n)。空位集合若为下标`[0,1,3,4,7]`，从 1 之后数第 3 个空位，依次 3、4、7，答案 7；若从 1 本身起计，答案会变 4。先看正式题面是否把起点算进去，环绕时是否包含当前位置，不能用样例猜定全部语义。

21.2.1 例 2：分块怎样跳，为什么是平方根而非常数

8 格分成每块 2 格：块[0,2)有 2 空，块[2,4)有 1 空，块[4,6)有 1 空，块[6,8)有 1 空。若从 0 起找第 4 个空，先看第一块 2 空，跳过，k 由 4 变 2；第二块 1 空，跳过，k 变 1；第三块扫描 4、5，第一个空在 4，返回 4。

一般块长 B，先跳整块最多 n/B 次，再在一个块内扫描 B 次，单次 O(n/B+B)，取 B 约√n 得 O(√n)。教师材料中笼统引导语若与实际复杂度冲突，应以扫描与跳块次数为准，不能宣布无条件 O(1)。修改一个座位只更新对应块计数，O(1)。

树状数组可把每座空位记 1、占用记 0，支持前缀空位数与第 k 个 1 查询。设起点前的空位数为 r、总空位数为 S，从起点后数 k 个空可转成全局秩`((r+k−1)%S)+1`，但 r 的定义必须与起点是否算入一致。找到位置后将该位从 1 改 0；总空位 S 也随之变化。二进制提升在树状数组上找第 k 个 1 为 O(log n)，不要再外套二分加前缀和变成不必要的 $O(\log^2 n)$，除非实现简洁优先且规模允许。

21.3 Gift：把选择拆成两半，重复和仍代表不同方案

每组从两个数选一个，n 组有 2^n 种选法。直接枚举全部，n=40 时约一万亿，通常太多。中间相遇将组分为两半，各自枚举约 2^(n/2)个半和，再统计左右半和配对。

21.3.1 例 3：两半列表完整配一次

四组为(1,4)、(2,5)、(0,3)、(1,2)，预算 P=7。左半和为[3,6,6,9]，右半和为[1,2,4,5]。对右半 1，左需≤6，有 3 种；对右半 2，左需≤5，有 1 种；对右半 4，左需≤3，有 1 种；对右半 5，左需≤2，0 种，总 5 种。

左半的两个 6 来自不同选法，必须保留两份。如果将半和去重，合法数会少算。负数是否允许不影响“排序后≤阈值计数”的基本方法，但可能影响其他基于总和单调剪枝的策略。

将左半排序，对每个 r 用 upper_bound(P−r)，返回的下标正好是合法左半项数。常见时间 O(n·2^(n/2))、空间 O(2^(n/2))，n 因子可能来自逐项枚举求和或排序对数。可用递推生成半和，每添一组，将已有列表分别加两个选择值；若旧列表有序，两份加常数后的列表各自仍有序，线性归并得到新列表。左右都有序后，双指针可在线性列表长度内统计配对，改善常数与对数项，但总体仍指数于 n/2。

计数最多 2^n，即使每个和只需 32 位，方案数也可能超过 32 位。和的范围要按组数乘最大绝对值估计，不能只按单个输入数的类型决定。

21.4 Graphics：二分前必须证明分界线不会乱序

若许多不交叉的分界线按顺序切分平面带状区域，查询点在哪个区，可比较点在线的哪侧，再二分。二维叉积 cross(u,v)=u_x*v_y−u_y*v_x，在固定坐标方向中正负表示转向。对有向线 AB 与点 Q，计算 cross(B−A,Q−A)，就能判断 Q 在有向线哪侧或线上。

21.4.1 例 4：先在一条竖线旁算出符号

A=(2,0)、B=(2,5)，Q=(4,3)。B−A=(0,5)，Q−A=(2,3)，叉积 0×3−5×2=−10。点在线的右侧。若 Q=(1,3)，叉积为 5，在线左侧。若 Q=(2,3)，为 0，点在线上，归属需正式题面规定。

多条分界线能够二分，要求对固定 Q，其侧别结果沿排序是单调的，例如先右侧后左侧。若线段交叉，分界顺序在不同高度可能交换，单一排序就未必保证谓词单调。证明要利用题面中“端点顺序一致”“不相交”等结构，不能只因最后使用二分就默认成立。

整数坐标可用精确叉积，避免浮点误差，但两个乘积可能溢出。坐标绝对值≤M 时，差向量分量可达 2M，乘积可达 4M²，差值可达 8M²量级；应据上界选宽类型。即使最终结果小，中间乘积也可能很大。

21.5 二维偏序：字典序排好，不代表全部可以串成链

三元组(a,b,c)，希望选择 a、b 都不减的一条链，使权重 c 之和最大。按(a,b)排序，只处理好了 a 的先后；例如(1,3,4)、(2,2,5)在字典序里先后明确，却因 b 下降不能接在同一不减链中。

21.5.1 例 5：先写平方 DP，再找到可维护的摘要

点为(1,3,4)、(2,2,5)、(2,4,6)。按字典序不变。f_i 为以第 i 项结束的最大链权，允许不接前项时为 c_i+max(0, 前面所有 b_j<=b_i 的 f_j)。

第一项 f=4；第二项 b=2，前面 b3 太大，不可接，f=5；第三项 b4 可接前两者，取较大 5，f=6+5=11。朴素对每项扫前面所有项，时间 O(n²)。

转移只需要“b 不超过某阈值的最大 f”。将所有 b 排序去重，压成 1 到 K 的秩，树状数组维护前缀最大值。处理当前项时先查询≤当前 b 的最佳值，再更新该 b 位置。每项 O(log n)，总 O(n log n)。与前缀和树状数组不同，这里聚合是 max，不存在通过两个前缀相减求任意区间最大值的通用办法。

若要求 a 严格增加，同一 a 的点必须先全部查询，再统一更新，避免同组互接；若 b 严格增加，查询要用秩−1 而非当前秩。等坐标是否可串、是否可重复用同一项、是否允许空链，都进入转移边界。

若等(a,b)允许任意顺序串接且仅求权和，可以合并同坐标正权；但要求至少取一项时，不能把所有负权直接删掉并输出 0。若任务是固定输入顺序子序列，而非任意重排成偏序链，又不能先按坐标排序改变顺序。类似词语下可能藏着不同问题。

PPT 的分治方案需要先解左半，再把左对右的贡献算好，最后解右半。若先把右半全部算完，再传入更好的左侧值，右侧内部后继可能来不及重新传播。这是带依赖 DP 的执行次序，不是普通归并排序的任意调度。

21.6 开始提交之前，把小例子变成检查器

Filename 可用完整 DP 核对带状 DP；Gift 可用小 n 枚举 2^n 个选择核对中间相遇；二维偏序可用 $O(n^2)$DP 核对树状数组；Graphics 用手算叉积核对左右与线上边界；Interview 可用小环直接走座位核对秩转换。小规模慢程序是独立的答案来源，能找到错误实现，不意味着正式程序已经被 OJ 验收。

练习 1。 Filename 的 A=AB、B=BA，插删距离是多少？如果替换成本 1 呢？

解答：LCS 长度 1，所以插删距离 2。替换两个位置也是 2，此例恰巧一样，不能据此推断两种模型总等价。A=A、B=B 时插删 2 而替换 1，才是区分操作集合的反例。

练习 2。 Gift 左半和[1,1,4]、右半和[2,3]、预算 4，有多少配对？

解答：右 2 要求左≤2，两份 1 都可，2 种；右 3 要求左≤1，仍 2 种，总 4。去重为[1,4]会误算 2 种。重复数值与重复方案是两件事。

练习 3。 二维链要求 a 严格增加、b 不减，同 a 的两个点(2,1,5)、(2,2,7)是否可以互相更新？

解答：不可以。两项 a 相等违反严格增加。应将同 a 组查询都基于前面更小 a 的结构，得到各自 f 后再统一写回。即使先按 b 排序后局部看起来不降，a 这一条件仍未满足。

21.7 偏序 DP 的分治顺序，用三个点检验

三点按 a 排序为(1,1,5)、(2,2,7)、(3,3,11)，全部可串，正确 f 依次 5、12、23。若分成左半第一点与右半后两点，先单独解右半，会得到 7、18；之后只把左点贡献给右边，会将第二点改 12、第三点最多直接改 16 或保留 18，漏掉第二点被提升后再贡献第三点的 23。

正确次序是先解左半，传播到右半初值，再解右半内部。或者采用按 a 排序的在线树状数组 DP，处理第二点时已经读到第一点的 5，处理第三点时已经读到第二点的 12。执行顺序本身在保证依赖已经准备好，不是一个任意可换的工程细节。

坐标压缩只保存顺序：b 值−3、4、100 映成 1、2、3，保持小于与不大于关系，却不保持距离。查询 b≤4 可转为秩≤2；若题目条件是距离不超过 5，就还需要在原值上算阈值后再二分映射，不能直接把压缩秩差 ≤5 当原坐标距离。

21.8 优化之前保存一份可解释的慢实现

带状 DP、树状数组与中间相遇都会压缩状态或改变处理顺序。先保留完整 DP、平方偏序转移与全部选择枚举，在小输入上逐状态比较，可以定位第一个分歧，而不只是最后输出不同。

正式提交只携带题目需要的代码，个人测试数据留在本地；教材中的独立示例应能够自己生成可核对的小数据，不依赖任何私人作业文件。

22 PS01：把“会背复杂度”变成“自己能数出来”

教师原始习题单：第 01 次习题共 32 题。原题中的公式、分段条件已经对照 PDF 页面核对。下面保留原题号，重新解释题目要你做什么，再把能落在纸上的计算、证明或实验办法写出来。编译、测时间和个人经历类问题没有一个替所有人编好的“标准经历”；这些题给出可照着做的操作和检查结果的办法。

读这一章时，准备一张纸。遇到程序，先写变量下一步变成多少；遇到证明，先试三个小输入；遇到复杂度，先决定到底数什么。答案里的上界、下界、平均值和实测时间是四种东西，不能相互顶替。

22.1 1 · 计算工具：算法不一定写在电脑里

算盘、尺规和纸笔都能执行计算。区别在于，它们允许的基本动作不同。算盘可以拨珠，尺规可以画直线、画圆、取交点，纸笔可以逐位写数。一个过程要说清楚，首先要说明输入如何放进去。例如用算盘算 $27+35$，先用十位和个位的珠位表示 27，再加 3 个十和 5 个一；个位从 7 加 5 得到 12，要留下 2 并向十位进 1，最后读出 62。只有“把数加起来”这句话还不是给执行者的完整步骤。

可以把十进制竖式加法写成程序：从最低位起，把两位数字和进位相加；结果对 10 取余作为本位，除以 10 取整作为新进位；移动到高一位，直到两串数字和进位都用完。每次处理一位，长度为 d 的输入需要 $O(d)$ 个这样的逐位动作。

C++把动作写成有精确语法的语句，变量、条件和循环都有规定的含义；自然语言描述更容易夹带“适当处理”之类的空白。反过来，尺规的“取两个圆的交点”是它的原语，在普通电脑里却要另外表示坐标、处理精度、求方程。比较两种算法快慢以前，需要把基本动作放在同一个计算模型里。

可以自己试一个检查：让完全没参与设计的人照步骤算 $99+1$。若他不清楚最后的进位要不要额外写一位，说明描述漏了边界，而不是他“应该懂”。

22.2 2 · 曹冲称象与刻舟求剑：保存的究竟是什么

曹冲称象需要的量是“大象使船增加的排水重量”。把大象放上船，在船舷记录水线；移走大象，再逐块放石头，直到同一条水线重新与水面重合。在船本身、船中其他载荷、水的密度等条件保持一致时，相同吃水意味着相同排水体积，因此石头总重量等于大象重量。最后称石头就把一个难称的大物体转成了若干容易称的小物体。

这里的记号标在船上有用，是因为水线要表示船相对水面的吃水状态。刻舟求剑要找的却是剑相对河床的位置；船移动后，刻线跟着船走，剑没有跟着走。记号保存了“剑落水时相对船的位置”，而人想恢复“现在相对河床的位置”，中间缺了船的位移。若剑在落水后还受水流影响，甚至连固定的河床位置也要另作假设。

用算法语言说，输入信息必须足以确定输出。设落剑时船在地面坐标 x_0，记号相对船的位置为 r，则剑的初始地面坐标是 x_0+r。船后来到 x_1，拿同一记号推断的位置是 x_1+r，误差正是 x_1-x_0。把坐标写出来，就能指出错误发生在哪一步。

这个题也提醒我们：程序可以按预定步骤顺利结束，却算错问题。有穷性保证会结束，正确性还需要输入、模型和推理都对。

22.3 3 · 死循环：局部能查，万能检查器做不出来

先看最常见的一类错误：

```
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] < target) lo = mid; // 长度为1时不再缩短
    else hi = mid;
}
```

若 `lo=4, hi=5` 且 `a[4]<target`，就有 `mid=4`，更新后还是 `[4,5)`，此后完全重复。修正通常是 `lo=mid+1`，但仍要检查这是否符合想要的返回语义。调试时把 `lo,hi,mid` 打印出来，比一直盯着循环体更容易看见状态没变。

证明一个循环终止，可以找“非负整数、每轮严格减少”的量。这里取 $hi - lo$。它不能降到负数，每次又至少减 1，因此轮数有限。只写“区间变小”不够，要检查每一条分支，特别是长度 0 和 1。机器整数溢出还可能让本应下降的量突然变负或绕回去，实际程序必须另查类型。

为什么不能写一个检查任意程序的万能算法？假设存在 `halts(P,x)`，能判断程序 P 在输入 x 上是否结束。构造程序 D：输入程序 P，若 `halts(P,P)` 说会结束，D 就永远循环；若说不会结束，D 立刻结束。再问 D 在输入自己时会怎样。判断说会结束，D 偏不结束；判断说不结束，D 偏结束，产生矛盾。因此这个假设不成立。

这不妨碍我们检查具体程序、有限状态程序，或要求循环带下降量的受限语言。正确的实践是组合使用不变量、边界测试、超时和审查；超时发现“这次没在限定时间结束”，本身不能证明永远不结束。

22.4 4 · Failstone：别因为长得像冰雹序列，就停止证明

原题递归在 p=q 时返回 1；否则变换参数并把返回值加 1。下面明确讨论正整数 $p, q \geq 1$，并假定算术精确、没有溢出。若允许 p=0,q>0，奇数 q 时会原样重复 `(0,q)`，结论就变了。“自然数”是否包含 0 必须先约定。

可以不用深递归，而是直接数经过多少个状态：

```
def failstone(p, q):
    if p <= 0 or q <= 0:
        raise ValueError("本实现只接收正整数")
    states = 1
    while p != q:
        if p > q:
            p, q = p - q, p
        elif q % 2:
            p = 2 * p
        else:
            q //= 2
        states += 1
    return states
```

例如从 `(1,3)` 开始，依次得到 `(2,3)`、`(4,3)`、`(1,4)`、`(1,2)`、`(1,1)`。共有 6 个状态，所以返回 6。要把末尾的基础情形也数一次；只数变换次数会少 1。

困难在于，单步最大值不总下降：`(2,3)` 变成 `(4,3)` 时还变大了。我们把有限的几步捆成一组。若 p<q 且 q 为偶数，减半 q 后，新的最大值小于原 q。若 p<q 且 q 为奇数，就不断将 p 加倍。若中途相等立即结束；否则一定能到某个 $p^* < q < 2p^*$，因为正整数反复加倍会超过固定 q，而且奇数 q 不可能等于偶数 $2p^*$。接下来三步是

$$(p^*, q) \to (2p^*, q) \to (2p^* - q, 2p^*) \to (2p^* - q, p^*).$$

最后一步合法，因为第二项 $2p^*$ 是偶数，且 $0 < 2p^* - q < p^*$。这组变换后两项都小于原来的 q。若起初 p>q，先走一步得到 `(p-q,p)`，再进入刚才的小项在前的情形，后续一组同样让最大值降到原 p 以下。

因此，每组都在有限步内结束，并让正整数最大值严格下降；这样的组不可能无限多。这就证明了原过程对正整数一定结束。它与尚未普遍解决的另一道递推问题相似，并不能把那个问题的未知状态搬过来。

22.5 5 · 几何分布：要数到第一次成功为止

每轮独立，成功概率固定为 p，直到第一次成功的轮数记为 K，且成功那轮也计数。那么

$$P(K = k) = (1 - p)^{k-1} p, \quad k = 1, 2, \ldots$$

前 $k - 1$ 轮失败，第 k 轮成功，两部分概率相乘。求期望有个很省事的办法：先做一轮，用掉 1；若失败，以后仍然面对同样的问题。因此 $E[K] = 1 + (1 - p)E[K]$，移项得 $E[K] = 1/p$。

例如独立地掷公平硬币直到第一次正面，平均需要 2 次；成功率为 0.2 的独立尝试平均需要 5 次。平均 5 次不表示第五次一定成功，前 20 次都失败的概率仍是 $0.8^{20} > 0$。

符合模型的计算例子是：从固定集合均匀独立抽样，直到抽到某个目标；或者每轮独立选择随机候选，命中合法集合的比例恒为 p。网络重试常有服务器状态变化、限流、相关故障，不能只因为“也在重试”就套几何分布。若允许最多重试 m 次，随机变量还变成了截断版本，期望也要重新算。

22.6 6 · 分解大整数：先忠于题目的蛮力算法

原流程依次试素数 2、3、5、7.....；找到因子就除掉，直到剩余数为 1。这里没有写“试到平方根便停止”，所以不能在分析中悄悄加上这项优化。

最坏例子可以取接近 10^{100} 的素数 N。每个小于 N 的候选素数都试不出因子，直到 N 本身才整除成功，测试量约为素数计数函数 $\pi(N)$。用素数分布的渐近估计 $\pi(N) \sim N/\ln N$，在这个量级约为 $10^{100}/(100\ln 10) \approx 4.34 \times 10^{97}$ 次。这个数只是数量级估计，不能当成对某一个具体 N 的精确素数数量；光生成所有候选素数还另有成本。

最好让待分解数是很小素数的幂。取 $N = 2^k$，每次都能除以 2，约经过 $k = \log_2 N$ 次成功除法。10^{100} 附近的 2 的幂需要约 332 或 333 次，具体取决于选的是哪一个幂。“同样约 100 位十进制数字”，计算量能相差几十个数量级。

若采用平方根停止并在最后输出剩余因子的改进版，素数输入只需试到 $\sqrt{N}$。那是另一个更好的算法，回答时应先分析原流程，再单独说明改进。大整数除法本身也不必是单位成本；若按位模型收费，还要乘上相应位长的运算成本。

22.7 7 · 图灵机递减：借位时到底改了哪几位

设纸带上是最高位在左的二进制正整数，读头先移到最低位。做减 1 时：读到 0 就把它改成 1 并向左，继续借位；读到第一个 1 就把它改成 0，借位结束。之后根据题目约定删去多余前导 0、复位读头、进入停机状态。

用 `1000` 减 1 逐步看：最低位 0 改 1 得到 `1001`，再向左把 0 改 1 得到 `1011`，再改一位成 `1111`，最后把最左的 1 改 0，得到 `0111`，去掉前导 0 即 `111`。中间的 `1111` 不是结果，因为“仍在借位”的状态还没有结束。

`1011` 减 1 只改最低位 1 为 0，得到 `1010`；`1` 减 1 得到 0。输入 0 若属于允许集合，不能继续向左无限找 1，应该单独输出边界错误或按明确规定返回 0。减法所需改写次数等于尾部连续 0 的数量再加 1，最坏与数字位数成正比。

图灵机的状态名不是装饰。至少要区分“去最低位”“借位”“返回起始处”“结束”，否则同一个符号 1 在不同阶段应执行什么动作就说不清。复位读头产生的移动也属于时间成本。

22.8 8 · 允许读头原地停留，会不会多出一种计算能力

只允许 L 和 R 时，每走一步位置的奇偶性都会改变。若读头最后回到起点，向左和向右的步数相等，移动总次数是两者之和，因此为偶数。这是对位移计数的结论，不是说程序中所有操作都天然成双。

加入 N 表示头不移动。设某条可达转移在读到某符号、处于某状态时，既不改符号也不改状态，且动作为 N，那么执行后整个配置与执行前完全相同。确定性的机器下一步还会选同一转移，于是永久重复。要让一条有效计算路径最终结束，这类转移必须改变符号或状态，或者本身就是停机规则。

时间应该按转移次数数，而不是只数头移动了多远。连续 100 次改状态但不移动仍做了 100 步，不能算零时间。N 也能由原模型模拟：先向一侧移动，进入一个记住待返回状态的临时状态，再向另一侧移回；邻格读到什么就原样写回。这样只增加常数倍转移，没有改变哪些问题可计算。

这项模拟说明模型的可计算能力相同，但不保证逐步时间完全相同。比较复杂度时要保留“至多常数倍模拟开销”这一层说明。

22.9 9 · RAM：用加减和跳转实现乘除

RAM 模型通常用一组可寻址寄存器保存数，用指令修改寄存器，用条件跳转控制执行位置。图灵机一次只读写头下的格子；RAM 可以按地址访问寄存器，所以它更接近我们写数组程序时的抽象。两者的时间度量还取决于是否限制字长，不能只凭外形决定。

非负整数乘法可以用“较小问题变一半”的办法。计算 13×11：11 的二进制是 1011，于是 $13\times11=13\times(8+2+1)=104+26+13=143$。程序每轮看乘数最低位，若为 1 就累加当前被乘数；然后被乘数翻倍、乘数整除 2。

```
def multiply(a, b):
    ans = 0
    while b:
        if b % 2:
            ans += a
        a *= 2
        b //= 2
    return ans
```

若这些单字加法、移位和测试都算常数，轮数是 $O(\log(b+1))$。若数能任意长，加法和移位也要按实际位数收费，不能照搬单位字模型的结论。

向下取整除法可以先把除数 b 倍增成 $b,2b,4b,\dots$，直到下一倍超过 a，再从大到小决定是否减去这一倍。例如 $37/5$：可用 20、10、5，依次减去 20 余 17，减 10 余 7，减 5 余 2，商为 4+2+1=7，余数 2。最后核对 $37=5\times7+2$，且 $0\le2<5$。

向上取整的商在余数不为 0 时比向下取整大 1，因此 $\lceil a/b\rceil=\lfloor a/b\rfloor+[a \bmod b\neq0]$。b 必须为正。`(a+b-1)/b` 虽常见，但定长整数里 `a+b-1` 可能先溢出。

C++还有类型、对象生命周期、异常、模板和库接口等语言规则。RAM 的一个抽象寄存器不能直接等同为任意一个 C++对象；分析代码时需要说明哪些语言操作被映射成什么成本。

22.10 10 · 小 o 与小 ω：比“同阶”更强的判断

$f(n)=o(g(n))$ 表示 $f/g\to0$，即 f 相对 g 越来越可忽略；$f(n)=\omega(g(n))$ 表示 $f/g\to\infty$。这里先假定 g 最终为正。大 O 只要求比值最终有一个固定上界，小 o 则要求不管选多小的正上界，最终都能低于它。

例如 $n/(n\log n)=1/\log n\to0$，所以 $n=o(n\log n)$，也就是 $n\log n=\omega(n)$。但 $3n$ 与 n 的比值恒为 3，所以 $3n=\Theta(n)$，却不是 $o(n)$。

再看一个容易误判的例子：$n/1000$ 在很多输入上比 n 小很多，但二者仍同阶，因为“1000”是固定常数。反过来 $n/\log n$ 与 n 的相对差别会随 n 增长，前者才是 $o(n)$。这些符号描述极限趋势，不表示某个小规模输入上哪一个实际更快。

22.11 11 · 乘法、除法能不能当$O(1)$

若课程明定“机器字内的整数乘除”属于 RAM 原语，分析时就可以算一次基本操作。但这个约定含有一个前提：操作数能放进规定字长，而且不把任意大整数悄悄塞进一个寄存器。

用只支持加减、移位的指令实现乘法，第 9 题展示的算法要按乘数位数做多轮；用小学竖式乘两个 L 位二进制数，最朴素需要 $O(L^2)$ 个逐位动作。这和“CPU 有一条乘法指令”的单位成本说法并不冲突，它们在不同模型里计费。

因此写复杂度报告时可以明确说：“以下 n 个元素均为固定宽度整数，比较和算术按单位成本计。”若题目让你处理有百万位的数，就必须把位长作为规模的一部分。用一句“乘法都是常数”会抹掉题目真正的难处。

22.12 12 · 取对数：正向可保留上界，反向只得到多项式关系

原题问的是：由 $T(n)=O(f(n))$，是否有 $\log T(n)=O(\log f(n))$，反向又怎样。先设 $T(n),f(n)\ge1$，且 f 最终至少为某个大于 1 的常数；更常见的充分条件是 $f(n)\to\infty$。这样对数非负，分母不会一直为 0。

若 $T\le Cf$，两边取对数得到 $\log T\le\log C+\log f$。在 $\log f$ 最终有正下界时，常数 $\log C$ 能被另一个常数倍的 $\log f$ 吸收，所以正向成立。若 f 恒为 1、T 恒为 2，虽然 $T=O(f)$，但 $\log f=0$、$\log T>0$，就不能写成后一关系。这说明前提不是可省略的装饰。

反向通常不成立。取 $T(n)=n^2,f(n)=n$：有 $\log T=2\log n=O(\log f)$，但 $T/f=n\to\infty$，所以 $T\neq O(f)$。对数把幂次从指数位置拉成了乘法常数，而大 O 恰好会忽略这个常数。

从 $\log T \le C \log f$ 真正能推出的是 $T \le f^C$，即 T 受 f 的某个固定次幂约束；不能擅自把这个幂 C 改回 1。核对方法很简单：每次把两边指数化，看看常数究竟跑到哪里去了。

22.13 13 · 指数枚举：输出数量本身可能就很大

n 个元素的每一个子集都可用长度 n 的 0/1 串编码：第 i 位为 1 表示取第 i 项。每位有两个选择，候选共有 2^n 个。枚举 n 个布尔变量的所有真值赋值、逐一检查所有子集的重量，也都有这层枚举成本。

n 从 20 增加到 40 时，候选量从约一百万增加到约一万亿；输入只翻倍，候选却平方。这正是先估算复杂度再写 PA 的理由。若每个候选还要重新扫描 n 项，时间是 $O(n2^n)$；若用增量维护，单个候选成本可能下降，但候选数本身没有消失。

要区分两个结论："这个算法枚举了 2^n 个候选"与"这个问题的所有算法都必须指数时间"。前者直接数流程就能证明，后者是问题下界，通常难得多。对于特定子集问题，折半枚举还能把主要枚举规模降到 $2^{n/2}$，这已经足以推翻"只能按原算法那么慢"的断言。

22.14 14 · 32 位、64 位无符号整数有几位十进制

正整数 x 的十进制位数是 $\lfloor \log_{10} x \rfloor + 1$。因为 d 位数恰好落在 $10^{d-1} \le x < 10^d$ 之间，两边取对数即可得到这个式子。

w 位无符号整数最大值为 $2^w - 1$。32 位最大值是 4294967295，共 10 位；64 位最大值是 18446744073709551615，共 20 位。估算时用 $\log_{10} 2 \approx 0.30103$，32 乘它约 9.633，64 乘它约 19.266，分别向下取整再加 1。

这里问的是"十进制字符要写几位"，不是"数有多大就需要多少字节"。64 位数在机器的定长二进制表示里仍占 8 字节，而把它打印成十进制文本最多要 20 个数字字符，若用 C 字符串还要给末尾零字符留位置。

22.15 15 · 时限预算：给自己的程序做一个能被反驳的估计

先从题面记下 n 的上限、时限、内存限额、编译器和允许的库，再写出候选算法的操作量。例如 n 为 20 万时，二重循环约 4×10^{10} 对，排序约 $n \log_2 n \approx 3.5 \times 10^6$ 次比较；即使常数有差异，这个数量级对比也足够先淘汰朴素二次方案。

不能把"每秒一亿次"当硬件定律。一次比较可能是两个整数，也可能比较长字符串；访问连续数组和随机追指针的缓存代价也不同。正确做法是在目标环境用与题目相似的数据做小型基准，确认随 n 增长的趋势，并保留一定余量。

内存也要算。例如两个长度 20 万的 64 位数组约占 $2 \times 200000 \times 8 = 3.2 \times 10^6$ 字节；若递归每层复制数组，或每个元素另建一个带指针的节点，实际占用会增加。记录的是你实际运行的配置和结果，不把本书的示例数字当作 OJ 实测记录。

22.16 16 · 典型循环：先给每个位置写"收费项目"

官方《习题解析》1-32 中的 16 段程序已经在第 32 章逐段展开。这道题练的是数法，建议先遮住答案，把每段程序分成"外层取哪些值""每个外层值的内层做多少次""条件判断本身贵不贵"三列。

例如 `for(i=1;i<=n;++i) for(j=1;j<=i;++j)` 的收费是 $1 + 2 + \cdots + n = n(n+1)/2$。把内层改成 `j*=2`，第 i 轮就只有 $lfloor \log_2 i \rfloor + 1$ 次，累计才是 $\Theta(n \log n)$。外层同样写着 n 轮，并不能决定整体是几次方。

特别留意循环条件中的函数：`while (i < n/G(i))` 每检查一次条件都可能重新计算 G。若 G 本身递归，不能因为它没写在花括号里面就免费。第 32 章的开方递归例正是把条件成本漏掉后容易答错的题。

22.17 17 · 分数级数怎样估整数次数

运行次数当然是整数。例如内循环按步长 i 扫到 n，真实次数是 $\lfloor n/i \rfloor$。分析时把它夹住：

$$\frac{n}{i} - 1 < \left\lfloor \frac{n}{i} \right\rfloor \le \frac{n}{i}.$$

对 i 从 1 加到 n，得到 $nH_n - n < \sum_i \lfloor n/i \rfloor \le nH_n$。上下界只差至多 n，而 $nH_n = \Theta(n \log n)$，因此真实整数和也为 $\Theta(n \log n)$。

拿 n=5 手算，次数为 5、2、1、1、1，总计 10；分数和为 $5(1+1/2+1/3+1/4+1/5)\approx 11.42$。我们没有声称程序做了 11.42 次操作，只是用一个更容易计算的数给真实次数定界。

22.18 18 · 调和级数：把一块块矩形放在曲线上

记 $H_n=1+1/2+\cdots+1/n$。函数 $1/x$ 单调下降，所以在区间 $[k,k+1]$ 上，函数面积不超过高度为 $1/k$ 的矩形；把这些区间拼起来，得到

$$\int_1^{n+1}\frac{dx}{x}\le H_n.$$

另一方面，从第 2 项起，每一项 $1/k$ 不超过区间 $[k-1,k]$ 下的面积，所以 $H_n\le 1+\int_1^n dx/x$。积分就是自然对数，于是

$$\ln(n+1)\le H_n\le 1+\ln n.$$

上下都是对数增长，故 $H_n=\Theta(\log n)$。换对数底只乘一个固定常数。若只记住“调和级数发散”还不够：发散表示最终能超过任意常数，这道题还要知道它慢到什么程度。

22.19 19 · 为什么 $\log(n!)$ 是 $n\log n$ 量级

把乘积取对数：$\log(n!)=\sum_{i=1}^{n}\log i$。上界很直接，每项至多 $\log n$，所以和至多 $n\log n$。下界只拿后半段，至少有 $\lfloor n/2\rfloor$ 项，每项至少 $\log(n/2)$，因此和至少约 $(n/2)\log(n/2)$。这已经证明了 $\Theta(n\log n)$。

Stirling 公式给出更细的近似：

$$n!\sim\sqrt{2\pi n}(n/e)^n,\qquad \ln(n!)=n\ln n-n+\frac{1}{2}\ln(2\pi n)+o(1).$$

例如 n=10 时，$\log_2(10!)\approx 21.79$。这表示区分 10 个互异元素的所有排列，二元比较树至少需要足够深，不能用十次左右的比较普遍完成排序。精细近似能估常数；只求渐近阶时，前面的上下夹逼更短，也更不容易记错。

22.20 20 · 递归空间：数同时没回家的调用，不数一共来过几次

线性递归 `sum(a,n)=sum(a,n-1)+a[n-1]` 在最深处会同时保留 n 层左右的调用帧。每层若只存固定几个下标、返回地址和局部量，辅助空间是 $\Theta(n)$。把输入数组本身排除以后，这些栈帧仍然是额外空间。

二分求和先算左半，再算右半。递归树总节点仍有 $\Theta(n)$ 个，因此总时间为 $\Theta(n)$；但任一时刻只在一条根到叶的路径上，深度为 $\Theta(\log n)$，辅助空间只有 $\Theta(\log n)$。顺序执行的左右子树不会同时保留各自所有内部调用。

迭代求和只用一个累计值和一个循环下标，空间为 $O(1)$。若每次递归都先复制子数组，再调用函数，复制出的数组可能还没释放，空间就不能只按“每层常数”算了。若并行启动左右递归，也要重新检查同时存活的任务和缓冲。

时间和空间有联系，但不必同阶。二分求和就是时间 n、空间$\log n$；缓存所有子问题可能多占空间却大幅省时间。判断时画“某个时刻哪些对象仍活着”，比把递归树所有节点直接相加更可靠。

22.21 21 · reverse：交换放在递归前后，为什么结果一样

采用闭区间 `[lo,hi]`。当 `lo>=hi` 时不做事；否则交换两端，再反转内部 `[lo+1,hi-1]`。交换的两个位置和内部递归处理的位置不重叠，因此可以调换这两步。

例如 `[1,2,3,4]`：先交换外端得到 `[4,2,3,1]`，再交换内部得到 `[4,3,2,1]`。先做内部则先得到 `[1,3,2,4]`，再换外端，同样得到 `[4,3,2,1]`。这不是“所有两行代码都能交换”，而是这里两步对互不重叠的数据起作用。

每层做一次常数时间交换，规模减 2，层数约 n/2，所以两种顺序时间都为 $\Theta(n)$。普通递归实现保留 $\Theta(n)$ 层帧。递归放最后时有尾调用形式，某些编译器可能优化；C++不保证一定消去栈帧，因此若必须保证常数辅助空间，明确写双指针循环更稳妥。

22.22 22 · Master 定理：从每层的账单证明，而不是背三行结论

先处理 $T(n)=aT(n/b)+g(n)$，其中 $a\ge 1,b>1$ 为常数，基础规模的成本为正常数。为看清结构，先取 $n=b^L$，省掉取整；常用取整版本还需保证递归规模确实下降。记 $d=\log_b a$。

第 i 层有 a^i 个节点，每个规模 n/b^i，本层非递归成本为 $a^i g(n/b^i)$；第 L 层叶数是 $a^L = n^d$。所以

$$T(n) = \Theta(n^d) + \sum_{i=0}^{L-1} a^i g(n/b^i).$$

若 $g(n) = O(n^{d-\varepsilon})$，代入第 i 层，有 $a^i g(n/b^i) = O(n^{d-\varepsilon} b^{i\varepsilon})$。这是向叶层增长的几何级数，和为 $O(n^d)$，叶层本身又提供 $\Omega(n^d)$，故 $T = \Theta(n^d)$。例如 $4T(n/2) + n$ 得 $\Theta(n^2)$。

若 $g(n) = \Theta(n^d)$，每层都为 $\Theta(n^d)$，共 L 层，故 $T = \Theta(n^d \log n)$。例如 $2T(n/2) + n$ 得 $\Theta(n \log n)$。

若 g 比 n^d 多一个固定正幂，并且存在 c<1 使 $ag(n/b) \le cg(n)$，第 i 层成本至多 $c^i g(n)$，整棵树的非递归成本至多 $g(n)/(1-c)$。根层又必须支付$g(n)$，叶成本可被 g 吸收，因此 $T = \Theta(g(n))$。例如 $2T(n/2) + n^2$ 得 $\Theta(n^2)$。

注意课程把根主导情况放在“第一种”，有些书把它编号为第三种。应按条件认定情形，不能只记编号。这个证明也没有覆盖 $n/\log n$ 一类恰在边界附近的所有函数，遇到它们要回到层成本和。

22.23 23 · 正则条件已经包含增长下界吗

假设 $ag(n/b) \le cg(n)$，其中 0<c<1，且基础区间中的 g 有正下界。把式子改写为 $g(n) \ge (a/c)g(n/b)$。重复 k 次，有

$$g(n) \ge (a/c)^k g(n/b^k).$$

选 k 使 n/b^k 落到固定基础区间，k 约为 $\log_b n$。因此 $g(n) = \Omega(n^{\log_b(a/c)})$。由于

$$\log_b(a/c) = \log_b a + \log_b(1/c),$$

后面多出的 $\varepsilon = \log_b(1/c) > 0$，正是所需的多项式间隔。若只讨论 $n = b^k$，证明直接沿这条序列成立；要推广到所有 n，需要基础区间统一下界等常规条件。

反过来，光有增长下界不能约束相邻尺度的起伏。取 a=b=2，在 $n = 2^k$ 上规定 g：k 为偶数时 $g(n) = n^2$，k 为奇数时 $g(n) = n^3$。无论哪种都有 $g(n) \ge n^2$，已比 $n^{\log_2 2} = n$ 多一幂。

但当 k 为偶数时，$2g(n/2)/g(n) = 2(n/2)^3/n^2 = n/4$，随 n 变大，连小于 1 都做不到，更别说统一小于某个固定 c。这个明确的反例说明，“整体长得够快”与“向下一层时有规律地衰减”是两个不同要求。

22.24 24 · 带对数幂的递推：别漏掉 p=−1

设 $g(n) = \Theta(n^k (\log n)^p)$，在足够大 n 上取对数，避免基础规模出现 log1=0。仍写 $n = b^L$。第 i 层的主要大小为

$$n^k (a/b^k)^i (L-i)^p,$$

其中换底常数已经吸收到 Θ 里。令 j=L-i，就能看见几何权重和幂次怎样叠加。

若 $a < b^k$，从根向下的几何因子衰减，任何固定对数幂都无法推翻固定多项式间隔，所以 $T = \Theta(n^k (\log n)^p)$。若 $a > b^k$，叶层主导，$T = \Theta(n^{\log_b a})$。

若 $a = b^k$，几何部分恰好抵消，剩下 $n^k \sum_{j=1}^{L} j^p$。当 p>−1，和为 $\Theta(L^{p+1})$；当 p=-1，和是调和级数，给 $\Theta(\log L)$；当 p<−1，和有常数上界，叶成本也同阶。对应地：

$$T(n) = \begin{cases} \Theta(n^k (\log n)^{p+1}), & p > -1, \\ \Theta(n^k \log\log n), & p = -1, \\ \Theta(n^k), & p < -1. \end{cases}$$

例如 $2T(n/2) + n/\log n$ 是 $\Theta(n \log\log n)$，不能机械套成 $\Theta(n)$。对于恰为平滑函数 $n^k (\log n)^p$，根主导时的比值最终小于 1；若只知上下常数包住它，原函数可能起伏，正则条件未必逐点成立，但可用平滑上下包络和递推比较得到同一渐近界。不要把“结论可由夹逼得到”写成“所有起伏函数都自动满足逐点正则条件”。

22.25 25 · Akra–Bazzi：先看它解决了哪种不等分

当递归是 $T(n) = T(n/3) + T(2n/3) + n$，子问题不再是统一的 n/b，前一题的同宽递归树公式不能原样照搬。Akra–Bazzi 处理的核心形式是

$$T(x)=\sum_{i=1}^{m} a_i T(b_i x)+g(x),\quad a_i>0, 0<b_i<1.$$

先求唯一实数 p 满足 $\sum_i a_i b_i^p=1$，再在 g 非负、增长满足相应正则条件、基础区间成本有界且为正等假设下，得到

$$T(x)=\Theta\left(x^p\left[1+\int_1^x \frac{g(u)}{u^{p+1}}\,du\right]\right).$$

p 是树的规模权重。一个规模 x 的节点分成孩子后，孩子的 x^p 权重总和是 $x^p\sum a_i b_i^p=x^p$，所以在同一尺度截面上这个量守恒。于是外部工作$g(u)$按尺度累计时，需要用 u^p 归一化；尺度的相对宽度是 du/u，两者相乘就产生积分中的 $g(u)/u^{p+1}$。这是公式来由的解释，还不等于一般定理的完整证明。

对本题常用的不等分线性合并，我们可以直接给完整的渐近证明。令固定 $\alpha\in(0,1)$，递归分成 $\alpha n,(1-\alpha)n$，节点成本为其规模。任何一层已展开节点的规模总和不超过 n，所以每层最多$O(n)$；最长路径每次至多乘 $\max(\alpha,1-\alpha)<1$，深度$O(\log n)$，得到上界$O(n\log n)$。前 $c\log n$ 层若取足够小正常数 c，最小孩子路径仍未触到基础规模，这些层都完整，规模总和恰好 n，于是下界 $\Omega(n\log n)$。上下合起来即 $\Theta(n\log n)$。

一般定理的严格证明还需要处理不齐的叶层、取整扰动和 g 的正则性，不应把上面一个特例冒充所有函数的证明。使用带 $h_i(x)$ 扰动的版本时，必须另外核对扰动界；若一个子问题实际变成 n-1，它不再是固定比例收缩。后面的具体应用都会先写条件，再代入积分，而不是拿定理名代替解释。

22.26 26 · 最大和区间：把“哪个区间”也放进状态

先采用非空区间，使用半开写法 `[l,r)`，并约定和相同时取起点更小者，再取终点更小者。答案要保存三元组 `(sum,l,r)`，所有更新都使用同一个比较规则。

线性算法中，记 `ending` 为“必须在当前下标 i 结束的最佳非空区间和”，记 `start` 为它的起点。读到 `a[i]` 时，只能二选一：独立从 i 开始，或接到前一个结束于 i-1 的区间后面。若旧 `ending<0`，延长只会更差，就令 `ending=a[i], start=i`；若旧和为 0，延长与重开同和，按“更小起点”应保留旧起点。

例如 `[-2,3,-1,3,-5]`：i=0 时当前和-2；i=1 重开得到 3、起点 1；i=2 延长得到 2；i=3 延长得到 5，最佳区间变为 `[1,4)`；i=4 降到 0，但全局最佳仍为 5。最后输出的不只是 5，还有元素 `[3,-1,3]`。

如果允许空区间，则全负数组的最优和为 0，可以约定输出 `[0,0)`。这需要改变初始化和并列规则；不能初始化为 0，却又在题面声称必须选至少一个元素。只多保存常数个下标，四种原有算法的时间阶不变，线性版额外空间仍为$O(1)$。

22.27 27 · 最大区间按 25% 与 75% 切，为什么没有变成二次

每个子问题在线性时间内求跨越分界的最佳区间，再加上左右递归。固定按 1/4 和 3/4 切，递推是 $T(n)=T(n/4)+T(3n/4)+\Theta(n)$。一层各节点代表互不重叠的输入区间，总长度约 n，所以合并工作每层线性。最长路径沿 3/4 那边下降，约经过 $\log_{4/3} n$ 层；这虽然比 $\log_2 n$ 深，但仍是常数倍的$\log n$。因此总时间为 $\Theta(n\log n)$，下界可按第 25 题的完整浅层论证得到。

对任意固定 $\alpha\in(0,1)$ 都一样。但“固定”不能删掉：若每次切成 1 与 n-1，最长路径有 n 层，层成本依次 n、n-1……，合计 $\Theta(n^2)$。看见两个递归分支，不能自动认定是 nlog n。

22.28 28 · 记忆化：缓存的键必须能决定整个子问题

以 Fibonacci 为例，直接递归 `F(n)=F(n-1)+F(n-2)` 会反复计算同一个 `F(3)`、`F(4)`。把已求得的 Fk存起来，再次遇到同一个 k 就直接读，只有 n 个不同状态，每个状态合并两项，因此单位字模型下时间$O(n)$、缓存$O(n)$。

可以手画 `F(5)`：左分支 F4里会计算 F3，右分支又要 F3。不缓存时重新展开，缓存后第二次只取已有结果。缓存节约的是重复子问题，不会神奇地消除互不相同的所有分支。

一个常见错误是求“从位置 i 出发、不许用已经选过的元素”的答案，却只用 i 作为键。两个调用虽然位置相同，已选集合不同，剩余合法选择也不同，不能共享答案。正确键可能是 `(i,used_mask)`，状态数也随之大增。先问“相同键是否保证以后面对完全相同的问题”，再写缓存。

22.29 29 · LCS 恢复：表里存的是长度，路径带出字符位置

设 $D[i][j]$ 表示 A 前 i 个字符与 B 前 j 个字符的 LCS 长度。若末字符相同，取 $D[i-1][j-1]+1$；否则取上格和左格的较大值。第一行、第一列为 0，因为有一串为空。

以 A=ABC、B=AC 为例，完整表是：

A 前缀 / B 前缀	空	A	AC
空	0	0	0
A	0	1	1
AB	0	1	1
ABC	0	1	2

从右下角 (3,2) 回溯：C 相同，记录 A 中下标 2、B 中下标 1，走左上到 (2,1)；B 与 A 不同，上格大于左格，向上到 (1,1)；A 相同，记录 (0,0)，再走左上。得到的是逆序的 C、A，把记录反转就是 A、C 及两串中的位置。

相等的候选格可能产生多条最优路径。规定“并列优先上格”便能得到一个确定答案，但它不自动保证字典序最小的 LCS；那是更强要求。保存完整长度表，计算时间$O(mn)$，回溯$O(m+n)$，额外输出长度不超过$\min(m,n)$。若原先只保留滚动两行求长度，直接回溯所需的信息已经丢掉；不能声称不增加信息就自然恢复路径。更省空间的恢复算法需要另讲分治技巧。

22.30 30 · 自顶向下和自底向上：同一个依赖图的两种走法

自顶向下从 D[m][n] 开始，需要哪个依赖就递归求哪个，算过的格子缓存。自底向上按行或列填写，保证填某格时左、上、左上已存在。两者使用同一递推关系，差别是访问顺序和实现开销。

LCS 的密集最坏情形中，两个方法都可能碰到 $\Theta(mn)$ 个状态。自底向上用连续数组，容易滚动行，且没有递归栈；自顶向下贴近定义，并可能避开没用到的状态。例如两串每个末字符都相等时，沿对角线就能一路下降，不必访问整个矩形。

但若依赖图是稀疏、不规则的搜索空间，先把所有可能状态铺成表可能很浪费；反过来，状态几乎全用到而且规则，逐格填表通常更便于控制常数。先比较访问了多少个不同状态，再比较每个状态的递归、查表与内存成本，不用“递归差、循环好”概括所有情况。

22.31 31 · 读源码：把接口的约定也读进来

打开课程代码包后，不必第一遍就追进每个模板细节。先选一个最小调用，例如对 [2,5,7] 求和，找到入口、参数、递归基和返回值。把调用链写成 sum(3)→sum(2)→sum(1)→sum(0)，再沿返回方向累计。

对每个接口写四件事：合法输入是什么、输出保证什么、哪些对象被修改、谁负责释放资源。例如 [lo,hi) 中 hi 不属于区间；若错按闭区间传参，复杂度公式完全正确也会越界。再用空数组、单元素、全相等、最大合法下标和非法边界分开测试。

如果本机平台无法直接打开 Visual Studio 工程，可以读取源文件并用合适编译器编译选定项目，但需要记录这个差别，不能虚构已运行指定 IDE。测试通过意味着这些输入符合预期；正确性证明还要覆盖其他合法输入。

22.32 32 · 看演示：先预测下一步，再按播放

演示最有价值的用法是让自己暴露误解。以反转或二分查找为例，先写当前数组、lo、hi，预测下一次交换或轴点；只播放一步，再比较。若预测错了，要指出自己把哪个区间端点或更新顺序理解错。

可以做一次具体记录：数组 [1,4,6,9]，目标 5，用返回最后一个不大于目标的位置的 C 版二分。第一次看 6，应把 hi 改到 2；再看 4，应把 lo 改到 2；此时 lo=hi=2，返回 1，对应值 4。返回值是前驱位置，不是“没找到，所以算法失败”。

动画只展示所选输入的一条执行轨迹。要把观察变成普遍结论，还要提炼不变量和终止量。纸笔预测、单步核对、写出理由，才是把演示变成可复用理解的过程。

22.33 本章练习：离开答案再做一次

1. $T(n)=n^3, f(n)=n^2$，判断 $T=O(f)$ 与 $\log T=O(\log f)$。说明两个判断为什么能不同。
2. 用 Failstone 规则追踪 `(3,5)`，数清返回值，并指出哪几步组成最大值下降的一组。
3. 非空最大和区间在 `[0,0]` 上，采用本章并列规则应输出哪个区间？若误把“旧和为 0”也当作必须重开，会违背哪一条规则？

第一题：T/f=n 无界，故第一个不成立；对数比值为 3/2，第二个成立。第二题依次为 `(3,5),(6,5),(1,6),(1,3),(2,3),(4,3),(1,4),(1,2),(1,1)`，返回 9；前三次变换后最大值从 5 降到 3，是一组。第三题应为 `[0,1)`，和 0、起点 0，并列时终点 1 比 2 小。若在第二项重开会选起点 1，损害先比起点的规则；若总覆盖为延长结果又会选 `[0,2)`，损害最后比终点的规则，所以全局比较也要一致。

23 PS02：向量里的每一次搬动，都要知道搬了谁

教师原始习题单：第 02 次习题共 43 题。统一把有效区间写为 `[0,n)`，含 0、不含 n；容量是能容纳多少项，规模是已经存在多少项。元素复制、比较暂按单位成本计；大对象的构造、移动和析构需要另计。

23.1 1 · 面向对象：从一个能自己维护状态的小类开始

对象是一次具体存在，例如某份成绩记录；类说明这些对象有哪些数据、能做哪些操作。属性保存状态，方法读写状态。构造函数建立合法初态，析构函数在对象生命结束时释放它独占的资源。把数据放进类，却允许外部随手把规模改成比容量大，仍没有保护合法状态。

```
template<class T>
struct Pair {
    T first, second;
    Pair(T a, T b) : first(a), second(b) {}
    T sum() const { return first + second; }
};
struct AddTo {
    int delta;
    void operator()(int& x) const { x += delta; }
};
```

`Pair<int> p(2,5)` 把模板参数 T 具体化成 int，`p.sum()` 得 7。`AddTo plus3{3}; plus3(x);` 看起来像函数调用，其实调用对象的 `operator()`，delta 则保存在对象里。

重载是在同一名称下提供不同参数签名；重写是派生类替换基类的虚方法实现。通过基类指针调用非虚方法，并不会自动按对象实际类型改选。阅读 Vector 模板时，先判断当前是哪个对象、哪个类型、哪份资源，再研究语法，别让一串名词把状态藏起来。

23.2 2 · std::vector：reserve 以后，为什么仍不能写 v[0]

空向量的 size 为 0。`reserve(10)` 只保证容量至少 10，不创建 10 个元素，所以 `v[0]=7` 仍越界。`push_back(7)` 才创建第一个有效元素；`resize(4)` 则把有效规模改成 4，新建整数按该接口的初始化规则得到相应初值。

照着做：空表→reserve(10)→push_back(7)→push_back(8)→erase(begin())。规模依次 0、0、1、2、1，最后元素为 8。打印每一步 size、capacity 和 data 地址；容量除标准明确的下界等保证外，不能把某个实现的数值照抄成普遍规律。

若保存元素指针，然后扩容迁移，旧指针不会自动指向新数组。释放后继续解引用违反有效性要求，即使偶尔还能读到旧数字也不能据此判断安全。只比较地址数值作观察，不去访问已经失效的对象。

标准原文：vector 容量与 reserve/resize，可核对规模不变、容量保证与重分配失效规则。

23.3 3 · 课程 Vector：测试报告必须对照哪个不变量

通常有 `_size`、`_capacity`、`_elem` 三项核心状态。首先检查 $0 \le size \le capacity$，其次检查有效对象恰好落在 `[0,size)`，最后检查谁拥有并释放底层数组。

从容量 4、元素[2,4,6]开始，依次头插 1、尾插 8、删中段、复制、修改副本。头插后应是[1,2,4,6]；尾插可能扩容；副本的修改不应改变原表。若改变了，复制很可能只复制了地址，没有建立独立资源。

把空表、单元素、刚好满、自赋值、空删除、删除到触发缩容分别作为测例。随机通过 100 次只说明这些轨迹没有暴露错误。阅读测试主程序的每条断言，确认它查的是内容、规模、地址失效，还是资源泄漏；同一个“通过”不代表所有性质都已验证。

23.4 4 · 批量接口：别把同一后缀反复搬开

向长度 1000 的表头插 100 项。一次批量插入，原 1000 项整体右移 100 格，再写 100 个新项，主要约 1100 次写入。反复在同一位置单项插入，搬移量为 $1000 + 1001 + \cdots + 1099 = 104950$，还要加 100 次新值写入，而且新项次序会反转。

若每次递增插入位置以保留新项顺序，之前新插的项可能不用再搬，但原 1000 项仍各搬 100 次，共 100000 次。具体公式随调用次序改变，重复移动的浪费却很清楚。

因此先做区间接口，再把单元素视作长度 1 的特例，可以共享一次搬移和边界处理。删除也一样：一口气删除头部 100 项，只需压缩一次后缀；逐个删会重复压缩。批量设计的收益来自减少真实工作，不来自函数名字更漂亮。

23.5 5 · insert：插入多少项，原后缀又有多长

令 $k = hi - lo$，原规模 n，在秩 r 前插入。没有扩容时，旧后缀 `[r,n)` 的 n-r 项右移 k 格，再复制 k 项，故成本为 $\Theta(1 + k + n - r)$。非平凡情况常省掉 1，写成 $\Theta(k + n - r)$；触发扩容还要计入旧数组迁移。

原表[10,20,30,40,50]，r=2，插入[7,8]。先从末尾搬：50 到 6、40 到 5、30 到 4，再写 7 到 2、8 到 3。结果为[10,20,7,8,30,40,50]，搬 3 项、写 2 项，对应公式里的 $n - r + k = 5$。

若源区间来自同一个向量，扩容可能让源地址失效，移动又可能覆盖尚未复制的项，必须遵守接口的别名约束或暂存源。不能在证明里假定源独立，实现里却把自身的一段直接传进去。

23.6 6 · 移动方向：右移倒着来，左移正着来

把[A,B,C,D]的 B、C、D 右移一格。若先搬 B 到 C 的位置，原 C 就丢了；下一轮读到的已经是 B，后面可能连着复制同一个值。正确顺序是先 D，再 C，再 B，让每次读到的源都还没被覆盖。

删除后的后缀左移则从前向后，因为目的地在源左侧，不会破坏更靠右的未读项。源和目的完全不重叠时，方向可以不同；判断依据是区间重叠关系。

`memcpy` 要求不重叠，`memmove` 允许重叠。但是含字符串等非平凡 C++对象不能简单按字节搬来搬去：构造、析构、移动和所有权都有类型规则。画整数格子帮助理解方向，写模板容器还要把对象生命期处理正确。

23.7 7 · remove：没删东西，却可能搬完整个表

删除 `[lo,hi)` 后，旧后缀 `[hi,n)` 左移 hi-lo 格。整数搬移模型的成本为 $\Theta(1 + n - hi)$，固定接口成本不能在 n=hi 时解释成“完全零时间”。

原循环如果不判空，lo=hi=0 时不删任何项，却让每个元素做一次自赋值，正好仍是线性最坏例。加 `if(lo==hi) return 0;`，便能让空区间常数返回。真实删除首项也要搬 n-1 项，仍为线性。

通用对象还可能需要析构被删项，所以仅以 n-hi 计费隐含了元素处理模型。多一次分支值不值，要看空删除频率与元素移动代价；能给出实验设计，不能凭空报提速百分比。

23.8 8 · 几何扩容：一次昂贵迁移由很多次追加分担

容量每次乘固定 $\alpha > 1$。从空表一路追加到 N 项，旧容量迁移量形成几何级数，累计小于约 $\alpha N/(\alpha - 1)$，加上 N 次新项写入仍为 O(N)，所以按单元素追加均摊 O(1)。

倍增时搬过 1、2、4、8 项，合计 15，不是每次都搬 16 项。$\alpha = 1.1$ 仍满足常数倍增长，但复制常数大致随 $1/(\alpha - 1)$ 增加；增长后装填率约 $1/\alpha$。更小的闲置空间通常意味着更频繁的迁移。

批量插入不能按“调用一次”无条件均摊 O(1)：一次插百万项，光写新项就得线性；在一个已有巨大满表上触发扩容，也可能有大迁移。应按完整操作序列和插入元素总数算账。标准库没有规定每个 vector 实现都恰好倍增，具体倍数要查所用库版本并打印 capacity 核对。

23.9 9 · 为什么不能只平均真正扩容的调用

倍增到 N，实际迁移约 log N 次，总迁移量为 $\Theta(N)$。只对这些昂贵调用取平均，结果是 $\Theta(N/\log N)$；把所有追加检查算进来，才是每次摊还 O(1)。两者换了分母，没有矛盾。

两次扩容之间的便宜操作为下次“搬家”积攒费用。若把这些操作从统计集合删除，自然不能继续引用原来的均摊结论。摊还也不要求随机输入；对任何遵守接口的合法序列，总账都受控制。

写分析时明确列出你平均的是所有 push_back、所有 expand 调用，还是仅有搬迁的 expand。先把样本集合讲清楚，后面的公式才有具体含义。

23.10 10 · 半满就缩会抖动，四分之一才缩留出了间隔

容量 C、规模刚低于 C/2 就缩成 C/2，接着只需插很少几项就再次满；删一点又缩。对手用常数次插删，就能迫使约 C/2 项反复迁移，昂贵操作之间没有足够便宜操作来分摊。

改成低于 C/4 才缩为 C/2，缩后约半满。要再扩容得插入约 C/4 项，要再缩还需删掉一批项。这个间隔使累计收费能够覆盖搬移。

例如容量 16、规模 8，删到 7 先不缩；删到 4 附近满足具体阈值时才缩为 8。“小于”还是“小于等于”会改变边界那一步，但不改变要留间隔的理由。最小容量要有固定下限，空表也不应反复分配无意义的零长数组。

23.11 11 · 去重里的哨兵：当前项本来就在停止位置上

查 `_elem[k]` 是否出现在 `[0,k)` 时，目标就在下标 k。若 find 靠第 k 格的哨兵让扫描最终停下，就不必再保存、写入和复原同一个目标，因为它已经在那儿。

每轮省常数次操作，n 轮省 O(n)。如果整体仍有二次比较或移动，总阶没有改变。常数优化值得解释，但不能说它把算法改成了线性。

查到前面的 i 时删 i，会保留较新的 k；改删 k 则保留先出现的一份。附加字段可能不同，所以这不只是下标替换。删 k 以后，下一个未处理项移到 k，不能还无条件 k++，否则会漏项。

23.12 12 · 无序去重：先减少移动，再讨论比较

若保留最后一次出现，可以让 i 扫描原表，find 检查未读后缀 `[i+1,n)` 是否还有同值项；有则跳过，没有才把整条记录写到 w 并令 w++。w 始终不超过 i，不会破坏未读后缀。

例：[A1,B1,A2,C1,B2]，A1 和 B1 后面还有同值记录，跳过；A2、C1、B2 依次写入，得到[A2,C1,B2]。每项最多写一次，移动不超过 n。已写前缀始终是已经确认不再有后继重复的记录。

全相等时，若 find 见到相等项立即返回，可每轮很快确认重复，整体 O(n)；全不同时仍要扫描各后缀，比较为 $n(n-1)/2$。减少移动没有自动减少所有比较。

若允许排序，可 O(nlog n)后线性去重，但顺序可能改变；允许散列表时可在相应散列模型下得到期望 O(n)，空间也增加；小整数值域可用位图。对于任意元素只提供相等测试的模型，不应承诺同一改进。题面所说移动比比较贵，是优化动机；通用临界长度还取决于元素类型和机器，不能编一个数字。

23.13 13 · 函数对象：累加到的是外部对象还是内部副本

求和访问器保存一个累加状态：

```
struct Sum {
    long long value = 0;
    void operator()(const int& x) { value += x; }
};
// 假设 traverse 按引用接收访问器
Sum s;
v.traverse(s);
std::cout << s.value;
```

对[2,-1,5]，状态 0⟶2⟶1⟶6。若 traverse 按值接收，内部副本变成 6，外部 s 仍是 0。先看函数签名，再判断它有没有工作。也可让接口返回访问器副本，但设计必须明确。

空表求和保持初值 0；整数元素多了以后可能超过 int，所以累加器范围仍要算。函数对象能把“访问时顺便保存状态”组织起来，不能免掉数值范围和复制语义。

23.14 14 · 用遍历做冒泡：访问器要记住前一项

一趟冒泡比较相邻元素。访问器保存上一项地址与交换标志；访问当前项时，若前项更大就交换，然后把上一项地址更新到当前项。对[3,1,2]，先交换 3 和 1 得[1,3,2]，再交换当前 3 和 2 得[1,2,3]。

若访问器只收到值的副本，就不能修改原表；要限制未排序区间，还需区间遍历或计数器。把循环换成 traverse 只是换了表达方式，邻接关系和边界状态仍然要维护。

一趟线性，最坏 n-1 趟，比较仍约 $n(n-1)/2$，总时间 $\Theta(n^2)$。接口复用与渐近效率要分别评价。

23.15 15 · 判等与比较：同分数的两条记录算相同吗

记录(score,id)按分数排序，(90,A)与(90,B)互不小于，属于同一排序等价类；若相等运算符还比较 id，它们又不相等。去重按分数还是按完整记录，要由业务定义。

严格弱序比较器的等价可写为 `!less(a,b)&&!less(b,a)`。这不一定与 `a==b` 一样。排序按 score、去重按全字段，并不会自动得到“每个分数只留一条”。

比较规则还必须在整个算法中一致。若同一个 a 满足 less(a,a)=true，或排序过程中临时改变顺序标准，二分与归并的证明都失去了前提。

23.16 16 · 缩容后：整数秩还在，旧地址不会自己更新

旧底层数组在 P，缩容后迁到 Q。秩 k 只是一个整数；再调用当前 `_elem[k]` 会用 Q 寻址，只要 k 仍合法，就能访问新数组。

旧指针 P+k 却不会自动改成 Q+k。原数组释放后它悬空；引用和某些迭代器也一样。删除还会改变秩：旧 k 可能现在指向原来的下一项，即使没越界，逻辑也可能错。

检查去重循环时分开问：“保存的是秩还是地址？”“删后下一项移到了哪里？”它们分别对应地址失效和扫描漏项，不能混成一句“扩容后小心”。

23.17 17 · 懒惰 uniquify：前面没重复时，为什么还写自己

read 向前读，write 指向最后保留位置；新值与保留值不同才推进 write 并写入。遇到首个重复以前 read=write，赋值只是自己写回自己。

可先找第一对相邻相等项；若没有便直接返回，找到了再开始压缩。例[1,2,3,3,4]的前三项不用写，最后 4 才移到下标 3。始终维持“输出前缀等于已读部分的去重结果”。

这减少写操作，却增加了首段检测或分支。对无重复、大量重复和昂贵对象，实际收益不同；总读次数仍线性，所以时间阶不变。优化不能以更难验证的边界代码为代价。

23.18 18 · 本题保留最后一份：按组扫描就能做到

原题把稳定约定为每组相等记录保留最后一项。它与排序稳定性的通常定义相关，但要求不完全相同，回答时要忠于这条题面约定。

有序表逐组扫描：i 是组起点，j 一路走到组末，把整条 a[j]写到输出位置。[(1,A),(1,B),(2,C),(2,D)]最终得到[(1,B),(2,D)]，不能只改 key 而丢掉附加字段。

每项被访问常数次，总时间 O(n)，几个下标只需 O(1)空间。从右向左也可选最后记录，但要把输出恢复到正常顺序，并把那部分移动计入成本。

23.19 19 · 分支次序：大 O 相同，不等于比较次数相同

一次查找要区分小于、大于、等于轴点三种情况。用两个 if 和末尾 else 表达，排在第一种的情况只用一次关键字比较，其余用两次。若本层三类概率为 p_L, p_R, p_E，把概率最大的类别放第一，理想化的期望比较次数为 $2 - \max(p_L, p_R, p_E)$。

无论怎么排，只要每次把区间按固定比例缩短，最坏迭代轮数仍是 $\Theta(\log n)$。但相等命中若极少，把相等测试放第一，就可能每层几乎都要多做一次比较。取百万个互异 uint32_t 关键字、目标在全部 2^{32} 个数中均匀抽取，总命中概率约 $10^6/2^{32} \approx 0.000233$，即 0.0233%。

这还不能推出每一层左右概率恰好各半。若关键字和查询都分布平衡，可粗估相等优先约每层 2 次，方向优先约每层 1.5 次；约 20 层时分别接近 40 和 30。若值集中、查询偏斜或比较代价不对称，估计会变。CPU 分支预测也会影响真实时间，所以这里估的是关键字比较，不能把它直接当纳秒。

23.20 20 · 左转 1、右转 2：课程为什么忽略循环下标的比较

原 A 版先测试 `e<S[mid]`，为真就左转；否则再测试另一方向，不等于才右转。按关键字比较计费，左转 1 次、右转 2 次。`lo<hi`、计算 mid 和更新下标也做工作，但课程把它们作为统一的控制开销，或把昂贵的关键字比较作为单独的分析对象。

若元素是长字符串，关键字比较可能远贵于一次整数下标运算，这个模型很有意义。若所有元素是短整数，要估实际运行时间，必须把这些成本和内存、分支都纳入。没有哪条物理定律保证右转恰好耗时左转两倍。

所以不是随意把 1:2 改成 2:3 或 3:4；那相当于在每轮额外加入了另一种固定收费。模型可以换，但模型换后，黄金分割最优常数的推导也应跟着换。

23.21 21 · 中点溢出：最终合法，计算途中也可能先出事

在 `0<=lo<=hi<=INT_MAX` 下，`hi-lo` 非负且不超过 INT_MAX，半差也在范围内，`lo+(hi-lo)/2<=hi`，所以这条中点公式不会因正整数加法超上界而溢出。

而 `(lo+hi)/2` 先算和。例如 32 位有符号数里 lo=1500000000、hi=2000000000，两者分别合法，总和 3500000000 已经超范围，不能指望最后再除 2 救回来。

若改用无符号类型，算术可能按模回绕，仍不等于算出了正确中点；若 lo、hi 来自不同符号类型，转换也要另查。库代码不应因为“当前机器放不下那么多元素”就省掉几乎不增加复杂度的安全写法；不过本题的具体小规模实测可能根本触不到这个边界。

23.22 22 · 重复项：“随机返回”其实是确定轨迹

A 版遇到等于轴点就返回。数组[2,2,2,2]、区间[0,4)第一次 mid=2，立即返回 2；区间换成[0,3)，mid=1，立即返回 1。同样的代码、输入和区间会重复返回同一个位置，所谓“随机”只是接口没有承诺首个还是末个。

需要第一项时寻找第一个不小于 e 的位置；需要最后一项时寻找第一个大于 e 的位置，再减 1 并检查是否等于 e。相等时继续向哪边收缩，决定了边界语义。

做测试时给重复元素附不同 id，才能看出返回的是哪一条。只打印 key，全是 2，很容易误以为测试已经验证了“保留哪份”。

23.23 23 · Fibonacci 切分：便宜的一边为什么分得更大

原 A 版左转收费 1，右转收费 2，轴点在 lo 加 $\lambda(hi-lo)$ 处。左边约占 λ，右边约占 $1-\lambda$。若最坏比较数近似 $C\log_2 n$，两种路径分别要求

$$C \geq \frac{1}{\log_2(1/\lambda)}, \qquad C \geq \frac{2}{\log_2(1/(1-\lambda))}.$$

要把两者的最大值压到最低，使两项相等，即 $\log(1-\lambda) = 2\log\lambda$，得到 $1-\lambda=\lambda^2$，正根为 $\lambda=(\sqrt{5}-1)/2\approx 0.618$。较大的子区间交给便宜的左转，最坏比较主项系数约 1.4404。

若镜像取 0.382，昂贵右转反而分到约 0.618，最坏系数约 $2/\log_2(1/0.618)\approx 2.8808$，并不仍然最优。这个最优性依赖 1 与 2 的收费模型，若使用三向比较器或不同硬件成本，结果要重新推导。

Fibonacci 数满足相邻两项比值趋向这个比例，可以用递减的 Fibonacci 跨度定位轴点，避免每层浮点乘法。但若关键字比较很便宜，维护额外状态的成本未必值得；“比较次数更少”是理论结论，是否更快还应实测。

23.24 24 · B 版为什么先判 e 小于最小值

课程 B 版维持至少一个候选，循环到区间长度 1，然后直接返回 lo 作为“不大于 e 的最后一项”。若 e 连最小值都不到，却不提前排除，最后仍会留下下标 0，错误地声称它是合法前驱。

例如[3,7,9]查 2。候选不管怎么缩都不可能出现合法元素，所以应返回起始 lo 减 1 的失败哨兵；这就是开头判断的作用。A 版查相等项，可以一直缩到空区间而报告失败；C 版允许空区间，并从统一边界减 1，故不需要同样的最小值特判。

原题 b 行写着 A、C“都需要如此特判”，与这两版实际代码的处理不同；应把源码行为讲清楚，不能因为题面一句话便给它们强加多余分支。B 版读取 `S[lo]` 前还必须保证非空，这又是另一项前置条件。

23.25 25 · 空区间：返回哨兵，不代表可以解引用哨兵

C 版求 upper_bound 后减 1。若目标小于最小项，所有比较都向左，最终 lo 停在起始位置，返回 lo-1。若起初 lo=hi，循环根本不执行，也返回起始 lo-1。

这只是表示“本区间没有不大于目标的项”。若对全表[0,n)得到-1，不能接着访问 `S[-1]`。若 Rank 改为无符号类型，减 1 会回绕，所以接口还需改用可表达失败的类型或显式结果。

A 版空区间自然不进入循环；B 版如果一开始就读 `S[lo]` 则越界，必须先判空或把非空写成前置条件。把“返回值有定义”和“该值可以当元素位置访问”分开，是这组题的重点。

23.26 26 · C 版的另一种证明：按区间长度归纳

C 版每次选 mid，若 e 小于轴点，则 `[mid,hi)` 全部大于 e，不可能包含前驱，剩下左区间；否则 `[lo,mid]` 全部不大于 e，它们已经可以归入确定的左侧，未知部分只剩 `[mid+1,hi)`。

以初始区间长度为归纳参数。长度 0 时，未知区域已经空，边界 lo 左侧全不大于 e、右侧全大于 e，返回 lo-1 正确。长度大于 0 时，两个分支都只删除已确定的一侧，并保留同样的边界含义，且剩余长度严格更小，因此可用归纳假设。

这个证明同时需要有序性、边界语义和严格缩短。只说“规模小了所以对”没有证明删掉的那一半不会藏着答案。若存在重复项，相等时也把 mid 归到左侧，正好确保最后找到最后一个不大于目标的项。

23.27 27 · 平均查找长度：先讲“平均哪些输入”

令 n 个关键字互异，成功时各关键字等概率，失败时 n+1 个间隙等概率；计费是 C 版循环中的关键字比较。若 $n = 2^k - 1$，决策树的 n+1 个终点间隙都在深度 k，所以任何查询均做 k 次比较，两个平均值恰好都等于 k。

一般 n 就不能把“同一主项”说成精确相等。n=2 时，间隙依次小于首项、两项之间、大于末项，比较次数为 2、2、1，失败平均为 5/3；目标等于两个关键字时为 2、1，成功平均为 3/2。两者已经不同。

设 $N = n + 1, k = \lfloor \log_2 N \rfloor$，这类对半决策树的间隙深度为 k 或 k+1。深层叶数为 $2(N - 2^k)$，因此叶深总和 $D = N(k + 2) - 2^{k+1}$，失败均值为 D/N。成功等于第 j 个键的轨迹与其右侧间隙相同，所以成功均值是 $(D - d_0)/n$，其中 $d_0 = \lceil \log_2(n + 1) \rceil$ 是最左间隙深度。

这些式子共同给出 $\log_2 n + O(1)$，即主项系数为 1。题面用大 Theta 串起的表达应按渐近意义理解；若要求任意 n 时两个精确均值相等，上面的 n=2 就是反例。对数底与概率模型也必须说明。

23.28 28 · 比较 A、B、C：别把三版的返回语义混起来

n 个互异有序键把数轴切成 n+1 个失败间隙，因此“失败类比成功类多 1”来自顺序结构，而不是某份二分源码的偶然。A 版成功是遇到相等项，B/C 版主要返回前驱；若需回答是否命中，还要在返回位置上再判等。

对 $n = 2^k - 1$，A 版成功键所在层 d 有 2^{d-1} 个。若只数访问层数，平均深度为

$$\bar{d} = \frac{\sum_{d=1}^{k} d\, 2^{d-1}}{2^k - 1} = k - 1 + \frac{k}{2^k - 1}.$$

但按原代码关键字比较，左 1 右 2，成功末层还需判等，平均约 $1.5\bar{d} + 0.5$；失败间隙等概率时平均约 1.5k。访问一次节点与做一次关键字比较不能混算。

C 版在这个规模上每次固定 k 个方向比较，成功与失败都 k。B 版循环到长度 1，另有最小值特判；大多数路径同样为 log n 量级，但其精确平均应按课件源码把开头特判和最终命中检查分别计入。第一个失败间隙常能在特判处结束，它与其他间隙并不对称。先列清收费表，再求平均，才能解释几版常数为何不同。

23.29 29 · 终止：为什么 lo 不会一脚跨过 hi

C 版在 lo<hi 时，取 `mid=lo+(hi-lo)/2`，总有 $lo \le mid < hi$。左分支令 hi=mid，仍 lo≤hi，长度从 m 降到 $\lfloor m/2 \rfloor$；右分支令 lo=mid+1，仍 lo≤hi，剩余长度也严格小于 m。

所以长度是非负、严格下降的整数，循环必停；出口因为不能 lo>hi，又不再 lo<hi，只能 lo=hi。长度 1 时左分支把 hi 落到 lo，右分支把 lo 落到 hi，两种都一步结束。

B 版的持续条件为长度大于 1，更新 lo=mid 时不会卡住，因为长度至少 2 保证 mid>lo；出口长度 1。若误把同样的更新配上 C 版 lo<hi 条件，长度 1 时就会死循环。三版更新规则与持续条件必须成套理解。

23.30 30 · 插值查找：一个极端值就能把猜测拉歪

取序列[0,1,2,...,n-2,M]，其中 M 远大于 n，例如 M=n³，查找靠近 n-2 的值。按端点线性插值，估计位置大致为 $(n-1)e/M$，常常接近区间起点；一次仅排掉一两个小值，反复执行就可能线性。

序列确实有序，坏的是“数值距离能预测秩距离”的假设。二分不依赖值分布，每次按位置砍半，因此最坏仍对数。插值实现可设置进展检查或回退二分，避免长期猜在边缘。

还要处理端点值相等的分母 0、乘法溢出、目标超范围和重复值。理论上在均匀模型下很快，不表示可以省略边界检查。

23.31 31 · 非均匀分布：用累计概率猜秩，而非硬套直线

若已知值服从分布 F，$F(x)$ 表示随机值不大于 x 的概率。长度 n 的独立样本里，不大于 x 的数量期望约 nF(x)，因此可用累计概率替代原始数值做插值。

对高斯分布，先标准化 $z=(x-\mu)/\sigma$，再用其 CDF 估计累计比例。中心很密、尾部很稀，直接按 x 的线性距离猜位置会失真，CDF 映射才反映这些密度差别。

但参数估错、有限样本起伏、离群点和重复值都会使预测不准。CDF 只负责提出一个候选轴点，仍需把它限制在合法区间，并靠有序比较维持正确性。不能把统计模型的期望当作每份数据的精确秩。

23.32 32 · 二分何时切顺序：做一个不会骗自己的实验

固定编译优化、元素类型、比较器和查询集合；分别测试长度 8、16、32、64 等小表，既有成功也有失败，既有均匀也有聚集值。每种重复足够多次，防止单次计时噪声淹没差异。

结果应累计到校验值并输出，避免编译器删除“算了却没人使用”的循环。把数据生成时间移出计时区，改变测试次序，报告中位数和波动，再选在代表数据上稳定的切换阈值。

顺序扫描虽然比较多，但控制简单、内存连续；小表可能胜过多分支二分。长字符串、缓存状态和 CPU 改变后，阈值可能变。这里给出实验步骤，没有把未经本机测量的某个长度冒充普遍答案。

23.33 33 · 冒泡提前停：一整趟没交换才证明已经排好

每趟开始 sorted=true，只要交换一次就设 false。若整趟结束仍 true，所有相邻项非降，因此整个序列有序，可以结束。

[1,2,3]第一趟无交换；[2,1,3]第一趟交换得到有序，但标志已经 false，若没有别的逻辑，还要下一趟确认；[2,3,1]中的 1 每趟只能向左移动一格，需要更多趟。全相等也应不交换，否则既损害稳定性又失去提前停机会。

最坏逆序仍二次，最好有序为线性。平均省多少趟，需要指定随机排列模型和是否把最后确认趟算入；只挑一个近乎有序样本，不能推出总体平均提速。

23.34 34 · last 跳跃：最后交换之后的区域，为什么可以不再碰

一趟从左向右比较，把最后一次交换的右端位置记为 last。本趟结束后，last 之后没有逆序交换需求，且大项已向右传递，因此下一趟可把活动右边界缩到 last。

例如[3,2,1,4,5,6]的乱序只在前三项，第一趟最后交换发生在前部，后面 4、5、6 可以直接排除。若只按 n-1、n-2 缓慢缩小，会重复比较这些已经安稳的后缀。

last 初值和边界含义要配套：若 last=0 表示未发生交换，就可以结束；若存左端下标，下一轮界限又差 1。画清“还比较哪些相邻对”，比套一个神秘赋值更可靠。逆序最坏仍二次。

23.35 35 · 非对半归并：固定比例仍是 nlog n

设固定 $\alpha\in(0,1)$，递推为 $T(n)=T(\alpha n)+T((1-\alpha)n)+\Theta(n)$。每层区间总长度不超过 n，最长路径 O(log n)，上界 O(nlog n)；前常数倍 log n 层都尚未触到基础规模，层成本为 Θ(n)，给出匹配下界。

例如 1/4 与 3/4 切分比对半树更深，但深度只差固定常数。若比例随 n 变化成 1 与 n-1，就变成 $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$。

固定比例偏斜通常使常数变差，还可能改变实际缓存表现；渐近同阶不代表实际一样快。先分清常数比例与每次只减固定个数。

23.36 36 · 复用归并缓冲：不必每个节点都 new 一遍

排序入口一次申请长度 n 的辅助数组，递归时把区间边界传下去，每次归并使用对应片段，退出整个排序后统一释放。顺序递归的兄弟区间不会同时使用同一片段，可以复用。

原来每个归并都申请、释放缓冲，分配调用次数可能达到线性数量；统一缓冲把这部分调用降为常数次。元素比较和迁移总量仍为 O(nlog n)，峰值辅助空间仍 O(n)，不能把少做分配说成空间阶从 n 降成常数。

若改成并行归并，要检查不同任务的片段是否重叠。C++还需选择缓冲中哪些元素已经构造、哪些需移动赋值，不能只把裸指针传来传去。计时应对同一输入副本比较两版，并把复制测试输入放在计时区之外。

23.37 37 · 一边耗尽：右半剩下的值为什么可能不用再复制

课程常把左半复制到缓冲，右半仍在原数组，写入位置从原区间左端向右移动。左半先耗尽时，写指针与右半读指针会重合，右半剩余项已经处在正确位置，无需原地复制。

例如左[1,2]、右[3,4]，前两次把 1、2 写回，随后读右半的位置恰好就是接下来输出的位置。若右半先耗尽，缓冲中的左半还没在正确位置，必须继续写。

如果实现把两半都复制到了外部缓冲，那么两边的剩余项都要拷回。答案取决于源、目的在哪以及读写指针的关系，不是“归并永远只处理一边耗尽”。

23.38 38 · 已经整体有序：只检查分界那一对就够吗

左右分别有序且非空。若左半最大值不大于右半最小值，即 `a[mid-1]<=a[mid]`，那么任意左项都不大于任意右项，整段已经有序，可以直接返回。

这个检查多一次比较；在已经有序输入上，每个递归节点都能跳过整段归并，总成本可降到线性量级的节点检查。普通输入若经常不满足，只是多付一个小常数，不保证所有机器都一定更快。

要先处理空段和长度小于 2，避免 mid-1 越界；比较器若只提供 less，用 `!less(a[mid],a[mid-1])` 表达非降关系。相等时不必交换，稳定性也能保留。

23.39 39 · inplace_merge：接口名字没有保证完全不申请缓冲

C++的 `inplace_merge` 表示结果仍放回原区间，不等于所有实现都禁止临时存储。要以标准规定的语义、复杂度条件和手头库源码为准，不能照题面一句“彻底避免辅助空间”推广到所有库。

一种低元素缓冲的思路是：在两段中选切点，把中间错位的两块通过旋转交换，再递归处理两个较小归并。旋转可用反转实现，能省大块元素缓冲，却增加移动和控制工作；递归本身还可能用 O(log n)栈空间。

例如[1,4,7]与[2,3,8]，可先找 4 在右段应插入的位置，把[4,7]与[2,3]旋转成[2,3,4,7]，再处理更小边界。要证明稳定，切点的 lower_bound 或 upper_bound 取法也必须正确。空间紧张、大对象移动昂贵、数据规模等因素共同决定是否值得，不能只看“原地”两个字。

标准对非并行 inplace_merge 明确区分额外内存是否足够：有足够内存时比较至多 N−1，否则允许 O(Nlog N)。核对合并接口与复杂度。

23.40 40 · 位图复位：清零所有格子，还是宣布旧记录失效

第 k 位在字节 `k/8`，偏移 `k%8`；置位用掩码或，清位用掩码取反再与。若值域大小 U，普通位图整体清零要处理约 U/字长个存储字，成本依赖 U。

Hopcroft 结构用稠密表 dense 保存当前存在的值，稀疏表 sparse 给出候选位置，另存逻辑长度 m。判断 x 存在，需要同时满足 `sparse[x]<m` 与 `dense[sparse[x]]==x`。只把 m 设为 0，就让所有旧候选位置失去校验效力，逻辑清空 O(1)。

另一种方法给每格存时间戳，清空时增加当前世代号，格子时间戳等于当前世代才有效。世代号有限宽，回绕时需要真正重置；首次分配、初始化和空间都不能消失。C++读取未初始化整数也不合法，所以“逻辑清空 O(1)”不等于首次创建一个巨大数组也完全免费。

23.41 41 · 普通位图去重并排序：时间里有值域这一项

把 n 个值的对应位设为 1，耗 O(n)；再从 0 到 U-1 扫描置位项并输出，耗 O(U)，合计 O(n+U)。扫描按数值递增，所以同时完成去重与排序。

若 U≤cn，便可简写为 O(n)。若 n=1000、U=十亿，扫描十亿位范围的需要处理的范围由 U 决定，不能只拿 1000 估计；位图还可能占约 125MB，具体单位和额外对象另计。

可以只在读输入时检查并保留首次出现来避免全域输出扫描，但那保留的是输入顺序，不再自动排序。输出要求不同，复杂度公式也不同；不能拿去重的改进偷偷替代“去重且排序”的原任务。

23.42 42 · 校验环删除：拿最后一项填洞

设 dense=[4,9,2]、m=3，sparse[4]=0、sparse[9]=1、sparse[2]=2。删 9 时，位置 1 会留下洞；把末尾 2 搬到 1，令 sparse[2]=1，再 m−，有效 dense 变成[4,2]。

旧 sparse[9]即使还等于 1 也没关系，因为 dense[1]现在是 2，双向校验失败，9 被判为不存在。若删的本来就是末项，直接 m−即可，交换自身的写入也可省掉。

每次只改常数个格子，因此删除 O(1)。前提是先确认 x 确实存在；若拿一个无效 sparse 位置直接搬，会破坏其他记录。删后 dense 的顺序可能变化，这也是它与保持插入顺序容器的差别。

23.43 43 · Hopcroft 位图：免掉全域扫描，付出什么代价

扫描 n 个输入，每个用双向校验判断是否已存在，不存在就加入 dense，并更新 sparse。已有结构的逻辑清空 O(1)，每个输入处理 O(1)，因此这一轮去重 O(n)，不用把整个值域逐格清零或扫描。

但直接寻址的 sparse 仍随 U 占空间；相较一位表示一个值，它常需要一整个整数位置，空间明显增加。首次建立可合法读取的存储仍有初始化成本，若把 U 也计入本次从零开始的总成本，不能声称完全与 U 无关。

dense 保存的是处理顺序或交换后的顺序，去重结果不再天然按数值排序。若还要求有序输出，需要另排序，或者重新扫描值域。节约的是反复清空和稀疏访问的时间，代价包括空间、顺序与初始化条件。

23.44 本章练习与核对

1. 容量 8、规模 8 的表从头插 1 项，为什么“追加均摊 O(1)”不能推出这次头插 O(1)?
2. 对[2,2,5]查 2，用 C 版列出 lo、hi、mid，返回哪一个 2？若查 1 又怎样?
3. dense=[6,1,8,3]，删除 1 后，至少需要改哪几项？旧 sparse[1]为什么可以暂时不擦?

第一题：几何扩容只控制整条追加序列的迁移账；头插还需要移动原后缀，单次本来就是 Θ(n)，扩容也可能单次 Θ(n)。第二题：起初[0,3)，mid=1，2 不小于 2，lo=2；再 mid=2，2 小于 5，hi=2，返回 1，即最后一个 2。查 1 先 hi=1，再 hi=0，返回-1，不可解引用。第三题：把 3 写到 dense[1]，sparse[3]改成 1，m 由 4 减到 3；旧 sparse[1]=1，但 dense[1]不再是 1，所以校验拒绝。

24 PS03：链表不是“数组换了几个箭头”

教师原始习题单：第 03 次习题共 37 题。这里把节点身份、逻辑次序和数据值分开：移动节点改变链接；交换 data 改变值属于哪个节点；删除并新建还会改变对象生命期。它们有时得到相同的数值序列，却不一定对外部指针、稳定性和资源管理产生相同效果。

24.1 1 · 静态与动态：看逻辑结构，不能只看地址动没动

静态操作查询现有数据，动态操作改变集合内容或结构。游标链表把节点放在固定数组格子里，用 succ 下标说明后继；把链接从 2→5→7 改成 2→7→5，物理格子都没挪，逻辑顺序已经改变，因此仍是动态操作。

反过来，底层把整个存储搬到新地址，只要抽象集合与顺序保持，用户看到的逻辑内容未必变化。接口分析应先说明以哪一层为准，而不是把“动态”误解成“指针数值变了”。

可以拿三个卡片做演示：卡片固定贴在桌上，只改“下一张编号”，输出顺序也能改变。这就是列表逻辑链接与数组物理邻接的差别。

24.2 2 · 秩与位置：写成 p+100，也还是走了 100 步

秩是第几个，位置是某个节点句柄。向量从首地址加秩乘元素大小可直接寻址；列表一般只能从一个已知节点沿链接走。给列表重载 `p+k`，可以把走 k 步封装得好看，不能因此让它变成常数时间。

在长度 n 的链表上循环访问 `L[i]`，若每次从头找第 i 项，总访问量是 $0+1+\cdots+(n-1)=\Theta(n^2)$。改为保存当前节点 p，每轮 p=p->succ，总共只走 n 次，就成线性。

因此课程不建议频繁用模仿数组的重载语法。阅读代码时主动问“这一行内部要走几条边”，不要让一个加号把复杂度藏起来。

24.3 3 · 两节点距离未知方向：两边同时试，不必先找头

假设 p、q 是同一合法双向链表内节点。若已知 p 在 q 前面，从 p 沿 succ 走，数到 q 即可。距离 d 按边数计算，p=q 时返回 0，整体可写 O(1+d)。

方向未知时，可以从 p 同时向前、向后扩展，每轮各走至多一步，任一方向碰到 q 就结束；碰到哨兵的方向停止，另一边继续。最多各探索 d 步，所以仍 O(1+d)。也可从 p、q 分别向后探索，看谁先遇到对方。

这需要同表与有效节点的前提。若 q 不属于该表，走到边界也找不到，必须返回错误，不能把哨兵当正常数据继续走。为了防御不可信句柄而先全表验证，可能另付 O(n)，与在已保证前提下的距离算法分开计费。

24.4 4 · 倒数第 k 项：双指针省的是重复定位，不是神奇少读所有节点

单链无长度时，先遍历数 n，再从头走 n-k 步，总边访问大致 $n+(n-k)$。双指针先让 fast 领先 k 步，再让两者同步；fast 到末尾时，slow 到倒数第 k 项。

以[10,20,30,40,50]找倒数第 2 项：fast 先越过 10、20，到 30；同步两轮后 fast 到 50 之后的尾边界，slow 到 40。初始化究竟把 fast 放在“第 k 项”还是“越过 k 项”取决于尾哨兵约定，必须按所用表示画一遍。

两种方法都 O(n)，双指针仍可能合计走近 2n 条边；其优势是一次向前流程、避免重新从头定位，也可能有不同缓存行为。k≤0 或 k>n 时必须拒绝。实验要固定内存布局和 k 的比例，而不是只比较一条很短链的偶然耗时。

24.5 5 · 哨兵：空表也有能接线的邻居

双向链表保留 head、tail，空表为 head↔tail。把 x 插入两者之间，用与一般位置同样的四条连接：x.pred=head，x.succ=tail，head.succ=x，tail.pred=x。删除唯一元素 x，再把 head.succ 指回 tail、tail.pred 指回 head，就恢复空表。

这消除了“首节点没有前驱”“尾节点没有后继”的多套分支，但并不把所有 O(n)接口改成 O(1)。它主要统一边界并减少错误机会，精确运行时间仍应实测。

要测试空表插入、删唯一项、连续清空两次、首尾移动和复制空表。哨兵不保存普通元素，不参与查找比较，也不该被普通 remove 删掉。若遍历到 tail 还读取 data，哨兵并没有替你保证安全。

24.6 6 · 四条链接的顺序：先保存旧关系，再改图

把新节点 x 插在 p 与 q 之间，先记住 q=p->succ，再写 x 的两条边，最后改 p 与 q 反向指向 x。若一上来 `p->succ=x`，随后又用 `p->succ` 当旧 q，就把旧后缀丢掉了。

删除 x 时先保存 p=x->pred、q=x->succ，再令 p->succ=q、q->pred=p，最后释放 x。释放后再读 x->succ 违反生命周期要求。两条互不依赖的写入可以交换，读旧边之前覆盖旧边则不行。

检验顺序的办法是每写一行就画出所有箭头，标出下一行还会读取哪条旧边。局部最终图相同不够，执行途中也不能读取已经丢失或释放的对象。

24.7 7 · 把节点移动到自己前面：这次应当什么都不做

若 `this==s`，通常把 move 定义成无操作，直接返回。若没有检查，先把 this 摘链，再试图插到 s 之前，s 恰好就是刚摘下的节点，后继和前驱已不再代表原位置，容易接成自环或断链。

相邻情形也要检查：x 本来就在 s 前面，再移到 s 前面应当保持原样；从另一个表转移，则还要同步两表的 size 和所有权。若接口不允许跨表或自身移动，应把它写成前置条件并断言，而非含糊地期待调用方永远猜对。

一个很小的测试表 head↔A↔B↔tail 就够，分别移动 A 到 A 前、A 到 B 前、B 到 A 前，逐次核对正向和反向遍历是否一致。

24.8 8 · insertFirst 复制：反着读，才能正着留下

原表 A→B→C，从头顺序读并每次 insertFirst，目标变化为空→A→B,A→C,B,A，结果反序。若从尾向前读，先 C、再 B、再 A，目标最终 A,B,C，顺序保持。

双向链表能直接沿 pred 反向遍历。单链若每次重新找前驱，会退化；可先用辅助栈保存节点或采用尾插，别为了使用 insertFirst 而制造无谓二次扫描。

独立复制 n 个节点需要 Θ(n)时间、Θ(n)新存储。即使额外工作空间是常数，新表本身的 n 个对象也必须分配；回答空间复杂度时说明是否把输出存储排除。

24.9 9 · 单链析构：先记住下一家，再拆当前这家

```
Node* p = first;
while (p != tail) {
    Node* next = p->succ;
    delete p;
    p = next;
}
```

单链没有 pred 不妨碍这个流程，因为销毁时只向前走。每个节点读一次后继、释放一次，时间 Θ(n)。若 tail 也是动态分配的哨兵，循环后再按所有权规则释放它；不能当普通节点提前删。

错误写法是先 delete p，再读 p->succ；另一个低效写法是每轮找最后节点并删除，每次都从头走，成本 n+(n-1)+…，变成 $\Theta(n^2)$。保存一个 next 就解决的问题，不需要用重新遍历补救。

24.10 10 · 链表无序去重：删一个节点，不必搬整个后缀

维护已经处理的无重复前缀。当前节点 p 在此前缀里找到同值 q，可以删 q 保留较新的 p；若改删 p，则必须先保存 p 的后继，再释放，随后从保存的节点继续，不能再访问 p。

全相等输入，每轮很快找到相等项，且删除只改常数条链接，最好 Θ(n)。全互异输入，第 k 项要查 k 个前缀节点，累计 $\Theta(n^2)$。课程原 Vector 版本即使比较少，也可能不断移动后缀，所以“同样的去重思路”在不同存储上有不同最好情况。

若改维护无重后缀，可从尾向前扫描，在后缀找重复；双向表能够对称实现。空表、单项、全等、首尾重复都要测。用哈希额外索引可改变期望比较成本，但已经增加了另一数据结构，不能不计其空间与假设。

24.11 11 · 遍历：访问器有没有资格修改链接

求和、计数、打印、验证有序性都可用访问器。计算平均值时让访问器保存 sum 与 count，遍历完再除；空表 count=0 时不能直接除 0。

若访问器仅修改 data，普通遍历一般还能沿 succ 继续；若它删除当前节点，遍历框架随后读取当前 succ 可能已经访问释放内存。需要安全删除协议：预存 next，允许回调返回新的位置，或明确禁止改链接。

因此“都能用遍历实现”不表示任何回调都能塞进同一个接口。先读参数是值、引用还是节点位置，再判断它能读写什么。

24.12 12 · 用 traverse 写 find/search：接口可能让你无法提前停

find 访问每个元素，若第一次相等就保存命中节点；search 在有序表里保存最后一个不大于 e 的节点。后者遇到第一项大于 e，本来就可以结束。

但若 traverse 的回调只返回 void、框架必定扫完整表，访问器虽然可以不再做比较，仍然发生后续指针移动。要真正提前停，可设计返回 bool 的回调或单独提供短路遍历。

例如[2,5,9,11]查 3，前驱是 2，看到 5 即可确认；通用遍历若继续走 9、11，只是功能正确而未达到同样的常数优化。接口能力和算法成本需要一起检查。

24.13 13 · 有序去重：不必为了少一行特判去冒险读哨兵

少于两项，没有一对真实相邻项可比较。先判 size<2 可以直接返回，避免构造第二节点位置后读到 tail。

也可以令循环显式要求当前 p 和其后继 q 都是真实节点。相等时删除其中一项，不等时推进；若删除 q，p 不动并重新取新后继，才能处理[1,1,1]连着三个相等值。

测试空、单项、两项同值、两项不同、整表全等、首尾各一组重复。无论哪种改写，证明都应是“已经越过的前缀无重复，且保持指定的保留规则”，而非单纯说代码行数少了。

24.14 14 · search 返回前驱：返回了一个位置，不等于找到了相等值

在有序区间(h,t)里，search 找最后一个不大于 e 的项。若 e 比所有项小，结果是头边界 h；插在它后面，就让 e 成为首项，顺序仍正确。

例如[2,5,9]查 7，返回 5 的位置。接着在其后插 7 得到[2,5,7,9]，这就是前驱接口的用途。若只想判断 7 是否存在，需要先确认返回值不是 h，再用一致的等价关系比较返回元素与 7。

查 2 返回 2、查 1 返回 h、查 12 返回 9，三个例子可以把“精确匹配”“前驱”“插入位置”区分清楚。把任意非空指针当命中会把后两种误判。

24.15 15 · 有序单链搜索：少比较，也未必少走路

只有相邻链接、无额外索引时，要到中间节点就得先经过前面的链接。即使关键字有序，对手也能把答案或最后确认位置放在很远处，最坏需要 Ω(n)次边访问。

在链表上做二分，比较次数可能降成 O(log n)，但找各次中点的总步数仍可为 O(n)，不会像数组那样获得 O(log n)总时间。若每次又从头定位中点，甚至可能更差。

增加跳表层、树索引或按秩定位结构能改进，但这已改变数据结构和维护成本。题目问原链表能不能做到，答案必须带着这个访问模型，不能用另一个结构的能力冒充。

24.16 16 · 选择排序的交换、平移和删建，是三种不同的动作

交换法每轮找到无序前缀最大项 m，与前缀末项 x 交换 data，再把边界左移。它只做常数次元素移动，不必分配节点。若用 remove 取出值、再 insert 新节点，会多做分配、析构和复制，原节点身份也丢了。

若“平移”指摘下同一个节点再接到后缀前面，则无需新建，代价主要是链接写入，且其他元素相对顺序可以保留。比较两个实现时必须读清 remove 返回的是值，还是只摘链返回节点。

例如外部保存了 m 的节点地址。交换 data 后，该地址里的值可能变了；移动原节点后，该地址和值仍然配对，只是邻居变了。只检查最终数值排序，检验不到这项接口语义。

24.17 17 · 最后一个最大值：能帮助稳定，但不能救任意交换

从后向前找最大，初始化为最右项，只在遇到严格更大值时更新，就自然保留最后一个最大值。相等项不会更新；某个值即使多次出现，也最多在它第一次严格超过现有最大值时触发一次记录更新。

若从前向后扫描，为了仍取最后一个最大值，要在大于或等于时更新。全相等长度 r 就会更新到每一项，最多 r-1 次（不计初始化）。扫描方向与相等处理必须配套。

选最后最大再把整个节点稳定地摘到后缀，可保留等值次序；但任意交换 data 仍可能破坏稳定。对[(2,A),(1,B),(2,C),(1,D)]，先将 2C 换到末尾，再将 2A 换到剩余末尾，可使 1D 跑到 1B 前面。因此要审查整套排序，不把 getMax 的一个好性质扩展成整体保证。

24.18 18 · 向量选择排序：二次比较，线性移动确实能同时成立

长度 r 的无序前缀找最大需要 r-1 次比较，r 从 n 降到 2，合计 $n(n-1)/2$。每轮最多做一次交换，而交换由常数次元素赋值组成，所以元素移动 O(n)。

例如[3,1,4,2]：先选 4 与末项 2 交换得[3,1,2,4]；再选 3 与 2 交换得[2,1,3,4]；最后换 2 与 1，得到有序。比较扫描很多，真正交换仅 3 次。

它适合需要控制移动次数的模型，但若比较也很贵，二次比较仍是明显缺点。普通交换版不稳定；若改成数组区间平移来保序，可能引入二次移动，不能同时保留原来的全部优点。

24.19 19 · 交换与轮换：先把排列画成箭头

将每个位置连向其元素的最终位置，整个排列分成若干不相交的环。已经就位的元素是长度 1 的环。交换两个位置，只改变这两个位置发出的归属关系，其余箭头保持。

若两位置在同一环上，交换会把环切成两个；若在不同环上，交换会把两环接成一个。可以用[2,3,1]：原箭头 1→2→3→1，交换位置 1 与 3 后成为[1,3,2]，拆成固定点 1 与环(2,3)。

因此一次交换让环数改变 1，奇偶性也改变。环不是“数组里相邻的一段”，两节点在环中相邻与在物理序列中相邻完全不同。

24.20 20 · 交换选择排序：为什么有效交换数是 n 减环数

无序前缀末位置 x 应该放当前最大值 m。因为 m 的目标就是 x，m 所在位置的箭头直接指向 x 所在位置，所以两者属于同一环且沿一个方向紧邻。交换后 m 就位成为固定点，原环少一个成员。

若初始有 c 个环，每个长度 l 的环需要 l-1 次这种有效交换，合计 n-c。课程若连最后长度 1 的步骤也做，每个环最终留下的那个固定点对应一次原地交换，所以无效交换共 c 次；若省去最后一步，计数相应少 1，必须统一约定。

互异均匀随机排列中，当前 r 项的最大值等概率位于任一位置，在末端的概率 1/r。对 r 从 1 到 n 求和，原地交换期望为 $H_n = \sum_{r=1}^{n} 1/r$。由逐输入“原地次数=环数”，得到期望环数也是 H_n，因而 Θ(log n)。

这不是假设每次事件独立。期望可相加，不需要独立；但最大位置均匀来自输入排列的随机模型，不能无条件套到人为构造数据。

24.21 21 · 换成平移：逐输入的环数等式已经失效

取[3,1,2]，原排列是一个三环。平移最大 3 到末尾后变成[1,2,3]，随后长度 2 与长度 1 的最大项都已经在末尾，发生两次原地动作。初始只有 1 个环，因此“原地次数恰好等于初始环数”不再成立。

但均匀随机排列中，删去最大项后，其余项保留相对次序，仍是均匀随机排列。因此当前 r 项的最大值处在末端仍以 1/r 概率发生，原地次数期望依旧 H_n（若从 r=2 计则 H_n-1）。

两个方法能有相同的平均数，却对同一个具体排列给出不同的次数。证明平均结论应重新基于剩余相对顺序的均匀性，不能拿已经失效的逐输入等式继续推。

24.22 22 · 在线与输入敏感：抓一张牌就插进去

插入排序看见新牌，就在已排好前缀里找位置并插入，任何时刻已收到的牌都有序，所以适合“还会继续来牌”的在线维护。用平衡搜索树接收新键也能在线维护顺序，但结构和输出方式不同。

输入敏感指成本还取决于某种输入难度。插入排序可按逆序对数 I 写成 O(n+I)的常见数组移动界；已经有序时线性，完全逆序时二次。带提前终止的冒泡也对某些接近有序输入更省。

选择排序每轮仍要扫描剩余集合，比较数不因输入已序而减少。它的优势是总交换 O(n)。因此不能只问“哪个排序更好”，应说清比较、移动、在线响应和稳定性分别要求什么。

24.23 23 · 数组插入：二分找到了位置，格子还是要腾开

保存当前值 x，把比 x 大的前缀元素依次右移，最后把 x 填入空位。例[2,4,7,3]：先存 3，7 右移，4 右移，再把 3 写到 2 之后，得到[2,3,4,7]。

逆序输入时，第 i 项要越过前 i 项，移动数是 1+2+...+(n-1)，所以 Θ(n²)。不能像链表那样只改常数条边，因为数组的逻辑连续性正依靠连续格子，必须给插入位置腾空间。

二分能把查位置的比较降到 O(log n)，却不能消除 Θ(n)后缀移动。链表则相反：按秩查位置不便宜，但已知位置后接入节点可以常数改链。每种结构省的工作不同。

24.24 24 · move 保留对象，remove 加 insert 可能重建对象

节点转移先摘下现有节点 x，再接到目标位置；x 的 data 对象可以完全不复制，外部持有的节点地址也保持。remove 若销毁节点再返回值，insert 又新建节点，则增加分配、构造和析构。

对于大字符串、不可复制对象或有身份的记录，这些差别比几条指针赋值大。对于只有几个字节的简单对象，差别可能主要是分配器常数，也应通过实际实现确认。

移动后扫描指针必须指向原后继，而不是沿 x 的新后继回到已经排好的前缀。先保存 next，再移动 x，是避免重复处理和无限循环的关键。

24.25 25 · 插入排序：原地不动几次，比整体有序容易得多

互异均匀随机排列中，完全有序只有 1 种，概率为 1/n!。但某次插入不移动，只要求新项是当前前缀最大值；在前 k 项中，每个位置等概率容纳最大值，所以该事件概率 1/k。

若排序从第 2 项开始，原地插入的期望次数为 $\sum_{k=2}^{n} 1/k = H_n - 1$；若把首项初始化也计一次，就是 H_n。n=4 时，期望为 1/2+1/3+1/4=13/12，不需要接近 1/n!那么罕见。

平均逆序对为 n(n-1)/4，标准插入排序每次右移消除一个相应逆序，所以平均仍 Θ(n²)。偶尔许多项无需移动，与总体二次工作并不矛盾。

24.26 26 · 为什么链表插入排序不直接二分

有序只说明比较后能排除一半候选，不保证可以常数时间跳到中点。链表从当前边界走到中点已经需要一段指针移动，所有层累计仍可能线性；每次从头重新找位置还会增加重复工作。

要真正像数组那样实现 O(log n)定位，需要高效随机按秩访问，或增加可跳跃的索引。同时插入后还要高效维护这个索引，否则定位省下的时间会在更新时付回。

即便对数组采用二分插入，移动仍可能二次；对链表普通顺序找位置虽然比较多，却能常数接链。先把“比较次数”“走链接次数”“移动数据次数”三张账分开，再决定该优化哪一个。

24.27 27 · 逆序对：每一对位置各自交一张账单

互异元素中，逆序是 i<j 却 a[i]>a[j]。共有 n(n-1)/2 对位置，完全递减时全部逆序，这是最大值。

随机排列中任意一对大小方向对称，成为逆序的概率 1/2。令每对的指示变量为 1 或 0，期望相加得 $E[I] = n(n-1)/4$，无需假设所有对之间独立。

例如[3,1,2]逆序为(3,1)、(3,2)，共 2；相邻交换 3 和 1 后剩(3,2)，再交换便有序。普通冒泡每次只消一个逆序；插入右移也能用这些对逐项收费，所以随机平均 Θ(n²)。

24.28 28 · 冒泡只跑 n 趟：先查清最后一趟是不是确认趟

原题长度为 2n，采用课件第 75 页提前终止版。该版在最后一次有交换之后，还要执行一趟“完全没交换”的确认，才让外循环停止。下面把这趟也计入 n，因此有交换的最多左移深度为 $d = n - 1$。

一个元素每趟最多向左移动一格。若需要 d 趟有交换，至少存在一个元素要越过 d 个较大前项，所以逆序至少 d。取前 d+1 项为[2,3,...,d+1,1]，其他值递增，恰有 d 个逆序，并需要 d 次有交换加一次确认。

每个有交换的第 i 趟至多比较并交换 2n-i 对，所以逆序总数至多

$$\sum_{i=1}^{d}(2n-i)=2nd-\frac{d(d+1)}{2}.$$

取前 d 项为最大的 d 个数、递减排列，后面其余数递增：每趟都能把一个大项推到末端，达到这个上界。n=15 时，d=14，最少 14、最多 315。最少例为[2,3,...,15,1,16,17,...,30]；最多例为[30,29,...,17,1,2,...,16]。若采用另一计数习惯，把“n 趟”仅指有交换的扫描，便应取 d=n，得到最少 15、最多 330。不要把两种趟数口径混写；课件这份源码可以实际打印 sorted 来确认。

24.29 29 · 交换远处逆序：能多消一些，但距离不是充分条件

互异数 a>b 位于 i<j。交换后，它们这对逆序消失；中间每个值 x 若满足 b<x<a，又会同时消掉两对逆序。因此逆序减少量为 $1+2k$，k 是中间落在两值之间的元素数。

距离远意味着可能跨过更多合适的 x，却不保证一定多消。例如中间值全大于 a 或全小于 b，仍可能只减少 1。若有重复值，还要单独计与端点相等的情况，不能照搬奇数公式。

任意交换模型下，环分解给出 n-c 次就能排序，至多 n-1。选择排序保证 O(n)次交换，堆排序和快排一般不保证这个线性界；冒泡只用相邻交换，完全逆序需 n(n-1)/2 次。允许交换的位置不同，最少次数也不同。

24.30 30 · 不等长二路归并：仍以两边总长度计上界

两有序段长 m、n。比较两头，输出较小者并推进对应一边；每次至少消耗一项，直到一边空。关键字比较至多 m+n-1，输出和访问总量 O(m+n)，并不要求 m 与 n 接近。

例如[1,100]与[2,3,4,5,6]，先出 1，连续出 2 到 6，最后接 100，长的一侧并没有被反复从头扫描。若链表剩余尾段可整段接上，某些输入的真实工作还可更少。

线性上界仍成立；但不能因此说所有输入都必须逐节点复制尾段。要按接口是否维护 size、尾指针、统计信息，判断一次接链之外还有没有必须访问的工作。

24.31 31 · 节点转移归并：保存 next，稳定地从左边先取

遇到右段 x 小于左段 y，把 x 摘到 y 前；先存 x 在原段的后继，再改链接，后续扫描从保存位置继续。若直接用移动后的 x->succ，它已经指向 y，可能反复处理错误位置。

相等时让左段先出，保留来自原左半的记录在前，才能稳定。对左[2A,4]、右[2B,3]，输出应为 2A、2B、3、4。

remove 加新 insert 也能造出正确值序，但会分配、复制并改变身份。原节点转移的优势是复用资源；如果还有外部节点句柄，语义差别尤其需要写清。

24.32 32 · 向量与列表归并：同为 nlog n，代价落在不同地方

向量连续，扫描常有较好的局部性，按秩分半方便；常见稳定归并需要 O(n)元素缓冲，并多次复制或移动值。列表可以拆链、重接原节点，避免大对象复制，额外元素存储可以很少。

列表节点却可能散在内存，走 succ 会发生缓存失效，节点分配本身也占额外指针空间。找中点若每层扫描，仍能并入每层线性成本，整体 Θ(nlog n)，但常数不同。

比较时使用同一元素类型、相同数据与实际内存布局。只有少量大对象和很多小整数两种场景，得到不同胜负很正常；不能用渐近符号决定所有实测结论。

24.33 33 · 固定 15/16 与每次 n−1：看似都偏斜，结论却不同

按 2/3 与 1/3、15/16 与 1/16，或任意固定 $\lambda:(1-\lambda)$ 切分，两边仍以固定比例缩小，每层总长度 O(n)、深度 O(log n)，整体 Θ(nlog n)。

15/16 那条长路径深约 $\log_{16/15} n$，常数比对半大，却仍是对数。若改成 n-1 与 1，路径 n 层，递推为 $T(n)=T(n-1)+\Theta(n)$，累计二次。

把单项不断与已排序的大段合并，效果接近逐项插入排序。是否从前还是后剥离，决定具体移动方向；判断渐近时关键是每次只减少 1，而非减少一个固定比例。

24.34 34 · 归并数逆序：右项先出时，整批左项一起记

左[2,5,7]、右[1,4,6]。先出 1，左边剩 3 项都比 1 大，增加 3；出 2 不增；出 4 时左边剩 5、7，增加 2；出 5 不增；出 6 时剩 7，增加 1，总跨段逆序 6。

递归先数两半内部逆序，再加跨段逆序，每层只扫线性，总 O(nlog n)。相等时先取左项而不加逆序，因为逆序要求严格大于。总数最大约 $n^2/2$，累加器要有足够位宽。

还可压缩秩后用树状数组：从右向左读，每次查询已有多少个更小秩，再插入当前秩，同样 O(nlog n)。若小整数值域允许更专门的操作，复杂度可变化；在一般比较模型里不应无条件宣称线性。

24.35 35 · 游标链表：数组下标充当地址

节点放进数组，每格保存 data、pred、succ，succ 是下一个节点的下标。另用一条 free 链管理空闲格。申请时从 free 头摘一格，释放时把该格接回 free 头。

例如有效链 2⟶5⟶1，物理下标既不递增也不连续。删除 5 只改 2.succ=1、1.pred=2，再回收 5；不需要把数组第 6 格以后整体前移。

它适合固定池、需要序列化引用或不便动态分配的场景。回收后的下标若被外部继续使用，可能指向后来新建的另一个对象；可以加代际号或限制句柄生命期。游标替代地址，不会自动解决悬空身份问题。

24.36 36 · Java：垃圾回收不会帮你接正确的链

interface 规定可调用能力，implements 表示类提供这些方法。Java 变量常持有对象引用，不像 C++某些写法那样直接复制整个节点对象；模板与泛型的具体机制也应按语言规则区分。

摘掉节点后，若没有任何可达引用，垃圾回收最终可释放它；若你仍无意保留一条引用，它可能继续存活。GC 不会自动把前驱接到正确后继，也不会修复环、错误 size 或逻辑重复。

对照课程版本时，把插入后前后遍历结果、size、外部引用指向对象身份逐项比较。运行测试只能报告实际运行的平台与版本，不能把“语言有 GC”当作算法正确性的证明。

24.37 37 · Python list：名字叫 list，底层却可能是数组

Python 内置 list 是动态数组，尾部追加或弹出通常有摊还常数表现，头部插删需要移动后缀；课程自己实现的链式 List 是另一类型。先确认 reverse.py 操作的是哪一种对象。

双指针原地反转只交换首尾，再向中间收缩，时间 O(n)、额外空间 O(1)。若递归版本每层切出剩余子串或子列表，再与首项拼接，每层复制长度 n、n-1……，就会出现二次时间和大量临时对象。

同样是“递归反转”，若仅传左右下标、不切片，时间仍可线性，只增加递归栈。解释性能差距时要读具体切片、拼接和访问成本，不能把原因缩成“Python 慢”或“递归一定慢”。

24.38 本章练习与详细答案

1. 对[2,3,1]画出置换环；任选一次把元素放回最终位置的交换，环数怎样改变？
2. 交换[5,2,4,1,3]首尾的逆序 5 与 3，会消掉几个逆序？先用中间值判断，再枚举核对。
3. 为长度 6、含最后确认趟共 3 趟的提前终止冒泡，各构造最少和最多逆序的输入。

第一题：位置 1⟶2⟶3⟶1，一个三环。交换位置 1 与 3 得[1,3,2]，拆成固定点 1 和环(2,3)，环数增加 1。

第二题：端点之间仅 4 严格位于 3 与 5 之间，故减少 1+2=3；原逆序为 7 个，交换成[3,2,4,1,5]后剩 4 个。

第三题：d=2，最少[2,3,1,4,5,6]含 2 逆序；最多[6,5,1,2,3,4]含 9 逆序，分别两趟交换后再一趟确认。通过逐趟打印数组，可检查自己有没有漏数确认趟。

25 PS04：先把栈里的东西写出来，再谈递归、表达式和单调结构

教师原始习题单：第 04 次习题共 35 题。表达式题采用课件第 41—42 页的实际优先关系表来解释原程序；特别提醒：原表的 ^ 对 ^ 是先归约，因此原程序是左结合。数学惯例通常右结合，修改版要明确说明，不能把两者混成同一个答案。

25.1 1 · 向量做栈：把栈顶搬到头部，哪些成本跟着变

原 Vector 尾端作为 top，push 就是尾插，pop 就是尾删；没有扩容时只改末项，几何扩容下 push 摊还 O(1)，top 读末项 O(1)。

若把首端作为 top，每次 push 必须把旧 n 项右移一格，每次 pop 把剩余项左移，单次 Θ(n)。连续压入 n 项，移动量 0+1+…+(n-1)，总 $\Theta(n^2)$。top 仍是读首项，仍 O(1)。

栈的“后进先出”规则没有变，改变的是底层接口代价。实现一种 ADT 不只要输出顺序正确，还要检查承诺的时间复杂度。可以用[10,20,30]各压弹一次，画出每次真正移动的格子。

25.2 2 · 列表做栈：双向与单向不能混答

双向表有首尾哨兵和尾指针，两端插入、删除都能常数改链，交换 top 与 bottom 的角色不会改变渐近阶。节点分配和缓存常数仍可能不同。

单链若有尾指针，尾插可以 O(1)，但尾删还要找到前驱，一般 O(n)；首插、首删则 O(1)。所以“列表两端对称”需要双向链接或额外结构，不是所有叫 List 的类型都成立。

读示例项目时找 push、pop 实际调用哪个接口，再看那个接口需要多少次走链接。先确认表示法，才知道对称性是不是实质的。

25.3 3 · 迷宫里的粉笔：调用栈不会记住所有来过的地方

调用栈保存“当前还没做完的路径”。函数返回后那一帧就消失，所以以后从另一路走到同一图节点，栈未必知道它曾被处理。

要避免图中的重复或环，可以把 visited 放在节点字段、外部数组或哈希集合里。C++并没有自动防止递归环的内建机制；静态或全局变量能跨调用保存状态，但也可显式传入一个共享的访问表，并非只能靠全局变量。

例如 A→B→A，进入 A 就标记，再进入 B，看到 A 已在当前访问路径上便识别环。只在返回时标“处理完成”，却不区分正在访问的灰色状态，可能还没返回就再次递归 A。粉笔记录的状态含义需要明确。

25.4 4 · 每帧减半，理想规模为什么可能平方

设调用栈预算 B，每帧 s 字节，允许深度约 B/s。固定比例分治若深度 $h = c\log n$，在同一栈预算下帧减半，允许深度翻倍，故 $c\log n' \approx 2c\log n$，得到 $n' \approx n^2$。

减到十分之一则允许约 10 倍深度，理想上 $n' \approx n^{10}$。这不是“内存容量变成原来的平方”，只是对数深度与输入规模之间的指数换算。

若算法实际是线性递归 h≈n，则帧减半只能把规模约翻倍。若输入数组、输出和堆缓冲占用了主要空间，也不能按这个理想算式承诺真实机器能力；题目已明确暂忽略这些消耗。

25.5 5 · 递归都能消去，但未必能消掉需要保存的信息

普通递归可以用显式栈模拟：每个帧保存参数、尚需使用的局部量、返回后该接着执行哪一段。while 循环读取栈顶状态，遇到递归调用就压一个新帧，遇到返回就弹出并把结果交给父帧。

例如二叉树后序遍历，帧需说明“左边还没做”“左边已做、等右边”“左右都完成”；只保存节点地址而不保存阶段，就不知道返回后该去哪。

这消除了语言层面的递归调用，却仍可能使用 O(h)辅助栈。一般程序需要记住哪些待办任务，是问题本身的信息需求，不会因为把函数改成 while 就自动变成 O(1)。

25.6 6 · 第二个递归是尾调用：可以省那条等待链

若某函数先递归处理左半，左半返回后直接返回右半的递归结果，右半调用是真尾位置，可以把当前帧参数改成右半并跳回循环起点，第一支仍正常递归。

这能减少“父帧等第二支结束”的堆叠，但第一支的深度仍存在。对快排一类过程，常见办法是递归较小一侧、循环处理较大一侧，可控制同时活跃的递归深度；需要证明每次递归那侧至多减半。

如果右递归返回后还要合并左右结果，它就不是尾位置，不能直接覆盖父帧；必须保留左结果和后续合并任务。看源码中的“调用以后还有没有实际需要的工作”，而非只看它是不是第二个调用。

25.7 7 · 阶乘：把待完成乘法装进一个累加器

普通阶乘 `fac(n)=n*fac(n-1)` 返回后还要乘 n。引入 acc，保持不变量“acc 乘上尚未处理的 1 至 n，等于原目标阶乘”，便能迭代：

```
def factorial(n):
    if n < 0:
        raise ValueError("阶乘输入须为非负整数")
    acc = 1
    while n > 1:
        acc *= n
        n -= 1
    return acc
```

n=4 时，(acc,n)依次(1,4)⟶(4,3)⟶(12,2)⟶(24,1)。固定宽度模型下只用常数个变量；若使用任意精度整数，保存 n!本身需要的位数仍在增长，不能叫实际字节 O(1)。

推广要检查剩余操作能否以固定大小状态组合，顺序能否改变、副作用是否允许。浮点乘法重排可能改舍入，打印输出的顺序也不能随意交换。“递归后只剩一行”不等于可以自动塞进一个标量。

25.8 8 · 头递归：参数推进也算计算

进制转换先调用 `convert(n/base)`，返回后输出 n%base。它的主要输出在递归之后，所以常被归为头递归；但实参中的除法已经使问题缩小，并不是调用之前完全没有计算。

如果所谓“纯头递归”是状态与参数完全不变，就再次调用同一个实例，那么确实不可能靠自己接近基例。例如 `f(n){if(n==0)return; f(n);}` 对 n>0 不断压栈。若参数改成 n-1，便已经有了推进，不能套这条断言。

去递归时，只需保存将来输出所需的余数，而不必保存编译器帧中的所有机器细节。例如十进制 13 转二进制，依次保存余数 1、0、1、1，反向弹出 1101。保存什么，取决于返回后还要用什么。

25.9 9 · 单链析构：后序删与顺序删都能线性

头递归可先销毁后继，再 delete 当前；它保证每个节点只删一次，但长 n 的链会同时压 n 层帧。调用前必须先判断 p 是否为空或尾哨兵，实参 p->succ 的读取本身也是工作，不属于“毫无状态推进”。

迭代版先保存 next，再释放当前，再 p=next。它不用返回旧节点处理剩余事，所以可常数辅助空间，时间仍 Θ(n)。释放前保存后继，释放后绝不能再从 p 读链接。

如果数据元素的析构必须遵循某种业务次序，头到尾与尾到头可能有语义差别；普通独立节点可选迭代，但不能把所有带副作用的析构顺序视为无关。

25.10 10 · 自定义括号：先分清字符串，再谈栈

词法阶段把输入识别成左记号、右记号与普通内容，栈保存(类型,起始位置)。遇左记号压栈；遇右记号，若栈空就报告多余关闭，类型不同就报告当前位置与期待的关闭类型；输入结束还剩左记号则报告未关闭的起点。

例如 `<a>[x)</a>`，读到右圆括号时，栈顶是方括号起点，应说明“这里期待]，它与位置 4 的[对应”。这比单纯返回 false 更能帮助修改。

JSON 字符串中的 "[" 是内容，不是结构括号，反斜杠转义也影响引号结束位置。HTML 还涉及空元素、属性、注释与复杂语法，简单标签栈不等于完整 HTML 解析器。扩展算法要先说支持哪一种受限语言，不能只按字符匹配就宣称解析所有格式。

25.11 11 · 加法结合方向：表中同类相遇的那一格决定顺序

课件 + 对 + 为大于，意味着栈顶先归约。2+3+4 读到第二个+时，先算 2+3=5，再把+压入等待 4，得到(2+3)+4。要改成 2+(3+4)，把这格改为小于，让新+先入栈，之后从右边归约。对只含合法加法的表达式，主循环不需换框架；若同时有减法，则必须检查相同优先级的交叉格，不能只改一个角落却声称整个语法都一致。精确整数无溢出时两者同值，浮点却不保证结合律。时间仍线性，但连续右结合运算符会积在栈里，某些输入的峰值空间可由常数增加到线性。只说“时间空间都没变”会漏掉这一输入族上的变化。

25.12 12 · 原程序的乘方是左结合，修改后才符合通常数学惯例

原表 ^ 对 ^ 为大于，因此 2^3^4 先算 2^3=8，再算 8^4=4096。通常数学写法解释为 $2^{(3^4)} = 2^{81}$，两者差很多。必须先报告原表实际做什么，再说要怎样修改。

把这格改为小于，连续^先入栈，最后从右向左归约。读取、弹出和计算框架不必改变，但测试应同时包含括号、阶乘与多个乘方；C++里的字符^本身是按位异或，这里只是自定义表达式语言的乘方 token。总时间依然 O(N)，最坏总空间依然 O(N)；但无括号纯乘方链原先可即时归约，改右结合后会积压 O(N)个运算符。复杂度类别与具体输入上的峰值，应分别说明。

25.13 13 · 右括号那一行为什么全空

当前右括号遇到左括号时只匹配并消去，不作为待执行运算符长期压栈。因此“栈顶已经是右括号，再看下一个 token”的状态不应出现，表中对应整行为空。

另一些空格表示语法不合法，例如没有左括号却遇到右括号，或输入结束还有未配对左括号。它们不是“优先级差不多，随便处理”，而是应报告的错误。

区分三种表项：小于表示移进，大于表示归约，匹配标记表示相消；空白表示不允许。若程序默认输入合法，可以断言；面向真实输入的解析器应返回位置和原因。

25.14 14 · 双阶乘：6!!不是(6!)!

定义 $n!! = n(n-2)(n-4)\cdots$，到 1 或 2 停止，并约定 0!!=1。6!!=6×4×2=48；(6!)!完全是另一个大得多的数。间接方案把一个新 token 映射成一元后缀双阶乘，计算器按步长 2 累乘。它与阶乘一样只弹一个操作数，必须检查非负整数定义域和溢出。直接读!!则需词法分析的最长匹配：先看下一个字符是否也是!，一次识别双阶乘。

若语法还允许两个连续普通阶乘，必须额外规定用括号如 `(6!)!` 表示，否则同一字符流有歧义。3!!!也要说明是 3!!后接!，还是别的扩展；不能依靠“人看着懂”让程序猜。

25.15 15 · 操作数顺序：先弹右边，后弹左边

读完 3、8 后，栈底 3、栈顶 8。遇减号先弹 b=8，再弹 a=3，算 a-b=-5；如果反过来做 b-a，就变 5。加法乘法的交换律会掩盖这个错误，所以测试要包含减、除和乘方。

一元阶乘只需一项；二元运算需两项；栈不足必须在执行前报告。除数为 0 也要先判断，不应偷偷压 0 继续算，因为后面看似合法的结果会隐藏真实失败。

整数截断、精确有理数、浮点是三种数值语义。第 30 章的表达式核对采用精确有理数；若课程 double 求值，最后小数应允许相应舍入，不把两种执行结果机械按字符串判等。

25.16 16 · 运算符种类有限，栈为什么仍能任意深

相邻待处理符号能同时入栈，是因为前一符号相对后一符号为移进关系。这个表关系不传递：+对(为小于，(对+也为小于，但+对+为大于。因此不能说“每层严格升高，符号只有九种，所以最多九项”。

构造很多尚未闭合左括号，栈就能线性深；运算数栈也能用嵌套的 `1+(1+(1+...))` 积累待算左值。输入结束前某个时刻的峰值需要计入，不能只看最终剩一项。

若问原优先表下“达到峰值 n 最少多少对括号”，计入底部结束哨兵时，一层括号内最多可暂留+、*、^、！四个不同优先级运算符；!后不能直接接(，所以每增加一个嵌套层最多增加四个待留项。无括号最多 5 项（含哨兵），故下界是 $\max(0, \lceil (n-5)/4 \rceil)$ 对，利用 `0+0*0^(0+0*0^0!)` 这类递归嵌套可达到相应峰值，其余 n 用较短运算链补齐。

这里把“补齐”实际做一遍。含底部哨兵时，下列四个式子都只用一对括号：

表达式	读到最内层!、尚未归约时的运算符栈（底→顶）	峰值
`(0+0*0^0!)`	`# ( + * ^ !`	6
`0+(0+0*0^0!)`	`# + ( + * ^ !`	7
`0+0*(0+0*0^0!)`	`# + * ( + * ^ !`	8
`0+0*0^(0+0*0^0!)`	`# + * ^ ( + * ^ !`	9

每加一个完整外层，外层可留下+、*、^三个运算符，再加左括号，共增加 4 项。!是后缀符号，原表不允许!后直接移进左括号，不能让每层都多留一个!再继续往里嵌套。对更深的目标，重复完整外层后，在最外层选上表对应的短前缀，就能补齐余数；例如峰 10 可在峰 9 的式子外再包一对括号。

若不计底部哨兵，最大峰变成 4P+4，最少括号相应为 $\max(0, \lceil (n-4)/4 \rceil)$；若先把^改成右结合，纯乘方链就能不加括号积到任意深，刚才的最少括号式不再适用。回答本题必须带上表版本与栈计数口径。

25.17 17 · 转 RPN：每个运算符都应等自己的孩子先输出

把表达式看成树，数字是叶，运算符是内部节点。后序遍历先输出左子树、右子树，再输出父运算符，就是 RPN。按树规模归纳：两个更小子表达式各自输出正确，接上运算符，就恰好表示它们的组合。

在线栈算法还没看到右操作数完整范围时，先保存运算符；只有后续 token 说明该子表达式已闭合，才归约输出它。因此数字顺序不变，运算符输出顺序遵从相同的树依赖。

例 *2(3+4)*：读 *2* 先输出，等待；括号内 3、4 先输出，再输出+；括号闭合后最后输出*，得到 `2 3 4 + *`。手工画树是整体分析，栈法是逐 token 保存尚未确定的工作，二者得到同一后序。

25.18 18 · 转换也要线性，RPN 还有什么价值

只计算一次时，中缀解析加 RPN 求值仍是线性，并没有把 O(N)降到更低阶。它的收益是将“理解括号与优先级”同“执行运算”拆开。

如果一个带变量表达式需要对一万组输入求值，语法只解析一次，后面反复执行简单指令序列；虚拟机也可以让每条指令注明弹入几个值。错误位置和类型检查可在转换阶段先完成。

这是一种可复用表示，而非复杂度魔法。若表达式只用一次、程序很小，直接求值也可能更简单；应根据任务选择，不必把 RPN 描绘成所有情形都更快。

25.19 19 · 21 个长表达式：用短片段先确定读法

完整 21 式的 RPN、数值与逐步栈追踪在第 30 章。这里先拆第一式：括号内 0+1+2-3+4+5+6 按左结合依次输出 `0 1 + 2 + 3 - 4 + 5 + 6 +`；另一个括号输出 `7 8 +`；两段相乘后输出，再读 *9*，最后输出。

合起来是 `0 1 + 2 + 3 - 4 + 5 + 6 + 7 8 + * 9 *`。数值第一括号 15，第二括号 15，所以 15×15×9=2025。转换与求值要分别核对：一个是 token 顺序，一个是算术结果。

含!!的式子不能逐字符当两个!；含连续减除的式子要按左结合；第 30 章统一用右结合乘方，但这 21 式没有无括号连续^造成的冲突。若另加 `2^3^4` 作为测试，必须标明使用原表还是修改表。

25.20 20 · 前缀 PN：从右往左读也能用栈

前缀 `- 8 * 2 3` 表示 8 减 2×3。倒序扫描 3、2 入栈，遇*弹出的第一个是左操作数 2、第二个是右操作数 3，压 6；再读 8 入栈，遇-弹 8 与 6，压 2。

这和 RPN 从左读时“先弹右操作数”的顺序恰好不同，因为扫描方向变了。固定元数让程序知道每个运算符要几个孩子；单目、双目应在 token 定义里标明。

每个 token 进出常数次，时间 O(N)，最坏栈 O(N)。也可递归按“运算符后接几个子表达式”解析，仍需检查 token 耗尽、操作数不足和最后剩余多份结果。

25.21 21 · 禁形：晚读入的大数，把中间数压在了小数上面

统一按输入 1、2、…、n 依次压入，输出按实际弹出顺序记录。设 i<j<k，目标输出却含 k...i...j。为了先弹 k，必须已经把 i、j、k 按输入顺序读入；若 j 尚未弹出，它就在 i 上面，因此 i 不能先于 j 弹出，矛盾。特例 j+1...i...j 只是令 k=j+1，机制相同。它们说的是三个元素的相对次序，不要求三个在输出中紧挨着。例如想输出 3,1,2：压 1、2、3，弹 3 以后栈顶是 2，要得到 1 必须先弹 2，却又要求 2 在 1 后面，卡住。输出 3,2,1 则可行。若教材图把最终输出画成另一只栈，视觉阅读方向可能反着，需要先换成实际弹出顺序，再使用禁形名称。

25.22 22 · 依最后或最先输出分类：两种视角不必有同样简单的公式

设最后输出 k。所有小于 k 的元素必须在 k 入栈前弹完，否则被 k 压住以后就不可能赶在最后 k 之前弹出。大于 k 的元素在 k 上方完成另一套独立栈过程，因此输出分为“小于 k 的混洗、大于 k 的混洗、最后 k”。两块分别有 C_{k-1}, C_{n-k} 种，分类互斥且穷尽，组合可逆，于是

$$C_n = \sum_{k=1}^{n} C_{k-1} C_{n-k}, \qquad C_0 = 1.$$

按第一个输出 k 分类时，要先压 1 至 k 再弹 k，剩余栈有 k-1 项，还有 n-k 个输入没读。后续不是两块简单相接，应该数从高度 k-1 回到 0、始终非负的入出栈路径。

令 u=n-k、h=k-1，可用完成数 F(u,h)递推：允许再压入时加 F(u-1,h+1)，栈非空允许弹时加 F(u,h-1)，u=0 时只能依次弹完，完成数为 1。也可由反射法得

$$F(n-k, k-1) = \binom{2n-k-1}{n-k} - \binom{2n-k-1}{n-k-1},$$

越界二项式视为 0。n=3 时按首项 1、2、3 分别为 2、2、1，总 5。不能为了得到熟悉的 Catalan 卷积，硬把这次分类也拆成并不存在的两块。

25.23 23 · CSTDSA：两个 S 不能直接让 Catalan 数除以 2

先给两个 S 编号，输入顺序 C、Sa、T、D、Sb、A。最后弹 Sa 时，C 不可能留在 Sa 下面等最后才出，所以 C 必须先出；中间 T、D、Sb、A 可独立混洗，故这类有 $C_4 = 14$ 种。

最后弹 Sb 时，A 在它上面，必须紧挨末尾先弹 A；更早四项已经完成，所以这类也是 14 种。再要求以 C 开头、末两项为 A 和 Sa 或 Sb，分别留下 3 项自由混洗，各 $C_3 = 5$ 种。

去掉 Sa、Sb 身份，只显示 S 后，这两组会有重叠。交集的可见输出是 CDTSAS、CSDTAS、CTDSAS、CTSDAS，共 4 种；于是末项为 S 的不同可见字符串为 14+14-4=24。两个特定 5 元素集合的对称差有 2 种，交集 4 种；各自独有 1 种。

可以用递归枚举全部 132 条六元素合法栈操作，再分别放入集合核对。去重的作用发生在输出字符串层面，不是在操作路径层面。因为重复字符的两种标号不一定都对应合法过程，直接 132 除 2 没有依据。

25.24 24 · 栈混洗判定：当前目标不在顶上，只有一个可能的补救

按目标输出逐项处理。如果栈顶就是目标，立刻弹；否则继续从输入压入，直到目标成为栈顶。若所有输入已压完仍不匹配，之后没有新动作能把埋在下面的目标提前取出，判失败。

贪心正确性来自动作被迫：想输出当前目标，栈顶若是其他值，不能把它先弹到输出；唯一可尝试的是继续压尚未读入的值。如果目标仍在输入里，压到它会成功；如果它已被压在错误的上层元素下面，过程就会卡住。

虽然外 for 最多 n 轮、某次 while 也可能压 n 项，但每个输入只压一次，每个输出只弹一次，内循环跨所有外层的总次数最多 n，整体 O(n)。二重循环不能不看变量变化就相乘。

25.25 25 · 括号与混洗：对应的是合法操作路径

入栈写(，出栈写)。任何前缀里出栈次数不能超过入栈次数，所以左括号数不少于右括号数；结束时各 n 个。这恰好是 n 对合法括号串的条件。

反向读一个合法括号串，遇)就压下一个输入，遇(就弹栈，非负前缀保证不会从空栈弹出。这个过程唯一，因此合法括号串与合法入出栈操作路径一一对应。

对互异且输入顺序固定的元素，可行输出也唯一决定其被迫的入出过程，于是输出混洗数等于 Catalan 数。两个字符相同后，可见输出可能由多条路径得到，刚才的 CSTDSA 正说明此时不能再把输出字符串与括号串一一对应。

25.26 26 · 枚举、第 k 个、均匀抽样：先知道每条分支下面有多少串

设已用 o 个左括号、c 个右括号。o<n 时可加左括号；c<o 时可加右括号。完成数 A(o,c)满足这两个合法后继的和，终点 A(n,n)=1，其余非法状态为 0。用表或记忆化计算，共 $O(n^2)$个状态。

按约定的字典序“(在)前”深度优先枚举，n=3 依次得到((()))、(()())、(())()、()(())、()()()。取从 1 编号的第 k 个时，先看左分支完成数 L：k≤L 就走左，否则 k 减 L 再走右。例如第 4 个跳过左分支的前三个，进入以()开头的部分，最终为()(())。

均匀随机抽样也按完成数分支：在总数 L+R 里均匀取整数，小于 L 选左，否则选右。每一整串的路径概率相乘后恰为 1/Cn。若每次在可选的左右之间直接各取一半，不同串经过的自由选择次数不同，概率会不等；n=3 的((()))可有 1/4，而(()())仅 1/8。

大数需要大整数计数和无偏随机整数，不能用溢出的机器整数或有偏取模。这里给出一套可验证的动态规划方案，解决原题三项任务；它不是把 Knuth 所列 P/U/W 算法的原始实现逐行复刻，阅读原书时还需比较其不同优化目标。

25.27 27 · 循环向量队列：出队只移动头，不搬整队

保存 head 和 size，容量 C>0 时，第 i 个逻辑元素位于 `(head+i)%C`。入队写到 `(head+size)%C` 后 size++；出队读 head，再令 head=(head+1)%C 并 size−。

容量 4，依次入 A、B、C，出 A、B，再入 D、E，物理数组可能是[E,旧 B,C,D]，逻辑顺序却是 C、D、E。旧格中留下的字节不等于仍有有效元素，size 决定有效范围。

扩容时按逻辑顺序把两段复制到新数组 0 起点，head 重置 0。记录 size 可区分 head==tail 时究竟空还是满；也可以保留一个空位，但容量含义改变。容量 0、恰好满、绕回和扩容后顺序都要测。

25.28 28 · FIFO 应用：按到达次序处理的真实任务

BFS 把刚发现的节点加入队尾，从队首取下一个处理，因而先处理距离近的一层。打印任务、生产者消费者缓冲、离散事件里同一优先级的等待队列，也常使用先进先出。

但“最早到达”与“最紧急”不同。需要先处理最高优先级时应使用优先队列；需要先处理时间戳最小事件时，时间顺序不等于消息到达顺序，也可能要堆。

设计应用例子时写清入队事件、出队事件、满时如何处理。只报“操作系统用队列”没有说明它在保存哪种待办任务。

25.29 29 · 双栈当队：三种证明的常数为何不一定一样

in 接收入队，out 负责出队；只有 out 空时才把 in 逐项弹出并压进 out。一个元素最多入 in 一次、从 in 弹出一次、入 out 一次、最后从 out 弹一次，总共常数个栈动作，所以任意 m 次合法队列操作总 O(m)。

势能取 $\Phi = 2|in|$：入队实际 1、势能增 2，摊还 3；out 不空出队实际 1、势能不变，摊还 1；倒 k 项再出队实际 2k+1，势能减 2k，摊还仍 1。

记账法若每次入队收更多费用，会得一个更松的常数；聚合法也可能把尚未出队元素按全流程预收。只要是上界，它们不必精确相等。比较常数前先统一 push/pop、空检查、返回值复制是否计费，而非看到 2 与 3 就认定证明互相矛盾。

25.30 30 · 最大直方图矩形：面积相同，左边界也要比

每个柱宽 1、高非负。矩形覆盖半开区间[l,r)，高度不能超过区间最矮柱，面积为 $(r-l)\min h$。逐根柱子向两边扫描直到更矮处，单调输入可使总扫描 Θ(n²)；枚举所有区间又每次重扫最小值的另一种朴素写法甚至 O(n³)，不要混为同一算法。

“最靠左”应明确定义为同面积时 l 最小；若 l 也相同，再规定 r 最小或最大，本章取 r 最小。更新答案时比较(-area,l,r)的字典序，或写等价的逐项条件。

例如[2,0,2]，左右两个单柱面积都为 2，应选[0,1)。有的栈实现先弹右边候选，仅用面积严格大于更新就会留错；有的扫描顺序恰好先见左侧，又碰巧符合，不能把偶然顺序当证明。课件 Excel 的实际选择应逐步运行核对，本书没有把未运行的演示声称为已验证。

25.31 31 · 单趟比两趟多做了什么即时工作

两趟方案先算每根柱左侧最近更矮位置，再反向算右侧边界，最后统一算面积。它便于把所有边界画在表上，但需要存储相应数组或中间状态。

单趟保持递增栈，当前更矮柱一到，就确定被弹柱的右边界，立即结算面积。对[2,1,5,6,2,3]，读到后面的 2 时，先弹 6 得面积 6，再弹 5 得宽 2、面积 10，不必等整份右边界数组建完。

单趟可流式前进、较早给出已结算候选，但仍可能把一整段递增输入存进栈。流式不等于常数空间，也不表示未来不可能出现更大的全局答案。

25.32 32 · 单趟线性：一次 while 很长，不代表每根柱都很长

聚合法：每根真实柱恰入栈一次，最多出栈一次，所以总 push≤n、总 pop≤n；外扫描 n 步，合计 O(n)。结尾追加高度 0 只负责把未结算部分清出，还需明确相等高度的保留规则。

记账法：每次入栈收两份，一份支付本次 push，一份存给未来 pop；从未弹出的柱保留信用，也不影响上界。每根柱收费常数，所以总线性。

势能法取栈大小。一步弹 k 根再压当前根，真实栈成本 k+1，势能变化 1-k，摊还为 2；再加常数比较等控制成本，仍常数。若把一次弹栈的比较也逐项收费，改变固定系数即可。这个证明完全不需要随机高度。

25.33 33 · 单时刻平均与全程峰值：不能把期望搬进 max

严格递增输入会让 n 根柱同时留在栈，最坏 Θ(n)；严格递减时每次弹掉旧项，最好峰值 O(1)。整个运行空间是 $M=\max_{1\le r\le n} S_r$，其中 Sr 是第 r 根真实柱处理后的栈长，最后清空之前的状态也必须算。

假设高度互异、相对顺序为均匀随机排列。栈恰保存此前缀从右往左看到的记录最小值；一个后缀长度 j，其中最左项成为最小值的概率 1/j，所以 $E[S_r]=H_r$。

要证明峰值期望也为 Θ(log n)，还需控制“某一步偶然特别大”的概率。先补上乘积公式所需的独立性：从右向左读一个长 r 的均匀随机排列，把第 j 个读到的数在此前 j 个数中的名次记为 R_j，最小者名次为 1。每个 R_j 可取 1 到 j。

为什么这些名次独立？给定一组 $R_1,\dots,R_r$，从 j=r 往前恢复：先在尚未使用的 r 个数中取第 R_r 小者，作为最后读入的数；删掉它，再在剩余 r−1 个数中取第 R_{r-1} 小者，依此继续。每组合法名次恰好恢复一个排列。名次组合一共 $1\cdot 2\cdots r=r!$ 组，而每个排列等可能，所以联合概率为 $1/r!=\prod_{j=1}^{r}(1/j)$。这就证明各名次独立且分别均匀；只证明单个事件概率 1/j，本来不足以得到独立性。

令 $B_j=1$ 表示 $R_j=1$，即这次读到了新的最小记录，否则为 0。于是这些 B_j 独立，且 $P(B_j=1)=1/j$；栈长 $S_r=\sum_{j=1}^{r}B_j$。独立变量的指数期望可以相乘，单项为 $E[e^{B_j}]=1+(e-1)/j$，因此

$$E[e^{S_r}]=\prod_{j=1}^{r}\left(1+\frac{e-1}{j}\right)\le e^{(e-1)H_r}\le Cr^{e-1}.$$

由 Markov 不等式，$P(S_r\ge t)\le Cr^{e-1}e^{-t}$；对所有 r 加起来，$P(M\ge t)\le Cn^e e^{-t}$。取 $t=A\ln n+s$、A>e，尾概率随 s 指数衰减，尾和给出 $E[M]=O(\log n)$。另一方面 M≥Sn，所以 $E[M]\ge H_n$，得到匹配下界。这些步骤解释了为什么答案成立，也暴露它依赖的随机模型。重复、相关或刻意排序的高度不能自动引用此概率界；最坏保证仍是线性。

25.34 34 · MaxQueue 摊还时间：每项只可能离开一次

影子队列按值单调递减。新项 x 到来，把尾部不可能在未来胜过 x 的较小项删掉，再把 x 接尾；原队列出队时，若影子头正对应离开的项，也删头。用索引或计数处理重复值，避免删错同值的另一项。

每项进影子一次，之后要么从尾被更大新值淘汰，要么从头因过期离开，至多一次。m 项总操作 O(m)，所以摊还 O(1)，不需要题名 Expected 暗示的随机假设。

记账法给每次加入预存一次未来删除费；势能取影子长度，一步删 k 项再加 1 项，实际 k+1 加势能 1-k 为 2。若维护原队列 X，其常数入出也另加，仍不改变摊还界。

25.35 35 · MaxQueue 空间：一扇窗口的平均，与长时间的最大值

固定窗口长 w，窗口内值独立连续同分布时，影子保存后缀最大记录。距窗口右端 j 项的位置成为记录的概率 1/j，影子在这一固定时刻的期望长度为 Hw，即 Θ(log w)。严格递减的一窗却会保留 w 项，最坏为 Θ(w)。

若观察越来越长的整段运行，不能仍把峰值期望写成 Hw。取互不重叠窗口，每窗恰好递减的概率 1/w!；观察 q 窗至少一次递减的概率为 $1-(1-1/w!)^q$，q 增加时趋近 1，因此固定 w 下的长期峰值趋近 w。另外，若原队列 X 显式保存全部 w 项，它本身就占 Θ(w)，即使影子平均仅 log w，整个数据结构也不是总共 log w 空间。需要明确题目是在分析额外影子，还是包含原始元素的完整队列。

25.36 本章练习与答案

1. 原优先表和右结合修改表分别如何转换 `2^3^2`？写 RPN 与数值。
2. 高度[2,1,2]的最大矩形覆盖哪里？把每次弹栈对应的宽度写出来。
3. n=3 按第一个弹出元素分类，为什么不是三个类别都一样大？

第一题：原表 `2 3 ^ 2 ^`，值 64；修改表 `2 3 2 ^ ^`，值 512。第二题：最后 0 清栈时，中间高度 1 的柱可以扩到[0,3)，面积 3；高度 2 的左右单柱面积各 2，所以最佳是整个区间，高度 1。第三题：先弹 1 后剩 2 项自由混洗有 2 种；先弹 2 后还有两种合法完成；先弹 3 时栈中剩 2 压着 1，只能 2 再 1，所以 1 种，总 5。分类计数要由剩余状态决定，不能凭对称感猜。

26 PS05：树的箭头、遍历顺序和编码成本，一步一步核对

教师原始习题单：第 05 次习题共 37 题。路径长度按边数，根深度 0，叶高度 0，空树高度约定为-1。先序为根左、右，中序为左、根、右，后序为左、右、根。“真二叉树”指每个内部节点恰有两个孩子，不等于每层都满。

26.1 1 · 路径按边计：空走一步都没走，就是 0

节点到自己的路径长 0。u 到 v 长 a，v 到 w 长 b，若可沿树中同一路径接起来，总长就是 a+b，不需要因为连接点 v 被数了两次再减 1。

例如根 R 到 A 一条边，A 到 B 一条边，R 到 B 深度 2。若按节点数，前两段各有 2 个节点，连接后却是 3 个，公式总要额外修正；按边计数与“每过一条链接深度加 1”一致。

计数约定本身都可以定义，但后续高度递推、路径拼接、复杂度边界必须统一。看到别的书叶高 1 时，先做整体偏移换算，再判断公式是不是矛盾。

26.2 2 · 深度加高度不超过全树高度

设 v 深度 d，从根到 v 有 d 条边；v 的最深后代距 v 为 h(v)，拼起来就是根到某个叶的路径，长 d+h(v)。全树高度 H 是所有根叶路径的最大值，因此 $d + h(v) \le H$。

若 v 的子树中含有全树某条最深路径，取等；若 v 在较短的侧枝，便可能严格小于。例：根左链深 4，右孩子就是叶。右孩子 d=1、h=0，总 1<4；左链中任一节点的深度加剩余高度都等于 4。

不能把全树最长路径误当“必须经过每个节点”。这条不等式是在比较 v 所在的那条候选路径与所有候选的最大值。

26.3 3 · 三种树表示：多存一条边，省下哪种搜索

只有 parent 时，判断 u 是否为 v 祖先，可从 v 向上走到根，最坏 O(h)。接入新节点时设置 parent 即可，但若要列出它的所有孩子，就可能扫描全体节点。删除整棵子树若无孩子索引，也需要先找到哪些节点属于它，不能只看摘根那一步。

只有孩子列表时，从 u 向下 DFS 能判断 v 是否属于其子树，最坏 O(size(u))；要从全树删除子树根，还需知道它在父节点孩子列表中的位置，若没有 parent 或外部位置，可能先从根搜索。已知插入父亲和孩子序号时，插入成本还取决于孩子容器是数组还是链表。

parent 加孩子列表兼顾上溯和下行，但每次接入、移动、删除都要两边同步。删除含 s 个节点的子树至少要处理这 s 个对象，故实际释放 Ω(s)。

求 LCA 时，有 parent 可先算深度、把更深节点上移，再同时上移，O(h)时间；没有 parent，可从根找两条路径再比较最后公共位置，普通实现 O(n)时间、O(h)路径空间。预处理倍增或时间戳能改进多次查询，但需要额外结构和预处理成本。

26.4 4 · 叶子总比双分支多一个：同一条边数两遍

非空树 n 个节点有 n-1 条边。另一方面，每个单分支节点向下贡献 1 条，双分支贡献 2 条，所以

$$n_1 + 2n_2 = n_0 + n_1 + n_2 - 1.$$

消去 n1，得到 $n_0 = n_2 + 1$。再写 $e = n - 1 = n_1 + 2n_2$，可看见单分支越多，边数里越多是单链贡献。

真二叉树 n1=0，于是 $n = 2n_2 + 1$、$n_0 = (n+1)/2$、$e = 2n_2$。完全二叉树可以有一个单孩子节点，不能不看结构就套真树公式。

例：根有两个孩子，其中左孩子还有一个孩子。n0=2、n1=1、n2=1，边 3=1+2，叶 2=双分支 1+1。手数一次，再对照公式，比直接背三个字母更稳。

26.5 5 · 长子—弟弟编码：一棵二叉树可能代表一片森林

约定左指针表示第一个孩子，右指针表示下一个兄弟。多叉树的根没有兄弟，因此它编码后的根右指针为空。任意二叉树若根有右孩子，按这套含义就代表根还有兄弟，更自然地对应森林，而非单棵树。

例如二叉根 A 的右指针指 B，左指针指 C，解码为两棵树：第一棵根 A、孩子 C，第二棵根 B。不能把 B 同时当 A 的普通孩子，那会混用左右指针的含义。

把整树根在某些接口里视为虚拟父节点的右孩子，是为统一“指向该子树的入口引用”的约定；它不让真实根凭空多一个父亲，也不意味着根在长子–弟弟编码里有真实的右兄弟。

26.6 6 · 有序森林与二叉树：正反变换都写出来才叫一一对应

森林各根按从左到右用右兄弟指针串起来；每个根的孩子列表，首孩子接左指针，后续孩子也用右指针串起来。所有节点只保留这两类指针，就得到二叉结构。

解码时，从二叉根沿右指针拆出森林的根序列；每个节点的左指针给第一个孩子，再沿这条孩子的右链列出其余孩子，逐个递归。顺序没有丢，层级也由左指针与兄弟链的位置唯一决定。

正变换后立刻反变换会恢复每个原孩子列表，反过来也恢复每一条二叉指针，所以既不会两个森林编码成同一结构，也不会遗漏某个二叉结构。无序树若本来没有兄弟顺序，就不具备这项同样的一一对应前提。

26.7 7 · 原题要求的真树：每个空位置都要补，包括旧叶子的两边

设原树 n=n0+n1+n2，n0=n2+1。原题要求新树双分支数 $2n_2 + n_1 + 1 = n$，叶数 $2n_2 + n_1 + 2 = n + 1$。正确转换是：把每个原节点都保留为内部节点，原来缺失的左、右孩子各补一个带标记的外部空叶。

原叶有两个空位置，要补两个；原单分支补一个；原双分支不用补。新外部叶总数 $2n_0 + n_1 = 2n_2 + n_1 + 2$；每个原节点如今都有两个孩子，新双分支数正是 n，单分支为 0。

例：只有一个原根 A。补后是 A 连两个外部叶，内部 1、外部 2。若只给单分支节点补另一边，A 一开始没有孩子，完全不补，这时仍只有一个叶，达不到题面要求的两个外部叶。只给单分支补孩子也是一种变成真树的方法，但它的节点数与本题指定的转换不同。

保留外部叶标记，逆变换删除所有外部叶即可唯一恢复原树。标记不能与普通数据值混用，否则真实节点恰好也存“空”字样时会有歧义。

26.8 8 · 高度更新：某处没变，往上的高度也不会变

高度由两个孩子高度决定，$h(v) = 1 + \max(h_L, h_R)$。一次局部修改只影响一条祖先路径；若更新到某祖先 a，发现 h(a)与旧值相同，那么 a 的父亲看到的这个孩子高度没变，另一孩子又没改，因此父高也不变，往上同理。

插叶、删叶、接入整棵子树、分离子树都适用这个逻辑，但必须从真正受影响的入口向上，并在改完孩子链接后重算。高度不变不表示 size 没变；若还维护节点数、和或其他统计，不能一并提前停。

例：根两边高度分别 3 和 1，在较短一边增加到 2，根高度仍 4，继续往更高处传播高度就是多余。最坏插在唯一最长链末端，高度一路变到根，优化仍 O(h)，只是某些修改能提前结束。

26.9 9 · 勤奋缓存与懒惰计算：用操作序列算总账

若每次求 height 都遍历子树，单次可能 O(n)；缓存 height 后查询 O(1)，但修改时需向上维护。size 也一样：即时维护使 size 查询常数，代价是每次接入、删除、旋转等都正确更新。

例如一天修改 100 次、查询一百万次，缓存常常合适；一天改一百万次、只查一次，按需遍历可能更简单。要把树高、子树规模和查询频率代入，而不是认为“勤奋一定更好”。

维护多个缓存还会增加一致性风险：旋转后必须先更新低节点，再更新高节点，使父读到的新孩子信息正确。测试除了最终值，还要在每次修改后用独立遍历结果核对缓存。

26.10 10 · 中序在单侧链上：空调用忽略后，看剩余工作

全右链中序先遇空左树，然后访问当前，再递归右树，右递归之后没有剩余访问工作，可按尾递归化。全左链则先递归左边，再返回访问当前，访问工作都积在返回路上，是头递归形状。

一般树左右都有非空分支，不能同时把两条路径都压成同一种单链循环。空子树调用仍有固定检查成本，讨论形状时可以忽略它，但实际计费不要说从未执行过。

以 A 右连 B 右连 C，中序就是 A、B、C，可直接循环；A 左连 B 左连 C，中序应为 C、B、A，要记住祖先或借助结构回溯，不能从根一路输出。

26.11 11 · 单链先序：只向前走，不需要 parent

每节点最多一个孩子时，先访问当前，再把 p 设为唯一的非空孩子，直到空。因为没有另一棵尚未处理的侧树，不需要返回祖先寻找分支。

时间 Θ(n)，辅助空间 O(1)，即使没有 parent 也完全可行。若递归版编译器没有尾调用优化，它会保留 n 层；显式循环才把空间保证写进实现。

这项简化依赖“最多一个孩子”。一般树中若先深入左边，右边还没处理，需要保存任务；删去 parent 并不会删去这些待办任务。

26.12 12 · size 访问器：只要不重不漏，哪种遍历都能数

定义 Count 含 value，visit 每到一个真实节点就加 1。先序、中序、后序、层序都访问每个节点一次，所以最终都为 n；空树为 0，外部空叶是否算节点必须按所求结构定义。

求所有 data 总和、检查每项是否在某范围，也可这样写。若要计算“每个节点子树大小”，单纯给全局计数器加 1 不够，需要把左右子结果汇回父节点，后序更自然。

函数对象按引用还是值传递也要核对，避免最后外部计数仍 0。遍历的抽象复用建立在任务真正只需要一次单节点访问的前提上。

26.13 13 · clone：中序、后序都能复制，但要保存结构位置

后序做法：先复制左子树得到 L，再复制右子树得到 R，然后新建根并接上 L、R。先序则先建根，再递归填两边。二者都能保持形状，且每个原节点对应一个独立新节点。

中序也可以先复制左边，保存结果 L，创建当前根，再复制右边并连接；不能因为“根在中间创建”就说不可能。要保存孩子身份，而非仅把中序值输出，再当作 BST 插入，那会改变原形状。

无论哪种都需 Θ(n)新输出存储，递归栈最坏 O(h)。若构造中途失败，还要释放已经复制的部分或使用资源管理对象，避免只在完全成功的路径上没有泄漏。

26.14 14 · 打印树：缩进只说明深度，还需要说明哪条边

若每层缩进两格，只能看出节点深浅，不一定知道它是哪个父亲的左或右孩子。完整打印还需保存左右分支身份，以及哪些祖先层还有未打印的兄弟，用来决定竖线是否继续。

可以先打印右子树、再根、再左子树，让树向侧面展开；也可以先序打印带“L/R”的路径。选择一种后把图例写清，避免读者把上方节点误当左孩子。

用一棵左右高度不同、有单孩子且有空位的小树核对，比用满树更能暴露错误。打印是显示接口，不能通过交换真实孩子指针来“画得好看”而改变树。

26.15 15 · 清空不只允许后序，但绝不能释放后再读

后序先清左右，再删根，最容易保证仍能找到未处理节点。先序也可先保存 left、right，再 delete 根，再用保存的地址清左右；删除根后不能再读它的字段。

中序可先清左，保存右指针，再删根，然后清右。对象析构若会自动递归释放孩子，就不能同时再手工递归，否则双重释放；所有权策略必须统一。

各种顺序都能做到 Θ(n)，差别在于保留哪些地址、析构副作用和代码可验证性。正确性由生命周期与剩余对象可达性决定，不是某个遍历名字天然授予安全。

26.16 16 · 从 RPN 建树：栈里放的从数字变成子树

遇操作数就建叶节点入栈；遇二元运算符，先弹右子树 R，再弹左子树 L，新建运算符根并接上 L、R；遇一元后缀运算则只弹一棵树。

例如 2 3 4 + *：先有叶 2、3、4，+弹 4 和 3，形成根+的子树；再弹这棵右子树和叶 *2*，得到 *2*(3+4)。最后栈恰有一棵树，便是答案。

执行前检查元数，结束检查剩余棵数；2 + 操作数不足，2 3 留下两棵，都不合法。固定 token 与元数后，树结构可唯一恢复；若同一符号有不明确的一元、二元含义，还需词法或语法信息。

26.17 17 · 沿右藤的“对称版”，会不会把先序变成根右左

直接访问根后一路深入右边、把左子树暂存，得到的是根–右–左，并非要求的根–左–右。若只是把 lc 与 rc 交换，必须同时承认遍历定义变了。

沿右藤分解时，可以把基本工作改成“访问藤上当前节点，完整处理它挂着的左子树，再沿右边走一步”。左子树仍必须先完成，不能把整条右藤预先访问完。直接写出来是：

```
def preorder_right_vine(node, visit):
    while node is not None:
        visit(node)
        preorder_right_vine(node.left, visit)
        node = node.right
```

while 负责沿当前右藤前进；递归负责侧面的整棵左子树。没有左孩子时，空调用立即返回；有左孩子时，调用结束前不访问当前节点的右孩子。这正好保持根–左–右。

取 A 左 B、右 C，B 右 D，C 左 E、右 F。访问 A 后，先进入左子树 B；访问 B，左边空，沿 B 的右藤访问 D。整个 B 子树完成，才回到 A 那一层沿右藤到 C；访问 C，递归访问 E，再沿右藤访问 F。输出为 A、B、D、C、E、F。若先扫 A、C、F 再补 B、D、E，先序就被破坏了。

这个版本把右方向改成循环，递归栈只为尚未处理完的左侧调用保存返回位置。全右链只需常数层调用，全左链却要 Θ(n)层；时间仍 Θ(n)，一般辅助空间 O(h)。沿左藤的迭代版则显式保存待办右子树，两者保存的位置不同。递归版在长链上还可能超过语言的调用深度限制。

若要把上述左递归也改成显式栈，离开当前节点进入左边之前，就保存它尚未继续的右藤起点；左侧完成再恢复：

```
def preorder_right_vine_iter(root, visit):
    resume = []
    node = root
    while node is not None or resume:
        if node is None:
            node = resume.pop()
            continue
        visit(node)
        if node.left is not None:
            if node.right is not None:
                resume.append(node.right)
            node = node.left
        else:
            node = node.right
```

栈内每项都是“当前左侧工作完成后，接着处理这段右藤”的任务，最后存入者最先恢复。它与常见的“先压右、后压左”任务栈在访问顺序上等价，但这里不必把立即要走的左孩子也压栈。两种写法都没有更好的普遍时间阶；优缺点要比较具体树形上的待办数量、调用开销与实现清楚程度。

26.18 18 · 先序正确性：按整树规模归纳，侧枝也要包括

空树无输出；单节点访问一次正确。对更大树，沿左藤访问根及各左后代，并把每处右子树压栈。最深处的右侧任务最后压入，最先弹出，正好符合先做完当前左子树内部，再返回更高祖先处理右树的顺序。

每个被压的右子树规模小于原树，且彼此不重叠，归纳假设保证它们各自按先序完成。藤节点与这些侧树恰好覆盖原树，所以不漏、不重、次序正确。

若只归纳藤长度，外挂侧树可能比整条藤大得多，原归纳假设未必覆盖它们。可以增强命题，让它对任意合法侧树也成立；不能只把参数名字从“规模”换成“藤长”就算证明。

26.19 19 · 不压空右树：减少无效栈动作，但加了一次判断

原先无条件压右孩子，可能压入 nullptr；弹出后又立刻发现为空。改成右孩子非空才压，就消去这些无效 push 和 pop。

对全左链，若原版每层都压空右树，会积累很多空任务；去掉后待办栈可以显著缩小。对几乎每个节点都有右孩子的树，新增判断很少省动作。

总时间仍 Θ(n)，在不同编译器与数据下实际净收益需要测量。能严格说少做了哪些栈操作，不应直接保证所有机器上运行时间都按同一比例下降。

26.20 20 · 先序栈空间：高度是上界，待办右树数更贴切

沿左藤版栈保存还没访问的右子树根。当前路径每经过一个“走向左孩子、右边还有任务”的祖先，最多增加一个待办，因此栈长不超过相应左祖先数，进而不超过 O(h)。底部哨兵或当前节点另加常数。

全右链高度 n-1，几乎没有左祖先，待办栈可常数；全左链若不压空右树，也可以常数。若长左链每个祖先还挂一个真实右叶，走到底时会同时积累约 h 个右任务，达到高度阶。

题目问两个上界之比的数学期望，必须说明树形与选点分布。左右方向独立对称的一条固定路径，其左转次数期望是路径长一半；但“全树最大左祖先数/全树高度”的期望不是因此自动一半。max 与期望以及比值都不能随意交换。没有随机模型，不能给出普遍数字。

26.21 21 · 中序归纳：左藤只是拆解入口，不是全部规模

中序先沿左藤把祖先压栈，直到最左节点；弹出当前访问，再把其右子树按同样方式展开。整棵树规模归纳可以直接覆盖左右两个更小子树。

若以左藤长度归纳，需要把“藤上每个节点挂着任意右子树时，算法仍正确处理它们”写进命题，并独立保证这些侧树的正确性。只讨论藤本身的访问顺序不足以证明整棵树。

用根 A、左 B、B 右 C、C 左 D 的小树测试：中序应 B、D、C、A。它很容易揭穿只按一条直链推理、漏掉弯曲侧枝的证明。

26.22 22 · 最后一个中序节点：一路向右后，再向上越过根

中序最后访问整树最右节点。它没有右子树，否则右子树中还有更晚节点。求 succ 时，若有右子树应取其最左节点；没有就向上，直到第一次从某祖先左侧返回，该祖先才是后继。

对于整树最右节点，向上始终处在祖先右侧，最终越过根得到 nullptr。因此算法应在访问 parent 前判空，不能再从根去读不存在的父节点。

最小树只有一个根，它也是最后节点，succ 直接为空。测试这个例子能检查代码是否把“根无父亲”的边界遗漏。

26.23 23 · 没有 parent，也能后序：自己保存返回阶段

栈保存(node,stage)。stage=0 时准备处理左边，改成 1 后压左孩子；stage=1 时左边已完成，改成 2 后压右孩子；stage=2 时左右都完成，访问当前并弹出。

```
def postorder(root):
    stack = [(root, 0)] if root is not None else []
    while stack:
        node, stage = stack.pop()
        if stage == 0:
            stack.append((node, 1))
            if node.left is not None:
                stack.append((node.left, 0))
        elif stage == 1:
            stack.append((node, 2))
            if node.right is not None:
                stack.append((node.right, 0))
        else:
            visit(node)
```

这相当于把语言调用帧的程序位置显式存出来，时间 Θ(n)、辅助空间 O(h)。parent 指针是获得返回方向的一种办法，不是唯一办法。

26.24 24 · 后序正确性：访问条件就是“两棵子树已完成”

对每个帧，阶段只按 0→1→2 向前推进。进入阶段 1 前左子树任务已执行完，进入阶段 2 前右子树也已执行完；因此访问根时，恰好符合左–右–根。

每个非空孩子只由唯一父帧压一次，树没有共享孩子或环，所以不会重复。空孩子直接跳过，不影响阶段含义。按子树规模归纳，两个子任务各自输出正确，接上当前根便正确。

如果输入其实是共享子图或有环，这份树遍历可能重复或不结束，需要 visited 和另一种语义。先把“是合法树”写成前提，证明才不会偷换对象。

26.25 25 · 迭代遍历总时间：一次下藤走 h 步，不能给每个节点都收 h 步

先序每个节点访问一次，其右任务至多压弹各一次；中序每个节点压一次、弹一次；阶段式后序每个节点只经历固定三阶段。合计都 Θ(n)。

内层 while 偶尔走很长一条藤，但这条藤上的节点随后不会在别处再次作为新的未访问节点下藤。把所有内循环迭代相加是 n 量级，不能看到嵌套循环就相乘。

若算法每次求 succ 都从根重新搜索，或每次重算整棵子树信息，已经不是这里被分析的迭代版本，可能增加成本。复杂度结论应绑定到真实的节点进出次数。

26.26 26 · 层序遍历：队列里为什么只会混着相邻两层

先把根入队。每次弹一个节点 v 访问，再按左后右把其孩子入队，孩子深度仅比 v 大 1。FIFO 保证当前层尚未处理节点在前，刚发现的下一层孩子排在后，所以队列深度非降，最多差 1。

归纳可知根先到，每个非根节点恰由唯一父亲发现一次。每个节点入队一次、出队一次，因此总 Θ(n)。同层左先右后来自孩子入队顺序；若调换左右，深度顺序仍正确，但同层阅读方向会变。

“左边更早”只能对同层按该有序树定义解释。一个很浅的右节点会早于深处左节点，不能把全树横向位置当成绝对处理次序。

26.27 27 · 层序空间：队列可能比某一层还稍宽

设最大层宽 w。队列可能同时包含当前层剩余部分与下一层已发现部分，故不一定恰好≤w，但最多是相邻两层的总宽，仍 O(w)。单链每次队列最多 1，完全树可线性大。

更精确地，队列每个节点代表一棵互不相交的尚未处理子树，每棵至少包含一个不同叶子，所以队列长度不超过叶子总数。非空二叉树叶子最多 $\lfloor (n+1)/2 \rfloor = \lceil n/2 \rceil$，完全二叉树可以达到这个数量级，且这种 BFS 队列也能达到该上界。

例如 n=5 完全树，根孩子 A、B，A 有 C、D。处理 A 以后队列为 B、C、D，长度 3；最大单层宽却只有 2。这一小例解释了为何空间 Θ(w)不表示每时每刻精确等于某层宽。

26.28 28 · 完全树放进数组：父子下标怎么算

零基存储时，节点 i 的左孩子 2i+1、右孩子 2i+2；i>0 的父亲为 $\lfloor (i-1)/2 \rfloor$。存在与否只需检查孩子下标是否小于 n，不能先访问再判断。

根 0，下一层 1、2，再下一层 3、4、5、6。这些区间连续，因此层序扫描局部性很好；沿一条深路径，下标却按倍增跳跃，不能保证每对父子都物理紧挨。

完全性保证中间无空洞；任意稀疏树硬放这种数组，深单链可能需要指数大的空位置。适用条件与缓存收益都要按访问模式说明。

26.29 29 · 真树先后序重建：第二个先序项给出左子树范围

假设标签互异，且给的是同一棵真二叉树。先序首项和后序末项必须是同一个根；若不止一个节点，先序第二项就是左子树根。到后序中找它，可得到整个左子树的末端与长度，从而把两个序列都切成左右两块递归。

例如先序 A,B,D,E,C，后序 D,E,B,C,A。根 A，左根 B 在后序下标 2，左块长度 3；递归左块得到 B 的孩子 D、E，右块只有 C。真树保证根不可能只有一支，消除了“这段到底左子树还是右子树”的歧义。

用标签到后序位置的字典可使定位期望常数，总体 O(n)；若每层线性找标签，退化树形可二次。还要验证左右块均合法、根标签一致、标签集合一致，不能把不一致序列强行拼成树。

26.30 30 · 增强遍历：空标记能救先序、后序，救不了中序

先序读一个 token，若是 # 返回空；否则建根，再依次读取左、右子树。对 `A B # # C # #`，先建 A，左建 B 并读两个空，右建 C 并读两个空，边界被递归结构唯一确定。

后序从末尾反读，遇根后先恢复右、再恢复左，同样唯一。每个真实节点和空位置读一次，时间线性，还要检查最后有没有多余 token。

中序增强不唯一：A 只有右孩子 B，与 B 只有左孩子 A，都会给 `# A # B #`。空标记本身不自动记录根在哪，所以“所有增强序列都能唯一重建”是错误推广。

26.31 31 · 组合重建：每个“唯一”都带着前提

互异标签下，先序给根、中序给左右分界，故先+中唯一；后+中同理。普通先+后对单孩子方向无解：A 左 B 和 A 右 B 的先序都 A,B，后序都 B,A。

加上真树条件可排除单孩子歧义，如前题。层序加中序也能逐层选出各区间最早出现的根来重建，但单独层序只有访问顺序，不能决定哪些边存在。

标签重复时，连先+中也可能不唯一，必须为不同节点加身份或另外约束。先检查输入条件，再选重建方法，不能只拿两个遍历名称就保证有解且唯一。

26.32 32 · 交换子树：成本差由总权与深度差决定

两棵互不为祖先后代的子树根深 d1、d2，总权 W1、W2。交换后各叶内部相对深度不变，第一棵整体移动 d2-d1 层，第二棵反向移动，因此

$$\Delta C = (W_1 - W_2)(d_2 - d_1).$$

若较大权 W1 原来更深，即 W1>W2 且 d1>d2，则差为负，成本下降。权相等或深度相等只保证不变，不能说严格下降。

例 W1=10、W2=3，深度 3 与 1，交换差(10-3)(1-3)=-14。若一棵包含另一棵，直接交换会破坏树，不能把同一公式不加结构限制地套用。

26.33 33 · Huffman 最优：贪心选择要能嵌进某棵最优树

先取最小两权 a、b。在一棵最优真编码树里选最深的一对兄弟叶，把 a、b 交换到这两个位置。最小权放得更深不会增加成本，故存在一棵最优树让它们为兄弟。

把这对兄弟缩成权 a+b 的叶，原树成本恰等于缩小树成本再加 a+b。若缩小树不是最优，换成更优的小树再展开，就会改进原最优树，矛盾。因此在缩小问题上继续做同样贪心，按叶数归纳得到最优。

对[2,3,7,8]，先合 2+3=5，再 5+7=12，再 12+8=20，WPL 为 5+12+20=37。展开树的叶深分别 3、3、2、1，直接算 2×3+3×3+7×2+8×1 也为 37。两个算法得到不同左右码字不一定有错，最优编码树可能不唯一。

26.34 34 · 框架与森林容器：换容器要重新带入成本

核心框架只有：取两个最小根、合并为新根、把新根放回，直到剩一棵树。它依赖优先队列能力，不必知道底层是无序数组、堆还是两个有序来源。

无序数组每轮线性找最小，n 轮可二次；小根堆每次取最小、插入均 O(log n)，总 O(nlog n)。初始建堆可以线性，但后面 n-1 轮合并仍在。

接口隔离让框架可复用；复杂度分析却不能对不同容器照抄一个数字。源码中找到森林类型、取最小和插回的位置，再把每项成本相加。

26.35 35 · 双队列 Huffman：线性的是已经有序之后的合并阶段

原始权重升序放在一个来源，新合并的权放第二个队列。每次只比较两个队首，从中取全局最小的两个。新合并权按生成顺序不减：下一轮取到的两个可用最小值，不会产生比上次合并和更小的新和。

原输入[1,2,4,8]：取 1、2 得 3，合并队列[3]；取 3、4 得 7；取 7、8 得 15。每个原项与合并项进出常数次，有序合并阶段 O(n)。

用栈存原权时，应让最小项在栈顶，即压入方向相反；换成升序队列也一样。若原权未排序，前面的排序一般 O(nlog n)，因此原题总复杂度仍 O(nlog n)，不能把后半线性当无条件总线性。

26.36 36 · 平均比较排序下界：少数短路径救不了所有排列

互异 n 项共有 n!个等概率排列，每个对应决策树一片可区分叶，深度 di 是比较次数。二叉树满足 Kraft 界 $\sum 2^{-d_i} \le 1$，且函数 2^{-x} 凸，所以

$$2^{-\bar{d}} \le \frac{1}{n!}\sum_i 2^{-d_i} \le \frac{1}{n!},$$

得到 $\bar{d} \ge \log_2(n!) = \Omega(n\log n)$。

也就是说，哪怕某些输入一眼就能认出，其他分支必须付出足够比较，均匀平均仍不能突破此界。若输入分布高度集中，平均下界应该用其信息量重新描述；若采用整数桶、位运算等超出比较模型的操作，也不能把这条下界照搬过去。

随机算法还需说明平均是在输入随机、内部随机，还是两者共同取期望。证明固定比较树的均匀平均后，对内部随机选择的树再平均，仍能得到相应下界。

26.37 37 · 编码构造下界：限定比较模型，再构造足够多的输出

这里讨论任意未排序正实权重、仅通过比较区分大小、输出带原字符标签的最优树。已排序权重的线性合并没有违反下界，因为排序信息已经作为输入给出了；受限整数模型也可能有其他手段。

可取一组增长极快的固定权重，后一权重远大于前面所有权重之和的 n 倍，再把字符标签任意置换。这时最大权必须在深度 1：让它多一层的损失，大于其他所有叶在至多 n-1 层内调整所能补偿的总量。移去最大叶后对剩余子树重复，最优树被迫呈梳状，依次用深度暴露各权重名次，只有最小两叶可互换。

所以至少有 n!/2 类必须区分的带标签输出，二叉比较树需 $\log_2(n!/2) = \Omega(n\log n)$ 深度。也可把问题看成：构造后在线性时间读出深度顺序，便得到这个受限排序族的排序信息。

Huffman 树是最优树的一种，因此同一组强分离输入也约束其构造。不能把“任意树的任意层序遍历一定按所有叶权排序”当成无条件事实；左右孩子可交换，同层叶也可能需要额外安排。下界应建立在明确可区分的信息上。

26.38 本章练习与核对

1. 原树仅 A 左孩子 B。扩充全部空位置后，内部、外部叶各几项？若只给 A 补右孩子，会漏什么？
2. 给先序 A,B 和后序 B,A，画出两棵可能树；加什么条件才能排除这种歧义？
3. 权重 1、2、4、8 做双队列 Huffman，每轮取哪两项，最终 WPL 多少？

第一题：原节点 n=2，扩后内部 2、外部叶 3；A 右边补 1 个，B 左右各补 1 个。仅给 A 补右边会让 B 仍是普通叶，达不到原题“每个原节点成为内部”的计数。第二题：A 左 B 与 A 右 B 均符合；要求真二叉树后，这个双节点输入本身不合法，并非随便选一棵就解决。第三题：1+2=3，3+4=7，7+8=15，WPL=3+7+15=25；叶深 3、3、2、1，直接 1×3+2×3+4×2+8=25 核对。

27 把算法写成程序以后，怎样知道它没有悄悄换题

纸上二分会收缩区间，程序可能因 lo=mid 在两项区间里原地打转；纸上方案数是整数，32 位变量却可能溢出；纸上删除链表节点只改两根链接，代码可能释放后又访问它。工程检查不是等证明失败以后才做的补丁，而是确认实际实现仍符合证明中的模型与状态。

27.1 先写规格，再写输入循环

以 lower_bound 为例，规格包括：输入 a 非降，返回 p 满足 0<=p<=n，p 之前所有元素<x，p 及以后所有元素≥x。规格描述答案性质，不依赖你准备用 while、递归还是库函数。

这使测试可以直接验证性质，而不必复制被测函数的每一行。对返回 p，检查边界，再分别检查左右关系；如果数组中目标不存在，仍应满足这些条件。若只测试“查到某个 3 就对”，第一与最后重复位置的错误会被漏过。

PA 规格还应列输入规模、值域、是否允许空、重复和负数、输出并列规则、运算语义、内存限制。输入最大 n=10^6 却每次开 n^2表，即使样例全对，也已经与规模要求冲突。判断复杂度要先于大量调试。

27.2 例 1：为二分造一套很小但有分辨力的数据

固定数组类型不够，要覆盖会走不同分支的输入。可以从值域{−1,0,1}生成长度 0 到 6 的所有非降数组，对每个 x∈{−2,−1,0,1,2}比较被测结果与独立线性扫描。

线性答案很简单：从左到右找第一个≥x，找不到返回 n。它比二分快，但小规模足够，而且思路不同，较不容易共享同一个区间错误。

具体反例常会非常短。若代码写成 while lo<hi、a[mid]<x 时令 lo=mid，数组[1,2]查 2 时 lo0,hi2,mid1 会移 lo1；接着 lo1,hi2,mid1 仍移 lo1，永不前进。两项数据比随机十万项更容易清楚定位这个问题。每轮记录区间长度并断言严格下降，也能直接发现原因。

要限制错误程序的运行时间，测试器可设置操作步数上限；否则一个无限循环会让整个测试停在第一个坏例子。验证器自己的边界同样要正确，例如空数组不能读 a[0]。

27.3 例 2：单调栈用慢答案核对“宽度差一”

对直方图，慢算法枚举所有区间[l,r]，维护区间最低高度 h，候选面积 h×(r−l+1)，取最大值。小 n 时 $O(n^2)$足够，与快算法的单调边界思路不同。

输入[2,1,2]的答案 3，来自高度 1 覆盖全部 3 柱；[2,2]答案 4，专查等高处理；[3,2,1]答案 4，专查连续弹栈；[1,2,3]答案 4，专查结尾清栈；全 0 答案 0，空直方图若允许也答 0。

如果快算法把宽度写成 i−left 而非 i−left−1，[2]补 0 时可能算出宽 2、面积 4，单元素就能抓错。如果忘记结尾清栈，[1,2,3]可能从未触发弹出，输出 0。测试要针对状态为何改变，而非只凑很多长度相近的数组。

随机测试可生成长度最多 8、高度 0 到 5 的直方图，几千次比较；穷举则能覆盖这个有限域全部输入。两者都不是无界数学证明，但每一个失败例子都是真实反证。若全过，应准确说“在这些范围内与独立枚举一致”，不能说“已证明任何 n 都正确”。

27.4 例 3：Gift 计数，先保留每一种选择的身份

快算法生成左右半和，排序后二分计数；慢算法直接枚举每个二进制选择掩码，对每组取对应项，再加总。用两种实现核对所有预算附近的结果。

组(1,1)、(2,2)有 4 种选择，所有和都为 3。预算 3 时答案 4，预算 2 时 0。这个数据专门抓“半和去重”的错法。再测负数，例如(−2,5)、(−1,3)，预算 0，合法和为−3、1、4、8 中的一项，答案 1；若程序使用“当前和超过预算就剪枝”的规则，会因为后续可能有负数而错删可行分支。

变形测试也有帮助：每组两个选项交换，方案总数应不变；所有选项同时加常数 c，总和统一增加 n×c，预算也增加 n×c 后答案应不变；预算增大，合法方案数不能减少。这些关系不需要另一个完整答案，却能发现不符合问题性质的输出。

27.5 大整数和内存，要算到中间值

逆序数最大 n(n−1)/2。若 n=100000，约 50 亿，超过有符号 32 位；写成 64 位变量还不够，若 n、n−1 先以 32 位相乘，再赋给 64 位，溢出已经发生。应在乘法前提升其中一个操作数的类型。

叉积也一样，两个大乘积相减可能结果很小，但中间值仍超范围。动态规划的“无穷大”不能取最大整数后无条件加 1，应判断是否可达，或选择留有安全余量的哨兵。无符号整数倒着循环到 0 时，`i>=0` 永真，减 1 会绕到最大值，常造成越界。

内存估计应乘实际元素字节数，并考虑数组份数、容器开销、递归栈和迁移时双份内存。一个 n×n 的 32 位表在 n=10000 时约 400MB，不含额外对象；若用每格独立对象的语言，实际可能更大。只写“空间 $O(n^2)$”不足以判断是否能在某台机器上运行。

27.6 调试时保存状态，不要只打印一句 wrong

二分打印 lo、hi、mid 和判定；单调队列打印下标及对应值；AVL 旋转后检查中序、parent、height、size；散列表记录探测序列与槽状态；图算法记录 dist 更新来自哪条边。每项日志都应能对应一个不变量，避免输出一屏无法解释的数字。

断言适合在本地捕捉不可能状态。例如 `0<=lo<=hi<=n`，堆中父值≥子值，双向链表相邻链接互相对应，树保存的 size 等于孩子 size 之和。大规模性能测量时可关闭昂贵的全结构断言，但保留必要输入检查与错误处理。

27.7 测性能，要测试算法最不舒服的形状

快排测升序、降序、全相等与交替值；BST 测递增插入；散列表测同桶键或大量墓碑；图测长链、星形、稠密图、不连通图；DP 测预算边界、长度差刚好 k 与 k+1；单调结构测严格递增和严格递减。

同一代码在 n 与 2n 上计时，平方算法可能接近 4 倍，线性接近 2 倍，但缓存、解释器启动、分配和系统调度都会影响小样本。应重复测量并用足够大的规模，不要用一次 0.001 秒与 0.002 秒就宣布证明了线性。操作计数往往比墙钟时间更适合验证理论增长。

27.8 本书示例检查器的作用与边界

`示例/算法检查.py` 用独立小规模答案检查二分、矩形、插删 DP、KMP、Gift、Huffman 和表达式等若干方法；`示例/复杂度计数.py` 检查习题 1-32 中辅助开方、阈值与随机取模概率等。它们支持书中具体算例，不是课程 OJ 的替代品，也没有证明未纳入检查的全部算法。

阅读测试输出时，要看测了什么和没测什么。一个检查器若用同一份函数计算“预期答案”和“实际答案”，通过一百万次也没有独立性。若测试为了配合实现而只验证实现本来就做的动作，也无法发现规格理解错了。

27.9 让检查器先自己暴露一次错误

写完独立答案后，可以有意造一个已知错误版本，确认测试真的会拒绝它。例如把矩形宽度少减 1，测试 [2]应立即报预期 2、实际 4；把 lower_bound 的 lo=mid+1 改成 lo=mid，测试应在[1,2]查 2 时超出操作上限；把 Gift 半和去重，等值选项例应报告 4 被误算 1。

这种小规模故障注入能检查测试是否真正执行了目标分支、失败是否会传出、比较对象是否选对。若改错以后测试仍全绿，就要先修测试，而非相信原实现已经安全。测试通过的可信度来自它能区分正确与错误行为，不能只来自打印了一句“通过”。

27.10 图算法的距离证据，除了数字还要核对边

给每个可达点保存 parent，恢复到源的路径后逐边求和，检查是否等于 dist。某条不存在的 parent 边、恢复形成环、路径没有回到源，都能暴露实现错误。还可对所有边检查 dist[v]≤dist[u]+w 在应适用的结束状态成立；只检查松弛不等式仍不足以证明所有有限距离来自真实源路径，所以路径证据也要有。

MST 可检查选边数为 V−1、连通且无环，再与小图枚举所有生成树的最小总权核对。只检查总权碰巧相同，可能接受一组不连通边；只检查连通无环，则可能接受不是最优的生成树。规格的每一部分都需要被验证器覆盖。

27.11 可复现实验需要保存输入，不只保存随机种子

固定种子有助于同一生成器重现序列，但库实现、调用顺序或并行调度变化可能让输入不同。发现失败时，应把完整最小输入与预期输出保存下来，并记录相关操作顺序。尤其数据结构错误常需要一串插删才出现，只保存最终集合无法重现历史状态。

公开示例可使用另造数据与最小反例；个人作业提交、含身份信息的日志或受限样卷原页不应顺手塞进仓库。验证方法可以公开，私有输入继续保留在本地原处，教材中的复现例子应足以独立运行且不依赖那些私有文件。

27.12 失败输出应包含第一次分歧

比较两份 DP 时，除了最终答案，还可以打印第一个不同的坐标和两格值；比较队列时，打印首次出队不符之前的完整操作前缀。越早定位分歧，候选原因越少。

修复以后，把这个最小反例加入长期检查，再跑相关原有用例。不要因为修正一个分支就扩大到所有无关项目测试，也不要只重跑单个失败输入而忘了它可能破坏相邻边界。

检查器捕获错误以后应以非零状态结束，避免自动构建把失败日志当成功。

27.13 练习与答案

练习 1。 给 KMP 设计三个不同目的的最小测试，而不是只写三个普通字符串。

解答：`T=AAAA`、`P=AA`，预期 0、1、2，测重叠匹配与完整匹配后回退；`T=ABABABAC`、`P=ABABAC`，预期 2，测失配边界链；T 空、`P=A`，预期无，测空文本；还可 `T=A`、P 空，按本书全部匹配约定为 0、1，测空模式接口。每个测试都针对一种状态转换。

练习 2。 AVL 中序正确，是否足够说明整棵树实现正确？

解答：不足。中序正确只说明键的顺序与部分链接可能合理，还需检查每个节点左右高度差、height 字段、size 字段、parent 互指、节点数与无环性。一个父指针断错的树，中序递归不使用 parent 时仍可能输出正确，后继查询却会失败。

练习 3。 随机测发现长度 100 的失败数组，如何把它变成容易分析的反例？

解答：不断尝试删除一段元素，若失败仍存在就保留删除；再尝试缩小数值、合并等值与减少操作。每次都用原规格和独立答案确认仍失败。最终可能缩成[1,2]的二分死循环或[2]的矩形宽度错误。缩减反例保持触发条件，比对着 100 个数猜原因更有效。

28 概率、摊还和归纳，怎样把省略的一步补上

“平均很快”“大概对数”“每次都变小”都不足以完成证明。本章集中处理几种容易被一句话带过的论证。你可以先读具体小例，再回看符号；每个符号都对应一个可计算量。

28.1 例 1：势能法，把暂时没付的钱写进账本

设第 i 次操作实际成本为 c_i，操作之后结构势能为 Φ_i。定义摊还成本

$$\hat{c}_i = c_i + \Phi_i - \Phi_{i-1}.$$

势能增加时，相当于这次多收一些，留给未来；势能降低时，释放此前积蓄，为这次昂贵操作付款。将 m 次相加，中间项互相抵消：

$$\sum_{i=1}^{m} c_i = \sum_{i=1}^{m} \hat{c}_i + \Phi_0 - \Phi_m.$$

若初始势能为常数且始终非负，实际总成本不超过摊还总成本加常数。势能不是物理能量，而是你为证明设计的一个非负计数函数；设计对了，贵操作会恰好释放足够额度。

双栈队列令 Φ=2×in 中的元素数，push 或 pop 各计 1。入队实际 1、势能增加 2，摊还 3；out 非空出队实际 1、势能不变，摊还 1；out 空而 in 有 k 项时，倒栈做 2k 次基本 push/pop，再弹 1 次，实际 2k+1，势能下降 2k，摊还仍 1。

具体序列入 A、入 B、出、出：实际成本 1、1、5、1，总 8；势能 0⟶2⟶4⟶0⟶0，摊还 3、3、1、1，总 8。中间某次贵到 5 并不违反每步摊还常数的结论。若序列结束时 in 仍有元素，未释放的势能使摊还总账大于实际账，但仍给可靠上界。

向量加倍也可设计势能。只追加时，容量 C、元素数 n，满后刚扩到 2C 的阶段可取与 2n−C 相关的非负函数，或直接用逐元素记账。不是每个问题都必须硬用势能；几何和已经清楚时，聚合法通常更直观。

28.2 例 2：随机置换的环数，线性期望不需要独立

排列把每个位置映射到另一个位置，可拆成互不相交的环。例如排列[2,1,4,3,5]对应环(1 2)、(3 4)、(5)，共 c=3 个环。将一个长度ℓ的环恢复到正确位置，需要ℓ−1 次任意两位置交换。所有环合计 n−c 次。

为什么至少要这么多？一次交换要么把一个环拆成两个，要么合并两个环，环数至多增加 1。目标恒等排列有 n 个单点环，从 c 增加到 n 至少需要 n−c 次。按环逐个拆恰好达到此数，所以最优。

对于 n 个互异标签的均匀随机排列，期望环数为 H_n。可以按标签从 1 到 n 构造随机环排列：加入第 k 个标签时，有 k 种等可能位置，一种让它独立成新环，其余 k−1 种把它插到已有某个标签后面。因此第 k 步新增环的指标变量期望 1/k，线性求和得到 E[c]=Σ1/k=H_n。

于是最少任意交换次数的期望为 n−H_n。这里不需要新增环指标相互独立，因为期望的线性性对相关变量也成立。若要把矩母函数写成乘积，那才需要额外独立性；这两个步骤的条件不同。

28.3 单调栈的某一时刻，与整段过程的峰值

假设输入 n 个互异高度的相对次序是均匀随机排列。处理前 r 项后，递增单调栈保留从右往左看的记录最小值：一个元素若比它右侧所有已处理元素都小，就尚未被后来的更小者弹掉。

长度 j 的后缀中，每个位置等概率成为最小值，其最左项成为记录的概率 1/j。因此栈长 S_r 的期望为 H_r=Θ(log r)。例如 r=3，6 种互异排列对应记录数的均值是 1+1/2+1/3=11/6，而最坏递增输入栈长仍为 3。

若问题问峰值 M=max(S₁,…,S_n)，不能把 E 移进 max，写成 `E[M]=max E[S_r]`。一般只有 E[M]≥max E[S_r]，反向上界需要另证。

28.4 例 3：给峰值一个真正的对数上界

随机排列中记录指标可表示为独立 Bernoulli 变量 B_j，P(B_j=1)=1/j。独立性可用相对秩的双射证明。依次读取一个 r 项均匀随机排列，记第 j 项在已经读到的 j 项里的相对秩为 R_j，最小为 1。任意合法秩序列都能唯一恢复原排列：从 j=r 倒着走，在尚未用的有序标签中取第 R_j 小者作为原第 j 项，再删掉这个标签。共有 1×2×…×r=r!组秩序列，对应 r!个等概率排列，因此每组秩序列概率等于各 1/j 相乘，R_j 相互独

立且各自在 1 到 j 均匀。成为记录最小值恰好是 R_j=1，所以记录指标独立，概率 1/j。把一个固定前缀从右向左读取仍是均匀排列，便得到所需后缀记录模型。

于是对固定 r，指数矩有

$$E[e^{S_r}] = \prod_{j=1}^{r}\left(1+\frac{e-1}{j}\right) \le \exp((e-1)H_r) \le Cr^{e-1}.$$

第一步用独立变量乘积，第二步用 1+x≤e^x，最后用 H_r≤1+ln r，C 是与 r 无关的常数。Markov 不等式给 `P(S_r>=t) <= C r^(e-1)e^(-t)`。再对 r=1 到 n 用并合界，不要求各 S_r 独立：

$$P(M \ge t) \le Cn^e e^{-t}.$$

取 t=A ln n+s，A>e，便有随 s 指数下降的尾部。整数非负变量的期望等于尾概率和，前约 A ln n 项每项不超过 1，后面几何级数总和为常数，故 E[M]=O(log n)。另一方面 M≥S_n，故 E[M]≥H_n=Ω(log n)，合得 Θ(log n)。

这个证明区分了两种独立性：固定前缀中的记录指标独立，用来算指数矩；不同时间的栈长并不独立，峰值上界用并合界绕开了它。若输入含重复值或不是均匀随机相对次序，整套概率模型需重看。

28.5 最大队列长期峰值，为何可能接近窗口宽度

固定一个宽 w 的窗口，若值独立连续同分布，单调最大队列的候选是从右向左的最大记录，期望 H_w。但一个严格递减窗口会留下 w 项，出现概率为 1/w!。

观察 m 个互不重叠窗口，各窗口相对排列独立，则从未出现递减窗口的概率为 `(1-1/w!)^m`，m 增长时趋 0。因此固定 w 而观察越来越长的流，峰值达到 w 的概率趋 1，期望峰值也趋 w。不能把“任意固定时刻平均 log w”说成“无限期运行的峰值也平均 log w”。观察范围也是随机变量定义的一部分。

28.6 例 4：平均比较排序下界，不只看最深叶子

比较决策树有 n!个叶子，对应等概率互异排列。叶深 d_i 表示该输入比较次数。前缀树满足 Kraft 不等式 `Σ2^(-d_i)<=1`。函数 `2^(-x)`是凸函数，Jensen 不等式得

$$2^{-\bar{d}} \le \frac{1}{n!}\sum_i 2^{-d_i} \le \frac{1}{n!},$$

因此平均深度 `bar d >= log₂(n!)=Ω(n log n)`。最坏下界只需叶子数量；平均下界则需要控制所有叶深之和，不能仅说“有一个叶很深，所以平均也深”。若输入分布不是均匀的，应按其概率加权，相关下界会与熵有关。

n=3 时一棵有 6 叶的比较树，可能有部分叶深 2、部分深 3；平均不必为 3。信息量 $\log_2 6$ 约 2.585 是下界，叶深必须整数也会增加限制。具体比较数应从实际算法树计算，渐近 Θ 并不等于精确均值相等。

28.7 强归纳：命题要覆盖你下一步真会用到的东西

证明算法对 n 规模正确，常假设所有更小规模都正确。归纳命题还必须包含输入条件与输出关系。例如列表反转只说“元素个数不变”太弱，下一步需要的是“已反转前缀顺序与原前缀恰好相反，未处理后缀仍保留原顺序”。树遍历若只对左链长度归纳，却允许链上挂任意大右子树，归纳参数并未覆盖全部递归结构。

可以对整树节点数归纳：左右真子树都更小；或增强命题，把任意合法侧子树的待办次序纳入。遇到归纳证明卡住时，先检查命题是否太弱，而不是马上做更复杂代数。

28.8 例 5：会下降的不是每一步，而是一段有限宏步骤

有些过程单步变量会增加，却仍能证明终止。课程 PS01 中的 Failstone 过程不能直接当作尚未解决的 Collatz 问题；它的具体分支可以单独分析。

对正整数状态(p,q)：若 p>q，转为(p−q,p)；若 p<q 且 q 为偶数，可减半 q；若 p<q 且 q 奇数，连续将 p 加倍，直到 p 第一次超过 q。由于 p 最初正，有限次就超过，且 $q<p<2q$。随后按分支执行减法与相应减半，可把状态归并为两个都小于旧最大值 q 的正整数；等号情形按题目终止规则处理。

仔细追分支时，应使用题面中参数更新的准确先后次序。常用的等价宏阶段写法是：在 p<q 且 q 奇数时，找到某次翻倍前的 p 满足 $p<q<2p$，再完成后续更新得到 $(2p-q,p^*)$，两项都小于 q。前一段翻倍不保证单步

max 下降，完整宏阶段才保证 max 严格下降。正整数最大值不可能无限严格下降，加上每个宏阶段有限，因此终止。

这类论证的要点是说明宏阶段怎样结束、阶段端点量怎样下降。不能只观察几个状态趋小，也不能将形式相似的别的程序的未知结论搬过来。

28.9 几何分布的期望，为什么是倒数概率

每次独立试验成功概率 p>0，令 K 为第一次成功轮次。至少进行 k 轮，等价于前 k−1 轮全失败，概率为 (1−p)^(k−1)。非负整数变量可按“第几轮还活着”计数，所以 E[K]=Σ_{k≥1}P(K≥k)=1+(1−p)+(1−p)²+...=1/p。

这个推导比直接背求导级数更接近执行过程：每一轮给仍未结束的运行再收 1 单位成本。若 p 每轮变化为 p_k，尾概率变成前 k−1 个失败概率的乘积，期望未必是某一个 p 的倒数；若试验相关，也不能把失败概率直接相乘。算法里随机重试的终止期望依赖这些条件。

成功概率 1/2 时 E[K]=2，但最坏轮数没有有限统一界。若设置最多 10 轮就返回失败，算法一定有限结束，却可能失败，输出承诺从“总会成功”变成“成功或报告未完成”。随机算法的错误概率、终止概率与期望运行时间是三项不同指标。

28.10 不等式用在哪一步，就把条件放在那里

Markov 不等式要求变量非负；指数矩 e^S 自然满足。并合界 P(至少一个事件发生)≤各事件概率之和不要求独立；将概率乘起来通常需要独立或条件概率展开。Jensen 的方向取决于函数凸凹，2^(−x) 凸给函数均值不小于均值处函数，对数凹则方向反过来。

同一证明中反复出现“平均”时，给变量一个名字能减少偷换。S_r 是固定时刻栈长，M 是整段峰值，H_w 是固定窗口候选数的期望；它们的下标与定义指出观察范围。把范围隐藏以后，一个本来正确的局部结论很容易被误用成长期保证。

28.11 期望不是保证某次靠近它

一个随机过程以很小概率付极大成本，期望仍可能有限；要回答延迟是否稳定，还需尾概率或高概率界。单调栈峰值证明特意使用指数矩，正是为了限制罕见但很大的峰值，而非只计算某时刻平均值。

若输入由固定人工模式产生，例如严格递增高度，随机排列期望不适用，最坏栈长仍为全部前缀长度。标出模型能让读者知道结果何时可用，条件不是可随意删除的啰嗦话。

期望有限并不提供统一最坏上限，使用时要把两种承诺分开。

28.12 练习与答案

练习 1。 对任意随机变量 X、Y，是否有 E[max(X,Y)]=max(E[X],E[Y])？给出两点分布反例。

解答：以 1/2 概率(X,Y)=(1,0)，以 1/2 概率为(0,1)。E[X]=E[Y]=1/2，但 max(X,Y)恒为 1，因此 E[max]=1，大于 1/2。峰值先对每次样本取最大，再求平均，和先分别求平均不同。

练习 2。 势能法若允许 Φ_m 为很大负数，会失去什么？

解答：实际总成本=摊还总成本+Φ_0−Φ_m。若 Φ_m 可无限负，右侧多出来的−Φ_m 可能巨大，即使每步摊还小，也无法约束实际总成本。所以要保证势能下界，常用非负；或者准确计入一个统一有限下界。

练习 3。 均匀随机 5 元素排列，期望环数和最少交换次数各多少？

解答：H_5=1+1/2+1/3+1/4+1/5=137/60。期望最少交换为 5−137/60=163/60。期望可以不是整数，因为它是不同排列整数结果的加权平均。若输入不是均匀排列，H_5结论不自动成立。

29 把几种工具接起来，完成一整道题

一道综合题经常不会在题目里写“请用单调队列”或“请用动态规划”。你先得到操作与约束，再决定需要维护什么信息。本章每个例子先用小输入明确答案，再选择状态，最后算复杂度。做练习时可以暂时遮住解答，先自己写一张状态表。

29.1 例 1：订单队列，既要先进先出，又要随时知道最高金额

操作依次是入单 A 金额 5、入 B 金额 2、查询最高、出单、入 C 金额 7、查询最高、出单、查询最高。普通队列的出单必须按 A、B、C 顺序，最高金额却要在当前所有待处理订单里取最大。

只用普通队列，每次查最高需扫描。可用 09 章的双最大栈：每个栈项保存自己的金额和从底到当前的最大金额，队列最大是两栈最大者。in 存新单，out 为空时再整体倒入。

入 A、B 后 in 为(5,5)、(2,5)，最大 5。第一次出单倒栈：out 底为(2,2)，顶为(5,5)，弹 A 后 out 只剩 B，最大 2。入 C7 到 in，in 最大 7，out 最大 2，总最大 7。下一次出 B 后，仍由 in 给最大 7。整个过程从未让 C 越过 B 提前出单。

每项入 in、转 out、出 out 至多一次，更新栈内最大 O(1)，m 次操作总 O(m)；查询最大最坏 O(1)，某次出单仍可能因倒栈花线性时间。若系统要求每次请求都严格常数延迟，摊还保证还不够，需要别的实现或分摊迁移方案。

29.2 例 2：最长不降成绩链，状态里的等号决定答案

记录按到达顺序为`[3,1,2,2,4]`，找最长不下降子序列，允许跳过，不能重排。一个答案是 1、2、2、4，长度 4。若要求严格上升，则长度只有 3，例如 1、2、4。

先写 $O(n^2)$DP：f[i]为必须以第 i 项结束的最长长度，初值 1；对所有 j<i 且 a[j]≤a[i]，候选 f[j]+1。逐项得到 f=1、1、2、3、4。严格上升时把≤改<，得到 1、1、2、2、3。

再维护 tails[k]：长度 k+1 的不降子序列，当前能达到的最小末尾值。新值 x 应放在第一个严格大于 x 的位置，即 upper_bound，而不是第一个≥x。过程：3→[3]；1 替换→[1]；2 追加→[1,2]；第二个 2 追加→[1,2,2]；4 追加→[1,2,2,4]。长度 4。

为何较小末尾更好？长度相同，末尾较小的序列能接上的未来元素至少不比末尾较大的少。tails 只保存最佳接续条件，不保证数组 tails 本身对应某一条完整真实子序列；要恢复序列，需记录每项前驱及每长度的代表下标。每项二分 O(log n)，总 O(n log n)、额外 O(n)。

29.3 例 3：道路封闭，为什么一次 DFS 能找到关键路口

无向网络边为 1-2、2-3、3-1、3-4、4-5。想找“删哪个顶点会让连通块变多”和“删哪条边会断网”。逐个删除后重跑连通性很慢；DFS 的 low 值保存绕过父连接的能力。

从 1 搜索到 2、3、4、5，发现序号 1 到 5。3 看到祖先 1，low[3]降到 1；5 无回边，low[5]=5；回到 4，low[4]=4；回到 3 时，孩子 4 满足 low[4]=4≥dfn[3]=3，所以 3 为割点，且 4>3 说明 3-4 为桥。4 的孩子 5 也满足条件，所以 4 为割点，4-5 为桥。2 的孩子 3 有 low[3]=1<dfn[2]=2，子树能绕回 1，因此 2 不是割点。

根 1 虽 dfn 最小，但只有一个 DFS 树孩子 2，不是割点。1 与 3 的邻接不增加根树孩子数，因为 3 已在 2 的搜索中被发现。这个例子同时检查 low 更新、根例外与桥条件的严格不等号。

29.4 例 4：在树中求第 k 小，并支持反复插入

输入操作：插 8、插 3、插 10、插 1、插 6，查询从 1 起第 3 小；删 3，再查询第 3 小。使用按键平衡 BST，每节点维护子树大小。

插入后有序序列 1、3、6、8、10，第 3 小为 6。根 8 左大小 3，目标 k=3 在左子树；在节点 3 左大小 1，3 本身是第 2，目标去右侧，并将 k 减 2 变 1，返回 6。

删除 3 若用后继 6 替换，左子树变成根 6、左 1；此时全序 1、6、8、10，第 3 小是 8。沿删除路径更新 size，根 8 左大小从 3 降为 2，立即知道自己是第 3 项。若只更新高度却忘 size，结构仍平衡，查询却可能走错。

AVL 查找、插入、删除与第 k 小均 O(log n)，但每次旋转都必须同步修正 height 和 size。这个需求不适合普通散列表，因为哈希桶没有提供全局秩顺序。

29.5 例 5：矩形中的点，只求个数与报告全部成本不同

点为(1,1)、(2,4)、(3,2)、(4,5)、(5,3)，查询闭矩形[2,5]×[2,4]。逐点核对得到(2,4)、(3,2)、(5,3)，计数 3。

若已有 kd 树节点包围盒完全落在查询矩形内，求个数可直接加节点 size，不必访问每个点；若必须输出坐标，即使完全包含，也仍需遍历输出所有点，至少花 k 步。若包围盒完全不交，可跳过；部分相交才继续递归。

不要把“整棵子树被包含就 O(1)”推广到报告全部。它只对能由摘要回答的聚合查询成立。线段树整段求和能 O(1)取节点值，也是因为答案已被摘要保存，而非节点内部真的没有元素。

29.6 例 6：最便宜的接线方案，不是源点距离之和最小

边 A-B1、A-C2、B-C2、B-D3、C-D1。Kruskal 先取 A-B1、C-D1，得到两分量；下一条权 2 的 A-C 连接两分量，选入，MST 总权 4。若先取 B-C2 也一样得到 4。

源点 A 的最短距离为 B1、C2、D3，最短路树可取 A-B1、A-C2、C-D1，总权也是 4，这里两种目标恰巧一致。但这种巧合不能当通用规律；17 章三角形 2、3、2 就给出不同结果。检验概念时应找能区分方案的反例，而不是只验证一个两者相等的输入。

如果边权全部互异，MST 唯一，可以用割上最轻边交换证明。边权有重复，仍可能唯一，也可能不唯一；“有重复就不唯一”也不成立。例如一条本身已经是树的图，所有边都权 1，生成树只能是它自己。

29.7 例 7：数据流里第 k 大，先找不会被未来推翻的保留规则

输入逐项到达，无法保存全部数据，要求每次报告第 3 大。保留容量 3 的最小堆，根是当前保留集合里最小者。扫描 4、9、1 后，保留 1,4,9，答 1；来 7，7>根 1，替换后保留 4,7,9，答 4；来 2，2≤根 4，丢弃，答仍 4；来 10，替换 4，保留 7,9,10，答 7。

不变式是堆里恰好保存当前前缀最大的 3 条记录。新项不大于根时，已有 3 项不小于它，未来只会加入更多候选，它不可能重新挤进当前定义的前三；新项较大时，旧根是应淘汰者。重复值按记录计名次，相等时保留哪条身份可按并列规则决定。

对正整数 k，统一每项 O(log(k+1))，整个流 O(n log(k+1))，空间 O(k)；k≥2 时常简写 log k，k=1 仍为线性扫描。尚未读满 k 项时不存在第 k 大，接口应报告未就绪。若要求每次输出所有前三项的降序列表，还要额外排序或遍历输出，堆本身只给第 k 大和候选集合，不能把内部数组当作已排序结果。

29.8 例 8：同一条路径，BFS 与 Dijkstra 会作不同选择

边 S→T 权 10，S→A 权 1，A→B 权 1，B→T 权 1。BFS 从 S 会在一条边后发现 T，认为最少边数距离 1，父亲 S。Dijkstra 先发现 T 候选 10，但会先确定 A1、B2，再将 T 改 3，路径三条边却权更小。

若所有边权都恰好 1，两者优化目标一致，Dijkstra 变成按层距离处理，BFS 更简单且无需堆。若边权仅 0 或 1，可用双端队列的 0-1 BFS：0 权改善放队头、1 权改善放队尾，使待处理距离维持适当顺序。不能将任意正权也按“放队尾”处理而继续声称线性最短路。

29.9 练习 5：把两个问题的状态放在同一张表上

题目给有序数组并允许任意次点修改，既要区间总和又要查询第一个≥x。能否只维护一棵普通求和线段树便支持所有要求？

解答：若点修改可能破坏全局有序，原二分前提失效；若修改保证仍有序，可继续对底层数组二分，同时用线段树处理和。但只保存 sum 的节点无法判断区间内是否有≥x：同和的[1,9]与[5,5]对阈值 8 给不同答案，需要最大值等额外摘要。接口组合会改变要维护的信息，不能因为已有一棵树就假定它能回答所有问题。

29.10 练习 1：带预算的字符串插删

给 A=ABCA、B=ACBA，只允许插删，预算 k=2。能否完成？给一种方案，并说明带状 DP 为什么只算|i−j|≤2 还不够。

解答：保留公共子序列 ABA 或 ACA，长度 3；最少 4+4−2×3=2 步。取 ABA 时删 A 中第三个字符 C，得到 ABA，再在第一个 A 后插 C，得到 ACBA。带内条件只排除长度差过大的状态，当前两串各前缀同长

度也可能字符完全不同，仍必须计算插删成本。若 k=1，即使终点在对角线上，真实答案 2 也应判不满足预算。

29.11 练习 2：用比较下界审查一个承诺

有人称“对任意 n 个互异实数，只做两项大小比较，最坏比较次数不超过 3n”。这个承诺在哪些规模上必然不可能？

解答：决策树至少 n!叶，最坏深度至少 $\log_2(n!)$。后 n/2 项均至少 n/2，故下界至少$(n/2)\log_2(n/2)$。当 $\log_2(n/2)>6$，即 n>128 时，这个粗下界已大于 3n，可否定无限规模承诺。实际更精确阈值可用 $\log_2(n!)$计算；题目不需要给最小矛盾 n，只要证明足够大时无法成立。

若算法读取整数位做计数或基数排序，就违反“只做两项比较”的模型限定，下界不能按原样套；如果 n 固定上限很小，线性形式的一个特定常数也未必与有限域矛盾。

29.12 练习 3：原地修改以后，旧索引还可信吗

向量[1,3,5,7]的某指针指向 5，在秩 1 插 2，未发生扩容。旧地址仍可能可读，它现在对应哪个值？这与链表中在 3 前插 2 有何不同？

解答：从后往前搬后，数组为[1,2,3,5,7]，原下标 2 地址现在是 3，不再代表原元素 5。是否仍可合法使用旧迭代器还要看容器失效规则，但逻辑身份已变化。链表只将新节点链接到 1 和 3 之间，原 5 节点地址不变；如果节点未被删除，指向它的引用通常仍指同一元素。地址可读与逻辑元素身份是两个检查点。

29.13 练习 4：随机平均与固定输入

随机快排对一个固定数组做很多次不同随机轴选择，得到平均 O(n log n)。这是否说明其中每一次都不超过 cn log n？是否需要输入本身随机？

解答：不是每一次，仍可能连续选极端轴而平方时间。随机期望可以针对每一个固定输入，只对内部随机选择求平均，不必另假设输入随机。若代码根本没有随机化、总选首项，则不能引用这个“内部随机轴”的期望证明；要么改算法，要么另给输入分布。

29.14 把时间和空间放进最终答案

综合题给出一种方法后，应写清初始化、每次操作、总操作数与输出量。维护前 k 大的堆占 k 项，若另外又把全部输入存了一份，实际空间便不是只有 k；流式读入才能兑现小空间方案。

同样，区间查询若先花线性时间建树，单次对数不代表处理第一问的总成本只有对数。多次查询值得预处理，一次小查询可能直接扫描更简单。把准备与使用分开记账，选择才与实际任务一致。

29.15 做综合题时留下的五份证据

先写一个自己能手算的小实例，再定义状态每一项代表什么。每次删候选、跳子树或确定距离时，写出让这步安全的条件；随后给完整状态追踪，最后把每个元素、边或状态被处理几次记账。答案、正确性、时间、空间和实现边界，应该能从这些具体证据中读出来。

若只需判断可行与否，找到第一份合法方案就可以停止；要计数则必须继续覆盖全部方案，停止规则不同会直接改变答案。

30 PS04 的 21 个表达式：每次弹栈究竟算了什么

只列 RPN 与最终值，会把最需要学习的中间过程藏起来。本章逐式展示运算发生时读到的操作数、局部结果及剩余栈。数字入栈的动作不另占表格一行；每一行运算都已包含相应数字按 RPN 顺序压入后的状态。栈栏从底到顶书写，二元运算先弹右操作数、再弹左操作数。

本章把!!解释为双阶乘，阶乘是高优先级后缀运算，除法保留精确有理数。教师原版优先表的^对^是先弹栈，即左结合；若修改成数学常用右结合，需改相等优先级规则。下面 21 式没有连续且未用括号区分的乘方链，所以这项结合性差异不改变其 RPN 与数值。真实课程浮点程序还可能有舍入；若改用 C++整数除法，含中间分数的表达式结果可能改变，不能混为一套答案。

例如 `8 3 -`中先弹 3 再弹 8，算的是 8−3。`3 4 /`给 3/4。`0 !`给 1，因为 0!=1；`3 !!`给 3，`4 !!`给 8，不能把!!当成重复执行两次!。

30.1 第 1 式

原式：`(0+1+2-3+4+5+6)*(7+8)*9`

先把左括号内的连加减从左到右算完，得到 15。另一个括号 7+8 也是 15，最后 15×15×9。两个独立子表达式只有在各自算完后才交给乘法。

```
0 1 + 2 + 3 - 4 + 5 + 6 + 7 8 + * 9 *
```

读到的运算符	取出的操作数与局部结果	运算后的栈
+	0 + 1 = 1	1
+	1 + 2 = 3	3
−	3 - 3 = 0	0
+	0 + 4 = 4	4
+	4 + 5 = 9	9
+	9 + 6 = 15	15
+	7 + 8 = 15	15, 15
*	15 * 15 = 225	225
*	225 * 9 = 2025	2025

最终值为 2025，数值栈峰值为 3 项。最后只剩一个值，满足完整表达式的结构检查。

30.2 第 2 式

原式：`(0*1+2)*3!/4!*5*6*(7+8)*9`

先算 0×1+2=2，阶乘 3!=6、4!=24。乘除同级按左结合：2×6÷24=1/2，再乘 5、6、15、9。中间出现分数并不代表错误，不能在这里擅自做整数截断。

```
0 1 * 2 + 3 ! * 4 ! / 5 * 6 * 7 8 + * 9 *
```

读到的运算符	取出的操作数与局部结果	运算后的栈
*	0 * 1 = 0	0
+	0 + 2 = 2	2
!	3! = 6	2, 6
*	2 * 6 = 12	12
!	4! = 24	12, 24

读到的运算符	取出的操作数与局部结果	运算后的栈
/	12 / 24 = 1/2	1/2
*	1/2 * 5 = 5/2	5/2
*	5/2 * 6 = 15	15
+	7 + 8 = 15	15, 15
*	15 * 15 = 225	225
*	225 * 9 = 2025	2025

最终值为 2025，数值栈峰值为 3 项。最后只剩一个值，满足完整表达式的结构检查。

30.3 第 3 式

原式：`0!+(1+2)!!*3-4-5!-6-7-8+9`

0!=1，括号 1+2=3，3!!=3，所以前面 1+3×3=10。另一个阶乘 5!=120，随后从左到右减 4、120、6、7、8，再加 9。注意双阶乘与连续两次阶乘的差别。

```
0 ! 1 2 + !! 3 * + 4 - 5 ! - 6 - 7 - 8 - 9 +
```

读到的运算符	取出的操作数与局部结果	运算后的栈
!	0! = 1	1
+	1 + 2 = 3	1, 3
!!	3!! = 3	1, 3
*	3 * 3 = 9	1, 9
+	1 + 9 = 10	10
-	10 - 4 = 6	6
!	5! = 120	6, 120
-	6 - 120 = −114	−114
-	−114 - 6 = −120	−120
-	−120 - 7 = −127	−127
-	−127 - 8 = −135	−135
+	−135 + 9 = −126	−126

最终值为-126，数值栈峰值为 3 项。最后只剩一个值，满足完整表达式的结构检查。

30.4 第 4 式

原式：`(0+1+2)/3!/4!*5!*6*(7+8)*9`

括号和为 3。依次除 3!=6 与 4!=24，得到 1/48，再乘 5!=120 得到 5/2，继续乘 6、15、9。若每次除法都截成整数，第一步 3/6 就变 0，整式会完全换掉。

```
0 1 + 2 + 3 ! / 4 ! / 5 ! * 6 * 7 8 + * 9 *
```

读到的运算符	取出的操作数与局部结果	运算后的栈
+	0 + 1 = 1	1
+	1 + 2 = 3	3
!	3! = 6	3, 6

读到的运算符	取出的操作数与局部结果	运算后的栈
/	3 / 6 = 1/2	1/2
!	4! = 24	1/2, 24
/	1/2 / 24 = 1/48	1/48
!	5! = 120	1/48, 120
*	1/48 * 120 = 5/2	5/2
*	5/2 * 6 = 15	15
+	7 + 8 = 15	15, 15
*	15 * 15 = 225	225
*	225 * 9 = 2025	2025

最终值为 2025，数值栈峰值为 3 项。最后只剩一个值，满足完整表达式的结构检查。

30.5 第 5 式

原式：`(0!^1/2*3!!/(4+5)*6-7-8)*9`

0!=1，1^1=1。接下来除 2、乘 3!!=3、除(4+5)=9、乘 6，连续乘除结果是 1。括号内再减 7、减 8 得−14，最后乘 9。这里的负值由明确的减法链产生。

```
0 ! 1 ^ 2 / 3 !! * 4 5 + / 6 * 7 - 8 - 9 *
```

读到的运算符	取出的操作数与局部结果	运算后的栈
!	0! = 1	1
^	1 ^ 1 = 1	1
/	1 / 2 = 1/2	1/2
!!	3!! = 3	1/2, 3
*	1/2 * 3 = 3/2	3/2
+	4 + 5 = 9	3/2, 9
/	3/2 / 9 = 1/6	1/6
*	1/6 * 6 = 1	1
-	1 - 7 = −6	−6
-	−6 - 8 = −14	−14
*	−14 * 9 = −126	−126

最终值为-126，数值栈峰值为 3 项。最后只剩一个值，满足完整表达式的结构检查。

30.6 第 6 式

原式：`((0*(1-2)+3+4)*5*6+(7+8))*9`

先算 1−2=−1，再乘 0 得到 0；随后加 3、加 4 得 7。7×5×6=210，另一括号 7+8=15，相加 225 再乘 9。乘零会消去一项数值，但解析器仍应正确识别其完整子表达式。

```
0 1 2 - * 3 + 4 + 5 * 6 * 7 8 + + 9 *
```

读到的运算符	取出的操作数与局部结果	运算后的栈
-	1 - 2 = −1	0, −1

读到的运算符	取出的操作数与局部结果	运算后的栈
*	0 * −1 = 0	0
+	0 + 3 = 3	3
+	3 + 4 = 7	7
*	7 * 5 = 35	35
*	35 * 6 = 210	210
+	7 + 8 = 15	210, 15
+	210 + 15 = 225	225
*	225 * 9 = 2025	2025

最终值为 2025，数值栈峰值为 3 项。最后只剩一个值，满足完整表达式的结构检查。

30.7 第 7 式

原式：`(0!-1!+2)*3!/4!*5*6*(7+8)*9`

0!和 1!都等于 1，所以第一个括号 1−1+2=2，后续链与第 2 式相同。这样可以把两个表达式做变形核对：不同写法确实产生同一个中间值 2。

```
0 ! 1 ! - 2 + 3 ! * 4 ! / 5 * 6 * 7 8 + * 9 *
```

读到的运算符	取出的操作数与局部结果	运算后的栈
!	0! = 1	1
!	1! = 1	1, 1
-	1 - 1 = 0	0
+	0 + 2 = 2	2
!	3! = 6	2, 6
*	2 * 6 = 12	12
!	4! = 24	12, 24
/	12 / 24 = 1/2	1/2
*	1/2 * 5 = 5/2	5/2
*	5/2 * 6 = 15	15
+	7 + 8 = 15	15, 15
*	15 * 15 = 225	225
*	225 * 9 = 2025	2025

最终值为 2025，数值栈峰值为 3 项。最后只剩一个值，满足完整表达式的结构检查。

30.8 第 8 式

原式：`((0-1)^2*(3!!/4-5+6*7+8))*9`

0−1=−1，其平方为 1。另一个括号中 3!!/4=3/4，6×7=42，合并成 3/4−5+42+8=183/4，最后乘 9，得到 1647/4。不要把 3/4 提前舍去。

```
0 1 - 2 ^ 3 !! 4 / 5 - 6 7 * + 8 + * 9 *
```

读到的运算符	取出的操作数与局部结果	运算后的栈
-	0 - 1 = −1	−1
^	(−1) ^ 2 = 1	1
!!	3!! = 3	1, 3
/	3 / 4 = 3/4	1, 3/4
-	3/4 - 5 = −17/4	1, −17/4
*	6 * 7 = 42	1, −17/4, 42
+	−17/4 + 42 = 151/4	1, 151/4
+	151/4 + 8 = 183/4	1, 183/4
*	1 * 183/4 = 183/4	183/4
*	183/4 * 9 = 1647/4	1647/4

最终值为 1647/4，数值栈峰值为 4 项。最后只剩一个值，满足完整表达式的结构检查。

30.9 第 9 式

原式：`((0-1)^2*(3!!/4-5-6+7*8))*9`

左侧平方仍为 1。右括号是 3/4−5−6+7×8=3/4−11+56=183/4，与第 8 式巧合相等，但子表达式不同，分别算清才能确认。

```
0 1 - 2 ^ 3 !! 4 / 5 - 6 - 7 8 * + * 9 *
```

读到的运算符	取出的操作数与局部结果	运算后的栈
-	0 - 1 = −1	−1
^	(−1) ^ 2 = 1	1
!!	3!! = 3	1, 3
/	3 / 4 = 3/4	1, 3/4
-	3/4 - 5 = −17/4	1, −17/4
-	−17/4 - 6 = −41/4	1, −41/4
*	7 * 8 = 56	1, −41/4, 56
+	−41/4 + 56 = 183/4	1, 183/4
*	1 * 183/4 = 183/4	183/4
*	183/4 * 9 = 1647/4	1647/4

最终值为 1647/4，数值栈峰值为 4 项。最后只剩一个值，满足完整表达式的结构检查。

30.10 第 10 式

原式：`(0!+1+2)!+(3!-4)^(5+6)-7*8+9`

0!+1+2=4，外面的阶乘得到 24。3!−4=2，5+6=11，乘方 2^11=2048。最后 24+2048−56+9=2025。阶乘作用于整个括号，而不是只作用于末项 2。

```
0 ! 1 + 2 + ! 3 ! 4 - 5 6 + ^ + 7 8 * - 9 +
```

读到的运算符	取出的操作数与局部结果	运算后的栈
!	0! = 1	1

读到的运算符	取出的操作数与局部结果	运算后的栈
+	1 + 1 = 2	2
+	2 + 2 = 4	4
!	4! = 24	24
!	3! = 6	24, 6
−	6 - 4 = 2	24, 2
+	5 + 6 = 11	24, 2, 11
^	2 ^ 11 = 2048	24, 2048
+	24 + 2048 = 2072	2072
*	7 * 8 = 56	2072, 56
−	2072 - 56 = 2016	2016
+	2016 + 9 = 2025	2025

最终值为 2025，数值栈峰值为 4 项。最后只剩一个值，满足完整表达式的结构检查。

30.11 第 11 式

原式：`(0+(1+2))^3/(4+5)*6!-(7+8)*9`

内括号 1+2=3，加 0 仍 3，立方 27。4+5=9，所以 27/9=3，再乘 6!=720 得到 2160；减去(7+8)×9=135，结果 2025。

```
0 1 2 + + 3 ^ 4 5 + / 6 ! * 7 8 + 9 * −
```

读到的运算符	取出的操作数与局部结果	运算后的栈
+	1 + 2 = 3	0, 3
+	0 + 3 = 3	3
^	3 ^ 3 = 27	27
+	4 + 5 = 9	27, 9
/	27 / 9 = 3	3
!	6! = 720	3, 720
*	3 * 720 = 2160	2160
+	7 + 8 = 15	2160, 15
*	15 * 9 = 135	2160, 135
−	2160 - 135 = 2025	2025

最终值为 2025，数值栈峰值为 3 项。最后只剩一个值，满足完整表达式的结构检查。

30.12 第 12 式

原式：`((0−1)^2!*(3!!/4−5+6*7+8))*9`

与第 8 式相比，指数由 2 变成 2!，二者恰好都为 2，因此数值仍 1647/4。RPN 必须保留“先算 2!再乘方”的过程，不能因为最后相同就抄成同一个记号串。

```
0 1 − 2 ! ^ 3 !! 4 / 5 − 6 7 * + 8 + * 9 *
```

读到的运算符	取出的操作数与局部结果	运算后的栈
−	0 - 1 = −1	−1

读到的运算符	取出的操作数与局部结果	运算后的栈
!	2! = 2	−1, 2
^	(−1) ^ 2 = 1	1
!!	3!! = 3	1, 3
/	3 / 4 = 3/4	1, 3/4
−	3/4 - 5 = −17/4	1, −17/4
*	6 * 7 = 42	1, −17/4, 42
+	−17/4 + 42 = 151/4	1, 151/4
+	151/4 + 8 = 183/4	1, 183/4
*	1 * 183/4 = 183/4	183/4
*	183/4 * 9 = 1647/4	1647/4

最终值为 1647/4，数值栈峰值为 4 项。最后只剩一个值，满足完整表达式的结构检查。

30.13 第 13 式

原式：`((0−1)^2!*(3!!/4−5−6+7*8))*9`

与第 9 式同理，指数 2!先得到 2；括号 3/4−5−6+56=183/4，结果 1647/4。这里可用独立简化式核对栈结果。

```
0 1 − 2 ! ^ 3 !! 4 / 5 − 6 − 7 8 * + * 9 *
```

读到的运算符	取出的操作数与局部结果	运算后的栈
−	0 - 1 = −1	−1
!	2! = 2	−1, 2
^	(−1) ^ 2 = 1	1
!!	3!! = 3	1, 3
/	3 / 4 = 3/4	1, 3/4
−	3/4 - 5 = −17/4	1, −17/4
−	−17/4 - 6 = −41/4	1, −41/4
*	7 * 8 = 56	1, −41/4, 56
+	−41/4 + 56 = 183/4	1, 183/4
*	1 * 183/4 = 183/4	183/4
*	183/4 * 9 = 1647/4	1647/4

最终值为 1647/4，数值栈峰值为 4 项。最后只剩一个值，满足完整表达式的结构检查。

30.14 第 14 式

原式：`(0!+1)*(2+3)*(4+5)/6*(7+8)*9`

三个括号分别为 2、5、9。乘除链先得 2×5×9/6=15，再乘 15 和 9，得到 2025。除法不是一个括号把后面所有乘项包起来，必须尊重左结合。

```
0 ! 1 + 2 3 + * 4 5 + * 6 / 7 8 + * 9 *
```

读到的运算符	取出的操作数与局部结果	运算后的栈
!	0! = 1	1

读到的运算符	取出的操作数与局部结果	运算后的栈
+	1 + 1 = 2	2
+	2 + 3 = 5	2, 5
*	2 * 5 = 10	10
+	4 + 5 = 9	10, 9
*	10 * 9 = 90	90
/	90 / 6 = 15	15
+	7 + 8 = 15	15, 15
*	15 * 15 = 225	225
*	225 * 9 = 2025	2025

最终值为 2025，数值栈峰值为 3 项。最后只剩一个值，满足完整表达式的结构检查。

30.15 第 15 式

原式：`(0!^1/2*3!!/(4+5)*6-(7+8))*9`

前半连续乘除仍得到 1；后面明确减去括号 7+8=15，所以括号值−14，乘 9 得−126。它与第 5 式的“先减 7 再减 8”在这个位置等价。

```
0 ! 1 ^ 2 / 3 !! * 4 5 + / 6 * 7 8 + - 9 *
```

读到的运算符	取出的操作数与局部结果	运算后的栈
!	0! = 1	1
^	1 ^ 1 = 1	1
/	1 / 2 = 1/2	1/2
!!	3!! = 3	1/2, 3
*	1/2 * 3 = 3/2	3/2
+	4 + 5 = 9	3/2, 9
/	3/2 / 9 = 1/6	1/6
*	1/6 * 6 = 1	1
+	7 + 8 = 15	1, 15
-	1 - 15 = −14	−14
*	−14 * 9 = −126	−126

最终值为-126，数值栈峰值为 3 项。最后只剩一个值，满足完整表达式的结构检查。

30.16 第 16 式

原式：`((0-1)^2*(3!!/4-5*(6-7-8)))*9`

0−1 的平方为 1。最内层 6−7−8 按左结合是−9，不是 6−(7−8)=7。于是 3/4−5×(−9)=183/4，最后乘 9，得 1647/4。

```
0 1 - 2 ^ 3 !! 4 / 5 6 7 - 8 - * - * 9 *
```

读到的运算符	取出的操作数与局部结果	运算后的栈
-	0 - 1 = −1	−1
^	(−1) ^ 2 = 1	1

读到的运算符	取出的操作数与局部结果	运算后的栈
!!	3!! = 3	1, 3
/	3 / 4 = 3/4	1, 3/4
-	6 - 7 = −1	1, 3/4, 5, −1
-	−1 - 8 = −9	1, 3/4, 5, −9
*	5 * −9 = −45	1, 3/4, −45
-	3/4 - −45 = 183/4	1, 183/4
*	1 * 183/4 = 183/4	183/4
*	183/4 * 9 = 1647/4	1647/4

最终值为 1647/4，数值栈峰值为 5 项。最后只剩一个值，满足完整表达式的结构检查。

30.17 第 17 式

原式：`((0-1)^2!*(3!!/4-5*(6-7-8)))*9`

与第 16 式相比，指数为 2!，仍等于 2；其他过程相同。这种配对适合检查解析器是否把后缀阶乘正确附到指数上。

```
0 1 - 2 ! ^ 3 !! 4 / 5 6 7 - 8 - * - * 9 *
```

读到的运算符	取出的操作数与局部结果	运算后的栈
-	0 - 1 = −1	−1
!	2! = 2	−1, 2
^	(−1) ^ 2 = 1	1
!!	3!! = 3	1, 3
/	3 / 4 = 3/4	1, 3/4
-	6 - 7 = −1	1, 3/4, 5, −1
-	−1 - 8 = −9	1, 3/4, 5, −9
*	5 * −9 = −45	1, 3/4, −45
-	3/4 - −45 = 183/4	1, 183/4
*	1 * 183/4 = 183/4	183/4
*	183/4 * 9 = 1647/4	1647/4

最终值为 1647/4，数值栈峰值为 5 项。最后只剩一个值，满足完整表达式的结构检查。

30.18 第 18 式

原式：`((0!+1!)^2+(3!-4)+5*6)*(7*8)+9`

0!+1!=2，平方为 4；3!−4=2；5×6=30，括号合计 36。7×8=56，36×56=2016，再加 9 得 2025。

```
0 ! 1 ! + 2 ^ 3 ! 4 - + 5 6 * + 7 8 * * 9 +
```

读到的运算符	取出的操作数与局部结果	运算后的栈
!	0! = 1	1
!	1! = 1	1, 1
+	1 + 1 = 2	2

读到的运算符	取出的操作数与局部结果	运算后的栈
^	2 ^ 2 = 4	4
!	3! = 6	4, 6
−	6 - 4 = 2	4, 2
+	4 + 2 = 6	6
*	5 * 6 = 30	6, 30
+	6 + 30 = 36	36
*	7 * 8 = 56	36, 56
*	36 * 56 = 2016	2016
+	2016 + 9 = 2025	2025

最终值为 2025，数值栈峰值为 3 项。最后只剩一个值，满足完整表达式的结构检查。

30.19 第 19 式

原式：`((0!+1)*2)!+(3!-4)^(5+6)-7*8+9`

0!+1=2，再乘 2 得 4，整个括号阶乘为 24。另一路 3!−4=2、5+6=11，得到 2048，最终 24+2048−56+9=2025。

```
0 ! 1 + 2 * ! 3 ! 4 − 5 6 + ^ + 7 8 * − 9 +
```

读到的运算符	取出的操作数与局部结果	运算后的栈
!	0! = 1	1
+	1 + 1 = 2	2
*	2 * 2 = 4	4
!	4! = 24	24
!	3! = 6	24, 6
−	6 - 4 = 2	24, 2
+	5 + 6 = 11	24, 2, 11
^	2 ^ 11 = 2048	24, 2048
+	24 + 2048 = 2072	2072
*	7 * 8 = 56	2072, 56
−	2072 - 56 = 2016	2016
+	2016 + 9 = 2025	2025

最终值为 2025，数值栈峰值为 4 项。最后只剩一个值，满足完整表达式的结构检查。

30.20 第 20 式

原式：`((0!^1+2)!!/(3!-4-5+6)-(7+8))*9`

0!^1=1，加 2 为 3，双阶乘 3!!=3。分母 3!−4−5+6 按左结合为 6−4−5+6=3，所以商 1；减 15 得−14，再乘 9 得−126。

```
0 ! 1 ^ 2 + !! 3 ! 4 − 5 − 6 + / 7 8 + − 9 *
```

读到的运算符	取出的操作数与局部结果	运算后的栈
!	0! = 1	1

读到的运算符	取出的操作数与局部结果	运算后的栈
^	1 ^ 1 = 1	1
+	1 + 2 = 3	3
!!	3!! = 3	3
!	3! = 6	3, 6
−	6 - 4 = 2	3, 2
−	2 - 5 = −3	3, −3
+	−3 + 6 = 3	3, 3
/	3 / 3 = 1	1
+	7 + 8 = 15	1, 15
−	1 - 15 = −14	−14
*	−14 * 9 = −126	−126

最终值为-126，数值栈峰值为 3 项。最后只剩一个值，满足完整表达式的结构检查。

30.21 第 21 式

原式：`((0!^1+2)!!/(3!-(4+(5-6)))-(7+8))*9`

分母的嵌套必须逐层算：5−6=−1，4+(−1)=3，3!−3=3。分子仍 3，故商 1，减 15 再乘 9 得−126。它与第 20 式分母数值相同，但 RPN 的减法次序不同。

```
0 ! 1 ^ 2 + !! 3 ! 4 5 6 - + - / 7 8 + - 9 *
```

读到的运算符	取出的操作数与局部结果	运算后的栈
!	0! = 1	1
^	1 ^ 1 = 1	1
+	1 + 2 = 3	3
!!	3!! = 3	3
!	3! = 6	3, 6
−	5 - 6 = −1	3, 6, 4, −1
+	4 + −1 = 3	3, 6, 3
−	6 - 3 = 3	3, 3
/	3 / 3 = 1	1
+	7 + 8 = 15	1, 15
−	1 - 15 = −14	−14
*	−14 * 9 = −126	−126

最终值为-126，数值栈峰值为 5 项。最后只剩一个值，满足完整表达式的结构检查。

30.22 合上表格，试着自己找错

练习 1。 第 2 式如果使用 C++整数除法，中间 `2*6/24` 会变成多少？最后还会是 2025 吗？

解答：2×6=12，整数 12/24 截为 0，后面无论乘多少仍为 0，最后结果 0。表达式字符串相同，数值类型与除法语义不同，计算问题就不同。本章有理数表适合精确核对数学值，不能直接冒充整数程序的输出。

练习 2。 第 16 式把 `6-7-8` 错读成 `6-(7-8)`，会怎样影响括号？

解答：正确为(6−7)−8=−9，错误为 6−(−1)=7。正确的 3/4−5×(−9)=183/4，错读后成 3/4−35=−137/4。RPN 中应是 `6 7 − 8 −`；若变成 `6 7 8 − −`，就是另一棵语法树。比较 RPN 可以在最终乘 9 之前就定位错误。

练习 3。 第 12 式与第 8 式结果相同，能否说明它们是同一个语法结构?

解答：不能。第 12 式需要先算指数 2!，第 8 式直接用 2；只是 2!=2 造成数值相同。换成指数 3 与 3!就会不同。测试解析器时需要既有结果不同的区分例，也有结果相同但结构不同的例子，避免把错误结构碰巧算对当成解析正确。

本章的数值栈峰值是求值 RPN 所需的空间，不是中缀转 RPN 时的运算符栈峰值。后者还取决于优先表、括号与底部哨兵计数；两种栈保存不同对象，不能拿一个峰值替另一个答案。

31 换掉数字以后，题目还剩下什么方法

考试题换一组数字、换一个变量名，背下来的答案就可能失效。能迁移的是你怎样决定循环轮数、怎样记录扩容账、怎样将树形转成递推，以及怎样检查归约的双向条件。本章使用重新设定的例子讲这些方法；本地课内样卷原页与完整题面不公开转载。

31.1 例 1：步长自己增加，先算累计走了多远

程序 `i=0,j=0; while(i<n){ i+=j; ++j; }`，看上去第一次 i 没有变化，很容易误判死循环。完整状态却从 (i,j)=(0,0)变(0,1)，下一轮会前进 1。

取 n=15，各轮结束状态依次(0,1)、(1,2)、(3,3)、(6,4)、(10,5)、(15,6)，共 6 轮。第 k 轮以后 j=k，i=0+1+…+(k−1)=k(k−1)/2。要退出需该值≥n，最小 k 是 `ceil((1+sqrt(1+8n))/2)`，在 n>0 时按对应整数边界理解，阶为 Θ(√n)。

这类程序的控制量不是每轮固定加 1，所以应先求累计进度。若改成 `i += j*j; ++j`，k 轮进度为平方和 Θ(k³)，轮数变 Θ(n^(1/3))；若 `i += 2^j`，累计进度为 Θ(2^k)，轮数变 Θ(log n)。先写和式，再反解阈值，方法是同一套。

真实整数计算精确轮数时，应避免浮点平方根在大数边界上的舍入；可先估算再用整数不等式修正，或直接二分最小 k。若题目只问渐近阶，无需为求根引入不必要的数值误差。

31.2 例 2：非几何扩容，按容量区段分别记账

容量 C 满了后增加约√C。每次扩容搬 C 项，但增长量越来越大，比固定加 1 好，又比固定比例增长慢。怎样算做 n 次追加的总复制？

先分容量段[M,2M]。在这段中，每次增加 Θ(√M)格，要跨过总长度 Θ(M)，需要 Θ(√M)次扩容；每次复制 Θ(M)项，因此该段复制总成本 Θ(M^(3/2))。

把容量段取 1 到 2、2 到 4、4 到 8……，各段成本按比例 2^(3/2)增长，总和由最后一段的常数倍控制，直到 n 附近得 Θ(n^(3/2))。再加 n 次写入，仍同阶。这比直接写积分更容易看清上、下界分别来自哪里。

积分估算也一致：容量增加 dC 时约经历 dC/√C 次扩容，每次搬 C 项，所以总成本近似 $\int C/\sqrt{C}\,dC = \Theta(n^{3/2})$。积分这里是连续化记账的辅助，严格离散界可由上面的分段夹逼保证。

一般增加 Θ(C^α)，固定 0≤α<1，同一容量段需 Θ(M^(1−α))次，每次 Θ(M)，总 Θ(M^(2−α))，全程 Θ(n^(2−α))。α=0 得到平方总量；α=1 的固定比例增长要单独按几何级数算，得到线性总量。若 α 随 C 变化、增长有额外跳跃，就不能无条件套同一公式。

31.3 例 3：表达式树的栈峰值，左右顺序会改变答案

固定先计算左子树、再右子树、最后做二元运算。叶子只压一个值，S(叶)=1。算右子树时，左边的结果还占着一格，因此

$$S(T) = \max(S(L), 1 + S(R)).$$

看三棵有 4 个叶子的树。`(((a+b)+c)+d)`依次压 a、b 做和，再压 c、d，峰值 2。`a+(b+(c+d))`先压 a 等待，再压 b 等待，再压 c、d，峰值 4。`(a+b)+(c+d)`左边需 2 格，留下 1 格；右边再需 2 格，总峰值 3。

它们节点数相同，树高与峰值也不是一一对应。右边有多复杂，决定左侧已算结果要等多久；递推比凭图形猜更可靠。如果运算可交换且允许调整求值顺序，可以选择先计算需求更大的一侧，得到更小峰值；但减法、除法仍要保留操作数左右身份，带副作用的语言表达式也不能随意重排执行。

若限制每个内部节点左子树规模至少右子树规模，要达到峰值 s，树至少需要约 2^(s−1)−1 个内部节点。理由：使峰值增加的是进入右侧时多占一格，而右子树能达到 s−1 时，左侧至少同样大。令最小内部节点数 N_s，$N_1=0$，递推 N_s≥2N_(s−1)+1，得到该指数下界。因此 k 个内部节点的峰值不超过 $1+\mathrm{floor}(\log_2(k+1))$ 附近的准确取整界。

完美二叉表达式树有 15 个内部节点时，叶子 16 个，从左到右后序求值峰值 5，恰好达到这个关系。向左退化的 15 内部节点长树仍只需 2 格，说明下界说的是“达到某峰值至少要多大”，不是所有同规模树都恰好该峰值。

31.4 例 4：逆序数要留多少在某段内部

长度 n 的数组，共有 n(n−1)/2 对下标。指定连续区间长度 m，完全在内部的对有 m(m−1)/2，其他所有对共有两者之差。因此若整个数组逆序数为 I，区间内部逆序数至少

$$\max\left(0, I - \frac{n(n-1) - m(m-1)}{2}\right).$$

取 n=6、m=4、I=13。全部 15 对，内部最多 6 对，外部或跨边界最多 9 对，所以内部至少 13−9=4 对。这个推理用的是“外部最多能容纳多少”，不需要知道每个具体元素值。

但这个下界未必对每种附加条件都能达到。若想说“最少恰好 4”，还要构造一个符合所有条件的实际排列，且总逆序数确为 13、内部为 4。相减给了必要界，存在性是另一份工作。

可以用一个可核对的构造练习：取数组`[6,4,5,3,2,1]`，总逆序数为 14，前 4 项内部逆序为 5。若区间位置、总逆序数都固定，要通过跨边界数值分布调节，而非任意指定“内部要几对、外部就放剩下几对”便自动可行。

31.5 例 5：线性归约，要证明真解与假解都跟着走

将互异实数 x 映射成平面点(x,x^3)。对三个不同 a、b、c，AB 斜率为 a^2+ab+b^2，AC 斜率为 a^2+ac+c^2。相减得到

$$(b-c)(a+b+c).$$

因为 b≠c，两斜率相等当且仅当 a+b+c=0。所以三个数和为 0，恰好对应三点共线。

取 a=−2、b=−1、c=3，和 0；三点为(−2,−8)、(−1,−1)、(3,27)。前两点斜率 7，第一与第三点斜率 35/5=7，确实共线。换 c=2，和−1，AC 斜率(8+8)/4=4，不共线。

证明归约还要检查：互异 x 映射后横坐标不同，不会合并输入；若原问题允许重复数字，需单独处理重复引起的退化；构造每点的 x^3 在实数 RAM 中可用常数次乘法，n 点总 O(n)，在大整数位模型中乘法成本又不同；从共线答案回取三项也需合法映射。只证明“零和能产生共线”不足以排除伪解，等价方向必须完整。

归约方向也决定结论：若 A 可快速转 B，B 的快算法能帮 A；已知 A 很难可用于限制 B，而不是反过来拿 B 难就说 A 一定难。箭头从“要解决的旧问题”指向“调用的新问题”。

31.6 遍历信息缺少时，先检查能不能唯一确定

给先序与后序，不含空标记，二节点 A-B 的孩子可以在左也可以在右，无法唯一恢复一般二叉树。给父指针与叶序，则还要知道孩子是否有序、标签是否互异、是否允许单孩子，才能判断有哪些结构可恢复。

某题要求线性遍历，先列出每步可用信息：当前节点、父节点、下一片待处理叶子、哪些边已经走过。复杂度按边记账。每条边下去一次、上来一次，总 O(n)；若每处理一片叶子都从根重新走一遍祖先链，退化树或梳形树可能多出高度因子。算法最后访问了每节点一次，并不意味着寻找“下一个节点”的成本也只一次。

31.7 对数级递归深度，不等于每层只做一次工作

求和分治有 log n 层，每层许多子问题；递归栈只同时保存一条路径，时间却要把整层全部节点算上。若每层总成本 n，时间 n log n，栈 log n；若每节点常数且叶数 n，时间 n，栈仍 log n。题目同时问时间空间时，先分别画调用树与当前活动路径，避免用一个图上的数代替两个指标。

表达式求值空间也要看顺序。已得左值放在数值栈中，处理右树时它仍占格；调用栈中则还保存父节点待返回的工作。显式 RPN 栈与递归语法树求值的峰值不一定逐项相同，比较前必须说明统计哪些对象。

31.8 反例怎样做得足够小

若要反驳“先序加后序唯一”，两节点就足够，左右单孩子是两种树。若要反驳“最后最大值选择排序稳定”，三项有时不够展示另一种相等键被跨越，06 章四记录反例能直接显示。若要反驳“logT 同阶 logf 推出 T 同阶 f”，T=n^2、f=n 即可。反例不需要覆盖所有失败方式，只需满足前提且违反结论。

给反例时逐项核对前提，特别是互异、非空、严格单调与允许操作。拿含负边图反驳“非负边 Dijkstra 正确”无效，因为它不满足前提；拿负边反驳“Dijkstra 对任意权正确”有效。一个严谨反例与一个严谨证明同样需要清楚的输入范围。

31.9 从程序返回值恢复复杂度时，画两层依赖图

第 32 章 G(m)返回 m^2却只花 m 步，外层 G(G(n))应先把内层值 n^2写到参数箭头上，再把外层成本 $T_G(n^2)=\Theta(n^2)$写到账本。若只看两个函数名就说两个线性过程相加，漏了第二次参数；若把返回 n^4当成本，又多算了不实际发生的工作。

同理，辅助函数在 while 条件里并非“免费判断”。条件每检查一次就真实调用一次，即使循环主体最后不执行，退出检查也可能花钱。只有程序缓存并复用结果时，才能把多次相同子问题降为一次，不能替源代码凭空假定它已经记忆化。

31.10 把精确答案和渐近答案分开交付

题目问循环恰好几次，应保留端点与取整；问渐近阶，可以在上下界之后简化。第一轮计数器不变、末尾退出检查多调用一次、确认有序的最后一趟，都可能让精确答案差一，却不改变 Θ。若题目正是在考这些边界，只报大 O 等于没有回答。

树高按边数时单节点为零，按节点数时为一；节点数固定，最小高度的取整公式会相差一。算法书中常同时出现两种传统，先把具体一节点与二节点例子代入检查，比靠记忆选择 floor 或 ceil 更稳妥。

证明“至少需要”通常给下界，证明“某构造可以做到”给上界。两者吻合才得最优；一个计数下界还没附构造时，不要直接用“最少恰好”结尾。31 章逆序区间与比较下界都在使用这一区分。

源程序若假设无限机器字长，可以在该模型里推导指数移位；真实实验要限制参数范围，不能让溢出后的怪异输出反过来否定抽象分析。两种模型各自对应一份明确问题，核对时先确认正在验证哪一份。

数学下界与实际可达构造需要分别核对，不能互相代替。

31.11 练习与答案

练习 1。 容量每次增加 Θ(C^(1/3))，做 n 次追加，总复制多少？

解答：容量段[M,2M]内需 Θ(M^(2/3))次，每次搬 Θ(M)，得 Θ(M^(5/3))；各倍增容量段相加由最后段主导，得 Θ(n^(5/3))。摊还每次为 Θ(n^(2/3))，不是常数。这里 n 指到达规模，若操作有大量删除需另作序列分析。

练习 2。 表达式(a+b)*(c+(d+e))固定左先右后，RPN 值栈峰值多少？

解答：左子树 a+b 需 2；右子树 c+(d+e)需 3；计算右侧时还留左值 1，因此根峰值 max(2,1+3)=4。直接追踪：a,b 合成一个值；压 c、d、e 时共 4 格，随后逐次合并。运算符栈峰值与此无关。

练习 3。 只知道全数组 I=5，n=6，指定 m=4，能否保证内部至少一对逆序？

解答：外部或跨边界最多 9 对，I−9 为负，下界截成 0，因此这个简单计数无法保证内部有逆序。实际反例可取[1,3,5,6,2,4]，指定前 4 项为区间，它们已升序，内部 0 逆序；跨界逆序为(3,2)、(5,2)、(5,4)、(6,2)、(6,4)，恰好 5 对。这样既满足总 I=5，又说明不能保证区间内至少一对。

32 习题 1–32：把十六段程序逐层数清楚

PS01 第 16 项让我们独立分析《数据结构习题解析》1-32。本章对应作者官方合体 PDF 第 446 到 455 页，亦即习题解析印刷页 24 到 33；第 456 页为空白。原程序可从作者教材页进入合体 PDF 阅读。下面保留题号顺序，用自己的推导解释每段执行成本，并把容易混淆的返回值、轮数与递归深度分别列出。

题目使用抽象机器，假设字长足够、移位是单位操作、递归不会因真实机器栈限制而失败。这里的 T 表示执行成本，V 表示返回值。把原程序直接交给固定字长 C++运行，可能先遇到溢出或非法移位；若用大整数，又要另算位成本。这些实现边界不改变题目指定模型下的推导，却决定实验能测到什么。

32.1 1：矩形循环区域，行数乘列数

第一段的核心是 n 轮外循环，每轮 n 次内循环。若把一次主体执行画成坐标(i,j)，正好是 n×n 方格，每格访问一次，总 n^2次。

n=4 时，每行 4 次，共 4+4+4+4=16 次。每个循环还要增加变量、检查条件，内层条件每行约 n+1 次，总也为 $\Theta(n^2)$，不会改变阶。基础操作不是只有函数体里面那一条，但主项可由它代表。

上界是常数倍 n^2，下界同样至少有 n^2次主体，因此为 $\Theta(n^2)$。若只写“有两层 for 所以平方”，这份理由在第 6 与第 8 段就会失效；这里正确是因为每行确实有 n 项。

32.2 2：三角形区域，逐行长度不同

第二段内层从当前外层位置扫到边界。各行长度依次 n、n−1、…、1，因此

$$N(n)=\sum_{i=0}^{n-1}(n-i)=\frac{n(n+1)}{2}=\Theta(n^2).$$

n=4 时为 4+3+2+1=10。它大约是第一段的一半，但渐近阶相同。判断下界可只看前 n/2 行：每行至少 n/2 次，已足以贡献常数倍 n^2。

如果循环端点改为不含当前位置，各行变 n−1、…、0，精确次数变 n(n−1)/2，仍平方级。端点差一会影响精确值，不能在考试问“执行几次”时直接忽略；问 Θ 时才可在证明后舍去低阶项。

32.3 3：内层步长 2013，常数再大也仍是常数

内层每次增加固定 2013，第 i 轮约执行 i/2013 次。取整误差每轮至多常数，总误差 O(n)，主和为

$$\frac{1}{2013}\sum_{i=1}^{n} i=\Theta(n^2).$$

例如将 2013 暂时换成便于手算的 3，外层上限 i 从 1 到 9，相应循环次数可能按端点约定取 ceil(i/3)，依次 1、1、1、2、2、2、3、3、3，总 18；外层规模扩大，仍是二次增长。这个小数值示例解释步长作用，不替换原题中的固定常数。

原常数很大，n 在几百时可能几乎看不到主体工作，测得曲线也可能像线性甚至常数。渐近结论讨论无界增长的 n，不由小规模测时曲线决定。若步长改成 n，就不再是固定常数，必须重新求和。

32.4 4：只做前半外循环，内层却仍然很长

外层只有约 n/2 轮，每轮内循环长度在约 n/2 到 n 之间。上下夹逼为

$$c_1n^2\le N(n)\le c_2n^2$$

其中 c_1、c_2为与 n 无关的正常数，所以 $\Theta(n^2)$。n=8 时，若内层长度为 8、7、6、5，总 26，已经能看出不是只执行 4 次。

这里不必为了得到 Θ 而求一个很长的精确闭式。行数 Θ(n)、每行长度 Θ(n)，且界对所选所有行都成立，直接相乘即可。若某些行短某些行长，不能只取最小或最大后就宣称紧界，应检查足够多行都提供所需下界。

32.5 5：后半个三角形，仍有平方数量的格子

外层走后半段，内层长度从约 n/2 逐步减到 1。总和约为 1+2+…+n/2，所以 Θ(n²)。n=8 的简化行长 4、3、2、1，总 10；n 翻倍时行数与最长行都翻倍，面积约变 4 倍。

旧解答旁的说明若写成另一个区间，应回到实际循环条件数格子，而不是同时抄两套互相冲突的文字。求和下标的范围是程序语义的一部分。

前 5 段的共同结论都是平方，但理由不全一样：矩形、完整三角形、固定步长三角形、梯形、半三角形。把每种区域画出来，后续遇到换边界的程序就能重新分析，而不必记住第几个答案。

32.6 6：外面 n 轮，里面翻倍到 n

内层状态 1、2、4、8……，在达到 n 以前执行约 $\log_2 n$ 次。外层每次都重新从 1 开始，所以每一轮都要重新支付这份成本，总 Θ(n log n)。

n=16 时，若条件为 j<n，内层取 1、2、4、8，共 4 次，外层 16 轮，总 64 次；若条件为 j≤n，则每轮多一次取 16，总 80 次。两者同阶，精确计数却不同。

不要因为内层变量翻倍就说“所有轮合起来是一个几何和”。这里每个 j 对应一次常数操作，而不是做 j 次操作；内层总成本是项数 log n。若主体改成重复 j 次，才需相加 1+2+4+...得到 Θ(n)，再乘外层 n 会变平方级。

32.7 7：翻倍到固定上限，关于 n 就是常数

内层翻倍却只到 2013，与输入规模 n 无关。因为 $2^{10}=1024$、$2^{11}=2048$，不论 n 多大，内层都只有约 11 次检查范围内的主体动作。外层 n 轮，总 Θ(n)。

第 6 与第 7 的代码可能只差一个上限，渐近结论就从 n log n 变为 n。分析要问“哪个量会随输入增长”，不能只搜源代码里的乘法或移位符号。

若常数 2013 本身变成另一个输入参数 M，则应写 Θ(n log M)，再根据 n、M 之间是否有关联决定是否可化简成单变量形式。

32.8 8：步长随外层变化，出现调和和式

第 i 轮内层跨步 i，遍历长度约 n，因此次数为 ceil(n/i)而非 i。外层 i 为 1 到 n−1 时，总数

$$\sum_{i=1}^{n-1} \lceil n/i \rceil = nH_{n-1} + O(n) = \Theta(n \log n).$$

n=8 时各轮 i=1 到 7 的次数 8、4、3、2、2、2、2，总 23。若外层也包含 i=8，再加 1。取整误差每项小于 1，n 项合计 O(n)，比主项 n log n 小。

H_n 为对数级，可把分母按 2 的幂分组：1 一组，2 到 3 一组，4 到 7 一组，每组总贡献在正常数范围内，共 Θ(log n)组。这个解释也说明为什么“大量后面很小的项”仍会累积出一个对数因子。

32.9 9：一个循环，累计步长是三角数

i、j 从 0 开始，每轮做 i+=j，再让 j 加 1。k 轮后 i=k(k−1)/2、j=k。n=10 时状态依次(0,1)、(1,2)、(3,3)、(6,4)、(10,5)，5 轮结束。

退出需要 k(k−1)/2≥n，故 k=Θ(√n)，每轮常数工作，总 Θ(√n)。第一轮 i 没变，但 j 变了；若只盯 i 就说“状态不动”，会漏掉下一轮增量来源。

这里的输入 n 以一个整数给出；若按 n 的二进制位数 b 计，√n 仍可约为 2^(b/2)。模型中所谓“平方根时间”是关于数值 n，不自动等于关于输入编码长度的多项式时间。

32.10 10：i 的指数也在翻倍，得到双对数

r 从 1 开始每轮加倍，i 从 1 开始每轮左移 r 位。左移 r 位是乘 2^r，不是乘 r，也不是只乘 2。

逐轮列出：初始 i=1、r=1；第一轮 i=2、r=2；第二轮 i=8、r=4；第三轮 i=128、r=8；第四轮 i=32768、r=16。k 轮后 i 的二进制指数是 1+2+4+...+2^(k−1)=2^k−1，所以

$$i = 2^{2^k-1}.$$

让 i 达到 n，需 2^k−1≥log₂n，故 k=Θ(log log n)。这是对数的对数，来源是指数本身在几何增长。真实 C++在移位位数达到类型位宽前就要特别处理，不能把这张抽象表无限照跑。

32.11 11：指数塔增长，反复取对数数回去

i 每次变为 2^i，状态 1、2、4、16、65536……。到达一个极大阈值 n，倒着问：对 n 反复取 $\log_2$，多少次降到不大于 1？这个次数记为 $\log_2^* n$，读作迭代对数。

例如 n=65536，依次取 log 得到 16、4、2、1，共 4 次；正向从 1 到 65536 也是 4 次指数化。成本 Θ(log* n)，虽然现实规模下数值很小，作为无界函数仍会增长，不能严格写成 Θ(1)。

它也不是 log log n。若 n 是高度不断增长的指数塔，双对数仍随塔高后的庞大数值变化，log*只记录需要剥掉多少层指数。

32.12 12：G 返回平方，第二次调用吃的是返回值

辅助 G(m)把前 m 个奇数相加：1+3+…+(2m−1)=m²，所以返回值 V_G(m)=m²，执行成本 T_G(m)=Θ(m)。主函数先算 G(n−1)，再将返回的(n−1)²作为另一次 G 的参数。

取 n=5，第一次 G(4)加 4 个奇数得到 16；第二次 G(16)加 16 个奇数得到 256。总加法次数约 4+16=20，而返回 256。一般返回(n−1)⁴，运行成本 Θ(n)+Θ((n−1)²)=Θ(n²)。

把“返回多大”和“执行多久”分成两列，就不会把四次方返回值误当四次方时间。相反，一个函数也可以花很久最后只返回 0，输出大小本身并不决定所有计算成本。

32.13 13：昂贵的是循环条件里的开方子过程

G 从输入 m 中依次减去 1、3、5……，第 k 次累计减去 k²，首次使剩余非正时返回 k，所以 G(m,0)=ceil(√m)，成本 Θ(√m)。

例如 m=10，依次减 1 后余 9，减 3 余 6，减 5 余 1，减 7 余−6，共 4 步，ceil√10=4。m=9 时减 1、3、5 恰到 0，共 3 步，边界也吻合。

F 在第 i 次检查中调用 G(i,0)，条件大致 i<n/G(i,0)。以 G 约√i 代入，外层能走到 I 满足 I^(3/2)约 n，所以 I=Θ(n^(2/3))。这只是条件被检查的轮数，不是总成本，因为每次检查本身花 Θ(√i)。

因此

$$T(n)=\Theta\left(\sum_{i=1}^{I}\sqrt{i}\right)=\Theta(I^{3/2})=\Theta(n).$$

平方根和为何是 I^(3/2)？上界 I 项每项至多√I；下界后 I/2 项每项至少√(I/2)，两者相乘即可。若只写外循环 n^(2/3)，会漏算条件调用；若再无脑乘一个最大√I，则能给上界，但还需后半段提供匹配下界。

原解答一处前面得 n^(2/3)，后面却写 n^(1/3)的文字，应以前面阈值与程序重新推导为准。整数截断版本的条件 `i < floor(n/ceil√i)`可准确变成`(i+1)ceil√i <= n`，便于用纯整数检查。

本书示例/复杂度计数.py 把逐个减奇数与独立整数平方根核对，再将逐项条件阈值与单调二分核对。它验证有限范围算例，没有拿排版结论自己核对自己。

32.14 14：F 只有一条递归链，G 却会越来越长

G(a,b)通过把 a 累加 b 次返回 ab，成本 Θ(b)。F 从 F(0)=1 起，每层通过 G 把前次返回值乘 2，故 F(n)=2^n。n=4 时，F 各层返回 1、2、4、8、16，对应 G 的第二参数依次 1、2、4、8，工作量总 1+2+4+8=15，加 F 自身 4 层常数工作，仍 Θ(16)。一般

$$T(n)=\Theta\left(n+\sum_{k=1}^{n}2^{k-1}\right)=\Theta(2^n).$$

这里指数时间来自每层 G 参数越来越大，不是 F 有两个递归孩子。G 自身的递归深度为 Θ(b)；代入最高层 b=2^(n−1)以后，最大深度才对 n 呈指数。不能把 G(a,b)直接说成深度 2^b，那又多套了一层指数。

32.15 15：孩子规模不同，总量甚至可以低于线性

递推为 T(n)=T(n/2)+T(n/4)+θ(1)，带常数基础情况。令 n=2^k，记 S_k=T(2^k)，得到 S_k=S_(k−1)+S_(k−2)+θ(1)，吸收常数后与 Fibonacci 同阶，所以 S_k=Θ(φ^k)，φ=(1+√5)/2。

换回 n，k=$\log_2 n$，得

$$T(n) = \Theta(n^{\log_2 \varphi}) \approx \Theta(n^{0.69424}).$$

例如规模 64 的孩子为 32、16；32 再分 16、8；16 再分 8、4。虽然每个非叶节点有两个孩子，孩子总规模只有父的 3/4，不像两半分治保留全部规模。只数孩子个数会丢掉缩小比例。

另一种解法令指数 p 满足 2^(−p)+4^(−p)=1，置 x=2^(−p)，则 $x+x^2=1$，正根 x=1/φ，得 p=$\log_2 \varphi$。整数向下取整与很小规模的基础分支，需用单调性和相邻二次幂夹逼处理，改变常数而非指数。

32.16 16：随机分支，把每轮期望加起来

外层 i 由 n 降到 1，每轮若随机数对 i 余 0，就执行 n 次内层。理想模型假设每个模数下余数均匀，触发概率 1/i。记该轮是否触发为 X_i，不管各轮是否独立，都有 E[X_i]=1/i。总成本含外循环 n 次及 nΣX_i，所以

$$E[T] = \left(\Theta n + n \sum_{i=1}^{n} \frac{1}{i}\right) = \Theta(n \log n).$$

n=4 时，触发概率依次 1/4、1/3、1/2、1，期望触发数 H_4=25/12，内层期望工作 $4H_4$=25/3。这个数可以不是整数，它是不同运行整数成本的平均。

最坏每轮触发，Θ(n^2)；最好仍需 n 轮外循环，且 i=1 必触发一次 n 内层，故 Θ(n)。同一程序给出三种不同答案，各自描述一个不同问题，不能把期望当最坏保证。

若真实 rand 在 0 到 R 均匀，余 0 的数为 0,i,2i,...,floor(R/i)i，共 floor(R/i)+1 个，因此准确概率

$$p_i = \frac{\lfloor R/i \rfloor + 1}{R+1}.$$

它一般不恰等于 1/i。如果 R+1≥n，每轮误差至多约 1/(R+1)，合计内层期望误差 O(n^2/(R+1))=O(n)，不改变 n log n 主阶。如果固定 R 而让 n 无限增长，i>R 时触发只对应随机数 0，概率固定 1/(R+1)，累积可能带来平方主项，不能继续引用同一理想渐近结论。

32.17 练习与答案

练习 1。一个函数返回 n^8，能否仅由这个式子推出它至少需要 n^8步？

解答：不能。在固定字或抽象无限字模型中，连续常数次乘法即可计算 n^2、n^4、n^8。位模型下要考虑输出位数与乘法位成本，但输出数值 n^8本身不等于执行次数。第 12 段正是在防止把返回值套进时间结论。

练习 2。第 13 段若预先用 O(1)表查得到每个 ceil√i，外循环时间还为 Θ(n)吗？

解答：单看已准备好的表且查询常数成本，循环轮数为 Θ(n^(2/3))，每轮条件成本变常数，总循环成本也变成该阶。但预处理表与内存成本要另算，不能拿修改后的程序成本冒充原程序。题目中昂贵 G 调用的存在是结论的一部分。

练习 3。第 16 段证明期望时，为什么没用各次随机分支独立？

解答：只求期望总和，线性期望永远成立，知道每轮触发概率就够。若求“所有分支都触发”的概率，或集中界，则可能需要独立性或其他相关条件。每份概率证明只使用它真正需要的假设，既不能少写，也不用额外乱加。

33 教材、习题解析、PS 与 PA，怎样对着解决同一个问题

遇到一段看不懂的证明，不必在同一页上反复硬读。先判断缺的是定义、具体执行、证明方法还是代码约定，再去对应材料补。教材给完整概念，课件常保留课堂推导的图与中间步骤，PS 提出容易混淆的条件，PA 则要求你把它们落实成在规模内可运行的程序。

课程名是数据结构。教材书名、旧版章号、课程课件节号与本书章号各有一套，不能拿某套号直接代替另一套。尤其 2026 教材目前可读部分重点覆盖算法概论到 AVL 的前六章，整套课件继续提供图、散列、堆、串等后续主题；旧版第三版与习题解析的章序又不同。

33.1 先知道自己翻的是哪一份书

本地 2026 教材为 352 页 PDF；作者官方合体 PDF 为 669 页，含第三版主教材与《习题解析》，习题解析从 PDF 第 413 页开始。官方入口是作者教材页，公开原件本地保存在`资料/教材与参考/数据结构（第 3 版）与习题解析（官方合体）.pdf`。

一份 PDF 中印刷页可能从前言重新编号，习题解析又从自己的页码开始。因此引用“第 24 页”不够；本书 32 章明确说“官方合体 PDF446 页、习题解析印刷 24 页”，两者定位到同一位置。搜索提取文字能帮助定位名词，涉及树形、箭头、求和上下限与分式时，仍应看对应原页。

下面按官方书签列出旧版习题解析的 292 道大题。许多题还有小问；它们与 PS01—05 的 184 个编号不是同一套，题量不能相加后就宣称全部完成。

旧版习题范围	大题数	官方 PDF 页	本书对应
1-1 至 1-32，绪论	32	424 至 456	01—03、22、28、31；32 专讲 1-32
2-1 至 2-41，向量	41	458 至 493	04—06、23
3-1 至 3-19，列表	19	496 至 508	06—07、24
4-1 至 4-26，栈与队列	26	510 至 524	08—09、25、30
5-1 至 5-30，二叉树	30	526 至 540	10—11、26
6-1 至 6-32，图	32	542 至 565	16—17
7-1 至 7-20，搜索树	20	568 至 578	12
8-1 至 8-20，高级搜索树	20	580 至 599	13—14
9-1 至 9-26，词典	26	602 至 615	15
10-1 至 10-22，优先级队列	22	618 至 632	11、18
11-1 至 11-10，串	10	634 至 640	19
12-1 至 12-14，排序	14	642 至 650	20

33.2 路线 1：复杂度题，先把“要数什么”固定

先读 01 章的计算模型，明确一条移位、大整数乘法和数组访问是否算常数。再读 02 章的三种和式与递归树，亲自写出每行次数。随后做 PS01：给程序一个小 n，列出循环状态，再写一般阈值。遇到 1-32，32 章按 16 段分别区分返回值、外层轮数和总调用成本。

例如第 13 段 G 算 ceil√i，主循环走到 i 约 `n^(2/3)`。若只看外循环轮数，会答错；把每次条件调用的√i 成本求和，才得到线性。看答案之前要问：“这里哪一句是在数轮数，哪一句是在数工作？”

到 PA 时再将复杂度与规模核对。n=40 的二选一组合不能轻易做 `2^40` 枚举，可考虑两半；字符串长度很大但预算 k 小，可考虑带状 DP。理论阶是选择算法的约束，不是写完代码后用来装饰的结尾。

33.3 路线 2：向量与列表，同一个插入动作付不同的账

读向量课件原图时，追踪容量、size 与复制量，不要只记“扩容摊还常数”。04 章给纯追加序列，28 章再把势能总账写成等式。然后用 PS02 检查不同增长策略、成功失败查找长度和二分接口。

转到列表，先在纸上画前驱、当前节点、后继，写出四根链接。07 章删除例子专门保存旧 next，防止释放后使用。PS03 再检查稳定性与比较次数：列表节点摘接便宜，不等于在有序列表中寻找插入位置也便宜。

若程序输出排序值正确，仍应给相等项加 A、B 标签检查稳定性。选择“最后一个最大值”后远距离交换，可能让另一种相等键反序；06 章给了可手算的反例。用具体记录比“这个算法通常稳定吗”的印象判断可靠。

33.4 路线 3：栈队列，先写状态再谈峰值

08 章先手算括号和 RPN，再读老师中缀优先表。课程原表的乘方左结合与数学常用右结合应分开。PS04 有些题要求原程序，有些要求修改程序，不能把修改后的规则倒灌回原版。30 章对 21 式列出每个运算后的数值栈，便于定位哪次减法或除法先算错。

09 章用窗口最大值和直方图说明删除候选的理由。之后读 PS04 里“单步空间”“整段峰值”“随机期望”的问题，再去 28 章看记录变量与并合界。固定一个窗口的平均候选数小，不代表无限长流的最大候选数同样小。

PA 中不一定直接出现栈，但同样需要问“哪些旧状态仍可能影响未来”。Filename 保留带内可达状态，Gift 保留有重复身份的半和，偏序 DP 保留每个 b 前缀的最佳值。这些压缩都应给出不会丢掉更优解的理由。

33.5 路线 4：树，把所有计数约定写在第一张纸上

先固定高度按边数还是节点数，空树高度取多少，二叉树是否区分左右，是否允许单孩子。10 章统一按边数，给增强先序解析；PS05 再检查普通遍历与带空标记遍历的唯一性。

扩充二叉树是在每个缺失孩子处补一个外部空叶。n 个原节点有 n+1 个空槽，所以补后外叶 n+1，所有原节点都成为双分支节点，共 n 个；不是只给原单孩子节点补一次。原叶的左右两个缺口也必须补。这个约定与增强先序中 n+1 个^是同一份结构信息的两种写法。

BST、AVL 先用 12 章小树执行删除与旋转，再回到教材的通用节点重构；不要只看最终树形。Huffman 先用 11 章算合并权重总和，再读贪心交换证明和比较下界。最优编码的成本、构造算法的复杂度、输出所有码字的长度，是三个不同的量。

33.6 原书十二章，各完整走一类代表题

以下题号都来自旧版《习题解析》，定位均为官方 669 页合体 PDF 的物理页。它们让你看到题目与本书的具体接点，并不代替原书所有题。题意采用转述，算例另设。

33.6.1 绪论 1–1，PDF424 页：尺规算法也要证明输出关系

任务是在直线 l 外给定点 P，作过 P 的平行线。取 l 上一点 A，以 AP 为半径作圆，与 l 交于 B；再分别以 B、P 为圆心、同半径作圆，两圆除 A 外的另一交点为 C，连接 PC。

为什么 PC 平行 l？AB=AP，且 BC=PC=AP，所以 A、B、C、P 依次构成菱形，PC 与 AB 平行，AB 又在 l 上。P 不在 l 上保证相关三角形不退化。算法的基本操作是定半径作圆、取交点、连直线，并非必须先写成 C++才叫算法。01 章说明输入条件与基本操作模型，尺规题同样要交代这些前提。

33.6.2 向量 2–2，PDF458 页：重叠复制怎样选方向

源区间与目标可能重叠时，按地址判断。数组[A,B,C,D,E]将前三项复制到从下标 1 起的位置，目标在源右侧，必须倒着：先 C 到 3，再 B 到 2，再 A 到 1，结果[A,A,B,C,E]。若正向，A 先覆盖 B，后面会不断复制 A。

反过来将下标 1 到 3 的 B,C,D 复制到 0 到 2，目标在左，正向得到[B,C,D,D,E]。若倒着，先 D 覆盖下标 2 的 C，再向前读时已丢原 C。一般规则是右移重叠区从后向前，左移重叠区从前向后；不重叠时两向都可。04 章的插入和删除正是在使用这条覆盖依赖关系。

33.6.3 列表 3–3，PDF496 页：唯一数据节点删除以后是什么

双哨兵链表H↔X↔T，X既是首数据节点也是末数据节点。先保存p=X.pred=H、q=X.succ=T；令p.succ=q、q.pred=p，再释放X，size从1变0。得到H↔T，恰好是空表的不变式。

没有特殊分支仍能成立，是因为H、T虽然不属于用户数据，却提供合法邻居。不能将X删除后再读X.succ，也不能把H或T当作可删除数据。07章的同一套remove对中间、首尾与单节点情形统一适用，前提是哨兵确实存在且不被当作数据。

33.6.4 栈与队列 4–1，PDF510 页：用列表实现栈，接口不止换个名字

令列表末端为栈顶：push(e)调用insertAsLast(e)，pop调用remove(last())，top返回last.data；size与empty沿用列表管理信息。双向列表有尾哨兵时，取末节点与在末端插删都是O(1)，因此标准栈接口都可最坏常数。

空栈的pop、top必须拒绝或返回失败，不能把尾哨兵的数据当结果。若还继承了find接口，应核对失败返回：列表返回空位置，向量可能返回负秩，调用代码必须相应修改。原页代码用last与insertAsLast，旁边个别“首元素”注释应按实际调用理解为末端；不能把互相冲突的注释当作两套同时成立的方向。

具体执行push A、push B、pop得到B，剩A，top仍A。若换成首端做栈顶也可，但遍历显示顺序会反过来，使用者若依赖“底到顶”的输出，需统一约定。08章强调的是后进先出关系，而非底层一定用数组。

33.6.5 二叉树 5–1，PDF526 页：深度加高度为何不超过整树高

取任意节点v，从根到v长depth(v)，再取v子树里最深叶x，v到x长height(v)。两段接起来是整树中一条根叶路径，所以depth(v)+height(v)=depth(x)≤height(T)。若v子树包含至少一个整树最深叶，便取等；若不包含，子树内所有叶都更浅，不等号严格。

例如根R左边有三条边长链，右孩子Q是叶。整树高3，Q深1、高0，相加1<3；左链上的节点均位于某条最深路径，相加3。证明不要求树平衡，也不要求最深叶唯一。10章统一的按边计高约定可直接用于这道题。

33.6.6 图 6–1，PDF542 页：关联矩阵乘自己转置，究竟保留了什么

无向链A-B-C，两条边分别为列，关联矩阵为`I=[[1,0],[1,1],[0,1]]`。两行点积共同计数两顶点关联的边，故`I I^T=[[1,1,0],[1,2,1],[0,1,1]]`。对角是度1,2,1，非对角为相邻边数；简单无环图下，非对角就是邻接矩阵。

有向链A→B→C，把出发端记−1、到达端记+1，则`I=[[-1,0],[1,-1],[0,1]]`，乘积为`[[1,-1,0],[-1,2,-1],[0,-1,1]]`。它记录总度与忽略方向的连接数，却无法恢复边方向，因为将某列整体取反不改变乘积。别误把这个对称乘积叫作原有向邻接矩阵。

应用例子是差分约束x_v−x_u≤w，转成边u→v权w。给各点一个超级源0权到达边，最短距离满足所有`d[v]≤d[u]+w`，可作可行赋值；若存在负环，将环上不等式相加会得0≤负数，矛盾。17章Bellman–Ford检测负环因此能处理这类可行性问题。矩阵方向编码负责表达约束，乘积本身并没有保留全部有向信息。

33.6.7 搜索树 7–1，PDF568 页：局部孩子大小还不够

BST要求左子树所有后代≤根，右子树所有后代≥根。中序按左、根、右列出，因此非降。反过来若中序非降，对任意根，所有左子树项都出现在根前且不大于根，所有右项都在根后且不小于根，故满足这种允许重复的BST定义。

仅检查孩子会漏错：根10左孩子5，5右孩子12，三个局部关系看似左小右大，但12位于10左子树且大于10，不是BST，中序5,12,10也不有序。12章先用互异键得严格递增；原书这题允许非严格关系，重复键的策略需另加约定。

33.6.8 高级搜索树 8–1，PDF580 页：重复键怎样保留身份并批量删除

一种完整实现是将每条记录的内部比较键定义为(用户键k,单调唯一序号id)，字典序严格比较；对外仍按用户键查找。相等用户键按id连续存放，中序不会混进别的键，旋转也不会破坏此序。

search(k)可以用lower_bound((k,最小序号))找第一个用户键k，找到后伸展到根；searchAll从该项沿中序后继走，直到用户键变化，输出全部k条记录，成本摊还O(log n+k)。插入每次给新id，按普通伸展插入。

remove(k)删找到的一条；removeAll 可先按用户键边界分裂成<k、=k、>k 三部分，销毁中间所有节点，再合并两侧。销毁 k 个节点本身要 Ω(k)，不能宣称整个批量删除始终 O(log n)。

例如插(5,A)、(3,B)、(5,C)，内部序为(3,idB),(5,idA),(5,idC)。查全部 5 得到 A、C；删其中一项后另一项仍在，不能只用一个布尔“键 5 存在”代表多条记录。13 章讲分裂合并与摊还条件，新增 id 只是把等值身份定义清楚，不免除节点释放与 parent 更新。

33.6.9 词典 9–2，PDF602 页：一座塔不要重复复制整个词条

跳转表某键高 4 层，若每层都复制一份长字符串和值对象，会浪费 4 份大载荷。可让底层数据节点唯一拥有 payload，高层索引只存指向同一 payload 或底层节点的引用及横纵链接。

查询在高层停止时，沿引用返回那一份数据；更新值只改一次，所有层看见同一记录。删除时先拆掉整座塔的索引链接，再释放唯一 payload，避免其他层留下悬空引用。若外部还持有引用，要由接口另行约定生命周期。链接数期望 O(n)的前提不变，但每层不再重复存可能很大的键和值；13 章的索引快车道因此与对象所有权接起来。

33.6.10 优先级队列 10–1，PDF618 页：四个基础实现，把三个接口都算一遍

无序向量尾部插入摊还 O(1)，找最大 O(n)，删最大可扫描后用末项填洞，总 O(n)。无序列表可常数插节点，找最大与删前寻找仍 O(n)。若缓存最大位置，getMax 可 O(1)，但删最大后重找下一最大仍需 O(n)，缓存没有免除这笔工作。

有序升序向量在正确秩插入需寻找并搬移，O(n)，最大在尾，getMax 与尾部删除 O(1)或按收缩策略给摊还界。有序双向列表插入需 O(n)寻找位置，末端取最大与删末节点 O(1)。保持全序使删除极值便宜，却使插入较贵。18 章的完全二叉堆只维护父子序，getMax O(1)，插入与删最大 O(log n)，正是放松全序换来的平衡。

33.6.11 串 11–1，PDF634 页：前缀、后缀与子串的坐标

以 S=ABCDE 为例，长度为 5，长 2 前缀 AB，长 2 后缀 DE。按原题 Excel 式从 1 编号，mid(S,2,2)取 BC；换成本书从 0 起的区间，就是 S[1:3]。相等比较要求长度相同且每个对应字符相同，ABC 与 AbC 在区分大小写的 exact 语义下不同。

这道看似简单的接口题在提醒你：课件的秩、语言切片与工具界面起点可能不同。字符长度也要看采用字节、码点还是其他单位；不能拿一个软件的显示长度直接当另一个 API 的字节偏移。19 章先固定字符模型，再定义 π 与匹配起点，正是为了防止这类坐标错位。

33.6.12 排序 12–1，PDF642 页：一个扫描指针也能完成轴点划分

原题版本 C 把轴值先放 lo，mi 指向已知小于轴的区域末尾，k 从 lo+1 向右扫。遇 a[k]<pivot，就先 mi 加 1，再交换 a[mi]与 a[k]；否则只前进 k。最后交换 a[lo]与 a[mi]，返回 mi。

以[4,7,2,5,1]、lo=0、轴 4 为例：7 不动；2 小，mi=1，交换得[4,2,7,5,1]；5 不动；1 小，mi=2，得[4,2,1,5,7]；最后 4 与下标 2 交换，得[1,2,4,5,7]，轴点秩 2。扫描中[lo+1,mi]严格小于轴，[mi+1,k)不小于轴，未知段从 k 开始；每步都缩小未知段，扫描线性。

这里返回单个轴点，而 20 章的三路划分返回整个等值区间，递归边界不能混用。若所有值相等，版本 C 每次 mi 仍为 lo，递归侧大小 n−1，仍可能平方；随机选轴无法改变“所有值相同”这个事实，三路划分才会一次跳过全部等值项。

33.7 范围说明

这本书按现有章节讲解核心概念，按 22—26 章逐项处理本学期五张 PS，32 章专门重做原书 1-32。取得 669 页原件并建立 12 章索引，不等于 292 道原书大题及所有小问都已经逐题完整解答。查到一个题号以后，仍要看对应章节是否实际给出了该题的证明与计算。

课内样卷中带使用限制的原件留在本地，31 章用另设实例提炼方法。个人作业、转系资料和群聊照片也不因写教材而进入公开仓库。公开教材中的例子用于教学与检查，不替代课程发布的正式约束与独立提交。

资料覆盖、原图出处与使用说明

这是一份独立组织、重新讲解的教材式初稿。课程范围首先按教师资料；并非教师审核的官方教材，也不把补充证明和预习部分自动列作考试要求。

章节覆盖表

教学来源	本书对应
教材/整套讲义第 1 章算法概论	01—03、22、28、31
第 2 章向量	04—06、23
第 3 章列表	07、24
第 4 章栈与队列	08—09、25、30
第 5 章二叉树	10—11、26、29
第 6 章 BST 与 AVL	12
第 7 章搜索树应用	14
第 8 章高级搜索树	13
第 9 章词典/散列	15
第 10 章图基础	16
第 11 章图进阶	16—17
第 12 章优先级队列	18
第 13 章串	19
第 14 章排序	20
已发布 PS01—PS05	22—26 逐题导航，题数 32+43+37+35+37=184
PA1 讲解 PDF 与 PPTX	21：Filename、Interview、Gift、Graphics、二维偏序
编程/白盒指引与模板	27：规格、复杂度、边界与验证
两份本地样卷	31：只提炼通用方法，不转载课内受限原题

原生解析与视觉核对

PDF 提取原生文字与页码，PPT/PPTX 提取文字和内嵌图片，同时原生兼容导出完整幻灯片以保留箭头、布局、曲线与公式。机器已为可用课件生成完整页级联系表；写入笔记的关键原图逐幅视觉核对。未声称已对数千页的每一个细节完成逐字人工审校。PPT 导出中缺失中文字体替换为 Noto Sans CJK SC，原件不覆盖。

原生提取和视觉复核需要互相补足：抽出的字不能告诉你箭头指谁，图片也不总能准确给出公式符号。课件中的省略或不一致处，本书在相关段落说明，不把它们无条件当作定理。

原图逐项出处

- `老师原图-向量.png`：`数据结构/slides/分章课件/02 向量.pdf`，PDF 第 29 页。直接渲染原始 PDF 页，不改图中结构。
- `老师原图-单调栈.png`：`数据结构/slides/分章课件/04 栈与列表.pdf`，PDF 第 95 页。直接渲染原始 PDF 页，不改图中结构。
- `老师原图-Huffman.png`：`数据结构/slides/分章课件/05 二叉树.pdf`，PDF 第 105 页。直接渲染原始 PDF 页，不改图中结构。

- 老师原图-增强遍历.png：数据结构/slides/分章课件/05 二叉树.pdf，PDF 第 87 页。直接渲染原始 PDF 页，不改图中结构。
- 老师原图-AVL.png：数据结构/slides/整套讲义/电子讲义(按章节).7z，DSA-2026F-Chapter/06.二叉搜索树.pdf，PDF 第 30 页。直接渲染原始 PDF 页，不改图中结构。
- 老师原图-KMP.png：数据结构/slides/整套讲义/电子讲义（按章节）.7z，DSA-2026F-Chapter/13.串.pdf，PDF 第 28 页。直接渲染原始 PDF 页，不改图中结构。

原创 SVG 图由工具脚本/教材/配图.py 生成，文字转为轮廓，可离线显示；它们是本书的补充示意，不伪装成教师原图。

版本和复核边界

2026-10-25，用新版 OneTHU 默认共享后台只读核验课程文件清单，三门课文件枚举均返回 complete=true。数据结构的综合资源接口对作业全量枚举仍给出未核实状态，因此本书不宣称覆盖了独立 OJ 未来/全部题目。随后已从作者官方入口取得 669 页主教材与《习题解析》合体 PDF；数据结构 32—33 章补充 1-32 的 16 种程序推导和全部 12 章交叉索引。取得全文不等于已经把其中 292 道大题所有小问逐一重做，具体覆盖按各章说明。

完整文字、公式与图在 Markdown 和 Typst 间共享；每次改正文后应运行构建器，Pandoc 公式转换警告将阻止成功构建。重要例题按推导、量纲、边界和独立小规模程序交叉检查，发现错误应同时修正文并重建两版本。

延伸阅读：MIT 算法课讲义。本书的章节编号与课程本地教材对应，不照搬另一门课的组织。

本版补充阅读的具体落点

33 章现在不仅列目录，还转述并独立讲解旧版习题 1-1、2-2、3-3、4-1、5-1、6-1、7-1、8-1、9-2、10-1、11-1、12-1，逐项给官方合体 PDF 物理页；相关页已结合原生文字与原页图形核对。13 章额外对照习题 8-2 的 PDF580 至 582 页，展开伸展访问引理的三种局部势能推导。32 章按 1-32 的 16 段分别给状态、阈值、求和与模型条件；这仍不表示 292 道原书大题及其全部小问已逐题解答。

30 章的 21 式从只有结果扩成每次实际运算与栈状态表，除法用精确有理数。教师原版乘方左结合与数学常见右结合现已分别说明；这 21 式没有无括号连续乘方链，数值不受该区别影响。代码中的乘方 token 使用自定义运算语义，不能直接按 C++异或解释。

原生提取用于准确检索定义和程序，视觉核对用于检查箭头、树形、上下限、缺失孩子和优先表；本版没有把生成联系表等同于已经逐页读完全部源材料。实际讲解范围以正文例题与题号为准。